SITE AUTOMATION

# Operations Runbooks

Health checks, troubleshooting, ServiceNow, POA&M and patching - with AAP and, later, ACT

Setup guide for new AAP admins | site-automation 0.28.0 | October 2026

**What this is.** A step-by-step guide to setting up the site's operations runbooks in Ansible Automation Platform: health checks (system, compliance, MariaDB / MySQL, certificates, patching), troubleshooting, ServiceNow tickets, POA&M status and patching, and later ACT (GenAI). It starts with what each AAP object *is* and walks through every click, saying what each step does, why, and what you should see.

**How to read it**

New to Ansible or AAP: read chapter 1 first, then do chapter 2 in order. Inventory already in AAP? Chapter 3 replaces the inventory parts of chapter 2. The later chapters are reference: open them when you build a workflow, read a finding, add ACT, change a setting, approve commands or set up the MariaDB / MySQL check.

Every setting, with its default and what it does - and where to put it - is in its own PDF, the *Settings Reference*: keep it open next to this one.

Rendered from `docs/` in the site-automation repository (version 0.28.0). The repository is the source of truth.

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

# Contents


**1. Start here: the pieces and how they fit 11**

What the runbooks do . . . 12
The ideas behind Ansible, in plain words . . . 13
The AAP objects, and what each one is for . . . 13
How a health check works, start to finish . . . 15
The findings: the one idea that makes everything modular . . . 15
Where things are . . . 16
The order to do things in . . . 16

**2. Setting up AAP, step by step 17**

Before you start . . . 17
Step 1. Put the repository in your work Git, with your inventory . . . 17
Step 2. Credential types . . . 18
Step 3. Credentials . . . 18
Step 4. Project . . . 19
Step 5. Inventory (filled from the repository) . . . 19
Step 6. The first job template: Health check . . . 20
Step 7. Run it on one host and read the output . . . 21
Step 8. Tune the thresholds . . . 21
Step 9. The other job templates . . . 22
Step 10. ServiceNow . . . 29
Step 11. Workflows and schedules . . . 30
Step 12. Who can do what . . . 30
If something goes wrong . . . 31

**3. Using the inventory you already have in AAP 32**

The easy way: your settings files, already named after your groups . . . 32
How variables work in AAP (two minutes) . . . 34
Step 1. Look at what you have . . . 34
Step 2. Create the groups the runbooks use . . . 35
Step 3. Settings for every host (inventory variables) . . . 36
Step 4. Settings for particular groups . . . 36
Step 5. Settings for one host (only when one host is different) . . . 37

Step 6. Use your inventory in the job templates . . . 37
Step 7. Check that it worked . . . 38
Which YAML files in the repository do I change? . . . 38
**4. ServiceNow: set up, test and verify 39**
What you need . . . 39
Step 1. Ask the ServiceNow admins for an API account . . . 39
Step 2. The addresses . . . 40
Step 3. The network path from AAP . . . 41
Step 4. Check from a shell first (optional, fastest) . . . 42
Step 5. The credential in AAP . . . 42
Step 6. Your settings . . . 43
Step 7. The ServiceNow - test ticket template . . . 43
Step 8. Run it: first a dry run, then for real . . . 44
Step 9. Check the ticket in ServiceNow . . . 45
When a step fails . . . 46
Next . . . 46
**5. Workflows and schedules 48**
How to build any workflow, click by click . . . 48
Adding ACT to a workflow: what "on fail" means, and where site_act goes . . . 49
Linux workflows . . . 50
Windows workflows . . . 59
Linux and Windows together . . . 62
VMware workflows . . . 63
Horizon workflows . . . 65
Schedules . . . 66
Things that trip people up . . . 68
**6. Every job template: what it is, and where its how-to is 69**
Linux servers . . . 70
Windows servers . . . 71
Certificates and trust stores . . . 72
VMware (vCenter) . . . 73
Horizon 8 (VDI) . . . 74
Network devices and storage . . . 75

ServiceNow, AAP and the reports . . . 75
What one job hands to the next . . . 76
**7. Dry runs: try anything, change nothing 78**
Where to set it . . . 78
What each template does in a dry run . . . 79
Examples . . . 80
What a dry run cannot tell you . . . 81
**8. The runbooks: every check and what to do 82**
Health check (playbooks/health_check.yml) . . . 82
Troubleshoot (playbooks/troubleshoot.yml) . . . 97
Certificate report (playbooks/cert_report.yml) . . . 98
Trust store report (playbooks/trust_report.yml) . . . 99
Windows trust store report (playbooks/win_trust_report.yml) . . . 100
VMware trust store report (playbooks/vmware_trust_report.yml) . . . 100
Certificate renewal: the CSRs (playbooks/cert_renew_csr.yml) . . . 100
POA&M status (playbooks/poam_status.yml) . . . 103
ServiceNow tickets (playbooks/servicenow_tickets.yml) . . . 104
ServiceNow - test ticket (playbooks/servicenow_test_ticket.yml) . . . 104
ServiceNow health (playbooks/servicenow_health.yml) . . . 105
Patch hosts (playbooks/patch_hosts.yml) . . . 105
Apply approved ACT fix (playbooks/act_fix_approved.yml) . . . 106
VMware jobs and reports (playbooks/vm_*.yml) . . . 106
Account lockout (playbooks/ad_lockout.yml) . . . 106
Horizon 8 (VDI) (playbooks/horizon_*.yml) . . . 106
VMware pick lists (playbooks/vm_pick_refresh.yml, vm_snapshot_delete_picked.yml, vm_reboot_picked.yml) 107
Windows hosts . . . 107
Not built yet (and why) . . . 107
**9. Adding ACT (GenAI) later 108**
How they fit: checks find WHAT, ACT finds WHY . . . 108
The three levels . . . 108
Turn it on . . . 109
Both endpoint formats and :probe (ACT 0.6.19 and newer) . . . 112
ACT for vCenter alarms (it runs on the AAP side) . . . 115

Write your own check (it works with the report, tickets and ACT automatically) . . . 115
Use ACT in a playbook of your own . . . 117
What ACT is told, and what it cannot do . . . 117
**10. Windows servers 118**
Step 1. WinRM on the servers . . . 119
Step 2. The execution environment . . . 119
Step 3. The credential . . . 120
Step 4. The Windows inventory . . . 120
Step 5. Test the connection . . . 121
Step 6. The job templates . . . 122
Step 7. Workflows and schedules . . . 125
Start small: the pilot order . . . 125
Patching through Software Center . . . 125
ACT on the servers (C:\ProgramData\act) . . . 127
Settings: what each check looks at, and how to change it . . . 128
If something goes wrong . . . 129
Not in this release (next) . . . 129
**11. VMware jobs (vCenter): restart, snapshots, VLAN, reports, alarms and ACT 130**
Before you start . . . 133
Which VMs a job acts on . . . 135
The templates . . . 136
Snapshots: the report, the cleanup, and the approval workflow . . . 137
Pick lists: choose what to restart, which snapshots to delete . . . 141
Health: every datacenter, cluster and host . . . 144
VMs created: the weekly report . . . 146
Alarms: the report, and ACT's analysis . . . 147
Outage watch: what is down or hung right now . . . 150
Capacity planning: overall and per cluster . . . 150
Deploy a VM from a ServiceNow ticket . . . 152
Trust stores: vCenter's trusted roots and the ESXi hosts' trusted CAs . . . 153
ESXi security settings (every night): SSH off, timeouts, lockdown . . . 153
Certificates of vCenter and the ESXi hosts: incidents and CSRs (weekly) . . . 155
Email the report . . . 159
What you see . . . 161

Troubleshooting . . . 162
Limits . . . 173
**12. VM outage watch: what is down or hung right now 174**
What it looks at . . . 174
What you see . . . 175
Set it up . . . 176
Settings . . . 177
Phase 2 - the response (not built yet) . . . 178
Event-driven, later . . . 178
Troubleshooting . . . 179
**13. Log analysis: the patterns in a group of servers' logs, explained by ACT 180**
What it does . . . 180
What it reads, per kind of host . . . 181
The settings . . . 181
Profiles . . . 183
Set it up in AAP . . . 184
Sizes and the GenAI quota . . . 184
**14. NetApp ONTAP health report 185**
What the report shows . . . 185
Before you start . . . 186
Set it up . . . 187
Settings . . . 188
Troubleshooting . . . 190
Limits . . . 191
**15. Network device certificates: report and renewal 192**
Which devices . . . 192
What is due . . . 193
The renewal . . . 193
Templates . . . 194
Troubleshooting . . . 194
**16. Trust stores: who still trusts a compromised or dropped CA 195**
The list . . . 195
What it reads, on a Linux host . . . 197

The findings . . . 198
Windows . . . 199
VMware . . . 200
Removing them . . . 200
Set it up . . . 201
When the fixes land . . . 201

**17. STIG drift: has a server moved away from its checklist? 202**

What it reports . . . 202
On the host . . . 203
STIG Manager: the login and finding each host's asset . . . 203
Setting it up . . . 204
Settings . . . 204
Not in this version . . . 204

**18. Network device configurations: backup and load 205**

Which devices . . . 205
The backup . . . 206
The load . . . 206
Templates . . . 208
Troubleshooting . . . 209

**19. The certificate inventory: CSV, JSON and SQL Server, for a dashboard 210**

What each certificate has . . . 211
Files: CSV and JSON . . . 212
SQL Server . . . 212
Schedule . . . 213
Troubleshooting . . . 213

**20. Certificate renewal through the PKI's ServiceNow catalog item 214**

What a run does . . . 214
Before you start . . . 215
Step 1 - the item's questions (their internal names) . . . 215
Step 2 - the settings . . . 216
Step 3 - the catalog test . . . 217
Step 4 - the renewals . . . 217
After the PKI issues the certificate . . . 218

When it goes wrong . . . 219
Limits . . . 219

**21. Installing the issued certificate: from the request, with an approval 220**

What it does . . . 220
Never automatic - a person installs it (BY HAND) . . . 221
Held for a person (HELD) . . . 222
Which service, which key (Linux) . . . 222
Setting it up . . . 223
Put the old certificate back . . . 224
When it goes wrong . . . 225
Limits . . . 225

**22. Emailed reports: the formatted email and its look 227**

Which jobs email . . . 228
Turning it on . . . 230
Also email the report to (asked at launch) . . . 231
Green or red: when a report finds problems . . . 232
What a report looks like . . . 233
The look: standard, command, portal or glass, your logo, the last runs . . . 233
Also on a web page . . . 234
Changing the look . . . 234
Emailing a report from another playbook . . . 234
Troubleshooting . . . 235

**23. The report website: every report as a web page 237**

What it writes . . . 237
Where the files go . . . 238
Turning it on . . . 241
The copy job: report templates in any execution environment . . . 241
Using the address in a workflow or another job . . . 243
Troubleshooting . . . 244

**24. Secrets: where they live 246**

Recommendation . . . 246
Where each secret lives . . . 247
Rules for every playbook in this repository . . . 248

The real trust boundary . . . 249
Keeping secrets out of Git . . . 249
Rotation checklist . . . 250
**25. Approved commands: what ACT may run by itself 251**
Read this first: two safety rules that you cannot switch off . . . 251
How a command becomes "approved": the three ways . . . 251
Add an approved command, step by step . . . 252
How to verify it worked . . . 254
Reference: what the guard rails are . . . 256
**26. The MariaDB and MySQL check, step by step 257**
The short version . . . 257
1. Which hosts the check runs on . . . 257
2. Tell it where MariaDB is . . . 258
3. Where to put the settings . . . 259
4. The login: the "MariaDB monitor" credential . . . 259
5. What it checks, and the limits you can change . . . 260
How to verify . . . 262
What was tested, and what was not . . . 262
If it does not work . . . 263
**27. Active Directory health report and account lockout 264**
What the report shows . . . 264
Before you start . . . 267
Set it up . . . 267
Settings . . . 268
Troubleshooting . . . 270
Limits . . . 270
Account lockout . . . 271
**28. Horizon 8 (VDI): reports, broken desktops, user help, image push 275**
Your Horizon version . . . 276
Step 1. The service account and its Horizon role . . . 276
Step 2. The credential type and the credentials . . . 277
Step 3. The job templates . . . 277
Workflow: fix broken desktops, with an approval . . . 278

Workflow: push a new golden image, with an approval . . . 279
User help (the service desk) . . . 280
Session cleanup . . . 281
Reports and schedules . . . 281
Messages and what to do . . . 282
**Appendix: credential types to paste into AAP 283**
ServiceNow API (servicenow_api.yml) . . . 283
MariaDB monitor (mariadb_monitor.yml) . . . 283
Keystore password (keystore_password.yml) . . . 284
STIG Manager API (stigman_api.yml) . . . 284
ACT model key (act_model_key.yml) . . . 285
ACT GenAI beta key (act_genai_beta_key.yml) . . . 285
STIG Manager database (stigman_database.yml) . . . 286
TLS certificate (tls_certificate.yml) . . . 286
Container registry login (hosts) (registry_login.yml) . . . 286
Teams webhook (teams_webhook.yml) . . . 287
SMTP relay (smtp_relay.yml) . . . 287
Report website (report_web.yml) . . . 288
Horizon API (horizon_api.yml) . . . 288

# 1. Start here: the pieces and how they fit

**Confused by all the moving pieces?**
Read HOW_IT_FITS_TOGETHER.md (or its PDF in `docs/pdf/`). It shows what the code, the inventory, the credentials and the survey each do, how a setting travels from a YAML file to a host, and how to update the repository safely.

This repository holds **runbooks**: jobs that check Linux servers and help troubleshoot them. You run them from **Ansible Automation Platform (AAP)**, by hand or on a schedule. This page explains the few ideas you need. SETUP_AAP.md then walks you through the setup, one click at a time.

## What the runbooks do

| Runbook | What it answers | When it runs |
|---|---|---|
| **Health check** | Is anything wrong on these servers right now? Disk, mounts, services, load/memory, time, network, logging, SELinux, fapolicyd, auditd, accounts, certificates, patching, MariaDB / MySQL | Daily (the "daily" set) and weekly (the "weekly" set) |
| **Database health** | Are the MariaDB / MySQL databases (on the host or in podman containers) running, with connection room, no stuck replication, and disk space? | Hourly or daily, on its own |
| **Troubleshoot** | Someone reports a problem on a host. What does it look like? | On demand |
| **Certificate report** | Which certificates expire soon, on every server, in one list? | Weekly |
| **Trust store report** | Which servers (Linux, and Windows with its own template), Java, Python, Node, containers and Go programs - and, with its own VMware template, vCenter and ESXi - still trust a compromised or dropped certificate authority? (TRUST_STORES.md) | Weekly (the check `trust` is in the weekly set), and when a CA incident is announced |
| **STIG drift report** | Has a server moved away from its STIG checklist? An OpenSCAP STIG scan of each Linux host against what STIG Manager has as Not a Finding (read-only) and against the host's last scan; the scans stay on the hosts (STIG_DRIFT.md) | Weekly, and after patching or a change window |
| **POA&M status** | Which POA&M items are overdue or due soon? Which open CAT I/II findings have no POA&M item? | Weekly |
| **ServiceNow tickets** | Opens or updates a ticket for each problem a check found. Notes the ones that cleared | After every check (as a workflow step) |
| **ServiceNow health** | Is ServiceNow answering? Are the MID Servers up? | Hourly |
| **Patch hosts** | Installs updates, and proves every service came back. Reboots only with `automatic_restarts: yes` | Monthly, after an approval |
| **Apply approved ACT fix** | Applies exactly the fix a person approved | Only inside a workflow, after an approval |
| **NetApp ONTAP health report** | Is any node, disk, port, volume, aggregate, SnapMirror or certificate in trouble on the storage clusters? (NETAPP.md) | Daily |

| Runbook | What it answers | When it runs |
|---|---|---|
| **VMware jobs** | Restart, shut down, snapshot, notes or change the VLAN of VMs through vCenter. Reports: secure boot, datastores, snapshots (with an approved cleanup), alarms and events, and ACT's analysis of the alarms: likely cause, fix and confidence (VMWARE.md) | On demand; the reports daily or weekly |
| **VM outage watch** | Is something down or hung right now: hosts, HA, datastores, VMs on a question, hung VMs (no heartbeat AND no answer on the network). Quiet when clean; a report and a ServiceNow incident per problem when not. Watch only for now: it restarts nothing (VM_OUTAGE_WATCH.md) | Every 5 to 15 minutes |
| **Log analysis** | Point it at a group of servers (Linux, Windows, a vCenter appliance) or at vCenter and its ESXi hosts: their log lines grouped into patterns, and ACT's likely cause, fix and confidence for each (LOG_ANALYSIS.md) | On demand, or daily |

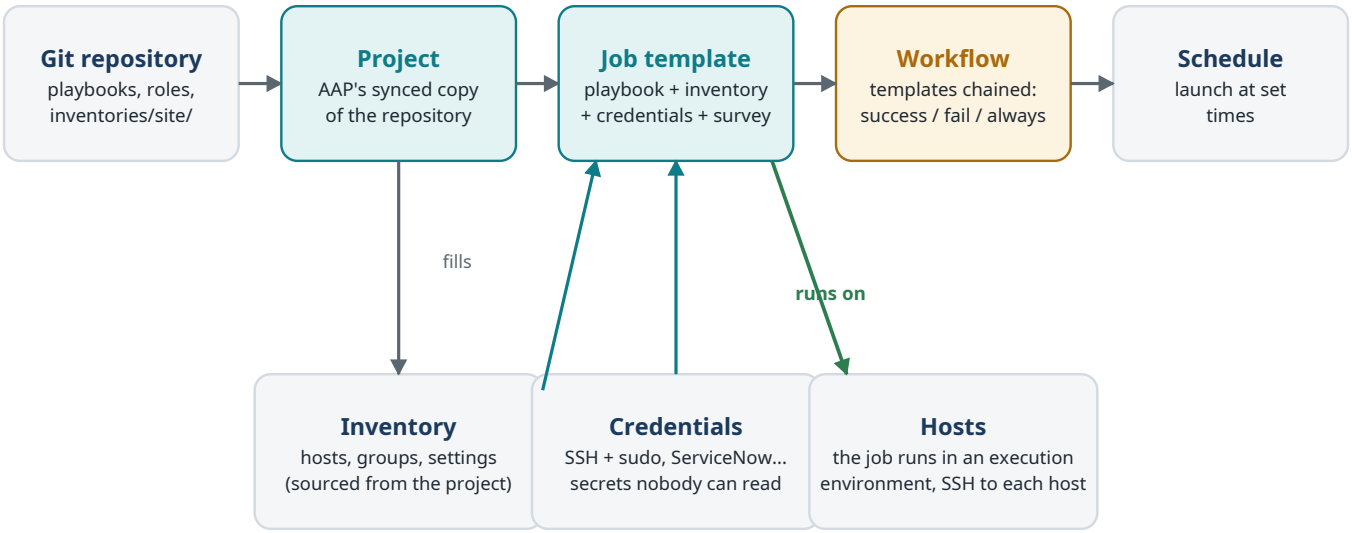
Every check is **read-only**. It looks and reports, and changes nothing. Patching changes servers, and waits for an approval first; the VMware jobs change only what an operator asked for, and never the AAP server's VM.

## The ideas behind Ansible, in plain words

- **Host**: a server Ansible connects to, over SSH, the way you would.
- **Inventory**: the list of hosts, sorted into **groups** such as `stigman` (the STIG Manager servers) or `mariadb`. The built-in group `all` holds every host. A group can also carry **settings (variables)**, for example "on the database hosts, the MariaDB container is called servicenow-mariadb".
- **Task**: one step, such as "run `df` and read the result". A **module** is the tool a task uses (`command`, `shell`, `uri`, `set_fact`...).
- **Role**: a folder of tasks that does one job, plus its settings. `roles/check_disk/` is "check the disks". Its settings, with their defaults, are in `roles/check_disk/defaults/main.yml`.
- **Playbook**: the file you run. It says which hosts, and which roles or tasks to run on them. `playbooks/health_check.yml` says "on every host, run the checks I was given".
- **become**: run as root through sudo. The checks need it to read root-only information (the audit log, /etc/shadow).
- **Check mode**: a dry run. Ansible shows what it would change and changes nothing. Every runbook here works in check mode, and the checks give the same answers in both modes.
- **Idempotent**: running it twice does no harm. The second run changes nothing.

## The AAP objects, and what each one is for

AAP is a web front end that runs playbooks for you. It keeps the passwords, records every run, and runs things on a schedule. You set up a handful of objects once. Each one points at the next:




*The AAP objects you create once. The job template is what you launch; a workflow chains templates; a schedule launches either.*

| AAP object | What it is | Here |
|---|---|---|
| **Project** | A copy of a Git repository that AAP keeps in sync | `site-automation` (this repository) |
| **Inventory** | The hosts and groups, and their settings | `Linux servers`, filled from `inventories/site/` in this repository, or the inventory you already have in AAP (USING_YOUR_AAP_INVENTORY.md) |
| **Credential type** | A form for a kind of secret, and how to hand it to the job | `ServiceNow API`, `MariaDB monitor`, `ACT model key`... (`aap/credential_types/`) |
| **Credential** | One filled-in form: the actual secret. Nobody can read it back | `Linux ssh (sudo)`, `ServiceNow API`... |
| **Execution environment** | The container a job runs in (Ansible, Python, tools) | The **Minimal** execution environment is enough. This repository needs no extra collections |
| **Job template** | "Run this playbook, on this inventory, with these credentials" plus the questions to ask (**survey**). This is what you launch | `Health check`, `Troubleshoot`, `Patch hosts`... |
| **Workflow** | Job templates chained together. The next step can depend on the last one's result (success, failure, always). It can stop for an **approval** | `Daily health` = Health check, then ServiceNow tickets |
| **Schedule** | "Launch this template or workflow at these times" | Daily health every day at 06:00 |

Three more words you will see:

- **Survey**: the questions a template asks when you launch it, such as "which checks?". Each answer becomes a variable for the playbook.

- **Limit**: run on only some hosts or groups, for example `snowdb01.example.mil` or `mariadb_hosts`.
- **Artifacts**: what a job hands to the next step of a workflow. The health check hands its findings to the ServiceNow step this way (`set_stats` in the playbook).

## How a health check works, start to finish

1. You click **Launch** on the *Health check* template (or its schedule fires).
2. AAP syncs the project, starts the execution environment, and connects to each host over SSH with the Machine credential.
3. On each host, each chosen check runs read-only commands and turns what it sees into **findings**. One finding is one thing that is wrong, in plain words. It includes a command you can paste to look further:

```
[CRITICAL] disk: /var is 96% full (critical at 95%), 1.2 GiB free of 40.0 GiB
      look: timeout 60 du -xh --max-depth=2 /var 2>/dev/null | sort -h | tail -15
```

4. The job prints a summary per host and for the whole fleet. It then **fails the hosts that have findings**. The red status is on purpose: it is how a workflow knows it should open tickets (or ask ACT, or ask for approval).
5. In the *Daily health* workflow, the *ServiceNow tickets* step runs next. It opens a ticket for each finding, or updates the ticket that is already open.

## The findings: the one idea that makes everything modular

Every check produces the same kind of finding:

| Field | Example | Used by |
|---|---|---|
| `check` | `disk` | reports, tickets |
| `id` | `disk:/var` | ServiceNow updates the same ticket every run, instead of opening a new one |
| `severity` | `critical` or `warning` | pass/fail, ticket urgency |
| `summary` | `/var is 96% full (critical at 95%)` | people |
| `hint` | `du -xh --max-depth=2 /var ...` | people, and ACT, which runs it to collect evidence |

The report, the ServiceNow tickets and ACT only ever read findings. That is why:

- a new check you write shows up in reports and tickets with no other change;
- ACT works on top of **any** check. Add `use_act=true` and ACT reads the findings and the hint commands' output, then explains the root cause (ADDING_ACT.md).

## Where things are

| Path | What |
|---|---|
| `playbooks/` | the files AAP runs (one per job template) |
| `playbooks/group_vars/` | **your settings**, one file per AAP group: edit these (VARIABLES.md, USING_YOUR_AAP_INVENTORY.md) |
| `roles/check_*/` | one folder per check. Its settings are in `defaults/main.yml` |
| `roles/site_findings/` | starts, reports and publishes findings, and decides pass/fail |
| `roles/servicenow/`, `roles/troubleshoot/`, `roles/patch/`, `roles/poam/` | the other runbooks |
| `roles/site_act/` | the bridge to ACT (GenAI) |
| `inventories/example/` | a sample inventory and settings. Copy it to `inventories/site/` |
| `aap/credential_types/` | the credential types to create in AAP |
| `poam/poam.csv` | your POA&M list (for the POA&M runbook) |
| `docs/` | this guide, SETUP_AAP.md, USING_YOUR_AAP_INVENTORY.md, VARIABLES.md, RUNBOOKS.md, MARIADB.md, WORKFLOWS_AND_SCHEDULES.md, JOB_TEMPLATES.md, ADDING_ACT.md, APPROVED_COMMANDS.md; PDFs in `docs/pdf/` |

## The order to do things in

1. **Set up and run one health check by hand** (SETUP_AAP.md, steps 1 to 8). Your inventory is already in AAP? Use USING_YOUR_AAP_INVENTORY.md for the inventory part. Read its output. Adjust thresholds in your settings files (VARIABLES.md) until the findings are the ones you care about.
2. **Add the other job templates** (step 9) and try each once. Every template, with the link to its how-to: JOB_TEMPLATES.md.
3. **Add ServiceNow** (step 10), and build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md).
4. **Schedule** the workflows.
5. **Later: add ACT** to the runbooks you trust (ADDING_ACT.md).

# 2. Setting up AAP, step by step

Every step says **what** you are doing, **why**, the **clicks**, and what you should **see** when it worked. Menu names are from AAP 2.7. Older versions (2.5, 2.6) may word a label or a menu differently: follow the meaning, not the exact word. If you have not yet, read START_HERE.md first. It explains the words used here.

## Before you start

You need:

- An AAP account that can create projects, inventories, credentials and templates in your organization (an **Organization admin** does it all).
- A **Linux account on the servers** that AAP logs in as, for example `svc_aap`. It needs an SSH key and sudo. With `NOPASSWD` sudo, add it to `check_accounts_nopasswd_allowed`, or the accounts check will report it.
- A **Git repository at work** that AAP can reach (GitLab, Bitbucket, GitHub Enterprise).
- For ServiceNow: an **API account** (step 10).

## Step 1. Put the repository in your work Git, with your inventory

> **Already have your inventory in AAP?**
> Then you do not need `inventories/site/`. Push the repository without it (skip items 1-3 below), skip step 5, and follow USING_YOUR_AAP_INVENTORY.md for the groups and settings to add.

**What.** Copy this repository into your work Git, and create your real inventory in it. (Already pushed an earlier release at work? Update it with `scripts/update-from-release.ps1` in VS Code, as in HOW_IT_FITS_TOGETHER.md, "Updating the repository at work". Then skip to step 2.)

**Why.** AAP reads playbooks from Git, never from your laptop. The inventory lives in the same repository, so host lists and thresholds are versioned and reviewed like code. Every change has a who, a when and a why.

1. Extract the release zip (right-click › **Extract All**). In the extracted folder, copy the folder `inventories\example` and name the copy `inventories\site`.
2. Edit `inventories/site/hosts.yml`. Replace the example host names with yours and keep the group names: the playbooks use them. A host can be in several groups.
3. Edit `inventories/site/group_vars/all.yml`. These settings apply to every host: the LDAP and SIEM servers each host must reach, the account AAP logs in as (`svc_aap`), and the thresholds. Every setting and its default is listed in VARIABLES_REFERENCE.md. **Two places can hold a setting:** this inventory file (it becomes AAP's *inventory* Variables box) and the project's own settings files `playbooks/group_vars/*.yml` (`all.yml` is only switched-off placeholders; a few group files, such as `stigman.yml` and `aap.yml`, ship real values). If a setting is in both, the project file wins. Use one place per setting: VARIABLES.md explains the order.
4. Put it in your work Git, with VS Code:
   - Create an empty repository on your Git server (for example `site-automation`) and copy its clone URL.
   - In VS Code: **Source Control › Clone Repository** › paste the URL › pick a folder, such as `C:\git\site-automation`.

- Copy everything from the extracted release folder into that folder.
- In **Source Control**: type a message, click **Commit** (answer *Yes* to "stage all changes"), then **Sync Changes** / **Publish Branch**.

The same from a command line: `git add -A`, `git commit -m "site-automation <version>"`, then `git push`.

5. Note the two values AAP needs in step 4: the **clone URL** (the one you pasted above) and the **branch** (shown at the bottom left of VS Code, usually `main`). AAP does not read them from your copy; you type them in once.

**You should see** the files in your Git server's web page, including `inventories/site/`.

## Step 2. Credential types

**What.** Teach AAP the kinds of secrets these runbooks use, one form per kind.

**Why.** AAP ships with types for SSH (Machine) and Git (Source Control). The ServiceNow login and the others are custom. A credential type says which fields the form has and how the secret reaches the job. Every type here passes it as an **environment variable** in the execution environment. The playbook must ask for it on purpose, and it never shows in job output or Git.

For each file in `aap/credential_types/` that you need now:

1. **Automation Execution › Infrastructure › Credential Types › Create credential type**.
2. **Name**: the name at the top of the file (for example `ServiceNow API`).
3. **Input configuration**: paste the `inputs:` part of the file, *without* the `inputs:` line.
4. **Injector configuration**: paste the `injectors:` part, *without* the `injectors:` line.
5. Click **Create credential type**.

| File | Name | Needed for |
|---|---|---|
| `servicenow_api.yml` | ServiceNow API | tickets, ServiceNow health (step 10) |
| `mariadb_monitor.yml` | MariaDB monitor | the MariaDB check on a containerized database (root login refused there); optional on host installs |
| `keystore_password.yml` | Keystore password | optional: certs check on a PKCS12 keystore |
| `stigman_api.yml` | STIG Manager API | optional: POA&M cross-check with STIG Manager; the STIG drift report's checklist comparison |
| `act_model_key.yml` | ACT model key | later, for ACT (ADDING_ACT.md): GenAI.mil and Ask Sage keys |
| `act_genai_beta_key.yml` | ACT GenAI beta key | only for ACT with the GenAI.mil beta provider |

**You should see** each new type in the Credential Types list.

## Step 3. Credentials

**What.** Store the actual secrets, once.

**Why.** This is the only place a password or key is kept. Templates point at the credential by name. Nobody, including you, can read the secret back, and it is never in Git or job output.

**Automation Execution › Infrastructure › Credentials › Create credential**:

| Name | Credential type | Fill in |
|---|---|---|
| Linux ssh (sudo) | Machine | **Username** `svc_aap`, **SSH Private Key**, **Privilege Escalation Method** `sudo` (plus the sudo password if sudo asks for one) |
| site-automation git | Source Control | only if the repository is private: user name and token, or an SSH key |
| ServiceNow API | ServiceNow API | instance URL, API user, password (step 10) |
| MariaDB monitor | MariaDB monitor | the read-only database account, for MariaDB **and MySQL**; needed when the database runs in a container (the ServiceNow database), usually needed for MySQL on a host, optional for MariaDB on a host. See MARIADB.md for the setup and the account to ask the DBA for |

**You should see** the credentials listed, with secret fields shown as `ENCRYPTED`.

## Step 4. Project

**What.** Tell AAP where the repository is.

**Why.** A project is AAP's copy of the repository. Templates pick their playbook from it.

1. **Automation Execution › Projects › Create project**.
2. **Name** `site-automation`. **Organization**: yours.
3. **Execution environment**: **Minimal execution environment** (`ee-minimal`). These runbooks use only modules that come with Ansible itself, so no extra collections or custom execution environment are needed (useful on a disconnected network).
4. **Source control type** `Git`. **Source control URL**: from step 1. **Source control branch/tag/commit**: `main`. Later, pin a release tag such as `v<version>` (the tags are listed on the repository's Releases page). **Source control credential**: `site-automation git` if the repository is private.
5. Tick **Update revision on launch**. Every job then syncs first, so a change you push is used right away.
6. Click **Create project**.

**You should see** the project's **Last job status** turn **Successful** after a few seconds. If it fails, open the sync job: it is almost always the URL, the branch name, or the credential.

## Step 5. Inventory (filled from the repository)

*(Skip this step if your inventory is already in AAP: see USING_YOUR_AAP_INVENTORY.md.)*

**What.** Create the `Linux servers` inventory and fill it from `inventories/site/hosts.yml`.

**Why.** Sourcing the inventory from the project brings in hosts, groups and the settings in `group_vars/`, all at once, and keeps them in step with Git. You do not type hosts into AAP by hand.

1. **Automation Execution › Infrastructure › Inventories › Create inventory › Create inventory**. **Name** `Linux servers`. Click **Create inventory**.
2. Open it › **Sources** tab › **Create source**:
   - **Name** `site inventory from Git`
   - **Source** `Sourced from a Project`
   - **Project** `site-automation`
   - **Inventory file** `inventories/site/hosts.yml`
   - Tick **Overwrite**, **Overwrite variables** and **Update on launch**.
3. Click **Create source**, then **Launch inventory update**. The rocket icon also starts it.

**You should see**, under **Groups**, `stigman`, `mariadb_hosts`, `patch_hosts`... Under **Hosts** you should see your servers. Open a group: its **Variables** are the ones from `group_vars/<group>.yml`.

## Step 6. The first job template: Health check

**What.** A template that runs `playbooks/health_check.yml`.

**Why.** A job template is the thing you launch. It fixes which playbook, which inventory and which credentials. Its survey asks which checks to run.

1. **Automation Execution › Templates › Create template › Create job template**.
2. Fill in:

| Field | Value | Why |
|---|---|---|
| Name | `Health check` | |
| Job type | Run | (Check also works: the checks read the same way) |
| Inventory | `Linux servers` | |
| Project | `site-automation` | |
| Playbook | `playbooks/health_check.yml` | |
| Execution environment | Minimal execution environment | |
| Credentials | `Linux ssh (sudo)` (+ `MariaDB monitor` if you made it) | how AAP logs in |
| Limit | leave empty, tick **Prompt on launch** | so you can run it on one host |
| Privilege escalation | ticked | the checks read root-only files |

3. Click **Create job template**.
4. **Survey** tab › **Create survey question**:
   - **Question** `Which checks?`
   - **Answer variable name** `health_checks`
   - **Answer type** *Multiple Choice (multiple select)*
   - **Choices**, one per line: `daily`, `weekly`, `all`, `disk`, `mounts`, `services`, `performance`, `time`, `network`, `logging`, `selinux`, `fapolicyd`, `auditd`, `accounts`, `certs`, `patching`, `mariadb`, `mysql`, `database`,

`containers`, `stig`

- **Default answer** `daily`. Tick **Required**. Click **Create question**.

**Made this survey with an earlier release?** AAP never updates a survey for you: open the **Health check** template › **Survey** tab › click the `health_checks` question › **Edit** › add `containers` on a new line under **Choices** › **Save**. (`daily` and `all` already include the containers check without that; the choice is for running it on its own. `stig`, the STIG drift scan, is in no shortcut: add it the same way to run it by name.)

5. Turn the survey **on** (the switch at the top of the Survey tab).

**You should see** the template in the Templates list with a rocket icon.

## Step 7. Run it on one host and read the output

**What.** Launch the health check on a single server.

**Why.** Start small. You will see what the checks find on a real host, and whether the thresholds suit you.

1. Click the rocket on **Health check**. **Limit**: one host name. Survey: `daily`. Click **Next**, then **Launch**.
2. The job output scrolls by. The parts to read:
   - `Check disk | run`, `Check services | run`...: each check running (green `ok` lines are normal).
   - `Report | result for this host`: the verdict in plain words. Either `HEALTHY. Checks run: ...` or a list of `[CRITICAL]` / `[WARNING]` findings. Each finding has a `look:` line: a command you can paste on the host to see more.
   - `Report | fleet summary`: one line for all hosts, for example `3 healthy, 1 with findings`.
   - A red `FAIL this host because it has findings` at the end. This is **on purpose**: a job with findings fails, so a workflow can react (open tickets). It does not mean the playbook broke.
3. The **Details** tab shows who launched it, when, and the exact revision of the repository.

**You should see** a finished job, green if the host is healthy, red with findings otherwise.

## Step 8. Tune the thresholds

**What.** Make the findings match what you care about.

**Why.** Defaults are a starting point. If a warning is expected on some host, change the setting for that host or group, not the code.

1. Change the setting in the project's settings file `playbooks/group_vars/<group>.yml` (every host: `all.yml`; one host: `playbooks/host_vars/<host>.yml`, create it). This is the place VARIABLES.md recommends, and it wins over the same setting in `inventories/site/group_vars/` or in an AAP group or inventory Variables box. The file name must be the exact group or host name. Examples:

```yaml
check_disk_overrides:
  /var/log/audit: {warn: 70, crit: 85}      # this filesystem: warn earlier
check_services_ignore_failed: ['^dnf-makecache']
site_fail_on: [critical]                    # warnings are reported but do not fail the job
```

(With `[critical]`, a check that could not run, `<check>:check-error`, still fails the host: a check that did not run is not a healthy result.)

2. Commit and push. `playbooks/group_vars/` is read straight from the project, which syncs on launch: nothing else to do. Only if you changed `inventories/site/group_vars/` instead, run the inventory

source again in AAP (**Inventories › Linux servers › Sources › Launch inventory update**): those files reach AAP only through the inventory source.

3. Launch the health check again.

**You should see** the finding gone, or at its new threshold.

## Step 9. The other job templates

Create each the same way as step 6: **Inventory** `Linux servers`, **Project** `site-automation`, **Execution environment** Minimal. For the ones that log in to hosts, tick **Privilege escalation** and leave **Limit** empty with **Prompt on launch** ticked. Only the differences are listed. A survey question is *Multiple Choice (single select)* unless it says otherwise.

> **Three templates run on the controller, not on hosts:**
> POA&M status, ServiceNow tickets and ServiceNow health. For these, do **not** tick Prompt on launch for Limit. A workflow's Limit (for example `patch_hosts`) is passed to every step that prompts for one. These playbooks run on `localhost`, so the Limit would match nothing and the step would silently do nothing, and still show green.

### Troubleshoot

On demand, when someone reports a problem. Always launch it with a **Limit**: the host with the problem. Findings do not fail this job.

| Field | Value |
|---|---|
| Playbook | `playbooks/troubleshoot.yml` |
| Credentials | `Linux ssh (sudo)` |
| Survey: `ts_area` | "What kind of problem?" Choices: `overview`, `disk`, `performance`, `service`, `network`, `selinux`, `fapolicyd`, `login`, `time`, `logs`, `mariadb`, `mysql`. Default `overview` |
| Survey: `ts_service` | *Text*, not required. "Service name (for area = service), e.g. nginx" |
| Survey: `ts_target` | *Text*, not required. "host:port it cannot reach (for area = network)" |
| Survey: `ts_since` | *Text*. "How far back to read logs". Default `-2h` |

### Database health - MariaDB / MySQL

The database check on its own, for **MariaDB and MySQL**, installed on the host or in **podman containers** (one, several, found automatically, rootless). Give it to the database team. It only reads: no data is read, nothing is changed. Set the databases up first (MARIADB.md): the container name in a settings file, and the account for the credential.

| Field | Value |
|---|---|
| Name | `Database health - MariaDB / MySQL` |
| Playbook | `playbooks/database_health.yml` |
| Credentials | `Linux ssh (sudo)` and `MariaDB monitor` (the read-only database account) |

| Field | Value |
|---|---|
| Privilege escalation | ticked |
| Job type | Run, with **Prompt on launch** ticked, so you can pick *Check* first |
| Limit | empty, **Prompt on launch** ticked. The job runs on the groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` and `mysql`; a Limit narrows that to one host or group. A Limit cannot add a server outside those groups: for that, put `target: <group or host>` in the template's **Variables** |
| Survey | none. (Optional: a *Multiple Choice* `use_act`, choices `no` and `yes`, to add ACT's explanation; then also attach the *ACT model key* credential, ADDING_ACT.md) |

Steps: create the template as in step 6, then launch it twice.

1. Launch with **Job type** `Check`. You should see, for every database host, a `MariaDB | status` line such as `MySQL 8.4.11 - container snow-mysql (podman, image ...): up 312.4 h, 23/151 connections ...`, then `HEALTHY` (or a list of findings, each with a command to look further), and at the end `N healthy, M with findings`. *Check* is safe here: the job only reads either way.
2. Launch with **Job type** `Run`. The result is the same. A host with findings shows as failed, so a workflow can open a ticket (workflow 8).

If a host prints `mariadb:connect` with `Access denied`, the credential is missing or the account lacks a privilege: MARIADB.md.

### Service watch

Two templates and a workflow: the steps are in SERVICE_WATCH_DEMO.md (Parts 5 and 6). The one field to decide here is which hosts it watches.

| Field | Value |
|---|---|
| Playbooks | `playbooks/service_watch.yml` (*Service watch - check*) and `playbooks/service_fix_approved.yml` (*Service watch - apply approved fix*) |
| Credentials | `Linux ssh (sudo)`, plus `ACT model key` on the check template |
| Variables | optional: `target: <group or host>`. Without it the jobs run on the group `stigman` - a **group inside the template's inventory**, not an inventory called stigman (if there is no such group, the job stops and lists the groups the inventory has). To watch the containers of other servers, put that group (or `all`) here, on **both** templates. Which containers are watched on each host is found by itself: every container that should be running, root's and each user's (SERVICE_WATCH_DEMO.md, "Which containers are watched") |
| Survey | on the workflow: `watch_mode` (`approval` / `self-heal`) |

### Certificate report

Weekly: one table of every certificate on every host, soonest expiry first.

| Field | Value |
|---|---|
| Playbook | `playbooks/cert_report.yml` |
| Credentials | `Linux ssh (sudo)` (+ `Keystore password` if you use it) |

| Field | Value |
|---|---|
| Survey: `check_certs_warn_days` | *Integer*. "Warn this many days before expiry". Default `30` |

### Trust store report

Weekly, and when a CA incident is announced: which trust stores still hold a compromised or dropped certificate authority (TRUST_STORES.md).

| Field | Value |
|---|---|
| Playbook | `playbooks/trust_report.yml` |
| Credentials | `Linux ssh (sudo)` |
| Variables | `check_trust_extra` for certificates of your own; `check_trust_severity` to change a severity; `check_trust_remove: true` to take the certificates out (run as Check first) |

### STIG drift report

Weekly, and after patching: each Linux host's OpenSCAP STIG scan against its checklist in STIG Manager (read-only) and against its last scan; the scans stay on the hosts (STIG_DRIFT.md).

| Field | Value |
|---|---|
| Playbook | `playbooks/stig_drift_report.yml` |
| Credentials | `Linux ssh (sudo)`, `STIG Manager API` (the client: your autostig one, or another with `stig-manager:collection:read` and a grant in the collections) |
| Variables | `aap/extra_vars/stig_drift_report.yml`: `check_stig_stigman_api`; each host's `check_stig_stigman_collection` in the inventory (per group or host) |

### VMware - trust store report

vCenter's trusted root store and each ESXi host's trusted CAs, against the same list, on its own.

| Field | Value |
|---|---|
| Playbook | `playbooks/vmware_trust_report.yml` |
| Credentials | `VMware vCenter` |
| Variables | `aap/extra_vars/vmware_trust_report.yml`: `vmware_trust_esxi: false` for vCenter only; `check_trust_remove: true` to delete (run as Check first) |

### Windows - trust store report

The Windows servers' certificate stores, Java and Python, against the same list (the Windows health check's weekly set runs the check `trust` too).

| Field | Value |
|---|---|
| Playbook | `playbooks/win_trust_report.yml` |
| Credentials | `Machine (Windows)` |

| Field | Value |
|---|---|
| Variables | `aap/extra_vars/win_trust_report.yml`; `check_trust_remove: true` to put the certificates in Disallowed and delete them (run as Check first) |

### Certificate renewal - CSRs

A new key and a CSR on the host for each certificate that expires (in a file, on a port, in a keystore), with the CSR attached to the certificate's ServiceNow incident (step 3 of workflow 7; RUNBOOKS.md, "Certificate renewal"). It sends no email.

| Field | Value |
|---|---|
| Playbook | `playbooks/cert_renew_csr.yml` |
| Credentials | `Linux ssh (sudo)`, `ServiceNow API` (+ `Keystore password` if the certificate report uses it) |
| Variables | `servicenow_assignment_group: <your PKI or Linux team>`: who gets a ticket this job opens (a certificate with no ticket yet). `cert_skip_hosts: [<your Satellite and Capsules>]` if they are not in a group `sat` (left out of the report too). `cert_renew_skip_hosts: [<IdM, AAP ...>]` for servers renewed by hand: reported, never a CSR. `cert_renew_self_signed: true` for CSRs for self-signed certificates too (never postfix's placeholder). `cert_renew_attach_ext: txt` when your instance refuses `.csr` attachments (a refused one is tried as `.txt` anyway). Better in `all.yml`, so the certificate report gets the same |
| Job type | try it as **Check** first: it makes and sends nothing |

**Windows:** *Windows certificate renewal - CSRs* (`playbooks/win_cert_renew_csr.yml`) does the same on Windows servers with `certreq`, with the Windows machine credential: WINDOWS.md.

### Certificate install - check / approved, and the restore

With the CSRs ordered from the PKI's catalog item, the issued certificate is installed from that request after an approval: *Certificate install - check* (`playbooks/cert_install_check.yml`: `Linux ssh (sudo)`, `ServiceNow API`) in a workflow with an **Approval** (Run on fail) and *Certificate install - approved* (`playbooks/cert_install_approved.yml`: the same credentials, the same inventory, **no** Limit and **no** Prompt on launch for Variables). *Certificate restore* (`playbooks/cert_restore.yml`) puts an old certificate back. The Windows three: `win_cert_install_check.yml`, `win_cert_install_approved.yml`, `win_cert_restore.yml`, with the Windows machine credential. The templates, the workflow and the settings: CERT_INSTALL.md.

### Network device certificate report / renewal

The certificates of switches, firewalls, printers, iLO / iDRAC ... read from the controller: no login to the devices. Both run on the controller (Limit empty, **no** Prompt on launch). Report: `playbooks/net_cert_report.yml`, no credential. Renewal: `playbooks/net_cert_renew.yml`, `ServiceNow API` (and a Machine credential for the Cisco devices with `net_cert_cisco_csr`). The devices, the templates and the settings: NETWORK_CERTS.md.

### Network config backup / load

Cisco IOS / IOS-XE (its CLI: a Machine credential that logs in to the devices) and Palo Alto PAN-OS (its XML API from the controller: the `Palo Alto PAN-OS API` credential). Backup: `playbooks/net_config_backup.yml`, with the `Network config git repository` credential and `net_config_git_repo` - a **private, internal** repository: the configurations hold the devices' secrets (or `net_config_store: share`). Load: `playbooks/net_config_load.yml`, with `ServiceNow API` - it needs a change request in Implement (`net_config_change`); run it in Check first. The devices, the templates and the settings: NETWORK_CONFIG.md.

### Certificate inventory (CSV / SQL Server)

Not a template: a setting of every certificate report (`cert_inventory_files`, `cert_inventory_sql`). For the SQL Server table, add the *SQL Server (certificate inventory)* credential to each report template: CERT_INVENTORY.md.

### POA&M status

Weekly. Runs on the controller and reads `poam/poam.csv` from the project.

| Field | Value |
|---|---|
| Playbook | `playbooks/poam_status.yml` |
| Credentials | none (+ `STIG Manager API` for the cross-check) |
| Privilege escalation | not needed |
| Limit | empty, and **no** Prompt on launch (it runs on the controller) |
| Survey: `poam_stigman_api` | *Text*, not required. "STIG Manager API URL (blank = skip the cross-check)" |

### ServiceNow - test ticket

Run once before the tickets step, and again after a password or certificate change. It opens a test incident, reads it back, adds a work note and resolves it, with PASS / WARN / FAIL for each step (SERVICENOW_SETUP.md). Runs on the controller.

| Field | Value |
|---|---|
| Playbook | `playbooks/servicenow_test_ticket.yml` |
| Credentials | `ServiceNow API` |
| Job type | Run, with **Prompt on launch** ticked, so you can pick *Check* (a dry run: it only logs in and reads) |
| Privilege escalation | not needed |
| Limit | empty, and **no** Prompt on launch (it runs on the controller) |
| Survey | none |

### ServiceNow - catalog test

Only with `cert_renew_servicenow: catalog` (the certificate renewals order the PKI's catalog item instead of opening incidents). It finds `cert_renew_catalog_item`, lists its questions by their internal names and checks `cert_renew_catalog_variables` against them, PASS / FAIL per step; `servicenow_catalog_test_order: true` also orders it once with a test CSR (CERT_RENEWAL_CATALOG.md). Runs on the controller.

| Field | Value |
|---|---|
| Playbook | `playbooks/servicenow_catalog_test.yml` |
| Credentials | `ServiceNow API` (the account that will order) |
| Variables | `cert_renew_catalog_item: <the item's name or sys_id>` - the rest: aap/extra_vars/servicenow_catalog_test.yml |
| Limit | empty, and **no** Prompt on launch (it runs on the controller) |
| Survey | none |

### ServiceNow tickets

Only useful as a workflow step after a check. Runs on the controller.

| Field | Value |
|---|---|
| Playbook | `playbooks/servicenow_tickets.yml` |
| Credentials | `ServiceNow API` |
| Privilege escalation | not needed |
| Limit | empty, and **no** Prompt on launch (it runs on the controller) |
| Survey | none |

### ServiceNow health

Hourly. Runs on the controller.

| Field | Value |
|---|---|
| Playbook | `playbooks/servicenow_health.yml` |
| Credentials | `ServiceNow API` |
| Privilege escalation | not needed |
| Limit | empty, and **no** Prompt on launch (it runs on the controller) |
| Survey | none |

### Patch hosts

Monthly, inside the *Patch with checks* workflow. Run it as *Check* first: it lists what would be updated and changes nothing.

| Field | Value |
|---|---|
| Playbook | `playbooks/patch_hosts.yml` |
| Credentials | `Linux ssh (sudo)` |
| Job type | Run, with **Prompt on launch** ticked, so you can pick *Check* (a dry run) |
| Variables | **required**: `target: <group>`, the name of a **group (or host) inside the inventory** - not the inventory's own name. For example `target: stigman`, or `target: all` when the template's inventory holds only the servers to patch. Without it the job looks for a group called `patch_hosts`, and stops with a message that lists the groups your inventory does have |
| Survey: `patch_security_only` | "Security updates only?" Choices `false`, `true`. Default `false` |
| Survey: `automatic_restarts` | "Reboot hosts automatically when the updates need it?" Choices `no`, `yes`. Default `no` (no automatic restarts: the job lists the hosts that need a reboot) |

### Apply approved ACT fix

Only inside the approval workflow (ADDING_ACT.md). Create it when you add ACT.

| Field | Value |
|---|---|
| Playbook | `playbooks/act_fix_approved.yml` |
| Credentials | `Linux ssh (sudo)` (no ACT key: it runs the approved commands itself) |
| Job type | Run, with **Prompt on launch** ticked, so a workflow can run it as *Check* (a dry run that only shows what it would run) |
| Variables | leave **Prompt on launch** unticked |
| Survey | none |

### VMware jobs and reports

They run against vCenter, not your hosts, and have their own setup: the VMware execution environment, a vCenter credential, one template per job. Follow VMWARE.md; the emails are in EMAIL_REPORTS.md. The VM deploy from a ServiceNow ticket has a guide of its own, setup included: VM_DEPLOY_GUIDE.md.

### VMware pick lists (choose what to restart, which snapshots to delete)

Three templates - `VM - refresh the pick lists` (`playbooks/vm_pick_refresh.yml`, scheduled every 30 minutes, with the **VMware vCenter** and the **Red Hat Ansible Automation Platform** credentials), `VM - delete picked snapshots` and `VM - restart picked VMs` - and two workflows (plan, approval, the same template again) whose survey has the multiple-select question the refresh fills. The AAP credential's user needs the **Admin** role on those workflows. Click by click: VMWARE.md, "Pick lists"; the workflows: WORKFLOWS_AND_SCHEDULES.md, 26 and 27.

### AAP - report email question ("Also email the report to")

One template - `AAP - report email question` (`playbooks/aap_email_question.yml`), with only the **Red Hat Ansible Automation Platform** credential (its user: **Admin** on the report templates and their workflows), any inventory, no Limit. Run it with **Check** first, then for real, once all your report templates exist (and again when you add one): it adds the optional box **Also email the report to** to every report template's survey and to the workflows that run them. What it does and what people can type: EMAIL_REPORTS.md, "Also email the report to".

### Report website - copy

Only when report templates run in an execution environment that cannot reach the report website's server (no WinRM or Kerberos in it) and `report_web_copy: true` is in `all.yml`. One template - `Report website - copy` (`playbooks/report_web_copy.yml`) - in the execution environment your Windows jobs use, with the inventory that has the web server (no Limit), its Machine credential (an account with Modify on the folder) and a **Red Hat Ansible Automation Platform** credential (its user: **Organization Auditor** on the report templates' organizations). *Concurrent jobs* off. Run it with **Check** first (it lists what it would copy), then for real, then give it a schedule (every 15 minutes). WEB_REPORTS.md, "The copy job".

### NetApp ONTAP health report

It reads the storage clusters' REST API, not your hosts: a read-only ONTAP account, a NetApp ONTAP credential, one template. Follow NETAPP.md.

### Active Directory - account lockout

For the service desk: is an account locked out, and which computer is locking it out (the DCs' Security logs), with an optional unlock. The Windows inventory and execution environment, an account that may read the DCs' Security logs, a survey that asks for the account, **Execute** for the service desk team. Follow ACTIVE_DIRECTORY.md.

### Horizon 8 (VDI) jobs

They read a Connection Server's REST API, not your hosts: an AD account with a Horizon role, a "Horizon API" credential, one template per job, two workflows with an approval. Follow HORIZON.md; run *Horizon - API test* first.

### Try each one

With a Limit of one host:

- **Troubleshoot**: `ts_area` = `disk`. Each command's output appears under a `### what it shows` line.
- **Patch hosts**: Job type **Check**. It lists what would be updated and changes nothing.
- **Certificate report**: read the table under `All certificates on all hosts, soonest expiry first`.

## Step 10. ServiceNow

**What.** An API account, its credential, and a test ticket that proves it works.

**Why.** The tickets step and the ServiceNow health check use ServiceNow's REST Table API with a user name and password. No plugin or collection is installed.

**The whole setup, step by step, is in SERVICENOW_SETUP.md**: the account to ask for (with a request you can copy), the instance URL and every API call, the network path (firewall, proxy, the DoD CA), the credential, the settings, the test ticket, and how to check the ticket in ServiceNow. In short:

1. Ask the ServiceNow admins for an **integration account** (web service access only) with:
   - `itil` (or a role that can create and update incidents);
   - `mid_server` (read the MID Server list), if you want the MID Server check.
2. Create the **ServiceNow API** credential (step 3) with the instance URL, for example `https://servicenow.example.mil`, and the account.
3. In your settings file (`playbooks/group_vars/all.yml`; the example inventory's `inventories/site/group_vars/all.yml` already has these two lines, so change the value in one place only), set who gets the tickets:

```yaml
servicenow_assignment_group: Linux Operations   # a group that exists in ServiceNow
servicenow_min_severity: warning                # or critical: tickets for critical findings only
```

4. Create **ServiceNow - test ticket** (step 9) and launch it, first as *Check*, then as *Run*. **You should see** six steps that say PASS, and a link to the test incident. It is resolved by the same job.
5. Launch **ServiceNow health** once: `API answers in N s`, then the MID Servers and their status.
6. Build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md). Its tickets step then runs after every check.

Try it safely first: launch the workflow **without** the ServiceNow API credential on the tickets template. The step then only prints `NOT CONNECTED - what would be opened` and sends nothing.

## Step 11. Workflows and schedules

See WORKFLOWS_AND_SCHEDULES.md: *Daily health*, *Weekly compliance*, *Patch with checks*, *Fix with approval (ACT)*, and when to schedule each.

**Going further, once the basics work:**

- ACT (the model): ADDING_ACT.md. If a model answers `HTTP 400`, see its section Both endpoint formats and `:probe`.
- Letting ACT run fixes you trust without asking: APPROVED_COMMANDS.md.
- Every setting you can change, and where to put it: VARIABLES.md and VARIABLES_REFERENCE.md (in print: the *Settings Reference* PDF).
- MariaDB and MySQL checks (also in podman containers): MARIADB.md.

## Step 12. Who can do what

Open a template or workflow › **User Access** (or **Team Access**) › **Add roles**:

| Role | Can |
|---|---|
| Execute | launch it (and answer its survey) |
| Approve | approve or deny its approval steps (workflows) |
| Read | see it and its job output |
| Admin | change it |

For example, give the help desk **Execute** on *Troubleshoot*, and only the server team **Execute** on *Patch hosts* and **Approve** on the patch workflow.

## If something goes wrong

| You see | It means | Do |
|---|---|---|
| Project sync fails | wrong URL, branch or Git credential | open the sync job and read the last lines |
| `UNREACHABLE! ... Permission denied (publickey)` | SSH key or user wrong for that host | check the Machine credential; test `ssh svc_aap@host` from a workstation |
| `Missing sudo password` / `a password is required` | sudo wants a password | put it in the Machine credential (Privilege Escalation Password) |
| `Unknown check(s): ...` | a survey choice that is not a check name | fix the survey choices (step 6) |
| `the <name> check could not run: ...` | that check hit an error. The others still ran | the message says why, often a missing command such as `semanage` |
| `skipped: the MariaDB/MySQL check is off for <host>` | the database check only runs on database hosts | add the host to `mariadb_hosts`, `mysql_hosts` or `database_hosts` in the inventory, or name its container (MARIADB.md) |
| Findings you do not care about | defaults do not fit that host | step 8: change the setting for its group or host |
| `NOT CONNECTED - what would be opened` | the tickets template has no ServiceNow API credential | attach it (step 10) |
| Tickets step fails: `No check results reached this job` | the check step before it failed before checking any host, or it was launched on its own | read the first red job in the workflow; run the tickets step only inside a workflow |
| A controller step (tickets, POA&M, ServiceNow health) shows `skipping: no hosts matched` | its template prompts for a Limit and the workflow passed one | untick Prompt on launch for Limit on that template (step 9) |
| A VMware job or report fails | see the troubleshooting tables | VMWARE.md, "Troubleshooting" |
| No email, or an email error | see the troubleshooting table | EMAIL_REPORTS.md, "Troubleshooting" |

# 3. Using the inventory you already have in AAP

You imported your inventory into AAP. You do not need `inventories/site/` in Git, and you can skip the Git inventory parts of SETUP_AAP.md (the `cp inventories/example ...` in step 1, and all of step 5). Your hosts and groups stay in AAP. Your **settings** can go in either of two places, and you can use both as long as **one setting lives in one place**:

- **this project's settings files** in `playbooks/group_vars/` (edited in VS Code, reviewed in Git, never overwritten by an update): the next section; or
- **AAP's Variables boxes** (inventory, group, host): the steps after it. What you paste there is YAML, and the blocks to paste are below.

Which place wins when a setting is in both, and the full list of places, is in VARIABLES.md. Every setting and its default: VARIABLES_REFERENCE.md.

The runbooks need two things from your inventory:

1. **Groups**: which hosts are which (steps 1-2). The safety lists are built in and already cover the usual group names (`aap`, `aap_hosts`, `sat`, `idm`, `netapp`, `netapp_console_hosts`...).
2. **Settings (variables)**: your LDAP server, your SIEM, the account AAP logs in as, the ServiceNow assignment group... (steps 3-5).

## The easy way: your settings files, already named after your groups

The repository has one settings file per AAP group, in `playbooks/group_vars/`. **The file name is the group name**: `mariadb.yml` applies to the hosts in AAP group `mariadb`, and `all.yml` to every host. Ansible reads these files by itself on every run, and the hosts and groups stay in AAP. **If your group has another name, copy the file to your group's name** (for example `mariadb.yml` to `mariadb_hosts.yml`, `aap.yml` to `aap_hosts.yml`, `netapp.yml` to `netapp_console_hosts.yml`, which are the names the example inventory uses). A file for a group your inventory does not have is never used, without any error. The files are:

| File | For AAP group | What is in it |
|---|---|---|
| `all.yml` | every host (and the ServiceNow / POA&M jobs) | placeholders, all switched off: the safety lists (already built in), ACT provider and models, LDAP/log server, service account, ServiceNow assignment group |
| `stigman.yml` | `stigman` | the containers service watch watches (`stigman`, `nginx`); the web certificate |
| `mariadb.yml`, `mysql.yml` | `mariadb`, `mysql` | placeholders: turn the database check on, and the container's name (MariaDB or MySQL; MARIADB.md) |
| `aap.yml`, `sat.yml`, `idm.yml`, `logstash.yml` | the same names | disk limits, certificates, services that must run |
| `netapp.yml` | `netapp` | nothing yet (the group is protected by the built-in lists) |
| `rhel8_all.yml` | `rhel8_all` | a placeholder for Python 3.9 (only with a newer AAP) |

**To change a setting:**

1. Open the file in VS Code (for example `playbooks\group_vars\mariadb.yml`).
2. A line that starts with `#` is **off**. To turn a placeholder on, delete the `#` in front of it (and in front of the lines that belong to it), and replace `CHANGE-ME` with your value. Keep the indentation exactly as shown: spaces, never tabs.
3. **Commit** and **Sync Changes**. AAP uses it on the next run: the project syncs itself when **Update revision on launch** is ticked.

Example (the full guide, with several containers and rootless ones, is MARIADB.md): MariaDB runs in a container called `snow-mariadb`. In `mariadb.yml`, change

```
check_mariadb_enabled: true
# check_mariadb_container: CHANGE-ME
```

into

```yaml
check_mariadb_enabled: true
check_mariadb_container: snow-mariadb
```

**A group that has no file yet** (`sn_prod` here is only an example name; use one of yours): create `playbooks/group_vars/sn_prod.yml` with the settings for it. The same name as the AAP group, `.yml` at the end. For one host only: `playbooks/host_vars/<host name>.yml` (create the folder).

**One thing the files cannot do: choose which hosts a template runs on.** That is decided before any setting is read. Each playbook therefore has a built-in choice, and the job template's own **Variables** box (the *Extra variables* on its Details page, not the inventory's) can change it with one `target:` line:

| Job templates | Variables box empty: runs on | To run on something else |
|---|---|---|
| Health check, Troubleshoot, Certificate report, Apply approved ACT fix | **every host** in the template's inventory (the built-in group `all`) | one run: the **Limit** at launch (tick **Prompt on launch** next to Limit). Always: `target: "rhel8_all:rhel9_all"` |
| Database health - MariaDB / MySQL | the groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb`, `mysql` | `target: <group or host>` (a Limit only narrows those groups; it cannot add a server outside them) |
| Service watch, Service watch - apply approved fix, STIG Manager - deploy | the group `stigman` | `target: <group>` |
| Patch hosts | the group `patch_hosts` only (no such group: the job stops and lists your groups) | **always set it**, for example `target: "rhel8_all:rhel9_all"`, and pick the hosts with the Limit at launch. `target` is a group or host **inside** the inventory, never the inventory's own name |

`target` and the Limit take group names, host names, or several joined with `:`. `stigman:mariadb` means either group, `rhel9_all:&sn_prod` only hosts in both, and `rhel9_all:!sn_dev` all of `rhel9_all` except `sn_dev`. Put a space after `target:`, quote a value that has a `:` in it, and use the group name exactly as the inventory's **Groups** tab shows it (`rhel9_all`, not `rhel9`).

**Is every host in this inventory a Linux server?** If some are not (firewalls, switches, appliances), an empty Variables box tries them too: they show as failed, and in a workflow that opens ServiceNow tickets each of them gets a ticket (a host that never reports is ticketed), on every run. Then put `target: "rhel8_all:rhel9_all"` on those templates **before** you schedule anything.

**These files are yours.** The update script adds a settings file you do not have yet, and never changes one you already have. A new release can therefore never overwrite your settings.

**Or put the same lines in AAP instead.** Everything below explains the AAP Variables boxes, which take exactly the same lines. **Keep each setting in one place.** If the same setting is in both a file here and an AAP *group* or *inventory* box, **the file here wins** (every job prints a note when that happens). The exception is one host: a *host's* Variables box in AAP beats a group file, and only a `playbooks/host_vars/` file beats the host box. Full order: VARIABLES.md.

## How variables work in AAP (two minutes)

A variable is a setting with a name, such as `check_disk_warn_pct: 85`. Where you type it decides which hosts get it:

| Where in AAP | Applies to | The Git-file equivalent |
|---|---|---|
| **Inventories › your inventory › Edit inventory › Variables** | every host in the inventory | `group_vars/all.yml` |
| **… › Groups › a group › Edit group › Variables** | the hosts in that group (and in its child groups) | `group_vars/<group>.yml` |
| **… › Hosts › a host › Edit host › Variables** | that one host | `host_vars/<host>.yml` |

- **Inside AAP the more specific one wins**: host beats group, a child group beats its parent group, and a group beats the inventory. A survey answer, or the template's own **Variables**, beats all of them. **Files in this project's `playbooks/group_vars/` beat the AAP group and inventory boxes** (see the previous section).
- **Anything you do not set uses the default** in `roles/<role>/defaults/main.yml`. Every setting is explained in VARIABLES_REFERENCE.md. Do not edit the role files: set the value in a settings file or in AAP instead. A new release would overwrite edits in the roles, but never your settings files or AAP variables.
- The boxes take YAML. Keep the indentation exactly as shown (spaces, not tabs). AAP refuses to save YAML it cannot read.

## Step 1. Look at what you have

**Inventories › your inventory**: open the **Groups** tab, then the **Hosts** tab. Write down which of your groups (or hosts) are:

- all your Linux servers;
- the AAP server(s);
- the NetApp Console host;
- the STIG Manager host(s);
- the ServiceNow database host(s);

- the ServiceNow MID Server host(s);
- the servers monthly patching may touch.

On the old ansible-core machine, `ansible-inventory -i <your inventory> --graph` prints the same tree.

## Step 2. Create the groups the runbooks use

A host can be in many groups. Creating these groups leaves your existing groups as they are.

| Create group | Put in it | Why | Needed? |
|---|---|---|---|
| `aap_hosts` | the AAP server(s) | **safety**: ACT never fixes anything here by itself, and patching never reboots it | **yes** |
| `netapp_console_hosts` | the NetApp Console host | **safety**: ACT only diagnoses here (vendor-managed) | **yes**, if you have one |
| `no_patch` | the groups `aap_hosts` and `netapp_console_hosts` (as children) | **safety**: *Patch hosts* skips these even when a job targets them | **yes** |
| `patch_hosts` | the servers monthly patching may touch | *Patch hosts* only patches these | only for patching |
| `mariadb_hosts`, `mysql_hosts`, `database_hosts` | the database host(s), MariaDB or MySQL (any of the three, or the groups `mariadb` / `mysql`; the check treats them alike) | the database check runs here, and the **Database health** template targets these by default | only for the database check |
| `stigman` | the STIG Manager host(s) | service watch and STIG Manager deploy run here | only for those |

**The clicks:**

1. **Inventories › your inventory › Groups › Create group**. Type the **Name** (exactly as in the table), then **Create group**.
2. Add hosts: open the new group › **Hosts** tab › **Add existing host** › tick the hosts › **Add**.
3. Or add one of your groups inside it: open the new group › **Related Groups** tab › **Add existing group** › tick your group › **Add**. Its hosts are now in the new group too.

**Keep your own names instead?** If you already have, say, an `aap` group and do not want a second group, tell the runbooks your names. Put these in the inventory **Variables** (step 3). A list you write **replaces** the shipped one, so repeat every name you want kept; `aap` and `aap_hosts` are always protected, whatever you write:

YAML

```
site_act_diagnose_only_groups: [aap, netapp]   # ACT never fixes these
patch_never_reboot_groups: [aap]               # patching never reboots these
patch_never_patch_groups: [aap, netapp]        # patching skips these
```

**Updated from 0.4.1 or older?** Your `playbooks/group_vars/all.yml` then still sets these three lists (and `site_act_provider`, `site_act_models`) as real lines, and that file silently wins over the inventory Variables box. Open it and put `#` in front of those lines (or delete them), or make your change there instead. The job's `NOTE - settings in this project's files win ...` line lists what the file still sets.

The checks run on every host of the inventory. To keep them to your Linux servers, put `target: linux_servers` (your all-Linux group) in the **Variables** of the Health check, Troubleshoot, Certificate report and Apply approved ACT fix templates.

## Step 3. Settings for every host (inventory variables)

**Inventories › your inventory › Edit inventory › Variables**. If something is already there, add below it; do not delete what is there. Paste, then change every line marked `CHANGE`:

```
---- what every host must be able to reach, from itself ----
check_network_dns_names: [ldap.yoursite.mil]              # CHANGE: names that must resolve
check_network_tcp:                                        # CHANGE: services every host needs
  - {name: LDAP, host: ldap.yoursite.mil, port: 636}
check_logging_remote:                                     # CHANGE: your SIEM / log collector
  - {name: SIEM, host: siem.yoursite.mil, port: 6514}

# ---- the account AAP logs in as ----
check_accounts_nopasswd_allowed: [svc_aap]                # CHANGE: your AAP service account
check_accounts_expiry_watch: [svc_aap]                    # CHANGE: the same account

# ---- STIG ----
check_fapolicyd_required: true                            # false if some hosts do not run
  fapolicyd

# ---- where tickets go ----
servicenow_assignment_group: Linux Operations             # CHANGE: a group that exists in
  ServiceNow
servicenow_min_severity: warning                          # or critical: tickets for critical
  only
```

The ACT provider and models are **not** set here: they are chosen in `playbooks/group_vars/all.yml` (lines ready as comments) or per template (ADDING_ACT.md, step 2). Do not also set them here: if the file ever sets them, the file wins and the value here is ignored.

Not sure about a value yet? Leave the line out. The check then uses its default, or checks nothing for it (no `check_network_tcp` means no TCP checks).

## Step 4. Settings for particular groups

Open the group › **Edit group** › **Variables**, paste, change the `CHANGE` lines, **Save**. Put these on your own groups or on the groups from step 2. Both work.

**The AAP server(s)**, for example on `aap_hosts`:

```yaml
check_disk_overrides:
  /var: {warn: 80, crit: 90}          # AAP keeps job output and container images under /var
```

**The STIG Manager host(s)**, on `stigman`:

```yaml
check_certs_endpoints:                  # the certificate browsers actually see
  - {name: STIG Manager, host: 127.0.0.1, port: 443, servername: stigman.yoursite.mil}   # CHANGE
check_certs_files: [/etc/stigman/*.crt, /etc/stigman/*.pem]                          # CHANGE:
  where your cert files are
watch_containers:                       # optional since 0.6.0: service watch finds the containers
  - name: stigman                       # that should run by itself; this list is watched first,
  - name: nginx                         # in this order (dependencies first), and a missing one is
    reported
```

If STIG Manager runs as systemd units (`systemctl list-units | grep -i stig` shows them), also add them, so the services check watches them: `check_services_required_extra: [<unit>, <unit>]`.

**The ServiceNow database host(s)** (MariaDB or MySQL): also add this group to `mariadb_hosts` or `mysql_hosts` (step 2).

```yaml
check_mariadb_container: servicenow-mariadb     # CHANGE: the container name from `sudo podman ps
  -a`
```

Then attach the **MariaDB monitor** credential to the **Database health**, Health check and Troubleshoot templates (it serves MySQL too). Container images give root a password, so the check logs in with a monitoring account instead. The whole setup (several containers, discovery, rootless containers, the account to ask the DBA for, and how to verify) is in MARIADB.md.

**The ServiceNow MID Server host(s):**

```yaml
check_services_required_extra: [mid]            # CHANGE: its unit name (systemctl list-units |
  grep -i mid)
check_certs_keystores:
  - path: /opt/servicenow/mid/agent/security/agent_keystore     # CHANGE: your MID Server's path
```

**The NetApp Console host:** nothing is needed. It is checked like every host; being in `netapp_console_hosts` and `no_patch` keeps automation from changing it.

## Step 5. Settings for one host (only when one host is different)

**Hosts › the host › Edit host › Variables**. For example, a host whose `/data` is always nearly full on purpose:

```yaml
check_disk_overrides:
  /data: {warn: 95, crit: 98}
```

Or a host with a known-broken vendor unit you want ignored:

```yaml
check_services_ignore_failed: ['^vendor-agent']
```

## Step 6. Use your inventory in the job templates

On every job template (and workflow), set **Inventory** to your inventory. That is the only link between them. Everything else (project, playbook, credentials, survey) is as in SETUP_AAP.md, steps 6 and 9.

## Step 7. Check that it worked

1. Launch **Health check** with **Limit** = one server and the survey set to `daily`.
2. In `Report | result for this host`, look for your own values. A network finding names *your* LDAP server; an accounts finding names *your* service account. If a value is not there, the variable did not reach the host: check its spelling, and which group or host it is on.
3. `skipping: no hosts matched` (a green job that did nothing) means the Limit or `target` names a group or host this inventory does not have. Check the spelling against the inventory's **Groups** tab, and that the template uses the right **Inventory**.
4. `skipped: the MariaDB/MySQL check is off for <host>` means the host is in none of `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` or `mysql` and has no container name set (MARIADB.md).
5. Findings you do not care about: change the setting for that group or host (steps 4-5), not the code.

## Which YAML files in the repository do I change?

| File | Change it? |
|---|---|
| `playbooks/group_vars/*.yml`, `playbooks/host_vars/*.yml` | **Yes, these are your settings** (the update script never overwrites them) |
| `inventories/example/...` | **No.** Only a reference to copy blocks from; AAP does not read it |
| `roles/*/defaults/main.yml` | **No.** Read VARIABLES_REFERENCE.md to see every setting; set values in a settings file or in AAP instead |
| `playbooks/*.yml` | **No** |
| `aap/credential_types/*.yml` | **No.** Paste their contents into AAP (credential types) |
| `ansible.cfg` | **No.** AAP ignores its `inventory =` line and uses the template's inventory |
| `poam/poam.csv` | **Yes.** Your POA&M export (see `poam/README.md`) |

# 4. ServiceNow: set up, test and verify

The runbooks open ServiceNow incidents for the problems the checks find (the **ServiceNow tickets** step), and check that ServiceNow itself is healthy (**ServiceNow health**). This page takes you from nothing to a **verified test ticket**, one step at a time:

1. an API account from the ServiceNow admins;
2. the addresses: the instance URL and the API calls;
3. the network path from AAP to ServiceNow;
4. a quick check from a shell (optional);
5. the credential in AAP;
6. your settings: which assignment group gets the tickets;
7. the **ServiceNow - test ticket** template;
8. a dry run, then a real run;
9. checking the ticket in ServiceNow.

**Sign-in method.** This integration uses **basic authentication only** (an account name and password over HTTPS). OAuth, CAC/PKI login and SSO are **not supported**. If your instance requires one of them for the API, ask the ServiceNow admins for an integration account that is allowed to use basic authentication for the Table API, and exempt from SSO.

Plan about 30 minutes, plus the time the ServiceNow admins need for step 1.

## What you need

| Item | Example | Who gives it to you |
|---|---|---|
| The instance URL | `https://yourinstance.servicenowservices.com` or `https://servicenow.example.mil` | the ServiceNow admins (or your browser's address bar) |
| An API account and its password | `svc_aap_sn` | the ServiceNow admins |
| Roles on that account | `itil` (and `mid_server` for the MID Server check) | the ServiceNow admins |
| The assignment group for the tickets | `Linux Operations` | the ServiceNow admins or your team lead |
| A network path from AAP to the instance | TCP 443 from the AAP execution nodes | the network or firewall team |
| Trust for the instance's certificate | the DoD root CA, if the instance uses a DoD certificate | you (step 3) |

## Step 1. Ask the ServiceNow admins for an API account

**What.** An account that only the automation uses, allowed to read, open and update incidents.

**Why.** The playbooks log in to ServiceNow's REST API with a user name and password (basic authentication). A dedicated account keeps the tickets traceable ("opened by svc_aap_sn"), and its access can be limited to what the automation needs.

Send them this (change the names in brackets):

```
Request: a ServiceNow integration account for Ansible Automation Platform (AAP)

- User ID: svc_aap_sn (or your naming standard)
- "Web service access only": ticked (API use only, no interactive login)
- "Internal Integration User": ticked, if our release has it
- Roles: itil (read, create and update incidents; read groups and users)
         mid_server (read the MID Server list; only if we want the MID Server check)
  No admin role.
- Authentication: basic authentication over HTTPS for the Table API. If MFA or single
  sign-on is enforced, this account must be exempt, as web-service accounts usually are.
- If IP address access control is on: allow the AAP execution nodes' addresses: [addresses]
- Password: set, with "Password needs reset" unticked. Tell me the expiry or rotation
  rule, so the AAP credential can be updated in time.
- Tickets go to the assignment group: [Linux Operations]. Please confirm its exact name.
- If our instance has custom incident states or close codes: the value for "Resolved"
  and a valid close code.
```

**You should get back** the user ID, the password (through your normal secure channel), the exact assignment group name, and, if they are custom, the resolved state and close code.

## Step 2. The addresses

**The instance URL** is the start of the address you see in your browser on ServiceNow: the `https://` and the host name, **nothing after it**.

| In the browser | Instance URL for AAP |
|---|---|
| `https://yourinstance.servicenowservices.com/now/nav/ui/home` | `https://yourinstance.servicenowservices.com` |
| `https://servicenow.example.mil/nav_to.do?uri=incident_list.do` | `https://servicenow.example.mil` |

**The API calls.** Everything goes to that host over HTTPS (TCP 443), with basic authentication and JSON. These are the only calls the playbooks make:

| Method and path | What for | Used by |
|---|---|---|
| `GET /api/now/table/incident?sysparm_limit=1` | log in and read (the connection test) | test ticket, health |
| `GET /api/now/table/sys_user_group?sysparm_query=name=<group>` | does the assignment group exist? | test ticket |
| `GET /api/now/table/sys_user?sysparm_query=user_name=<caller>` | does the caller exist? | test ticket |
| `GET /api/now/table/incident?sysparm_query=correlation_id=...` | is a ticket for this problem already open? | tickets |
| `POST /api/now/table/incident` | open an incident | test ticket, tickets |
| `GET /api/now/table/incident/<sys_id>` | read an incident back | test ticket |
| `PATCH /api/now/table/incident/<sys_id>` | add a work note; resolve | test ticket, tickets |

| Method and path | What for | Used by |
|---|---|---|
| `GET /api/now/table/ecc_agent` | MID Servers: up and validated? | health |

With `cert_renew_servicenow: catalog` the certificate renewals order the PKI's catalog item instead of opening incidents (CERT_RENEWAL_CATALOG.md). Those jobs, and the catalog test, make these calls:

| Method and path | What for |
|---|---|
| `GET /api/sn_sc/servicecatalog/items?sysparm_text=<name>`, `GET /api/sn_sc/servicecatalog/items/<sys_id>` | find the item; its questions |
| `GET /api/now/table/sys_user`, `/cmn_location` (or another reference's table) `?sysparm_query=name=...` | a reference answered by name: its sys_id |
| `GET /api/now/attachment?sysparm_query=file_nameSTARTSWITH...` | has this certificate's CSR been ordered already? |
| `GET /api/now/table/sc_req_item/<sys_id>`, `?sysparm_query=request=<sys_id>` | that request's state; the requested item of a new order |
| `POST /api/now/attachment/file` | the CSR, for the item's attachment question (or on the requested item) |
| `POST /api/sn_sc/servicecatalog/items/<sys_id>/order_now` | the order |
| `GET /api/now/attachment/<sys_id>` | is the CSR on the requested item? |

## Step 3. The network path from AAP

**Which machine talks to ServiceNow.** The ServiceNow jobs run on the **AAP execution node**, inside the execution environment container, not on your managed hosts. To see the nodes, open **Automation Execution › Infrastructure › Instances** (or **Instance Groups**). Those are the addresses the firewall (and ServiceNow's IP access control, if any) must allow.

**Firewall.** Allow the execution nodes › the instance, TCP 443.

**A proxy.** If the execution nodes reach the outside only through a proxy, give the jobs the proxy settings. Go to **Settings › Automation Execution › Job › Edit › Extra Environment Variables**, and set (add to what is already there):

```
{"https_proxy": "http://proxy.example.mil:8080", "no_proxy": "localhost,127.0.0.1,.example.mil"}
```

This applies to every job. Leave hosts that must not go through the proxy in `no_proxy`.

**The certificate.** A ServiceNow instance with a DoD certificate is only trusted if the execution environment trusts the DoD root CA. Choose one:

| Option | How | When |
|---|---|---|
| **A. The CA in the execution environment** | have the AAP admins build the execution environment with the DoD root CAs in its trust store | best: every job trusts it |

| Option | How | When |
|---|---|---|
| **B. A CA file in the repository** | put the CA certificate (PEM) in the project, for example (an example path: the folder does not exist until you create it) `playbooks/files/ca/servicenow-ca.pem`, and set `servicenow_ca_path: "{{ playbook_dir }}/files/ca/servicenow-ca.pem"` (step 6) | when you cannot change the execution environment. CA certificates are public, so this is fine |
| C. Switch the check off | `servicenow_validate_certs: false` | only for one test, never for real use |

## Step 4. Check from a shell first (optional, fastest)

On the AAP execution node, or any Linux machine on the same network:

```shell
curl -sS -u svc_aap_sn -H 'Accept: application/json' \
  'https://yourinstance.servicenowservices.com/api/now/table/incident?sysparm_limit=1&sysparm_fiel
    ds=number'
```

`curl` asks for the password, so it is not left in your shell history.

**You should see** `{"result":[{"number":"INC0012345"}]}` (any number). Anything else:

| What you get | What it means |
|---|---|
| `SSL certificate problem: unable to get local issuer certificate` | the CA is not trusted (step 3, the certificate) |
| `Could not resolve host` | DNS: check the host name |
| nothing, then `Connection timed out`, or `Connection refused` | firewall or proxy (step 3) |
| `{"error":{"message":"User Not Authenticated"...}}` (HTTP 401) | wrong user or password, the account is locked, or it is not allowed to use basic authentication |
| `{"error":{"message":"Operation Failed","detail":"ACL Exception..."}}` (HTTP 403) | the account lacks the `itil` role |
| an HTML page | the URL has something after the host name, or a login page (SSO) answered: the account must be exempt from SSO |

## Step 5. The credential in AAP

1. If you have not yet, create the **ServiceNow API** credential type (SETUP_AAP.md, step 2). It uses the file `aap/credential_types/servicenow_api.yml`.
2. **Automation Execution › Infrastructure › Credentials › Create credential**:
   - **Name** `ServiceNow API`;
   - **Credential type** `ServiceNow API`;
   - **Instance URL**: from step 2 (only `https://` and the host name);

- **API user name** and **API password**: from step 1.

3. **Create credential**.

**You should see** the credential listed, with the password shown as `ENCRYPTED`. The job gets the three values as `SN_HOST`, `SN_USERNAME` and `SN_PASSWORD`. They never appear in job output, and never reach your managed hosts.

## Step 6. Your settings

In VS Code, open `playbooks/group_vars/all.yml`. (If your inventory comes from Git, the same lines in `inventories/site/group_vars/all.yml` end up in AAP's inventory Variables box, and the project's file wins over them: keep each setting in **one** place. See VARIABLES.md.) In the ServiceNow block, delete the `#` in front of the lines and fill in your values:

```
---- ServiceNow tickets ------------------------------------------------------------

servicenow_assignment_group: Linux Operations   # the exact name from step 1
servicenow_min_severity: warning                # or critical: tickets for critical findings only
# servicenow_caller: svc_aap_sn                  # optional; empty = the API account is the caller
# servicenow_ca_path: "{{ playbook_dir }}/files/ca/servicenow-ca.pem"   # step 3, option B
```

**Fields your instance makes mandatory.** If your incident form requires more fields (a red asterisk: Location, Subcategory, Channel ...), set them for every new ticket, and the fields a resolved ticket needs for the resolve step. Use each field's **column name**: right-click its label on the form and choose **Show - '<column name>'** (Channel is `contact_type`). Names and choice labels work as values, as they appear on the form:

```yaml
servicenow_category: Software
servicenow_fields:
  location: "Building 100"
  subcategory: "Operating System"
  contact_type: "Self-service"         # Channel
servicenow_close_code: "Solution provided"     # Resolution code, as your instance has it
servicenow_resolve_fields:
  resolved_by: svc_aap_sn
```

Mandatory fields enforced only by the form (UI policies) do not stop the API, but the ticket then cannot be closed by hand until someone fills them in, so set them here anyway. A data policy does stop the API: step 3 or 6 of the test then fails with `Data Policy Exception` and names the field.

**Commit** and **Sync Changes**. Every other ServiceNow setting (urgency, impact, resolve, close code) is described in `roles/servicenow/defaults/main.yml`.

**All settings**, with their defaults: servicenow.

## Step 7. The ServiceNow - test ticket template

**Automation Execution › Templates › Create template › Create job template**:

| Field | Value |
|---|---|
| Name | `ServiceNow - test ticket` |
| Inventory | any (for example `rhel-inventory`); it runs on AAP itself, not on these hosts |

| Field | Value |
|---|---|
| Project | `site-automation` |
| Playbook | `playbooks/servicenow_test_ticket.yml` |
| Credentials | `ServiceNow API` |
| Job type | Run, with **Prompt on launch** ticked (so you can pick *Check*, a dry run) |
| Limit | empty, and **no** Prompt on launch |
| Privilege escalation | not needed |

**Create job template**.

## Step 8. Run it: first a dry run, then for real

**The dry run. Launch** › **Job type** `Check` › **Launch**. Steps 1 and 2 run for real, because they only read. Steps 3 to 6 are only printed:

```
PASS 1 connection: the instance answers and svc_aap_sn logged in and read the incident table (0 s)
PASS 2 settings: assignment group "Linux Operations" exists; caller not set: the tickets show the
  API account as the caller
DRY RUN (Check mode): steps 1-2 ran (they only read). Steps 3-6 would:
3 open: POST https://yourinstance.servicenowservices.com/api/now/table/incident {...}
```

Nothing was created in ServiceNow.

**The real run.** Low priority does not mean silent: ServiceNow may still send its usual emails, to the assignment group for the new incident and to the caller when it is resolved. Warn the group first, or point the test at a test group (`servicenow_assignment_group: <test group>` in the template's **Variables**), or at a test instance if you have one.

**Launch** › **Job type** `Run`. At the end it prints a summary like this:

```
ServiceNow test at https://yourinstance.servicenowservices.com as svc_aap_sn:
PASS | 1 connection | the instance answers and svc_aap_sn logged in and read the incident table (0
  s)
PASS | 2 settings | assignment group "Linux Operations" exists; caller not set: ...
PASS | 3 open | opened INC0012346 (sys_id 5f1c...)
PASS | 4 read back | INC0012346: state New, priority 5 - Planning, assignment group Linux
  Operations, ...
PASS | 5 work note | added a work note to INC0012346 (see its Activity stream)
PASS | 6 resolve | resolved INC0012346 (state 6)
```

Step 4 also prints a link: `Open it in ServiceNow: https://.../nav_to.do?uri=incident.do?sys_id=...`.

| Result | Meaning |
|---|---|
| PASS | that step works |
| WARN | it works, but look at it (for example, no assignment group is set, or the instance would not resolve the ticket). The job stays green |
| FAIL | that step does not work. The job is red, and the line says why and what usually causes it |

To leave the test ticket open and look at it first, add `servicenow_test_resolve: false` to the template's **Variables**.

## Step 9. Check the ticket in ServiceNow

1. Click the link from step 4 of the job output, or go to **Incident › All** and search for the number (`INC0012346`).
2. Check:
   - **Short description**: `[site-automation test] test ticket from AAP job <number> - safe to close`;
   - **Assignment group**: yours (step 6);
   - **Caller**: your `servicenow_caller`, or the API account;
   - **Urgency** 3 and **Impact** 3, so normally **Priority** 5 - Planning, the lowest (your instance's priority table decides). The real tickets use `servicenow_urgency` and `servicenow_impact`;
   - **Activity**: the work note `Test work note from AAP job ...`;
   - **State**: Resolved, with the close notes `Test ticket from AAP job ...: resolved automatically by the same job` (unless you set `servicenow_test_resolve: false`).
3. To find every test ticket: filter **Incident › All** on **Correlation display** = `site-health-test`.

**The test passes when** all six steps say PASS (or WARN for step 6, if you do not resolve tickets automatically), and the ticket in ServiceNow looks like the list above.

## When a step fails

| Step and message | Cause | Fix |
|---|---|---|
| 1: `Attach the "ServiceNow API" credential` | the template has no ServiceNow credential | step 7: add it under Credentials |
| 1: `HTTP -1 ... certificate verify failed` | the execution environment does not trust the instance's CA | step 3, the certificate |
| 1: `HTTP -1 ... Name or service not known` | DNS | the Instance URL's host name; DNS from the execution node |
| 1: `HTTP -1 ... timed out` or `Connection refused` | firewall or proxy | step 3 |
| 1: `HTTP 401` | wrong user or password; account locked or inactive; "Password needs reset"; basic authentication not allowed | step 1 with the admins, then update the credential |
| 1: `HTTP 403` | the account may not read incidents | the `itil` role |
| 1: `HTTP 404` | the Instance URL has a path after the host name | only `https://` and the host name |
| 2: `assignment group "..." does not exist` | the name differs from ServiceNow's, even in capitals or spaces | copy the exact name from ServiceNow into `servicenow_assignment_group` |
| 2: `could not look up ...` (WARN) | the account may not read groups or users | fine if step 4 shows the group; otherwise ask for read access to `sys_user_group` |
| 3: `HTTP 403 ... ACL Exception Insert Failed` | the account may read but not create incidents | the `itil` role |
| 3: `HTTP 400` | ServiceNow refused a value, and says which | usually `servicenow_category` or a custom field |
| 4: `did not stick` (WARN) | the ticket was opened without the group | the group name (step 6) |
| 5: `HTTP 403` | the account may create but not update | the `itil` role, or an ACL on work notes |
| 6: `could not resolve ... Data Policy Exception` (WARN) | the instance needs other values to resolve | ask the admins for the resolved state and close code; set `servicenow_resolved_state` and `servicenow_close_code`. Only needed if you use `servicenow_resolve_cleared: true` |

## Next

- **ServiceNow health** (hourly): the instance's API and the MID Servers (RUNBOOKS.md).
- **The tickets step** in the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md): one incident per problem, updated on later runs, noted or resolved when it clears.

- Keep **ServiceNow - test ticket**. Run it again after a password change, a new certificate, or a ServiceNow upgrade.

# 5. Workflows and schedules

A **workflow** chains job templates. Each arrow says when the next step runs:

- **Run on success**: only if the step before it succeeded.
- **Run on fail**: only if it failed.
- **Run always**: either way.

The checks *fail on purpose* when they find something. So "Run on fail" means "a problem was found", and "Run always" means "whatever happened". A step also receives what the steps before it published (**artifacts**). That is how the tickets step learns what the health check found.

This chapter first shows how to build any workflow, then a catalog of ready-made workflows for Linux, for Windows, and for both, and how to schedule them.

## How to build any workflow, click by click

1. **Automation Execution › Templates › Create template › Create workflow job template**.
2. **Name** (e.g. `Daily health`), **Organization**.
   - **Inventory: leave it empty.** Each step then uses its own template's inventory, so one workflow can hold Linux and Windows steps. A workflow inventory only reaches steps whose template prompts for the inventory.
   - Tick **Prompt on launch** next to **Limit** if you want to pick hosts at launch.
   - Leave **Variables** empty: workflow variables override every step's own answers.
3. **Create workflow job template**. The **workflow visualizer** opens.
4. **The first step: Add step** › **Node type**:
   - *Job Template*: one of the templates (the usual case);
   - *Approval*: a person must click Approve (give it a **Timeout**);
   - *Workflow Job Template*: another workflow, as one step;
   - *Project Sync* / *Inventory Source Sync*: refresh the code or the inventory first.

Pick the template, then fill in **this step's** answers:

- **Survey answers**, e.g. `health_checks` = `daily`;
- **Job type** `Check` for a dry run (DRY_RUNS.md);
- a **Limit**.

The wizard only offers what the template asks in its survey or has **Prompt on launch** ticked for. › **Finish**.

5. **Next steps:** hover over a box › **Add step and link** (the plus sign) › the node, and the **link type** (success / fail / always).
   - **Two things at once:** add a second first step with **Add step**. Both start together.
   - **Two arrows into one step:** open the step › **Convergence**: *Any* runs it when one parent finishes as linked, *All* waits for all of them.
6. **Save** (top right). Closing without saving throws the graph away.
7. **Try it:** launch it once with the risky steps set to `Check` (step 4), then set them back.

**Who may approve:** open the workflow › **User Access** (or **Team Access**) › **Add roles** › **Approve**. Pending approvals are in **Automation Execution › Administration › Workflow Approvals**, and in the workflow job's view.

**Which templates fit where:**

| Template | Fails when | Publishes (for the next step) | Typical next step |
|---|---|---|---|
| Health check / Windows health check | a finding (`site_fail_on`), or ACT proposes a fix | the findings (`site_health`) | ServiceNow tickets (**always**); an approval (**on fail**) |
| Certificate report / Windows certificate report | a certificate expires soon | the findings | ServiceNow tickets (**always**) |
| POA&M status | an item is overdue / due soon | the findings | ServiceNow tickets (**always**) |
| Troubleshoot / Windows troubleshoot | never for findings (a look, not a verdict) | the findings | a health check, tickets if you want them |
| Patch hosts / Windows patch | a host fails, or its services do not come back | - | a health check (**always**) |
| Apply approved ACT fix | the problem is still there | the findings of the re-check | ServiceNow tickets (**always**) |
| Service watch - check | a container is down (NEEDS APPROVAL) | the fix to approve | an approval (**on fail**) |
| ServiceNow tickets | problems exist (it opened or updated tickets) | - | - (the end) |

## Adding ACT to a workflow: what "on fail" means, and where site_act goes

**You do not add `site_act` to the workflow.** There is no ACT template and no ACT box. ACT runs **inside the check job**. The check playbooks (Health check, Troubleshoot, Database health, Service watch) already include `roles/site_act` - the Windows ones not yet (WINDOWS.md, "Not in this release") - and it runs when that job's answers say `use_act` = `yes`, at the level in `site_act_level`. In the workflow you only give the check box those two answers.

**"On fail" is the check job's own result.** Nothing extra is checked. The check playbook decides by itself, against its thresholds (the role defaults and your `group_vars`: disk full %, certificate days, ...). The check box ends **red** when:

- it found a problem at or above `site_fail_on` (default: `critical` and `warning`), or
- ACT (`diagnose`) proposed a fix that waits for a person: `NEEDS APPROVAL ... ACT proposes: <commands>`, or
- a host could not be checked (sudo failed, the job stopped on it), or no host at all could be reached. One host that cannot be reached does not make it red: it is listed as NOT REACHED and gets its "no result" ticket (`site_unreachable_fails: true` = red, as before).

A **Run on fail** link from the check box therefore leads to the approval. **Run on success** means the check box ended green; **Run always**, either way.

| Box | Link into it | Runs when |
|---|---|---|
| 1. Health check (`use_act` = `yes`, `site_act_level` = `diagnose`) | (start) | always |
| 2. Approval | **Run on fail**, from 1 | the check job ended red |
| 3. Apply approved ACT fix | **Run on success**, from 2 | a person clicked Approve |
| 4. ServiceNow tickets | **Run always**, from 1 and from 3 | after the check, and after the fix |

The steps in words:

1. **Health check** box: answers `use_act` = `yes`, `site_act_level` = `diagnose`. This needs those two survey questions on the template (ADDING_ACT.md, step 3).
2. **Approval** box, **Run on fail** from 1.
3. **Apply approved ACT fix** box (`playbooks/act_fix_approved.yml`), **Run on success** from 2. It needs no answers: AAP hands it ACT's proposed commands from the health check job, and it runs exactly those.
4. **ServiceNow tickets** boxes, **Run always**, from 1 and from 3.

The approver opens the health check job and reads `ACT | what ACT says` and the red `NEEDS APPROVAL` line. If there is no such line (the job is red because of findings ACT had no fix for), there is nothing to approve: **Deny**. Approving does no harm either: the apply step skips every host with nothing approved.

**Without approvals:** `site_act_level` = `self-heal` with the fixes you allow in `site_act_allow`. ACT applies those itself, inside the check job, and the checks run again. Then the workflow is simply `[Health check] --Run always--> [ServiceNow tickets]`.

**A playbook of your own** gets ACT in the **playbook**, not in the workflow: it includes `site_findings` (start, your check), then `site_act`, then `site_findings` report. See ADDING_ACT.md, "Use ACT in a playbook of your own". Its job template then gets the same two survey questions, and it fits the same workflow.

## Linux workflows

### 1. Daily health




*Daily health: every run hands its findings to the tickets step, healthy or not.*

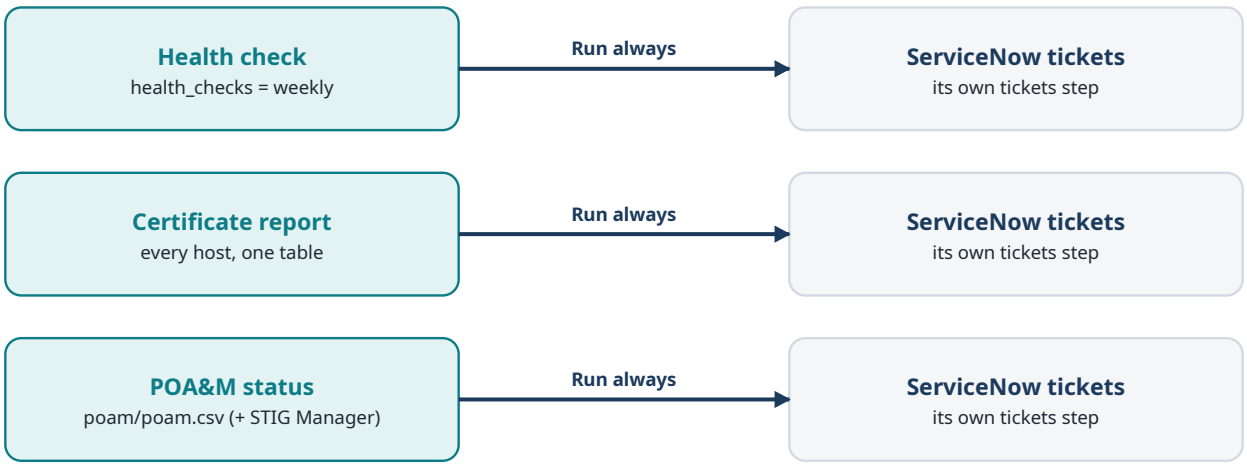
| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Health check | `health_checks` = `daily` | (start) |
| 2 | ServiceNow tickets | none | **Run always**, from step 1 |

**Why "always".** The tickets step opens tickets for what failed, and also notes the tickets of problems that cleared. That needs the healthy runs too. It fails when problems exist, so the workflow shows red until they are fixed.

**Schedule:** every day at 06:00.

### 2. Weekly compliance

Three independent chains in one workflow. Add each first box with **Add step** (not "Add step and link"), so they all start together:



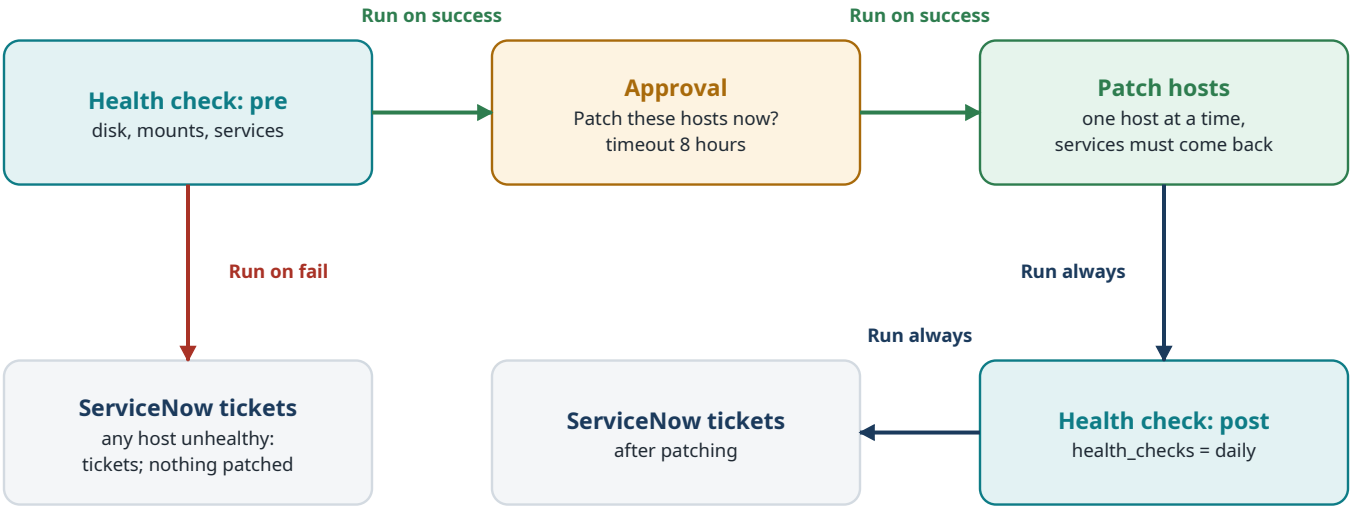

*Weekly compliance: three independent chains, each with its own tickets step (a step with two parents would get only one parent's findings).*

Give each chain **its own** tickets box. A box with two parents gets only one parent's findings: the artifacts overwrite each other.

The `weekly` set includes the trust check (a compromised or dropped CA still trusted somewhere on the host). For the one table of every store on every Linux host, add a fourth chain, *Trust store report* -> *ServiceNow tickets*, or run that template on demand. vCenter and the ESXi hosts have their own report, *VMware - trust store report*, which does not feed tickets (TRUST_STORES.md).

**Schedule:** Mondays at 07:00.

### 3. Patch with checks




*Patch with checks: every host healthy first (one failing host stops the whole run), a person approves, patch one host at a time, then check again.*

| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Health check | `health_checks` = `disk`, `mounts`, `services` | (start) |
| 2 | *Approval* | Name `Patch these hosts now?`, **Timeout** 8 hours | **Run on success**, from 1 |
| 3 | Patch hosts | `patch_security_only` as you like; `automatic_restarts` (see below) | **Run on success**, from 2 |
| 4 | Health check | `health_checks` = `daily` | **Run always**, from 3 |
| 5 | ServiceNow tickets | | **Run always**, from 4 |
| 6 | ServiceNow tickets | | **Run on fail**, from 1 (tickets for what the pre-check found) |

**Why these links:**

- **Healthy first.** Patching a host that is already failing makes the next problem impossible to diagnose. The pre-check must pass before anyone is asked. **If any one host fails it, the whole run stops before the approval and no host is patched.** Fix that host (the ticket says what is wrong), or launch again with a Limit that leaves it out. Warnings count too while `site_fail_on` includes `warning`. A disk at 86% blocks patching, so clear it before the maintenance window.
- **A person approves.** The approval step waits (up to its **Timeout**). Deny, or no answer, stops the workflow and nothing is changed.
- **Always check after.** The post-check runs whatever happened in patching, so a half-patched host is still reported.

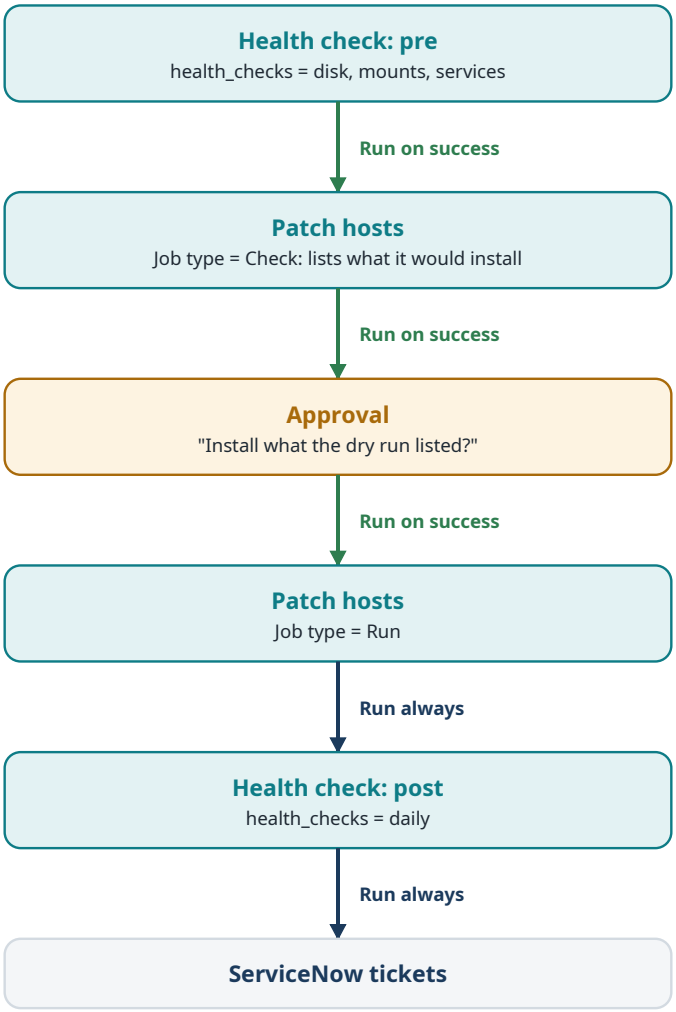
**No automatic restarts** unless you ask for them. Patching installs the updates; a host that then needs a reboot is **not** rebooted, and the job says so. The post-check's `patching` finding ("reboot needed") keeps it

visible until someone reboots it. To let the job reboot hosts that need it, give step 3 `automatic_restarts` = `yes`. Add a survey question for it on the Patch hosts template (*Multiple Choice*, `no` / `yes`, default `no`), or put `automatic_restarts: true` in the step's variables. Hosts in `patch_never_reboot_groups` are never rebooted either way.

**Launch** with **Limit** `patch_hosts`, or one group at a time (`stigman` this week, `servicenow_mid_hosts` next week). **Schedule:** monthly, in your maintenance window, for example the second Wednesday at 20:00. The approval then waits for the on-call person.

### 4. Patch with a dry run first

The same as 3, but the approver sees exactly what will be installed before approving:



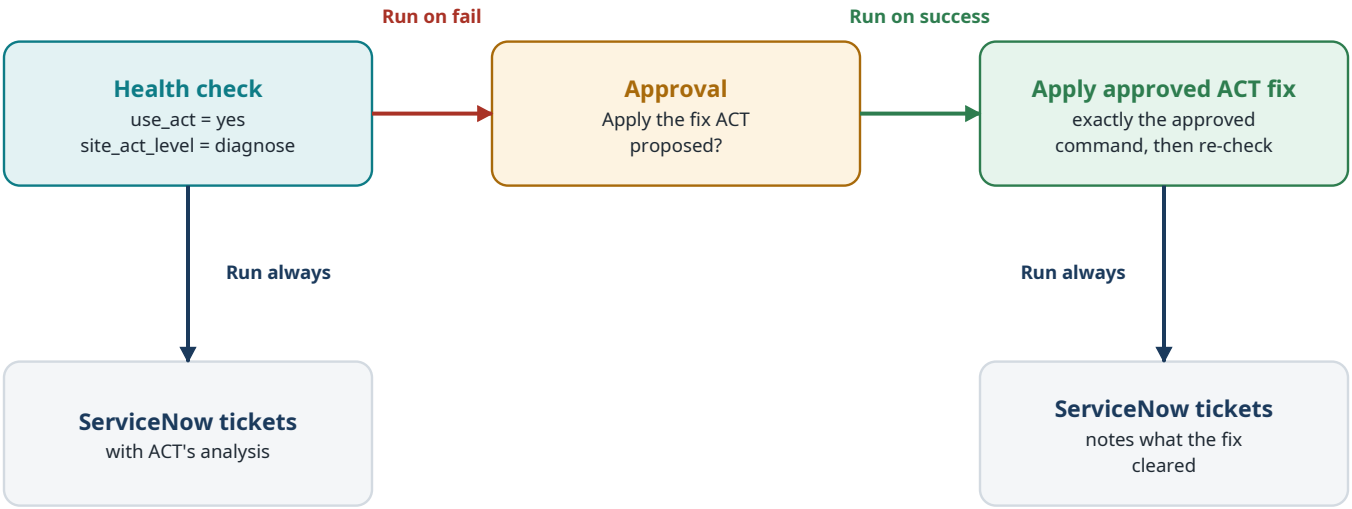

| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Health check | `health_checks` = `disk`, `mounts`, `services` | (start) |
| 2 | Patch hosts | **Job type** `Check` | **Run on success**, from 1 |
| 3 | *Approval* | Name `Install what the dry run listed?`. Description: `Open the "Patch hosts" dry-run job: it lists every package per host.` | **Run on success**, from 2 |

| Step | Template | Answers | Link |
|---|---|---|---|
| 4 | Patch hosts | **Job type** `Run`; `automatic_restarts` as you decide | **Run on success**, from 3 |
| 5 | Health check | `health_checks` = `daily` | **Run always**, from 4 |
| 6 | ServiceNow tickets | | **Run always**, from 5 |

Needs **Prompt on launch** for **Job type** on the Patch hosts template (steps 2 and 4 set it).

### 5. Fix with approval (ACT)

For when you use ACT. See ADDING_ACT.md for the three levels.




*Fix with approval (ACT): the check job fails with NEEDS APPROVAL only when ACT proposed a fix; deny or timeout changes nothing.*

| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Health check (with the ACT survey questions) | `use_act` = `yes`, `site_act_level` = `diagnose` | (start) |
| 2 | ServiceNow tickets | | **Run always**, from 1 |
| 3 | *Approval* | Name `Apply the fix ACT proposed?`, **Timeout** 30 minutes. Description: `Read ACT's report and the NEEDS APPROVAL line in the health check job first.` | **Run on fail**, from 1 |
| 4 | Apply approved ACT fix | | **Run on success**, from 3 |
| 5 | ServiceNow tickets | | **Run always**, from 4 (notes the tickets whose problem the fix cleared) |

**Caveat: not yet proven in a live AAP.** The whole approve-and-fix path depends on the check job's result (the proposed commands) reaching the apply job through the Approval node. The playbooks were tested on their own, but that hand-over inside AAP itself has not been run end to end. Rehearse it first with the dry run below, and read SERVICE_WATCH_DEMO.md (Part 9) if the apply job says `Nothing to apply`.

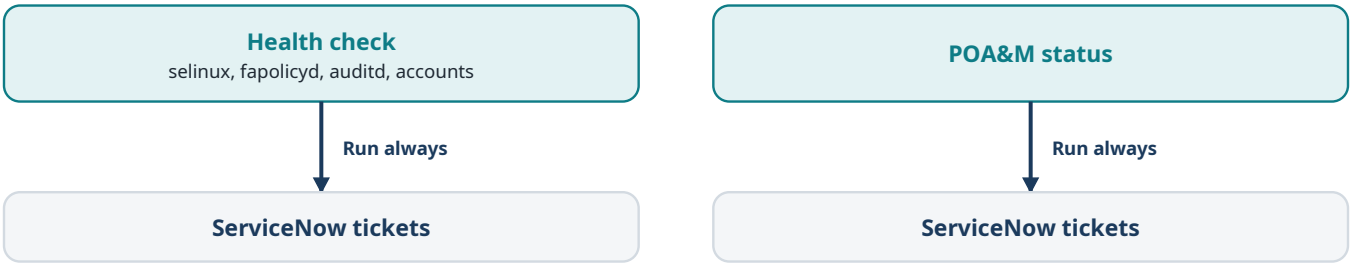
The approver reads, in the health check job, `ACT | what ACT says` (root cause, evidence, the fix) and the red `NEEDS APPROVAL ... ACT proposes: <command>` line. **That exact command** is all the apply step can run: it runs it itself (no model, no key), stops at the first command that fails, and runs the same checks again. The tickets carry ACT's analysis too.

**Rehearse it: see ACT's fixes, change nothing.**

- **One job, no workflow.** Launch **Health check** (or **Troubleshoot**) with `use_act` = `yes` and `site_act_level` = `diagnose`. ACT investigates with read-only commands and proposes fixes; every change is refused. The job ends red with `NEEDS APPROVAL ... ACT proposes: <commands>`: that list is what would run. Nothing on the host changed.
- **The whole workflow, as a dry run.** On the **Apply approved ACT fix** template (and *Service watch - apply approved fix*) tick **Prompt on launch** next to **Job type**. In the workflow visualizer click the apply box › **Edit** (pencil) › **Job type** `Check` › **Save**. Now run the workflow and approve: the apply job prints `DRY RUN on <host>: approved, and would run: <commands>. Nothing was changed (Check mode).` and ends green. When you trust it, set that node's **Job type** back to `Run`.

### 6. Security posture (weekly)

The STIG-related checks, and the POA&M list, with tickets:

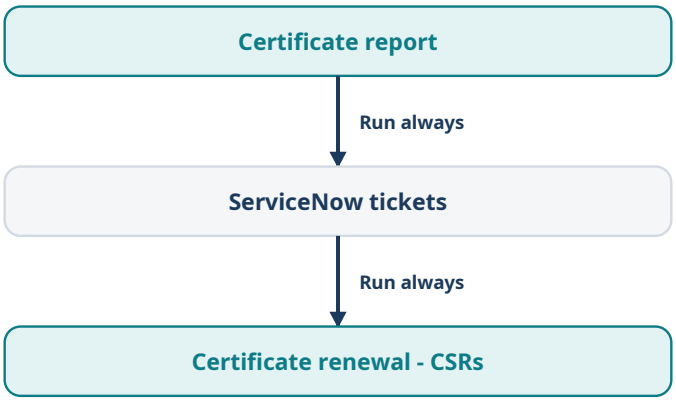



| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Health check | `health_checks` = `selinux`, `fapolicyd`, `auditd`, `accounts` | (start) |
| 2 | ServiceNow tickets | | **Run always**, from 1 |
| 3 | POA&M status | | (start, with **Add step**) |
| 4 | ServiceNow tickets | | **Run always**, from 3 |

**Schedule:** Wednesdays 07:00. Use it instead of the `weekly` set in *Weekly compliance* if your security team wants these on their own day.

### 7. Certificate watch (daily)

Certificates expire on a day, not in a week:




| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Certificate report | | (start) |
| 2 | ServiceNow tickets | | **Run always**, from 1 |
| 3 | Certificate renewal - CSRs (optional) | | **Run always**, from 2 |

**Schedule:** every day 06:30. Put `check_certs_warn_days` (survey, 30) on the **workflow**, so every step uses the same number. Tickets then start a month ahead. The de-duplication keeps it to one ticket per certificate.

**Step 3** makes a new key and a CSR on the host for each certificate that expires (in a file, on a port, in a keystore - never the trust store's), and attaches the CSR to that certificate's ticket from step 2 (RUNBOOKS.md, "Certificate renewal"). It must be **Run always**: steps 1 and 2 are red on purpose while certificates expire. Hosts in `cert_skip_hosts` or `cert_skip_groups` (Satellite) are left out of every step: set them in `all.yml`, or on the workflow, so every step gets them.

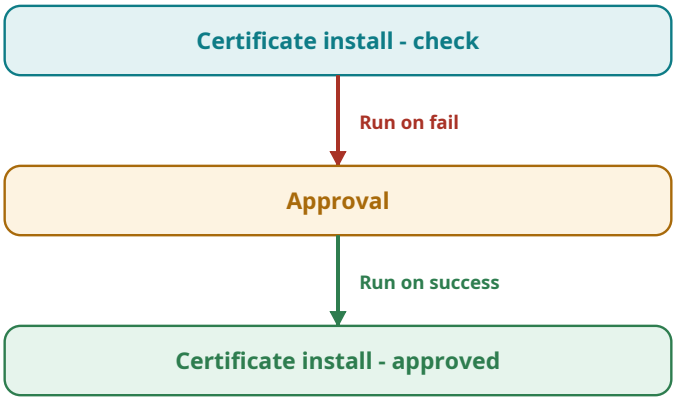
**No incidents** (`cert_renew_servicenow: catalog` - the CSRs ordered from the PKI's catalog item): leave step 2 out and link step 3 to step 1, **Run always** (CERT_RENEWAL_CATALOG.md).

**Install the issued certificates** (with the catalog item): workflow 29, after this one.

**Network devices:** *Network device certificate renewal* on its own, daily - it reads the devices, opens the incidents and emails the report (NETWORK_CERTS.md). **A dashboard:** with `cert_inventory_files` or `cert_inventory_sql` in `all.yml`, each certificate report also writes every certificate it read (CERT_INVENTORY.md).

**Network configurations:** *Network config backup* on a daily schedule - a commit in the backups' repository when a device changed, optionally an incident for it. A **load** as a workflow: *[Network config load]* as **Check** › **Approval** › *[Network config load]* as **Run**, the change and its lines in the workflow's Variables: the approver sees the dry run's report - exactly what will be sent - before anything is. Click by click: NETWORK_CONFIG.md, "A load, with an approval".

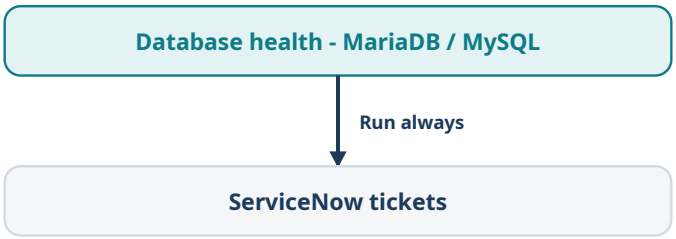
### 29. Certificate install, with an approval (daily)




| Step | Template | Link |
|---|---|---|
| 1 | Certificate install - check | (start) |
| 2 | *Approval*: `Install the certificates the check listed?`, **Timeout** 8 hours. Description: `Open the check job: its READY lines say what each one changes.` | **Run on fail**, from 1 |
| 3 | Certificate install - approved (the check's inventory, no Limit) | **Run on success**, from 2 |

**Schedule:** every day 07:00, after workflow 7. The check reads each renewal's request in ServiceNow for the certificate the PKI put there and fails on purpose when one is ready (`NEEDS APPROVAL`); nothing ready, it stays green and the approval never comes. The install plans each one again, copies the old files aside, reloads the service, checks the port, and puts the old files back if anything is wrong. Windows: the same with *Windows certificate install - check / approved*. Everything else - what stays for a person, the restore: CERT_INSTALL.md.

### 8. Database watch: MariaDB and MySQL (hourly)



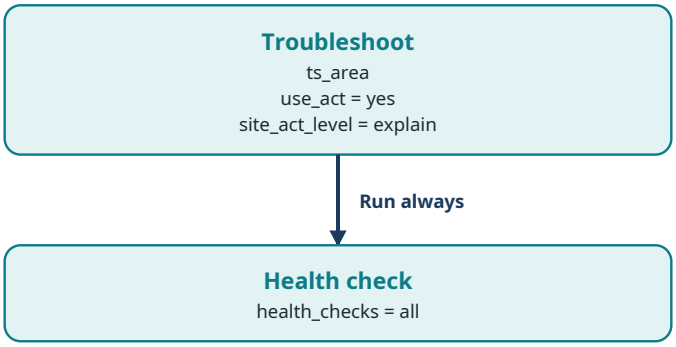

| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Database health - MariaDB / MySQL | none; it targets `mariadb_hosts`, `mysql_hosts` and `database_hosts` by default (a **Limit** narrows it) | (start) |
| 2 | ServiceNow tickets | | **Run always**, from 1 |

The database template runs only the MariaDB / MySQL check (host installs and podman containers), so it is quick and the database team can own it and its tickets (MARIADB.md). Prefer to keep it in the general Health check? Use the *Health check* as step 1 with `health_checks` = `disk`, `services`, `mysql` (`mariadb` and `database` are the same check) and a **Limit** of your database group. The template *Database health* also supports `use_act=true`.

**Schedule:** hourly. It is quick, and a ticket comes within the hour of a problem.

### 9. Investigate a host (on demand)

One launch gives the troubleshooting commands, ACT's explanation and every check:

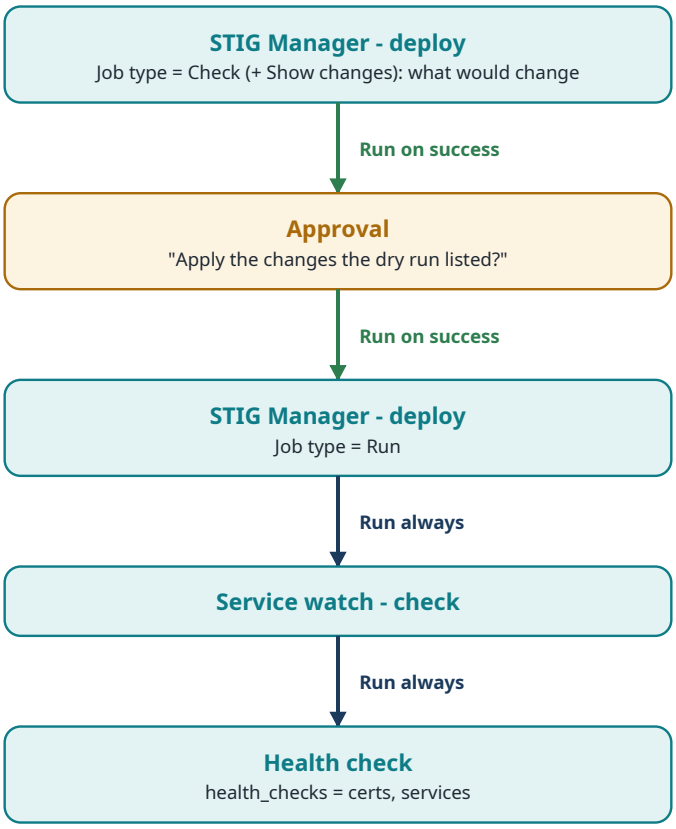



| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Troubleshoot | `ts_area` = `overview` (or asked at launch), `use_act` = `yes`, `site_act_level` = `explain` | (start) |
| 2 | Health check | `health_checks` = `all` | **Run always**, from 1 |

Give the workflow **Prompt on launch** for **Limit** and launch it with one host. No tickets: you are already looking. Add `[ServiceNow tickets]` after step 2 (**Run always**) if you want them.

### 10. STIG Manager deploy with checks

**The template** *STIG Manager - deploy*: playbook `playbooks/stigman_deploy.yml`; the inventory with the group `stigman`; credentials Machine (ssh + sudo), *STIG Manager database*, *TLS certificate* (and *Container registry login (hosts)* when you pull with an account); **Prompt on launch** ticked for **Job type** and **Show changes**, so the workflow can run it as Check, then as Run. Its Variables: `aap/extra_vars/stigman_deploy.yml`.




| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | STIG Manager - deploy | **Job type** `Check` (and **Show changes**) | (start) |
| 2 | *Approval* | `Apply the changes the dry run listed?`, **Timeout** 8 hours | **Run on success**, from 1 |
| 3 | STIG Manager - deploy | **Job type** `Run` | **Run on success**, from 2 |
| 4 | Service watch - check | | **Run always**, from 3 |
| 5 | Health check | `health_checks` = `certs`, `services` | **Run always**, from 4 |

The dry run shows which files and units would change, and how. The approver approves only changes they expected. (This dry run has not been tried on a real server yet: run step 1 on its own once, on the test server, before you build the workflow.)

### 11. Service watch

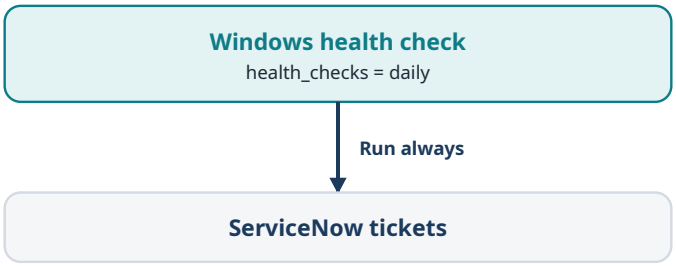
The approval or self-heal workflow for containers: SERVICE_WATCH_DEMO.md, Part 6.

If you schedule it every 15 minutes, turn **off** "Enable concurrent jobs" on the workflow (edit the workflow), because its approval waits up to 30 minutes: without that, a new run starts every 15 minutes while the last one still waits for a person.

## Windows workflows

Build them the same way, with the Windows templates (WINDOWS.md). The ServiceNow tickets step is the same template as for Linux: it runs on AAP itself.

## 12. Windows daily health




**Schedule:** every day 06:15.

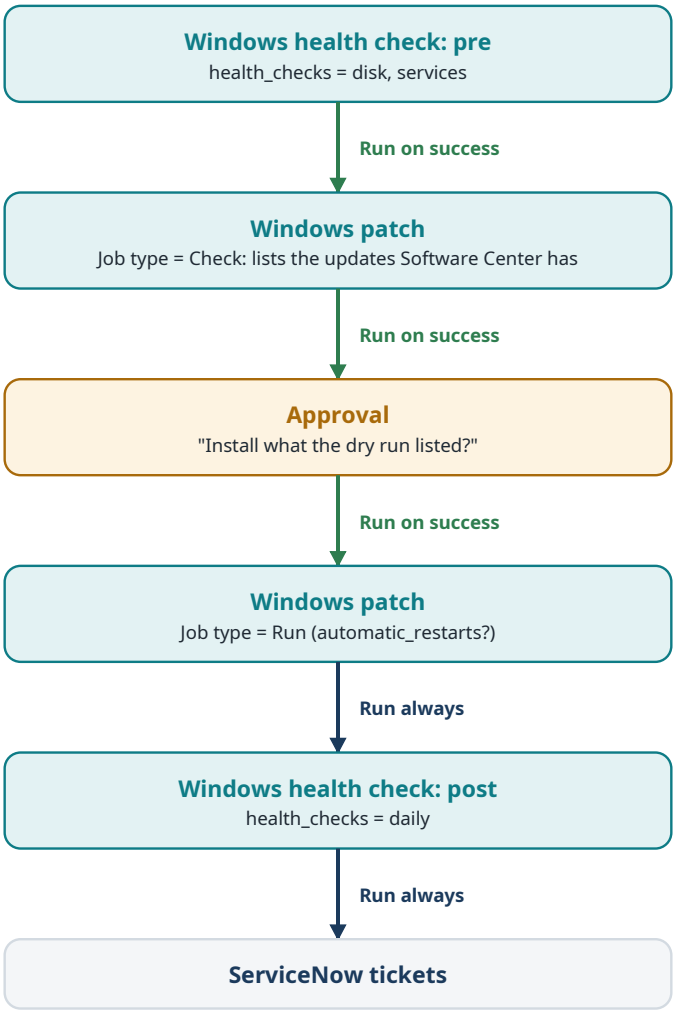
## 13. Windows weekly compliance




The last step is optional: a CSR for each of the server's own certificates that expires, on its ticket (WINDOWS.md).

**Schedule:** Mondays 07:15.

## 14. Windows patch with a dry run first

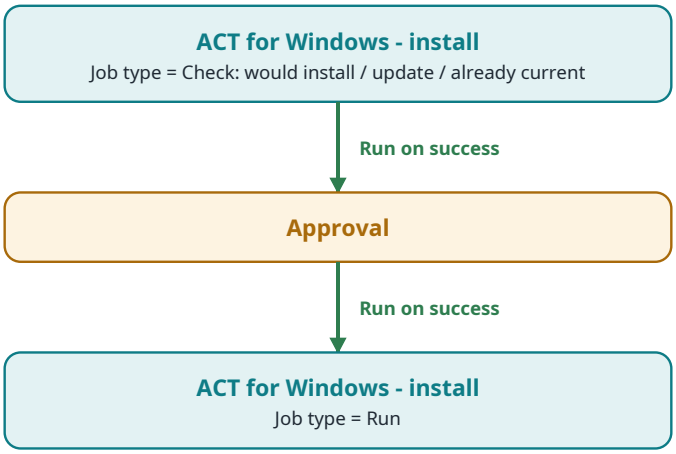



| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | Windows health check | `health_checks` = `disk`, `services` | (start) |
| 2 | Windows patch | **Job type** `Check` | **Run on success**, from 1 |
| 3 | *Approval* | `Install what Software Center offers (listed in the dry run)?`, **Timeout** 8 hours | **Run on success**, from 2 |
| 4 | Windows patch | **Job type** `Run`; `automatic_restarts` as you decide | **Run on success**, from 3 |
| 5 | Windows health check | `health_checks` = `daily` | **Run always**, from 4 |
| 6 | ServiceNow tickets | | **Run always**, from 5 |

**No automatic restarts** unless step 4 gets `automatic_restarts` = `yes`: the servers that need a restart are listed, and the post-check's `patching` finding ("a restart is pending") keeps them visible. **Schedule:** inside the servers' ConfigMgr maintenance window. Outside it, ConfigMgr holds the updates and the patch step stops and says so.

### 15. ACT for Windows rollout

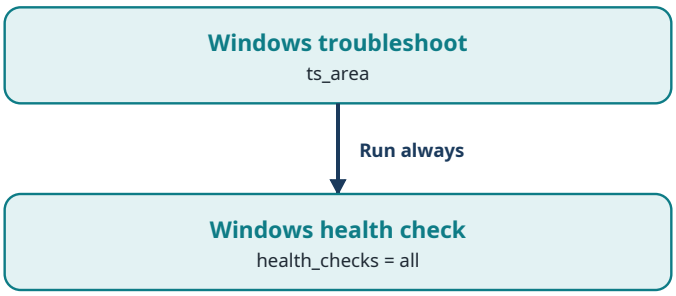
**The template** *ACT for Windows - install*: playbook `playbooks/win_act_install.yml`; your Windows inventory and its **Machine (Windows)** credential; the execution environment your Windows jobs use; **Prompt on launch** ticked for **Job type** and **Limit** (WINDOWS.md, "ACT on the servers").




| Step | Template | Answers | Link |
|---|---|---|---|
| 1 | ACT for Windows - install | **Job type** `Check` | (start) |
| 2 | *Approval* | `Install or update ACT on these servers?`, **Timeout** 8 hours | **Run on success**, from 1 |
| 3 | ACT for Windows - install | **Job type** Run | **Run on success**, from 2 |

The dry run lists, per server, whether ACT would be installed, updated, or is already current. Launch the workflow with a Limit (a group; tick **Prompt on launch** for Limit on the workflow too) to roll out group by group. Run it again after every update of this repository.

### 16. Windows: investigate a server (on demand)

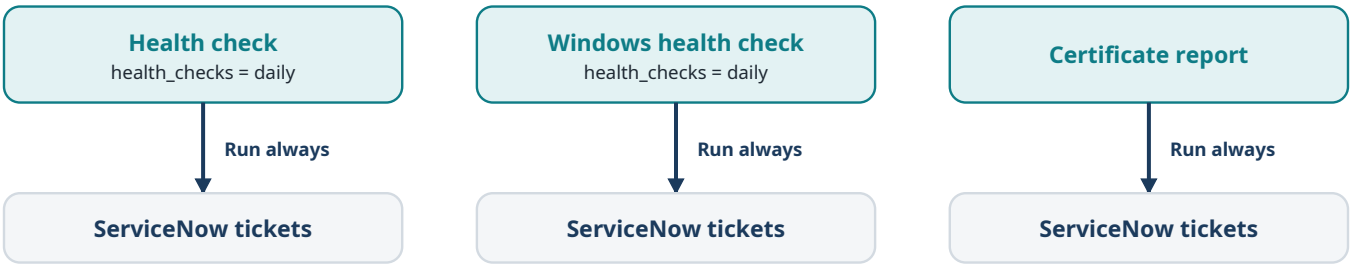



Launch with one server in the Limit.

## Linux and Windows together

### 17. Everything, every morning

One workflow, two independent chains (each first box added with **Add step**). Each step uses its own template's inventory, so leave the workflow's Inventory empty:

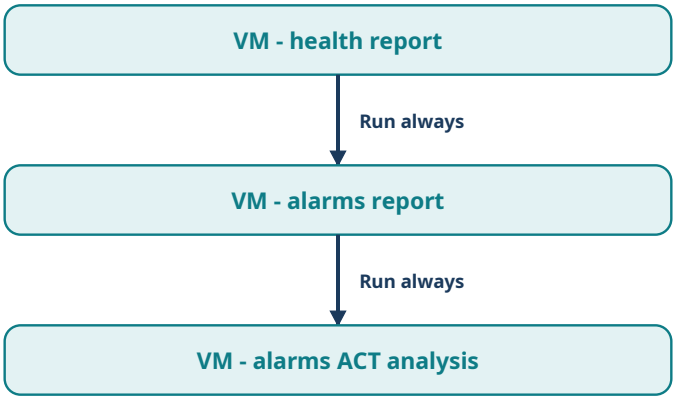



**Schedule:** every day 06:00. It replaces workflows 1 and 12, and the first two steps of 7 (keep 7 for its CSR step).

## VMware workflows

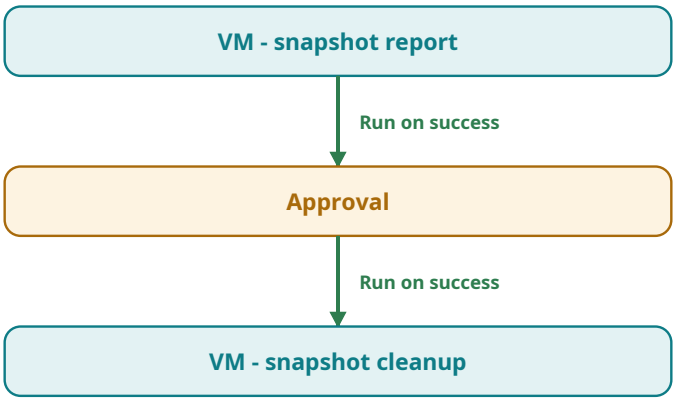
The VMware templates run on the AAP side against vCenter: no Limit, and each uses its own template's credentials (VMWARE.md).

### 18. VMware daily checks: health, alarms, then ACT's analysis (daily)




**Always**, not On success: a report is red when `vm_health_fail` / `vm_alarm_fail: true` is set, and the next step should run anyway. The health report covers every datacenter, cluster and host (HA, DRS, vCLS, hardware); the alarms report the alarms and the day's events; the analysis ACT's likely cause and fix for each. Each step sends its own email (`report_email_to` on each template, or a workflow survey question `report_email_to` for all). **Schedule:** every day 06:00. Only the analysis is wanted? Schedule `VM - alarms ACT analysis` on its own: its email lists every problem.

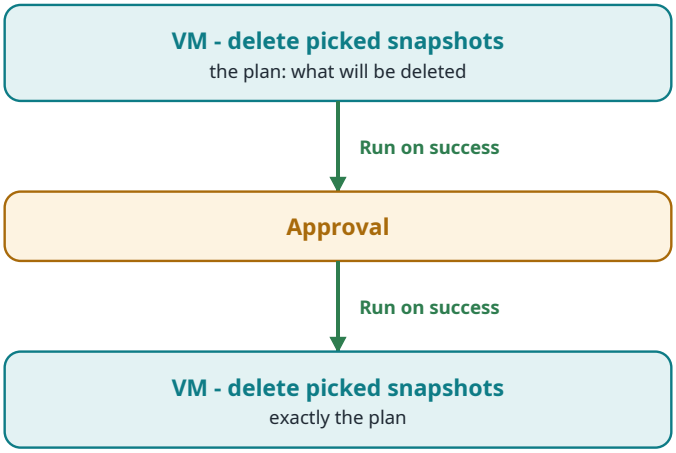
### 19. Snapshot cleanup with approval




Click by click, with the survey and the dry run: VMWARE.md, "Snapshots".

### 26. Delete picked snapshots, with approval (on demand)

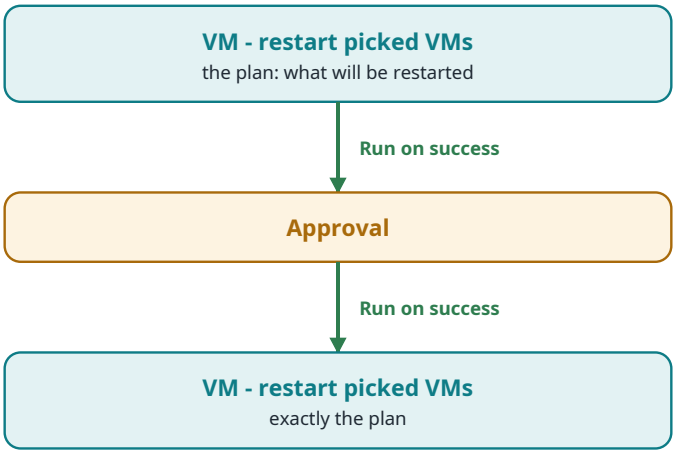
The workflow's survey: tick the snapshots. Its choices come from *VM - refresh the pick lists*, on a schedule every 30 minutes.




The **same** job template twice. Its survey question `vm_snapshot_picks` (Multiple choice, multiple select) is on the **workflow** - inside a workflow only the workflow's survey is asked, and an approval takes no input, so the picking is at launch. The first run plans (it never deletes inside a workflow), the approval reads that plan, the second run deletes exactly what it listed, each snapshot checked again live. Nothing left that can be deleted = the plan step fails and the workflow stops. Click by click: VMWARE.md, "Pick lists".

### 27. Restart picked (or hung) VMs, with approval (on demand)

The workflow's survey: tick the VMs, and How (guest or hard).




As 26, with `vm_reboot_picks` and `vm_power_mode` (`guest` / `hard`) on the workflow's survey. For a service desk list of **hung** VMs (no heartbeat, or VMware Tools not running): the refresh with `vm_pick_reboot_only_hung: true`, and How = `hard` - a guest restart needs VMware Tools running. Never the AAP server or a protected VM, whatever is picked.

### 20. ESXi security settings (nightly)

No workflow needed: a schedule on the `ESXi - security settings` template, every day at 00:00 in **your** time zone (VMWARE.md, "ESXi security settings").

### 22. vSphere capacity planning (weekly)

No workflow needed: a schedule on `VM - capacity planning`, e.g. every Monday at 07:00. One template per cluster (`vm_capacity_clusters`) gives each cluster's owners their own report (VMWARE.md, "Capacity planning").

### 23. VMs created this week (weekly)

No workflow needed: a schedule on `VM - VMs created report`, every Friday at 12:00 in your time zone (Repeat frequency *Week*, on *Friday*). It reports the VMs created in the 7 days up to the run (VMWARE.md, "VMs created").

### 25. vCenter and ESXi certificates (weekly)

No workflow needed: the `VMware - certificate renewal` template opens the incidents and attaches the CSRs itself. A schedule on it, e.g. every Monday at 07:00 in your time zone - and, if you want the list by email without tickets, one on `VMware - certificate report` (VMWARE.md, "Certificates of vCenter and the ESXi hosts").

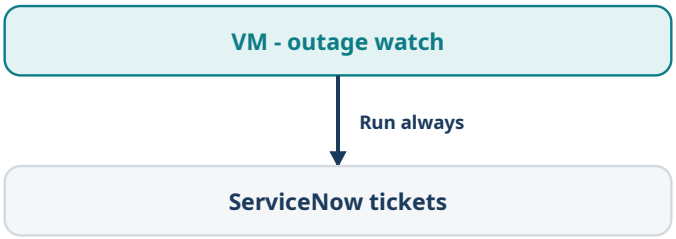
### 21. NetApp ONTAP health (daily)

No workflow needed: a schedule on the `NetApp ONTAP health report` template, e.g. every day at 06:00 in your time zone (NETAPP.md). With `ontap_report_fail: true` the job turns **red** when it finds a problem: put a notification on the template (its **Notifications** tab: on failure) to hear about it. The *ServiceNow tickets* job does not take this report's results - it only works after the checks that hand it findings (JOB_TEMPLATES.md, "What one job hands to the next").

### 24. Active Directory health (daily)

No workflow needed: a schedule on the `Active Directory health report` template, every day at 06:30 in your time zone, with `ad_report_hours: 24` (the default) so each change is in exactly one report (ACTIVE_DIRECTORY.md). With `ad_report_fail: true` the job turns **red** when it finds a problem: put a notification on the template (its **Notifications** tab: on failure) to hear about it. The *ServiceNow tickets* job does not take this report's results - it only works after the checks that hand it findings (JOB_TEMPLATES.md, "What one job hands to the next").

### 28. VM outage watch (every 5 to 15 minutes)

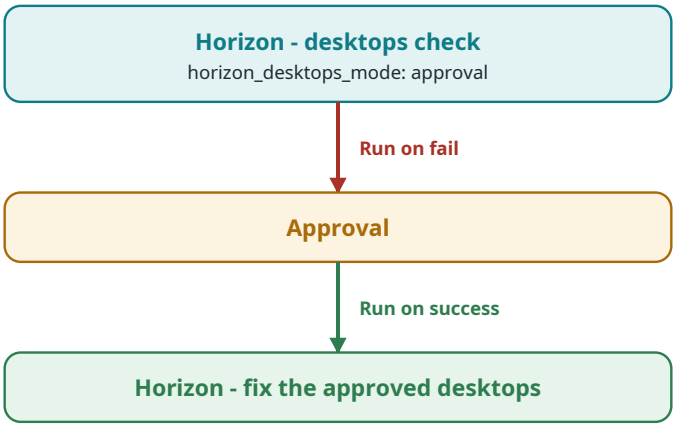



*Run always*, so a clean run reaches the tickets job too and the incident of a condition that cleared is noted (or resolved). **Concurrent jobs** off on the workflow, and `vm_watch_window_minutes` set to the schedule's interval. Click by click: VM_OUTAGE_WATCH.md, "Set it up".

## Horizon workflows

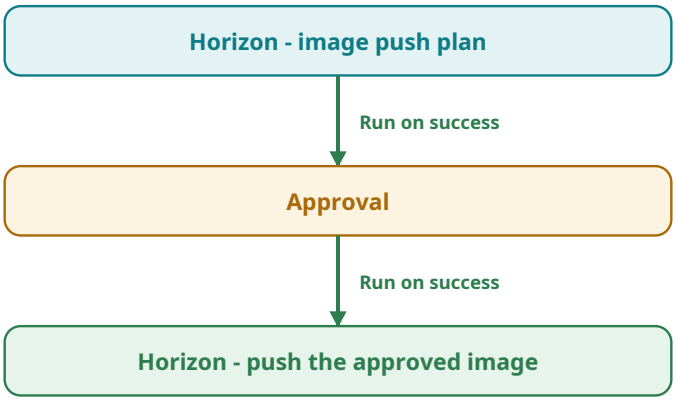
The Horizon templates run on the AAP side against a Connection Server: no Limit, the "Horizon API" credential (HORIZON.md).

### 30. Fix broken desktops, with an approval




The check fails on purpose when there is a desktop to fix (`NEEDS APPROVAL: ...`), and hands the list on. The fix job reads each desktop again and fixes only the approved ones still broken. Click by click, and a rehearsal as Check: HORIZON.md. **Schedule:** e.g. every hour in working hours.

### 31. Push a new golden image, with an approval




A survey on the workflow (the pools, the golden image VM, its snapshot, the start, the log-off policy). A plan with a problem fails and never reaches the approval (HORIZON.md).

### 32. Horizon reports and the session cleanup

No workflow needed: schedules on `Horizon - health report` (every day 06:00), `Horizon - capacity report` (every Monday 07:00) and `Horizon - session cleanup` (every night 02:00, after a first run as Check).

## Schedules

**What.** A schedule launches a template or workflow at set times.

**How.** Open the workflow (or template) › **Schedules** tab › **Create schedule**:

1. **Name**, **Start date/time**, **Time zone** (use your site's, not UTC, so "06:00" means 06:00).
2. **Repeat frequency** and its details: *Day* (every 1 day), *Week* (on Monday), *Month* (on the second Wednesday)...
3. **Prompts**: the survey answers, the Limit and the Job type **for this schedule**. Without them, the schedule uses the template's **defaults**. A schedule can only change what the template **prompts on launch** or asks in its survey.

4. **Finish**. The schedule's page shows the next run times. Check them once.

| What | When | Launches |
|---|---|---|
| Daily health (or *Everything, every morning*) | every day 06:00 | workflow 1 (or 17) |
| Certificate watch | every day 06:30 | workflow 7 |
| Database watch | hourly | workflow 8 |
| ServiceNow health | hourly | template *ServiceNow health*, with an email **notification on failure** |
| Weekly compliance | Monday 07:00 | workflow 2 |
| Security posture | Wednesday 07:00 | workflow 6 |
| Patch preview | the Monday before patch night | template *Patch hosts*, **Prompts › Job type** `Check` |
| Patch with checks | monthly, in your window | workflow 3 or 4 |
| Service watch | every 15 minutes (with **Enable concurrent jobs** switched off on the workflow) | workflow *Service watch - approve or self-heal* |
| Windows daily health / weekly compliance | 06:15 daily / Monday 07:15 | workflows 12 and 13 |
| Windows patch | monthly, in the ConfigMgr maintenance window | workflow 14 |
| VMware daily checks (health, alarms, ACT analysis) | every day 06:00 | workflow 18 |
| Active Directory health | every day 06:30 | template *Active Directory health report* (24) |
| vCenter and ESXi certificates | Monday 07:00 | template *VMware - certificate renewal* (25) |
| Pick lists (the surveys of workflows 26 and 27) | every 30 minutes | template *VM - refresh the pick lists* |
| Snapshot cleanup | Monday 07:00 | workflow 19 (VMWARE.md, "Snapshots") |
| ESXi security settings | every day 00:00 | template *ESXi - security settings* (20) |
| NetApp ONTAP health | every day 06:00 | template *NetApp ONTAP health report* (21) |
| vSphere capacity planning / VMs created | Monday 07:00 / Friday 12:00 | templates (22, 23) |
| Certificate install | every day 07:00 | workflow 29 |
| Horizon: broken desktops / reports | hourly in working hours / daily | workflows 30 and 32 |

| What | When | Launches |
|---|---|---|
| VM outage watch | every 5 to 15 minutes (**Concurrent jobs** off) | workflow 28 |
| Report website - copy | every 15 minutes | template *Report website - copy* (WEB_REPORTS.md) |
| VM deploy tickets | every 15 minutes | template *VM - pick up deploy tickets* (VM_DEPLOY_GUIDE.md) |
| Network config backup | every day | template *Network config backup* (NETWORK_CONFIG.md) |

**Notifications** (optional). **Automation Execution › Administration › Notifiers › Create notifier** (type *Email*, your mail relay). Then on a template or workflow › **Notifications** tab › switch on **Failure**. Use it where a ticket cannot help. ServiceNow health is the main case: if ServiceNow is down, it cannot take a ticket about itself.

## Things that trip people up

- **Red is not broken.** A check job with findings is red by design. Read `Report | result for this host`: findings mean the playbook worked.
- **Save the visualizer.** Closing without **Save** loses the graph.
- **One tickets box per check box.** Artifacts from two parents overwrite each other.
- **A step can only be set to Check** (or given a Limit, or survey answers) if its template has **Prompt on launch** for it, or a survey.
- **Schedules use survey defaults** unless the schedule sets its own answers (Prompts step).
- **Approvals need a timeout.** Otherwise a forgotten approval keeps the workflow open forever. After the timeout it counts as denied.
- **Workflow variables override node answers.** Leave the workflow's **Variables** empty, and set answers on each step.
- **Leave the workflow's Inventory empty** unless every step should use it: it only reaches steps whose template prompts for the inventory, and a Linux inventory on a Windows step matches no host.
- **Limit on the workflow** applies to every step whose template prompts for a Limit. The controller-side templates (ServiceNow tickets, ServiceNow health, POA&M status) must **not** prompt for one, or a workflow Limit makes them match no host and silently do nothing.
- **A tickets step fails when nothing reached it.** If the check step died before checking any host (credential, project sync, execution environment), the tickets step fails too, so a broken schedule shows red instead of green. Read the first red job.
- **No automatic restarts** unless a patch step gets `automatic_restarts` = `yes`.

# 6. Every job template: what it is, and where its how-to is

One line for each playbook in `playbooks/`, under the name the docs give its job template. Each line says what kind of job it is and links to its how-to.

**Every template starts the same way**: **Project** this repository, the **Playbook**, an **Inventory**, an **Execution environment**, the **Credentials**. Its **Variables** come from its own file in `aap/extra_vars/`: the file's first lines name the template, the playbook and the credentials. Copy the file into the template's Variables box and switch on only what you change (SETUP_AAP.md).

**The kinds.**

- **Report**: runs, reports, emails and publishes. Its how-to is its settings file; the linked section says what to schedule and what the report shows.
- **On demand**: a person launches it, with a Limit or a survey.
- **Workflow step**: it is one step of a workflow and hands results to the next step (the table What one job hands to the next).
- **After an approval**: the step after an approval. It does exactly what was approved, and nothing when launched on its own.
- **Watch**: on a schedule, often every few minutes.
- **Setup or test**: run once, or after a change, to set something up or to prove a connection.

## Linux servers

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| Health check | playbooks/health_check.yml | Workflow step | SETUP_AAP.md, the checks: RUNBOOKS.md, workflows 1-5: WORKFLOWS_AND_SCHEDULES.md |
| Troubleshoot | playbooks/troubleshoot.yml | On demand | SETUP_AAP.md, workflow 9: WORKFLOWS_AND_SCHEDULES.md |
| Patch hosts | playbooks/patch_hosts.yml | Workflow step | SETUP_AAP.md, workflows 3 and 4: WORKFLOWS_AND_SCHEDULES.md |
| Apply approved ACT fix | playbooks/act_fix_approved.yml | After an approval | SETUP_AAP.md, workflow 5: WORKFLOWS_AND_SCHEDULES.md |
| Database health - MariaDB / MySQL | playbooks/database_health.yml | Watch | MARIADB.md, workflow 8: WORKFLOWS_AND_SCHEDULES.md |
| Service watch - check | playbooks/service_watch.yml | Workflow step | SERVICE_WATCH_DEMO.md, the workflow: Part 6 |
| Service watch - apply approved fix | playbooks/service_fix_approved.yml | After an approval | SERVICE_WATCH_DEMO.md |
| POA&M status | playbooks/poam_status.yml | Report | SETUP_AAP.md, RUNBOOKS.md |
| STIG drift report | playbooks/stig_drift_report.yml | Report | STIG_DRIFT.md |
| STIG Manager - deploy | playbooks/stigman_deploy.yml | Setup or test | workflow 10: WORKFLOWS_AND_SCHEDULES.md |
| Log analysis | playbooks/log_analysis.yml | On demand | LOG_ANALYSIS.md |

## Windows servers

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| Windows connection test | playbooks/win_connection_test.yml | Setup or test | WINDOWS.md |
| Windows health check | playbooks/win_health_check.yml | Workflow step | WINDOWS.md, workflows 12 and 13: WORKFLOWS_AND_SCHEDULES.md |
| Windows troubleshoot | playbooks/win_troubleshoot.yml | On demand | WINDOWS.md |
| Windows patch | playbooks/win_patch.yml | Workflow step | WINDOWS.md, workflow 14: WORKFLOWS_AND_SCHEDULES.md |
| ACT for Windows - install | playbooks/win_act_install.yml | Setup or test | WINDOWS.md, workflow 15: WORKFLOWS_AND_SCHEDULES.md |
| Active Directory health report | playbooks/ad_health_report.yml | Report | ACTIVE_DIRECTORY.md |
| Active Directory - account lockout | playbooks/ad_lockout.yml | On demand | ACTIVE_DIRECTORY.md |

## Certificates and trust stores

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| Certificate report | playbooks/cert_report.yml | Workflow step | SETUP_AAP.md, workflow 7: WORKFLOWS_AND_SCHEDULES.md |
| Certificate renewal - CSRs | playbooks/cert_renew_csr.yml | Workflow step | SETUP_AAP.md, RUNBOOKS.md, with the PKI's catalog item: CERT_RENEWAL_CATALOG.md |
| Certificate install - check | playbooks/cert_install_check.yml | Workflow step | CERT_INSTALL.md, workflow 29: WORKFLOWS_AND_SCHEDULES.md |
| Certificate install - approved | playbooks/cert_install_approved.yml | After an approval | CERT_INSTALL.md |
| Certificate restore | playbooks/cert_restore.yml | On demand | CERT_INSTALL.md |
| Windows certificate report | playbooks/win_cert_report.yml | Workflow step | WINDOWS.md, workflow 13: WORKFLOWS_AND_SCHEDULES.md |
| Windows certificate renewal - CSRs | playbooks/win_cert_renew_csr.yml | Workflow step | WINDOWS.md |
| Windows certificate install - check | playbooks/win_cert_install_check.yml | Workflow step | CERT_INSTALL.md |
| Windows certificate install - approved | playbooks/win_cert_install_approved.yml | After an approval | CERT_INSTALL.md |
| Windows certificate restore | playbooks/win_cert_restore.yml | On demand | CERT_INSTALL.md |
| Trust store report | playbooks/trust_report.yml | Report | TRUST_STORES.md |
| Windows - trust store report | playbooks/win_trust_report.yml | Report | TRUST_STORES.md |
| VMware - trust store report | playbooks/vmware_trust_report.yml | Report | TRUST_STORES.md |
| VMware - certificate report | playbooks/vmware_cert_report.yml | Report | VMWARE.md |
| VMware - certificate renewal | playbooks/vmware_cert_renew.yml | Watch | VMWARE.md |
| Network device certificate report | playbooks/net_cert_report.yml | Report | NETWORK_CERTS.md |

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| Network device certificate renewal | playbooks/net_cert_renew.yml | Watch | NETWORK_CERTS.md |

The certificate inventory for a dashboard (CSV, JSON, SQL Server) is a setting of the certificate reports, not a template of its own: CERT_INVENTORY.md.

## VMware (vCenter)

Every VMware template: no Limit, the **VMware vCenter** credential, an execution environment with the VMware collection (VMWARE.md).

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| VM - restart | playbooks/vm_reboot.yml | On demand | VMWARE.md (each template's survey) |
| VM - shut down | playbooks/vm_shutdown.yml | On demand | VMWARE.md |
| VM - snapshot | playbooks/vm_snapshot.yml | On demand | VMWARE.md |
| VM - delete snapshot | playbooks/vm_snapshot_delete.yml | On demand | VMWARE.md |
| VM - notes | playbooks/vm_notes.yml | On demand | VMWARE.md |
| VM - change VLAN | playbooks/vm_vlan.yml | On demand | VMWARE.md |
| VM - secure boot report | playbooks/vm_secure_boot_report.yml | Report | VMWARE.md |
| VM - datastore report | playbooks/vm_datastore_report.yml | Report | VMWARE.md |
| VM - snapshot report | playbooks/vm_snapshot_report.yml | Workflow step | VMWARE.md, workflow 19: WORKFLOWS_AND_SCHEDULES.md |
| VM - snapshot cleanup | playbooks/vm_snapshot_cleanup.yml | After an approval | VMWARE.md |
| VM - refresh the pick lists | playbooks/vm_pick_refresh.yml | Watch | VMWARE.md |
| VM - delete picked snapshots | playbooks/vm_snapshot_delete_picked.yml | Workflow step | VMWARE.md, workflow 26: WORKFLOWS_AND_SCHEDULES.md |
| VM - restart picked VMs | playbooks/vm_reboot_picked.yml | Workflow step | VMWARE.md, workflow 27: WORKFLOWS_AND_SCHEDULES.md |
| VM - health report | playbooks/vm_health_report.yml | Report | VMWARE.md, workflow 18: WORKFLOWS_AND_SCHEDULES.md |

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| VM - alarms report | playbooks/vm_alarm_report.yml | Report | VMWARE.md |
| VM - alarms ACT analysis | playbooks/vm_alarm_act_analysis.yml | Report | VMWARE.md |
| VM - outage watch | playbooks/vm_outage_watch.yml | Watch | VM_OUTAGE_WATCH.md, workflow 28: WORKFLOWS_AND_SCHEDULES.md |
| VM - vCenter log analysis | playbooks/vm_log_analysis.yml | On demand | LOG_ANALYSIS.md |
| VM - capacity planning | playbooks/vm_capacity_report.yml | Report | VMWARE.md |
| VM - VMs created report | playbooks/vm_created_report.yml | Report | VMWARE.md |
| ESXi - security settings | playbooks/esxi_security.yml | Watch | VMWARE.md |
| VM - deploy settings finder | playbooks/vm_deploy_inventory.yml | Setup or test | VM_DEPLOY_GUIDE.md |
| VM - deploy from ServiceNow ticket | playbooks/vm_deploy_from_ticket.yml | Workflow step | VM_DEPLOY_GUIDE.md, the approval workflow: step 8 |
| VM - pick up deploy tickets | playbooks/vm_deploy_poll.yml | Watch | VM_DEPLOY_GUIDE.md |

## Horizon 8 (VDI)

Every Horizon template: any inventory (the jobs run on the AAP side), the **Horizon API** credential, read only or acting (HORIZON.md).

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| Horizon - API test | playbooks/horizon_api_test.yml | Setup or test | HORIZON.md |
| Horizon - health report | playbooks/horizon_health_report.yml | Report | HORIZON.md |
| Horizon - capacity report | playbooks/horizon_capacity_report.yml | Report | HORIZON.md |
| Horizon - desktops check | playbooks/horizon_desktops_check.yml | Workflow step | HORIZON.md |
| Horizon - fix the approved desktops | playbooks/horizon_desktops_fix_approved.yml | After an approval | HORIZON.md |

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| Horizon - image push plan | playbooks/horizon_image_push_plan.yml | Workflow step | HORIZON.md |
| Horizon - push the approved image | playbooks/horizon_image_push_approved.yml | After an approval | HORIZON.md |
| Horizon - user help | playbooks/horizon_user_help.yml | On demand | HORIZON.md |
| Horizon - session cleanup | playbooks/horizon_session_cleanup.yml | Watch | HORIZON.md |

## Network devices and storage

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| Network config backup | playbooks/net_config_backup.yml | Watch | NETWORK_CONFIG.md |
| Network config load | playbooks/net_config_load.yml | Workflow step | NETWORK_CONFIG.md |
| NetApp ONTAP health report | playbooks/ontap_health_report.yml | Report | NETAPP.md |

## ServiceNow, AAP and the reports

| Job template | Playbook | Kind | How-to |
|---|---|---|---|
| ServiceNow - test ticket | playbooks/servicenow_test_ticket.yml | Setup or test | SERVICENOW_SETUP.md |
| ServiceNow - catalog test | playbooks/servicenow_catalog_test.yml | Setup or test | CERT_RENEWAL_CATALOG.md |
| ServiceNow tickets | playbooks/servicenow_tickets.yml | Workflow step | SETUP_AAP.md, RUNBOOKS.md |
| ServiceNow health | playbooks/servicenow_health.yml | Watch | SETUP_AAP.md |
| AAP - report email question | playbooks/aap_email_question.yml | Setup or test | EMAIL_REPORTS.md |
| Report website - copy | playbooks/report_web_copy.yml | Watch | WEB_REPORTS.md |

## What one job hands to the next

In a workflow, a job hands its results to the steps after it as **job artifacts** (`set_stats`; on a finished job: **Details** > **Artifacts**). Only these hand-overs exist. Without its hand-over, a job after an approval stops and says why - nothing is changed. The exceptions work on their own too: the snapshot cleanup finds its own candidates, and a pick-list template's first run is its plan (VMWARE.md).

| From | Hands over | To | Workflow |
|---|---|---|---|
| Health check, Windows health check, Troubleshoot, Windows troubleshoot, Database health, Certificate report, Windows certificate report, Trust store report, Windows - trust store report, POA&M status, VM - outage watch | `site_health`, `site_health_targets` (the findings, per host) | ServiceNow tickets | 1, 2, 6, 7, 8, 12, 13, 17, 28 |
| Health check or Troubleshoot with ACT (`use_act`, `site_act_level: diagnose`) | `act_triage` (ACT's proposed fix), `site_health` | Apply approved ACT fix | 5 |
| Service watch - check | `service_watch_fix` | Service watch - apply approved fix | SERVICE_WATCH_DEMO.md, Part 6 |
| Certificate install - check, Windows certificate install - check | `cert_install_plan` | Certificate install - approved, Windows certificate install - approved | 29 |
| VM - snapshot report | `vm_snapshot_cleanup_candidates` (only for the cleanup in the same workflow) | VM - snapshot cleanup | 19 |
| VM - delete picked snapshots (its plan run) | `vm_snapshot_pick_plan` | the same template, after the approval | 26 |
| VM - restart picked VMs (its plan run) | `vm_reboot_pick_plan` | the same template, after the approval | 27 |
| Horizon - desktops check (`horizon_desktops_mode: approval`) | `horizon_desktops_fix` | Horizon - fix the approved desktops | 30 |
| Horizon - image push plan | `horizon_push_plan` | Horizon - push the approved image | 31 |
| VM - deploy from ServiceNow ticket (proposal) | not an artifact: the ticket's `[AAP] Checked` work note | the same template, after the approval | VM_DEPLOY_GUIDE.md, step 8 |
| every report job, with `report_web_copy: true` | `site_report_page` (the page, packed) | Report website - copy, read through AAP's API - no workflow | WEB_REPORTS.md |

The other reports keep their results as artifacts too (`vm_health`, `ad_health`, `log_analysis`...), for your own use - no step here reads them. They do **not** feed *ServiceNow tickets*: put a notification on failure on those templates instead (the `*_fail: true` settings make them red when they find a problem).

The numbers are the workflows in WORKFLOWS_AND_SCHEDULES.md, with every link and every answer.

# 7. Dry runs: try anything, change nothing

A **dry run** runs a job without changing anything. Ansible calls it *check mode*; AAP calls it **Job type: Check**. Every step that would change something only says what it *would* change. The steps that only read (the health checks, looking at logs, asking ServiceNow what is open) still run, so the answers are real.

Use a dry run the first time you use any template, on one host, and before every change you are not sure about: patching, applying an approved fix, installing ACT.

## Where to set it

There are five places. Pick the one that fits:

| Where | How | Good for |
|---|---|---|
| **1. The template** | Template › **Edit** › **Job type** `Check` | a template that is *always* a dry run, e.g. a copy named `Patch hosts (dry run)` that more people may launch |
| **2. At launch** | Template › **Edit** › tick **Prompt on launch** next to **Job type**. Then at launch, pick `Check` or `Run` | the usual way: the same template, you choose each time |
| **3. A workflow step** | (needs 2 on the template) Workflow › **Visualizer** › click the step › **Edit** (pencil) › **Job type** `Check` › **Save** | rehearsing a whole workflow, or a "dry run first, then real" workflow |
| **4. A schedule** | (needs 2) Template or workflow › **Schedules** › the schedule › **Prompts** › **Job type** `Check` | a regular preview, e.g. "what would patch night install" a week before |
| **5. Command line** | `ansible-playbook ... --check` (add `--diff` to see file changes) | testing from a shell |

**Show changes.** Tick **Show changes** on the template (or `--diff`) and a dry run also shows the line-by-line changes it would make to files, for example in STIG Manager's configuration.

**How to tell.** The job's **Details** tab says **Job type: Check**. Our playbooks also print `DRY RUN (Check mode): would ...` where it matters.

## What each template does in a dry run

| Template | In a dry run it... |
|---|---|
| Health check, Database health, Troubleshoot, Certificate report | runs **exactly as usual**: they only read, so you get the same findings. Nothing is written: no syslog lines, no disk-forecast sample, and on Windows no event log entries |
| POA&M status | as usual: reads the CSV and asks STIG Manager |
| ServiceNow tickets | asks ServiceNow what is already open, then prints `would open: ...`, `would add a note to: ...`, `would resolve: ...`. **Nothing is sent** |
| ServiceNow health | as usual: only reads |
| ServiceNow - test ticket | logs in and checks the assignment group and caller (it only reads), then prints the incident it would open, note and resolve. **Nothing is created** |
| Patch hosts (Linux) | lists the packages dnf would update. Installs nothing, restarts nothing |
| Apply approved ACT fix | `DRY RUN on <host>: approved, and would run: <commands>`. Runs nothing. A command the apply-time guard would refuse (reboot, `mkfs`, ...) shows `Refused on <host>: ...` and that host goes red, as in the real run |
| Service watch - check | checks the containers and shows what it would propose. Writes no incident, does not call ACT |
| Service watch - apply approved fix | `DRY RUN on <host>: approved, and would run: <commands>` (or `Refused on <host>: ...`, as above) |
| STIG Manager - deploy | says which files and units would change (with Show changes: how), and verifies nothing (nothing was started). Changes nothing. *Not yet tried on a real server: try it once on the test server* |
| Windows health check, troubleshoot, certificate report, connection test | as usual: they only read |
| Windows patch | asks Software Center to rescan (without starting anything) and prints `DRY RUN (Check mode): would install <updates>` |
| ACT for Windows - install | `DRY RUN: would install ACT in C:\ProgramData\act because ...`, or that it is already current |
| VMware jobs (restart, shut down, snapshot, delete snapshot, notes, change VLAN) | finds the VMs and runs every check (the AAP VM is refused as in a real run), then prints `Would restart ...`, `Would take snapshot ...`, `would delete ...`, the notes `AFTER`, `would move ...`. **vCenter is not changed**, and no report email is sent (`DRY RUN: would email ...`) |

| Template | In a dry run it... |
|---|---|
| VM - secure boot report, VM datastore report, VM snapshot report, VM alarms report | as usual: only read. No report email is sent |
| VM - alarms ACT analysis | reads vCenter and lists the problems it would give ACT (`A dry run (Check): ACT was not called`). ACT is not called, no email is sent |
| NetApp - ONTAP health report | as usual: it only reads (GET requests), and prints the same report. No email is sent |
| Horizon - every template | reads as usual and says what it would do: `would recover`, `would log off`, the push it would schedule. Nothing is sent to Horizon but reads - not even a login for an action. No email is sent |
| VM - capacity planning | as usual: it only reads. With `vm_capacity_act`, ACT is not called (`ACT did not run: A dry run`). No email |
| VMware - certificate report | as usual: only read (it never talks to ServiceNow). No email is sent |
| VMware - certificate renewal | reads every certificate and the incidents, then lists what it would do (`would open: ...`, `would make the CSR, and would attach it to INC...`). **No CSR is made** (vCenter and the hosts keep any key waiting), nothing is sent to ServiceNow, no email |
| ESXi - security settings | reads every host's SSH service, timeouts and lockdown mode and lists what it would change (`ESXi security settings (dry run): 3 of 40 host(s) would change`). **No host is changed**, no email is sent |
| VM - snapshot cleanup | lists what it would delete (`WOULD DELETE ...`), deletes nothing, sends no email. In the approval workflow: set the cleanup step's Job type to Check (docs/VMWARE.md, "Dry run") |

**ACT does not run in a dry run.** It would call the model, and ACT could change things. A dry run with `use_act` = `yes` reports the checks and says ACT gave no result. To see ACT's ideas without changes, do a *real* run with `site_act_level` = `diagnose`: ACT then only reads and proposes (ADDING_ACT.md).

## Examples

**Before patching one server.** Launch **Patch hosts** › **Job type** `Check`, **Limit** `web01`. The output lists every package that would be updated. Nothing is installed.

```
TASK [patch : Patch | what was updated (dry run - what WOULD be updated)]
ok: [web01] => {"msg": "DRY RUN (Check mode), nothing was installed - would make 4 change(s):
  Installed: expat-2.5.0-6.el9_8.5.x86_64; Installed: libxml2-2.9.13-14.el9_8.5.x86_64;
  Removed: libxml2-2.9.13-14.el9_8.4.x86_64; Removed: expat-2.5.0-6.el9_8.3.x86_64"}
```

(`Removed` is the old version that the new one replaces.)

**Before patch night, every month.** On **Patch hosts** add a schedule for the Monday before patch night, with **Prompts** › **Job type** `Check`. The job output shows what patch night will install on every host, in time to

exclude something.

**A dry-run copy for the team.** Template **Patch hosts** › **…** › **Copy**. Rename the copy `Patch hosts (dry run)`, set **Job type** `Check`, untick **Prompt on launch** for it, and give your team **Execute** on the copy only. They can preview; only you can patch.

**Which tickets would a run open?** In the *Daily health* workflow click the **ServiceNow tickets** step › **Edit** › **Job type** `Check`. Run the workflow: the tickets job prints what it would open, note and resolve. Set it back to `Run` when the tickets look right.

**A Windows server, before its first real patch.** Launch **Windows patch** › **Job type** `Check`, **Limit** one server:

```
"DRY RUN (Check mode): would install 2026-09 Cumulative Update (KB5040001); Malicious Software
  Removal Tool (KB890830)"
```

**The approve-and-fix path, end to end.** See WORKFLOWS_AND_SCHEDULES.md, "Fix with approval (ACT)": set the apply step to `Check`, run the workflow, approve. The apply job shows what it would run.

**From a shell** (on a machine with Ansible and this repository):

```shell
ansible-playbook -i <inventory> playbooks/patch_hosts.yml --check --diff -l web01
ansible-playbook -i <windows inventory> playbooks/win_patch.yml --check -l web01
```

## What a dry run cannot tell you

- Whether the updates will **install cleanly**, or how long they take: it only lists them.
- Whether a **restart** would succeed: nothing restarts in a dry run.
- Anything that **depends on an earlier change**. In a dry-run workflow the post-check after "patching" still sees the unpatched host, because nothing was patched.
- What **ACT** would say (see above).

# 8. The runbooks: every check and what to do

For each runbook: what it answers, its settings (defaults in `roles/<role>/defaults/main.yml`; change them in your settings files or AAP's Variables boxes, as VARIABLES.md shows), and each finding with what usually fixes it. The `look:` command printed with a finding is always the first thing to run.

## Health check (playbooks/health_check.yml)

Runs the checks you pick on each host: a list of names, or the shortcuts:

- `daily` = disk, mounts, services, performance, time, network, logging, mariadb (the database check: MariaDB or MySQL; `mysql` and `database` also pick it), containers (podman containers, rootful and rootless)
- `weekly` = selinux, fapolicyd, auditd, accounts, certs, trust, patching
- `all` = both

A host with a finding at a severity in `site_fail_on` (default: critical and warning) fails. That is what a workflow reacts to. A template that only emails its report to people: `site_fail_on: []` keeps the job green (EMAIL_REPORTS.md). A check that itself breaks becomes a warning finding (`the X check could not run: ...`, id `<check>:check-error`) and the other checks still run. That finding fails the host even when `site_fail_on` is `[critical]` (a check that did not run is not a healthy result); only `site_fail_on: []` never fails. Its ServiceNow incidents from earlier runs stay open: a check that did not run to its end clears nothing it found before (no "passes again" note, nothing resolved).

**A host that cannot be reached** (switched off, no network, ssh or WinRM refused, its login refused) does not fail the job. It is listed as `NOT REACHED - <why>` and left out; the other hosts are checked as usual. It still counts as checked-for, so the ServiceNow step opens its "no result" ticket. The same goes for the certificate report and renewal, database health, the service watch and troubleshoot, Linux and Windows. Two cases still fail the job:

- **no host at all is reached**: that is what a wrong or expired machine credential looks like;
- `site_unreachable_fails: true`: a host not reached fails the job, as before.

Patching, the ACT and service fixes, the STIG Manager deploy and the Windows connection test always fail for a host they cannot reach: a workflow must not go on as if it had been patched or fixed. A host lost later in a job (it went down halfway) still fails it.

**All settings** of this job, with their defaults: site_findings.

### disk

| Finding | Means | Usually |
|---|---|---|
| `/var is 96% full` | local filesystem space (`check_disk_warn_pct` 85, `check_disk_crit_pct` 95) | find what grew with the `look:` command: old logs, core dumps, a runaway log file. `journalctl --vacuum-size=500M` for the journal |
| `... has used 97% of its inodes` | too many small files (space may still be free) | usually a directory with millions of small files (session/cache/spool); `look:` finds it |
| `/var will be full in about 3.2 days` | growth-rate forecast from earlier runs (needs a day of history) | same as above, before it becomes an outage |
| `could not read filesystem usage` | `df` failed or timed out | a hung mount; see the mounts check |

Per-mount limits: `check_disk_overrides: {/var/log/audit: {warn: 70, crit: 85}}`. The forecast keeps a small history in `/var/lib/site-health/disk-history.json` (the only file the checks write; a Check-mode run does not write it).

**All settings** of this check, with their defaults: check_disk.

### mounts

| Finding | Means | Usually |
|---|---|---|
| `/data (xfs) is in /etc/fstab but is not mounted` | it did not mount at boot, or was unmounted | `journalctl -b` for the mount error; a missing disk or LUN; a typo in fstab |
| `/var is mounted READ-ONLY` | the kernel remounted it read-only after disk/filesystem errors | **urgent**: `journalctl -k` for I/O errors; storage team; plan an fsck |
| `network mount /mnt/share does not answer within 5s` | NFS/CIFS server down, network, or a stale handle | check the server and the network; diagnose only (NetApp is vendor-managed) |
| `network mount ... is 96% full` | the share is full | the storage owner |

**All settings** of this check, with their defaults: check_mounts.

### services

| Finding | Means | Usually |
|---|---|---|
| `systemd unit X has FAILED` | a unit crashed or exited with an error | `systemctl status X`, `journalctl -u X`; fix the cause, then `systemctl restart X` |
| `required service X is inactive` | a service that must run (`check_services_required` + `_extra`) is stopped | same; find out why it stopped before starting it |
| `required service X is not installed` | the unit does not exist here | remove it from the list for this group, or install it |
| `required service X is disabled` | it will not start after a reboot | `systemctl enable X` |
| `X has restarted 5 times` | a restart loop: it keeps crashing | its log shows the crash |

Per group: `check_services_required_extra: [mariadb]`. To ignore a failed unit: `check_services_ignore_failed: ['^dnf-makecache']`.

**All settings** of this check, with their defaults: check_services.

### performance

| Finding | Means | Usually |
|---|---|---|
| `15-minute load is 14.2 on 4 CPUs (3.55 per CPU)` | more work than CPUs, for 15 minutes | the top CPU processes are in the finding; a runaway job, or undersized |
| `only 4% of memory is available` | memory is nearly exhausted | the top memory processes are in the finding; a leak, or undersized |
| `63% of swap is in use` | the host is short of memory | as above |
| `CPUs spend 35% of their time waiting for disk` | slow or overloaded storage | `iostat -x` shows which device; storage latency |
| `the hypervisor takes 15% of CPU time (steal)` | the VM host is overcommitted | the virtualization team |
| `the kernel killed processes for lack of memory` | OOM kills in the last 24 h (with the victims) | the victim's memory limit, or the host's memory |
| `27 zombie processes` | a parent process is not reaping its children | restart the parent (`look:` shows it) |

**All settings** of this check, with their defaults: check_performance.

### time

| Finding | Means | Usually |
|---|---|---|
| `the clock is NOT synchronized` | chrony uses no time source | `chronyc sources -v`: unreachable servers, firewall (UDP 123) |
| `chrony is installed but not answering` | chronyd is not running | `systemctl start chronyd` |
| `the clock is 2.4s ahead of NTP time` | offset above `check_time_offset_warn` | chrony corrects it slowly; very large offsets break Kerberos/AD logins at 5 minutes |

**All settings** of this check, with their defaults: check_time.

### network

| Finding | Means | Usually |
|---|---|---|
| `there is no default route` | the host cannot reach other networks | NetworkManager connection, gateway |
| `this host's own name ... does not resolve` | its FQDN is not in DNS or /etc/hosts | DNS record |
| `ldap.example.mil does not resolve (DNS)` | a name in `check_network_dns_names` fails | `/etc/resolv.conf`, DNS server |
| `LDAP (ldap.example.mil:636) does not answer within 5s` | a TCP port in `check_network_tcp` is unreachable **from this host** | firewall (local or network), the service down, routing |

**All settings** of this check, with their defaults: check_network.

### logging

| Finding | Means | Usually |
|---|---|---|
| `the journal is kept in memory only` | `/var/log/journal` missing: logs vanish at reboot | `mkdir /var/log/journal && systemctl restart systemd-journald` (or `Storage=persistent`) |
| `log server SIEM (...) does not answer` | forwarding target unreachable | network/firewall; the SIEM collector |
| `rsyslog has 850 MB of undelivered messages queued` | the log server has been unreachable for a while | as above; the queue drains once it answers |
| `1203 error messages in the journal` | something complains a lot (top senders listed) | look at the top sender first |

**All settings** of this check, with their defaults: check_logging.

### selinux

| Finding | Means | Usually |
|---|---|---|
| `SELinux is Permissive, it must be enforcing` | not enforcing (STIG) | `setenforce 1` after checking denials; `SELINUX=enforcing` in `/etc/selinux/config` |
| `/etc/selinux/config says SELINUX=permissive` | a reboot would change the mode | fix the config file |
| `locally added permissive domain(s)` | SELinux only logs for those domains | `semanage permissive -d <domain>` once the policy is right |
| `12 SELinux denial(s) since yesterday - top: nginx (httpd_t) denied read on a file labeled user_home_t` | SELinux blocked something | a wrong label (fix: `restorecon -Rv <path>`), a boolean (`setsebool -P ...`), or a real policy need. `audit2why` says which |
| `SELinux boolean X is off, expected on` | a boolean in `check_selinux_booleans` differs | `setsebool -P X on` |
| `N file(s) under /etc/nginx have the wrong SELinux label` | a dry-run `restorecon` would relabel them (`check_selinux_relabel_paths`) | `restorecon -Rv <path>` |

**All settings** of this check, with their defaults: check_selinux.

### fapolicyd

| Finding | Means | Usually |
|---|---|---|
| `fapolicyd is inactive` | application allow-listing is off | `systemctl start fapolicyd`; its log says why it stopped |
| `fapolicyd is in permissive mode` | it logs but blocks nothing | `permissive = 0` in `/etc/fapolicyd/fapolicyd.conf` |
| `N file(s) no longer match the fapolicyd trust database` | files changed outside RPM | `fapolicyd-cli --update`, or re-add the file's trust entry |
| `N fapolicyd denial(s) since yesterday - top: /usr/bin/bash -> /opt/app/tool` | it blocked programs | a legitimate program needs a trust entry (`fapolicyd-cli --file add <path>`), after review |

`check_fapolicyd_required: true` (the example inventory sets it) makes a missing fapolicyd a finding.

**All settings** of this check, with their defaults: check_fapolicyd.

### auditd

| Finding | Means | Usually |
|---|---|---|
| `auditd is failed` | nothing is being audited | `systemctl status auditd`; often a full audit log partition |
| `auditing is DISABLED in the kernel` | `auditctl -s` says enabled 0 | `audit=1` on the kernel command line |
| `only 1 audit rule(s) are loaded` | the STIG rules are missing or failed to load | `augenrules --check`, `augenrules --load`; a syntax error in `/etc/audit/rules.d/` |
| `N audit event(s) were LOST` | the kernel backlog overflowed | raise `-b` (backlog) in the rules |
| `the audit log filesystem is 92% full` | auditd may stop, or halt the host, when it fills | rotate/offload; check `space_left_action` |
| `audit records are not forwarded` | au-remote is not active (`check_auditd_remote_required`) | configure audisp-remote |
| `audit volume is 850 MB a day (... events in 24 h, at most N in one hour) - mostly account svc_x (60%), rule key delete (55%), program /usr/bin/python3 (40%)` | more audit records than `check_auditd_volume_warn_mb_day` (200), excluded accounts not counted | what the named account/program does that the named STIG rule records: a job deleting temp files in a loop, an app in a rootless container (its processes keep the user's login ID) |
| `the audit logs on disk reach back only 9.5 h (5 files of 8 MB, then the oldest is deleted)` | with `max_log_file_action = ROTATE` older records are gone (`check_auditd_retention_warn_hours` 24) | bigger `max_log_file` / more `num_logs` in `/etc/audit/auditd.conf`, or fewer records (the finding names who makes them) |
| `auditd dropped records N time(s) ... on the way to its plugins` | the queue to au-remote / the SIEM forwarder was full: the SIEM copy is missing records | fewer records; a larger `q_depth` in `auditd.conf` |
| `audit rule(s) stop recording auid=1234 (someone) altogether` | a `-a never` (or `-a always,exclude`) rule hides an account that is not in `check_auditd_exclude_accounts` | remove the rule, or approve the account (below) |
| `too much to break down by account within 120 s` | very large logs | `aureport` with the `look:` command, or a larger `check_auditd_volume_timeout` |

**How much is audited, and the vulnerability scanner.** The volume, retention and rule findings come from the audit logs themselves. The breakdown (which account, which STIG rule key, which program) is read only when the volume or the retention is over its limit, at low priority, and is always printed in the job output (`Auditd | how much is audited`), with excluded accounts shown separately.

A credentialed vulnerability scan logs in to every host and runs thousands of commands with sudo: during a scan its account usually makes most of the audit records. List it in `check_auditd_exclude_accounts` (in `playbooks/group_vars/all.yml`, so it applies to every host) and it no longer counts toward the volume warning. It still counts for retention and disk space, which are physical: if the scan rotates a week of records

away in an hour, that is still reported, with the scanner named as the cause. The job output shows how much of the scanner's volume comes from syscall rules and how much from sudo/PAM records, which tells you what an audit rule exclusion would remove.

To stop auditd recording that account at all (a host change, not part of the health check):

- It is a deliberate exception to the STIG audit requirements: get it approved and recorded first.
- The rules `-a never,exit -F arch=b64 -S all -F auid=<UID>` and the same with `arch=b32` (the form of audit's own sample rules), placed before the STIG rules, remove the syscall records: a file like `/etc/audit/rules.d/05-exclude-scanner.rules` (`augenrules` merges the files in name order). The sudo/PAM records (USER_CMD, USER_START, CRED_*) come from user space and need `-a never,user -F auid=<UID>`. Leaving that one out keeps the sudo records, which still show what the account ran.
- Use the numeric UID. A name is looked up when the rules load at boot, before IdM/AD (sssd) can answer, and a rule that fails to load can stop the rest of the rules from loading.
- STIG hosts make the rules immutable (`-e 2`): a new rule takes effect only at the next reboot.
- The auditd check then reports rules like this only for accounts that are NOT on `check_auditd_exclude_accounts`.

**All settings** of this check, with their defaults: check_auditd.

### accounts

| Finding | Means | Usually |
|---|---|---|
| `account(s) other than root have UID 0` | full root power under another name | **investigate first**; remove or renumber the account |
| `account(s) with an EMPTY password` | log in without a password | `passwd -l <user>` |
| `3 account(s) not used for 35+ days` | inactive and not locked (STIG) | lock them (`usermod -L`), or list break-glass accounts in `check_accounts_ignore_inactive` |
| `sudo without a password (NOPASSWD) for: alice` | a sudoers entry skips the password | remove it, or allow it on purpose in `check_accounts_nopasswd_allowed` (e.g. `svc_aap`) |
| `the password of svc_aap expires in 6 days` | an account in `check_accounts_expiry_watch` expires soon | **change it before it breaks automation** (and update the Machine credential) |

**All settings** of this check, with their defaults: check_accounts.

### certs

| Finding | Means | Usually |
|---|---|---|
| `certificate CN=... in /etc/stigman/stigman.crt expires in 9 days` | a certificate file expires within `check_certs_crit_days` (critical) / `warn_days` (warning) | renew it. For STIG Manager: update the *TLS certificate* credential and run *STIG Manager - deploy*; it restarts only nginx |
| `... in STIG Manager (https://127.0.0.1:443) expires in ...` | the certificate a TLS endpoint serves | the service's certificate, even if the file on disk was already replaced (restart the service) |
| `... in /opt/.../agent_keystore (alias mid) expires in ...` | a Java keystore entry | renew in the keystore (MID Server: ServiceNow's procedure) |
| `could not check the certificate in X` | unreadable file, endpoint not answering, or a PKCS12 keystore without its password | fix the path, or add the Keystore password credential |
| a certificate file is not in the report at all | its folder is not searched (a home folder, an application's own folder), or it is a bundle of more than 10 certificates (listed under *Found but not checked*) | add the folder to `check_certs_discover_extra_dirs` (two levels deep: `[/home, /root]` finds `/home/you/x.crt`), or the file to `check_certs_files` |

What it looks at: `check_certs_files` (globs), the usual directories (`check_certs_discover_dirs`, plus your own in `check_certs_discover_extra_dirs`; skipping CA bundles), `check_certs_endpoints`, `check_certs_keystores`. Text (PEM) and binary (DER) certificate files both, the names in any case (`.crt`, `.CRT`, `.pem`, `.cer`, `.der`). For a PKCS12 keystore, name the variable that holds the password (`password_env: KEYSTORE_PASSWORD`), never the password.

**All settings** of this check, with their defaults: check_certs.

### trust

Which trust stores on the host still hold a certificate authority that was compromised or that the browsers dropped - the list ships with the check (October 2026: the eight eMudhra intermediates and their roots; yours in `check_trust_extra`) - and the software that carries its own trust logic. Every certificate is matched by its fingerprint and by its public key. TRUST_STORES.md.

| Finding | Means | Usually |
|---|---|---|
| `the system trust (tls-ca-bundle.pem and 1 more, ca-certificates ...) trusts emSign ECC Root CA - G3 - ...` | the system bundles still hold a dropped root: every OpenSSL / Go / Python / curl client on the host accepts the compromised chains | update `ca-certificates`; or blocklist it (a copy in `/etc/pki/ca-trust/source/blocklist/`, `update-ca-trust`) |
| `... trusts EM DV TLS CA - G3B - its private key is computable from the certificate ...` (**critical**) | a compromised intermediate itself is in a store or an application's bundle | take it out; find out who put it there |
| `2 Python certifi bundles (...) trusts ...` | requests / pip carry their own roots, per environment | `pip install -U 'certifi>=2026.10.19'` in each environment the finding lists |

| Finding | Means | Usually |
|---|---|---|
| `/usr/lib/jvm/.../cacerts 17.0.9 (Java cacerts) trusts ...` | a JDK's own trust store | update the JDK, or `keytool -delete`; a `cacerts` linked to `/etc/pki/java/cacerts` follows the system trust |
| `/usr/bin/node (built-in roots) 22.23.1 ... trusts ...`, `Node.js 22.23.1 is below 22.23.4` | Node carries its own roots | update Node, or start it with `--use-openssl-ca` |
| `container web (root) ... trusts ...` | the image's own bundle | pull or rebuild the image, restart the container |
| `3 Go programs built with a Go before the fix: ...` | a Go program verifies certificates itself; 1.27.3 / 1.26.10 reject the compromised keys | rebuild each with a fixed Go, or get the vendor's build |
| `NSS 3.126.0 is below 3.131.1` | the roots NSS carries (Firefox, NSS clients without p11-kit) | update `nss` |
| `no system CA bundle was read on this host` | none of `check_trust_bundles` exists here | name the host's bundle in `check_trust_bundles` |

A certificate that is in the blocklist is *blocked*: no finding. *AAA Certificate Services* (Sectigo's root that cross-signed the compromised intermediates) is a **notice** by default - in the report, no finding - because nearly everything chains to it; `check_trust_severity: {"AAA Certificate Services": warning}` once you decide to blocklist it. The **Trust store report** (below) shows every store read on every host, and vCenter and ESXi.

**All settings** of this check, with their defaults: check_trust.

### patching

| Finding | Means | Usually |
|---|---|---|
| `12 security advisories pending, 2 of them rated Critical` | updates that fix security issues are waiting | patch (the *Patch with checks* workflow) |
| `a reboot is needed to finish earlier updates` | new kernel or core libraries are not in use yet | reboot in a window |
| `no package has been updated for 60 days` | older than `check_patching_max_age_days` | patch |
| `cannot check for updates` | dnf cannot reach its repositories (Satellite, mirror) | repository configuration, network, subscription |

**All settings** of this check, with their defaults: check_patching.

### mariadb and mysql (health only): the database check

Checks **MariaDB and MySQL** (5.7, 8.0, 8.4; MariaDB 10.x and 11.x; Percona). Pick it in the Health check as `mariadb`, `mysql` or `database` (the same check), or run it alone with the **Database health - MariaDB / MySQL** template (`playbooks/database_health.yml`: only this check, on every database server it is pointed at; it fails a host that has findings, so a workflow can open a ticket). In the Health check it runs on hosts in `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` or `mysql`, and on any host that has a container name set. It runs status queries only: no data is read and nothing is changed. It handles a database installed on the host and one in **podman containers** (one, several, found automatically, or rootless): the client inside each container is used. **How to set it up, step by step, and how to verify it: MARIADB.md.**

| Finding | Means | Usually |
|---|---|---|
| `MariaDB is not running` | the service or container is down | its log; disk full; a crash |
| `MariaDB is running but the status query failed` | cannot log in, or it hangs | the login (see below); `max_connections` reached |
| `MariaDB restarted 12 minutes ago` | a restart nobody may have planned | the error log: crash, OOM kill |
| `143 of 151 connections in use (95%)` | clients are about to be refused | the `look:` query shows which user and client hold them; a connection leak in an app |
| `N client connection(s) were REFUSED` | `max_connections` was reached since the last start | as above; raise `max_connections` only if the load is real |
| `40 queries are executing at this moment` | a pile-up: locking or one slow query | `look:` lists the oldest |
| `2 query/queries running longer than 300s` | long-running queries (user / db / seconds / state; never the query text) | the owner decides; `KILL <id>` only after that |
| `replication (default) is broken: IO thread No` | a replica stopped copying | `Last_IO_Error` / `Last_SQL_Error` in the finding |
| `replica ... is 1900 seconds behind` | replication lag | load on the replica; long transactions on the primary |
| `Galera node is not healthy` / `cluster has 2 node(s), expected 3` | Galera cluster trouble | the node's error log; network between nodes |
| `MySQL Group Replication is not healthy` / `has 2 ONLINE member(s), expected 3` | a Group Replication member is not ONLINE, or the group is short (`check_mariadb_group_size`) | `SELECT ... FROM performance_schema.replication_group_members` (in the finding); the member's error log |
| `could not read the replication status` / `... Group Replication members` | the monitoring account lacks a privilege | the grant named in the finding (MARIADB.md) |

| Finding | Means | Usually |
|---|---|---|
| `only 91% of reads come from memory` | `innodb_buffer_pool_size` may be too small | the DBA |
| `3 [ERROR] line(s) in the MariaDB log` | errors in its log in the last day (latest shown) | read them |
| `the MariaDB data directory is on a filesystem 93% full` | MariaDB stops writing when it fills | disk space |

**Performance** (all warnings; `check_mariadb_performance: false` turns them off). Counters that count since the server started are judged as an average since then, and only after it has run `check_mariadb_perf_min_uptime_hours` (2): cold caches right after a start are normal. Old transactions, lock waits, purge lag and open files are judged as they are now. The job's status line prints the numbers either way: `performance since start, 6 days: 100 queries/s, 3 slow/h, 4% temporary tables on disk, ...`.

| Finding | Means | Usually |
|---|---|---|
| `mariadb:slow-queries: 504 slow queries per hour ...` | queries slower than `long_query_time`, on average since the start | the slow query log (the finding says whether it is on) or `performance_schema` (the `look:` query lists the costliest statements) shows which; an index or a rewrite |
| `mariadb:tmp-disk: 60% of the temporary tables ... went to disk` | sorts and GROUP BYs spill to disk | `tmp_table_size` and `max_heap_table_size` (the smaller one counts); TEXT/BLOB columns always spill |
| `mariadb:full-joins: 1,440 joins per hour ... read a whole table` | joins with no usable index | the DBA adds the index; they get slower as the tables grow |
| `mariadb:table-locks` | MyISAM / Aria tables locked as a whole are busy | convert them to InnoDB (the `look:` query lists them) |
| `mariadb:row-locks: queries waited on average 2,000 ms for a row ...` | transactions wait for each other's row locks | long transactions, or many writers on the same rows |
| `mariadb:deadlocks` | InnoDB rolled back one transaction of each deadlock | the application's order of updates; `SHOW ENGINE INNODB STATUS` shows the latest |
| `mariadb:lock-waits: waiting for a lock for 30 s or longer right now ...` | sessions blocked NOW (metadata, table or row locks; who / db / seconds / state, never the query) | the session holding the lock: usually an open transaction or an `ALTER TABLE`. The owner decides before any `KILL` |
| `mariadb:long-transactions: 2 transaction(s) open longer than 600 s ...` | transactions left open hold their locks and stop purge; `idle (Sleep)` = the application never committed | the application owner; a connection pool that does not commit |
| `mariadb:purge-lag: 2,500,000 old row versions wait to be purged` | InnoDB history list length: reads slow down, undo grows | the long transaction above |

| Finding | Means | Usually |
|---|---|---|
| `mariadb:open-files` | close to `open_files_limit` | raise `open_files_limit` (and the unit's `LimitNOFILE`) |
| `mariadb:table-cache` | the table cache is full and tables are reopened all the time | `table_open_cache` |
| `mariadb:bufferpool-wait`, `mariadb:log-waits` | InnoDB waited for a free buffer pool page / a full log buffer | `innodb_buffer_pool_size` / `innodb_log_buffer_size` |
| `mariadb:aborted-connects` | many failed connection attempts per hour | wrong passwords, a health probe that does not log in, or a scan |

The performance query reads `information_schema.INNODB_TRX` (needs `PROCESS`, which the monitoring account already has) and `INNODB_METRICS`; without the privilege those findings are simply not made, the rest still are.

**Logging in** (the full steps are in MARIADB.md). Without a credential, the check logs in as root through the local socket. That is MariaDB's default for a host install on RHEL (no password needed with sudo); MySQL's root usually has a password. **A database in a container usually gives root a password** (the official images do), and the check then reports `Access denied ... attach the "MariaDB monitor" credential`. For containers, or whenever you prefer not to use root, ask the DBA to create a monitoring account with **no data access**, inside that database with host `localhost`, and attach the *MariaDB monitor* credential to the Health check template:

```sql
CREATE USER 'aap_monitor'@'localhost' IDENTIFIED BY '<password>';
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';   -- MySQL; MariaDB before
  10.5.9
GRANT PROCESS, REPLICA MONITOR ON *.* TO 'aap_monitor'@'localhost';      -- MariaDB 10.5.9 and
  newer instead
GRANT SELECT ON performance_schema.* TO 'aap_monitor'@'localhost';       -- MySQL Group
  Replication only
```

`PROCESS` shows other sessions (for long queries). Replication status needs `REPLICATION CLIENT` (MySQL) or, on MariaDB 10.5.9 and newer, `REPLICA MONITOR`: `REPLICATION CLIENT` **alone is not enough there.** The password reaches the client only through its environment, never its command line.

**All settings** of this check, with their defaults: check_mariadb.

### containers: podman containers, root's and every user's

Finds the podman containers on the host by itself - you list nothing - and checks each one. Hosts without podman are skipped (`skipped: podman is not installed`, no finding). It is read-only: it runs `podman ps`, one `podman container inspect` and `systemctl show`, and nothing that starts, stops or creates anything.

**Which containers.** Root's (rootful podman) and each user's (rootless podman: only that user can see them). A user is looked at when a podman process runs as them, when they have *linger* on (a file in `/var/lib/systemd/linger/`), or when they are in `/etc/subuid` and have container storage in their home (`~/.local/share/containers/storage`). A user's containers are read **as that user** (`runuser -u USER -- env XDG_RUNTIME_DIR=/run/user/UID podman ...`), and only when `/run/user/UID` exists: otherwise podman would create it, and a check must not change anything. Such a user is listed as `not checked` instead (and reported, below).

**Which should be running.** A container should be running when a systemd unit runs it and starts it at boot (a Quadlet `.container` file with an `[Install]` section, or an enabled unit from `podman generate systemd` or written by hand), or when its restart policy is `always` or `unless-stopped`. A container that has neither and stopped with exit code 0 was stopped on purpose: no finding. A Quadlet or `--new` unit removes its container while the unit is stopped, so the check also reads the Quadlet files and unit files: a container that should run but does not exist at all is found too (state `missing`).

**Pods and compose projects too.** Some units start several containers whose names the unit does not say: a Quadlet `.kube` file or a `podman-kube@` unit (`podman kube play` of Kubernetes YAML), a unit that runs `podman pod start`, `podman-compose` / `podman compose` / `docker-compose`, or `podman run` without `--name`. Each such unit is a row of its own (the name of the unit, kind `pod`, `compose project` ...), and it is down when its unit is not active. A `Type=oneshot` unit that finished successfully since boot (`podman-compose up -d`) counts as up: its containers run on without it. Root's units are read from `podman_discover_unit_dirs` (`/etc/systemd/system`, `/usr/local/lib/systemd/system`), a user's from `~/.config/systemd/user`. Podman's own tools (`podman-auto-update`, `podman-restart`, the API service) are left out. The containers of a running pod are listed as containers when they carry the unit's label (Quadlet sets it).

The output has a table of what was found:

```
OWNER        NAME                      STATE             UNIT                          SHOULD
  RUN
root         systemd-db                running           db.service (active)           yes
root         web                       running           -                             yes
alice        web                       missing           web.service (failed)          yes
bob          web                       exited (3)        -                             no
not checked: carol - no /run/user/1003 (the user is not logged in and linger is off): ...
```

Every finding names the owner, and its `look:` command runs as that owner (paste it as root):

| Finding | Means | Usually |
|---|---|---|
| `container X (owner O) should be running (...) but it does not exist / it exited with code N` (critical) | a container that should run is down, or its systemd unit failed | read the `look:` output (the unit's status, the container's log, the journal). Start it the right way: `systemctl start UNIT` for a unit (as a user: `runuser -u O -- env XDG_RUNTIME_DIR=/run/user/UID systemctl --user start UNIT`), `podman start X` for a plain container. Service watch does this for you, with approval |
| `required container X ... does not exist` / `is not running` (critical) | a container in `check_containers_required` is missing or stopped | create or start it; fix its name in the setting |
| `container X (owner O) is running but its healthcheck reports unhealthy` (critical) | its own healthcheck fails | `podman inspect X` shows the last healthcheck output (`State.Health.Log`); the container's log |

| Finding | Means | Usually |
|---|---|---|
| `container X (owner O) stopped with an error: it exited with code N, 2.5 hours ago` (warning) | a container that is not meant to run all the time stopped with an error in the last `check_containers_recent_hours` (24) hours | its log. Older crashes are not reported |
| `container X (owner O) has restarted N times - a restart loop` (warning) | more restarts than `check_containers_restart_warn` (3), counted by podman or by its systemd unit | its log shows why it keeps stopping |
| `container X (owner O) was killed by the out-of-memory killer` (warning) | the kernel killed it for memory | more memory on the host, or a higher memory limit on the container |
| `user U (uid N) has containers that should run (...) but linger is off` (warning) | the user's containers stop when they log out and do not start at boot | `loginctl enable-linger U` (as root) |
| `user U (uid N) has podman containers ... and N unit(s) that should start at boot, but they could not be checked: no /run/user/N` (warning) | the user has units that should start at boot (a Quadlet with `[Install]`, an enabled unit) but is not logged in and has no linger: none of their containers run now. A user with only container storage (an admin who once ran podman) is listed under `not checked:` and gets no finding | if they should run: `loginctl enable-linger U`. If not, leave them out: `podman_discover_ignore: ['U/.*']` |
| `the containers of O could not be read: podman ps failed ...` (warning) | podman (or systemctl) failed for that owner: a blind spot | run the `look:` command to see the error (storage problems, `podman system migrate` needed after an upgrade ...) |

**Settings** (roles `podman_discover` and `check_containers`; VARIABLES_REFERENCE.md):

| Setting | Default | What it does |
|---|---|---|
| `podman_discover_enabled` | `true` | `false` = do not look for containers (the check says `skipped`) |
| `podman_discover_rootless` | `true` | also look at users' containers |
| `podman_discover_users` | `[]` | more users to look at, e.g. `[appsvc]` when the rules above miss one |
| `podman_discover_unit_dirs` | `[/etc/systemd/system, /usr/local/lib/systemd/system]` | where root's own units that run podman are looked for (a user's: `~/.config/systemd/user`) |

| Setting | Default | What it does |
|---|---|---|
| `podman_discover_ignore` | `[]` | containers to leave out: regular expressions that must match the **whole** name or `OWNER/NAME`, e.g. `['test-.*', 'alice/.*', 'root/scratch']` |
| `check_containers_required` | `[]` | containers that must exist and run: a name (any owner) or `OWNER/NAME`, e.g. `[stigman-api, alice/web]` |
| `check_containers_recent_hours` | `24` | how recent a crash must be to be reported |
| `check_containers_restart_warn` | `3` | restarts above this = a restart loop |

**To verify it** on one host: run the Health check with `health_checks: containers` and a Limit of that host; compare the table with `sudo podman ps -a` and, for a user, `sudo runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/$(id -u alice) podman ps -a`. Service watch uses the same way of finding containers (SERVICE_WATCH_DEMO.md).

**All settings** of this check, with their defaults: check_containers, podman_discover.

### containers, database, service watch: which one does what

Three jobs look at podman containers. They find the containers the same way (`roles/podman_discover`), but each answers a different question:

| | containers (Health check) | mariadb (Health check, or *Database health* alone) | Service watch |
|---|---|---|---|
| Asks | is every container alive and stable? | is the database *inside* working, and fast? | something is down: why, and what fixes it |
| Looks at | every container, pod and compose project (`podman ps -a`, Quadlet files, systemd units): down, unhealthy, crashed, restart loops, OOM kills, linger | connections, long queries, replication, Galera / Group Replication, its log, its disk, and performance (slow queries, lock waits, open transactions ...) | the containers that should run: down or unhealthy |
| Then | a finding (a ticket) | a finding (a ticket) | ACT's root cause and a recommended fix, which waits for approval (`watch_mode: approval`, the default) |
| Run it | daily, every host | daily with the Health check; hourly alone for the database team | from its AAP workflow, when you want a diagnosis and a fix to approve (on demand, or on a schedule for the hosts that matter) |

**One outage, one finding.** When a database container is down and both checks run in the same Health check, the outage is reported once: as `containers:down` (or `:unhealthy`), whose summary then says *it is a*

*MariaDB/MySQL database container, so the database is down too (mariadb:down is not reported separately)*. The containers check has the better finding for that: how the container is run and how to start it. The database check still reports `mariadb:down` when the containers check does not (it is not running in the same job, or it does not report that container - one that has no unit and no restart policy, so nothing says it should run), and the *Database health* template always reports it.

The same goes for the **services** check: root's systemd unit that runs a container (or a pod) that is down is reported by the containers check, and the services check's `unit-failed:UNIT` (or `unit-down` / `unit-flapping`) for that unit is left out and named in that finding (*the services check sees its systemd unit web.service fail too (unit-failed:web.service is not reported separately)*). A failed unit that runs no container stays the services check's.

Service watch and the Health check can both see the same container down: the watch acts on it (within minutes), the Health check records it (once a day). On hosts where Service watch runs, a daily `containers:down` for a watched container usually means the watch could not bring it back.

## Troubleshoot (playbooks/troubleshoot.yml)

For "something is wrong on this host": pick the area and the job runs the commands an experienced admin runs first. Each is printed under a line saying what it shows, and the matching health checks run too. Read-only. Findings do not fail this job: you asked for a look, not a verdict.

| Area | Commands (what they show) | Checks |
|---|---|---|
| overview | uptime, failed units, disk, memory, recent errors, recent reboots | disk, mounts, services, performance, time, logging |
| disk | space and inodes, biggest directories on the fullest filesystem, deleted-but-open files, files over 1 GiB, journal size | disk, mounts |
| performance | load, top, vmstat, memory users, disk latency (iostat), load history (sar), OOM kills | performance |
| service (`ts_service`) | its status, why it stopped (result, exit code, restarts), its log, dependencies, unit file, listening ports | services (plus that unit) |
| network (`ts_target` host:port) | addresses, routes, DNS servers, name lookup, TCP connect, route taken, firewall, connection counts | network (plus that target) |
| selinux | mode, today's denials, audit2why, changed booleans, local file labels | selinux |
| fapolicyd | status, today's denials, rules, enforcing? | fapolicyd |
| login | sshd log, faillock lockouts, failed logins, SSSD domain status, authselect, clock offset | accounts |
| time | chrony tracking and sources, timedatectl, configured servers | time |
| logs | errors, top error senders, rsyslog and its queue, journal size | logging |

| Area | Commands (what they show) | Checks |
|---|---|---|
| mariadb (or mysql) | (the MariaDB / MySQL health check) | mariadb |

Add `use_act=true` and ACT reads everything collected and explains the root cause (ADDING_ACT.md).

**All settings** of this job, with their defaults: troubleshoot.

## Certificate report (playbooks/cert_report.yml)

The certs check on every host, and **one table**, soonest expiry first: days left, date, host, subject, and where the certificate is. Hand it to whoever renews certificates. It fails the hosts with a certificate inside `check_certs_warn_days`, so a workflow can open tickets (`site_fail_on: []` keeps it green: a report only).

**Emailed** with `report_email_to` (Linux and Windows: `win_cert_report.yml`). In the email the table is coloured: **red** = expired, **amber** = expires within `check_certs_warn_days` (30), **blue** = within `check_certs_notice_days` (60), the rest plain with a green "ok". Windows: `win_check_certs_warn_days` / `win_check_certs_notice_days`.

**Found by itself** (the report only; `cert_report_discover: false` turns it off):

- **TLS ports:** every listening TCP port is asked once for its certificate, on the address it listens on, with a 3-second limit (`check_certs_probe_timeout`), at most 25 ports (`check_certs_discover_ports_max`). Ports that are plain text are never touched (`check_certs_discover_ports_skip`: 22, 25, 53, 80, 389, 3306, 5432 ...), nor the ones you listed in `check_certs_endpoints`. Shown as `TLS port 8443, java (https://127.0.0.1:8443)`.
- **Java keystores:** `*.jks`, `*.keystore`, `*.p12`, `*.pfx` in `check_certs_discover_keystore_dirs` (`/etc`, `/opt`, `/srv`, Tomcat's folders), and the ones running Java programs use (`-Djavax.net.ssl.keyStore=...`). A JKS is read without its password; a PKCS12 that needs one is listed under **Found but not checked** - no password is ever guessed. To check it, add it to `check_certs_keystores` with `password_env` and a *Keystore password* credential. The JDK's own trust stores (`cacerts`) are skipped.
- **The trust store:** every file in `check_certs_trust_dirs` (`/etc/pki/ca-trust/source/anchors`, where `update-ca-trust` takes the CA certificates the host trusts, such as the DoD roots), whatever its name, text or binary. Shown as `/etc/pki/ca-trust/source/anchors/DoD_Root_CA_3.crt (trust store)`, with its date and colour, but **never a finding, a ticket or a CSR**. A certificate there is not this host's own: its CA renews it. When it does, put the CA's new certificate in that folder and run `update-ca-trust`. The system bundle `update-ca-trust` builds from it (`/etc/pki/tls/certs/ca-bundle.crt`) is never read: it holds hundreds of public roots.
- **Bundles:** a found file with more than 10 certificates (`check_certs_discover_max_per_file`) is listed under **Found but not checked**, with how many. To check each certificate in it, add it to `check_certs_files`.

**Windows** (`win_cert_report.yml`): every machine store is read, not only Personal, Web Hosting and Remote Desktop: Trusted Root, Intermediate Certification Authorities, Other People, ... (where *Automatically select the certificate store* puts a `.cer` file). A certificate there with its private key is checked like the others; one without is listed as `... (trust store)` when it expires within `win_check_certs_notice_days`, never a finding or a ticket (`win_check_certs_trust`, details in WINDOWS.md). Remote Desktop's certificates are left out of the report and the renewal (Windows renews them): `win_check_certs_remote_desktop: true` puts them in.

The weekly health check (`certs` in `health_checks`) does not search by itself, so it opens no tickets for certificates nobody listed; set `check_certs_discover_ports` / `_keystores: true` on it when you want that.

**Placeholders:** postfix makes its own certificate when it is installed, `/etc/pki/tls/certs/postfix.pem` (subject `O=SomeOrganization, OU=SomeOrganizationalUnit, C=--`), self-signed for one year and never renewed. On every host with postfix it expires a year after the install. A found one is listed under **Found but not checked** (one row for all the hosts) and is never a finding, a ticket or a CSR, in the report and in the health check (`check_certs_placeholder_subject`). A real certificate put in its place (a host that relays mail) has a real subject, so it is checked and renewed. A ticket already open for a placeholder gets the usual "passes again" note on the next run (resolved with `servicenow_resolve_cleared`).

**Left out:** hosts whose certificates something else renews, such as Satellite and its Capsules (`satellite-installer`, never by hand). They are listed under **Found but not checked** and get no ticket. Name them in either setting, or both; each takes a list or one string with commas:

| Setting | Leaves out | Example |
|---|---|---|
| `cert_skip_hosts` | hosts by name, as in the AAP inventory: any case, `*` and `?` wildcards, a short name matches the full one | `[sat01, '*capsule*']` or `sat01, *capsule*` |
| `cert_skip_groups` | every host in these AAP inventory groups (default `[sat]`) | `[sat, appliances]` |

Set them in the template's Variables, or in `all.yml` so the report and the renewal both get them. The health check still checks these hosts (with `[sat]`: the Satellite web UI's certificate, `playbooks/group_vars/sat.yml`).

**Reported, but renewed by hand** - IdM, AAP, any server whose certificates are put in place by its own tool: name it in `cert_renew_skip_hosts` (or its group in `cert_renew_skip_groups`), not in `cert_skip_hosts`. It stays in the report, and the Certificate watch still opens its tickets, each one saying *Renewed by hand: this host is named in cert_renew_skip_hosts (idm01) - the certificate renewal job makes no CSR for it*. The renewal lists its certificates with that reason and makes no key, no CSR. The same forms: a list or one string with commas, any case, `*` and `?`, short names.

## Trust store report (playbooks/trust_report.yml)

The trust check on every Linux host, and **one table** of every trust store and program read - the system bundles, p11-kit, Java cacerts, Python certifi, Node, podman containers, an application's own files, Go programs - with what each holds, worst first: the proof of what was looked at, and the checklist of what to update. Read-only - with `check_trust_remove: true` it takes the listed certificates out (the system trust blocklisted, Java entries deleted, certifi and application bundles cut, every file backed up first; Check mode shows the plan) and reads the hosts again. Fails the hosts that (still) hold a compromised or dropped CA, so a workflow can open tickets. Emailed (`report_email_to`). Weekly, and on demand when a CA incident is announced (add its certificates to `check_trust_extra`). TRUST_STORES.md.

## Windows trust store report (playbooks/win_trust_report.yml)

The Windows servers' certificate stores (Trusted Root, Intermediate CAs, the roots Windows Update keeps, Trusted Publishers, the connecting account's), every Java `cacerts` and Python `certifi` bundle, against the same list as the Linux report; the same tables, worst first, emailed. A certificate in the Disallowed store is blocked. `check_trust_remove: true` puts the certificate in the Disallowed store (Windows' blocklist, it wins over every other store) and deletes it from the store it was in, deletes Java entries, cuts certifi bundles, backups first; Check mode shows what would go. The Windows health check's `weekly` set runs the check `trust` too. TRUST_STORES.md.

## VMware trust store report (playbooks/vmware_trust_report.yml)

vCenter's trusted root store and each ESXi host's trusted CA certificates, against the same list, from the controller with a *VMware vCenter* credential (no inventory; `vmware_vcenters` for several). One table, worst first, emailed as the other VMware reports; the job fails when vCenter or a host holds a listed certificate. `vmware_trust_esxi: false` reads vCenter only. `check_trust_remove: true` deletes the chain from vCenter's trusted roots and replaces each host's trusted CA list without it (Check mode shows what would go; the account needs *Certificate Management* > *Manage*). TRUST_STORES.md.

## Certificate renewal: the CSRs (playbooks/cert_renew_csr.yml)

For every certificate the certificate report flags (it expires in 30 days or less, `check_certs_warn_days`, or has expired), the job makes a new private key and a certificate signing request (CSR) on the host: a certificate in a file (text or binary), one served on a port, one in a keystore. It attaches the CSR to the certificate's ServiceNow incident. The renewal sends no email: the CSR goes on the ticket (ServiceNow's own notifications tell its assignment group). Making a CSR changes nothing in use: the old certificate works until it expires.

**No incidents, the PKI's catalog item instead:** with `cert_renew_servicenow: catalog` each CSR is ordered from your PKI's catalog item (one request per certificate, the CSR in its CSR question): CERT_RENEWAL_CATALOG.md. It applies to the Windows, VMware and network device renewals too. vCenter's and the ESXi hosts' certificates have a job of their own, which opens the incidents and makes the CSRs on vCenter and the hosts: `VMware - certificate renewal` - and a report of them alone, `VMware - certificate report` (VMWARE.md, "Certificates of vCenter and the ESXi hosts"). The three use the same window: the report flags, the CSR is made and the incident is opened at 30 days. Change it on the workflow (`check_certs_warn_days`) and all three follow. The PKI person submits the CSR and attaches the issued certificate to the same ticket. This job changes nothing a service uses. With the catalog item, the issued certificate is then installed from its request after an approval - *Certificate install*, Linux and Windows (CERT_INSTALL.md); with incidents, a person installs it.

**Per certificate:**

1. It reads the certificate: the subject and the names (subjectAltName: DNS, IP, email, URI) - from its file (a binary, DER, file too), from the port that serves it, or from its keystore entry (the entry with a key; a keystore password from `check_certs_keystores`).
2. It makes a new key on the host and a CSR with exactly the same subject and names, signed with `cert_renew_digest` (sha256).
   - RSA keys are at least `cert_renew_key_bits` (2048); a bigger old key keeps its size.
   - An EC certificate gets a new key on the same curve.

- It uses the `openssl` command, so it works on RHEL 8 (OpenSSL 1.1.1) and RHEL 9 (3.x), in FIPS mode.

3. It checks the CSR before keeping it: the CSR verifies, its key is the new key, and its subject and names are exactly the old certificate's. If not, it keeps nothing and says "make this CSR by hand".
4. It keeps the files in `cert_renew_dir` (`/etc/pki/tls/renew`, made 0700):
   - `<name>-<8 characters>.key` (0600);
   - the `.csr`;
   - a `.json` with where it came from, the old serial and expiry, and the AAP job.

The key never leaves the host and never appears in the job output.

5. It finds the certificate's open incident, or opens one. The incident is the one *Certificate watch* opened, found by the same correlation ID (workflow 7). It attaches the CSR (`<common name>-<8 characters>.csr`) and adds a work note: the certificate, where it is, who issued it, the names, where the new key is, and the next steps. Every CSR gets a ticket, whatever `servicenow_min_severity` says: critical (14 days or less, `check_certs_crit_days`, or expired) or warning (15 to 30 days).

**Run again:**

- The pending key and CSR are reused, so the CSR the PKI already has stays valid.
- A CSR already on its ticket is not attached again. A daily run adds nothing until the certificate changes.
- Once the new certificate is installed, the old one is no longer flagged.
- The folder keeps its files until you remove them.

**One certificate in several places** (its file, a chain file, served on a port) gets one CSR, and the work note lists every place. The watch opens a ticket for each place; the CSR goes on one of them, and each of the others gets a work note saying which ticket has the CSR.

**Where the issued certificate goes** - step 3 of the next steps on the ticket:

| The certificate is | Step 3 says |
|---|---|
| in a file | install it with the new key in place of the file |
| in a binary (DER) file | the same, converted first: `openssl x509 -in <issued> -outform DER -out <file>` |
| in a Java keystore or a PKCS12 file | the commands that put the new key and the issued certificate in the keystore, under the same alias, after a copy of the keystore (`cp -p <keystore> <keystore>.bak`, `openssl pkcs12 -export`, `keytool -importkeystore`) |
| served on a port, its file not found | find where that service keeps its certificate and key, and replace both |

**A certificate served on a port** is matched to its file when the file is in `check_certs_files` or in one of the folders the report searches (`check_certs_discover_dirs`, two levels deep: `/etc/pki/tls/certs`, `/etc/pki/nginx`, `/etc/nginx`, `/etc/httpd/conf.d`, `/etc/stigman`, and `/etc/stigman/nginx/tls` where *STIG Manager - deploy* puts nginx's certificate, plus `check_certs_discover_extra_dirs`). Its file not found, it gets its CSR from the port, and the ticket says to find the file. Add the file to `check_certs_files` and the next CSR names it. Certificates of services that renew their own (AAP's internal CA, for one) are left out by their issuer: `cert_renew_exclude_issuer`. To see what the job would do with a certificate that is not expiring yet, run it with **Check**, a **Limit** of that host and `check_certs_warn_days: 400`: it says "would make the CSR" or why not, and changes nothing.

**Step 3 of the next steps** says to install the issued certificate in place of the file. When something else deploys that file, a copy by hand is overwritten, so `cert_renew_install_notes` writes another step 3 for paths that start with a given folder. The default covers STIG Manager (`/etc/stigman/`): put the issued certificate and the new key in the AAP credential *TLS certificate*, then run *STIG Manager - deploy*. Add your own: `{'/opt/app/certs/': 'App: ...'}`.

**Never renewed here** (each listed in the job output with the reason):

| What | Why |
|---|---|
| a host in `cert_skip_hosts` or `cert_skip_groups` (`[sat]`) | Satellite renews its own certificates with `satellite-installer`; left out of the certificate report too |
| a host in `cert_renew_skip_hosts` or `cert_renew_skip_groups` | renewed by hand (IdM, AAP ...): still in the certificate report, and its tickets say no CSR comes from here |
| a file found only in `check_certs_discover_extra_dirs` (`/home`, `/root` ...), unless `cert_renew_extra_dirs: true` | those folders are searched so the report shows what is there (a test certificate, a copy someone left); a CSR for each would only add work. A file there that is also under `check_certs_discover_dirs`, or listed in `check_certs_files`, is renewed |
| a path matching `cert_renew_exclude` | something else renews it: Satellite and Capsule (katello, foreman, candlepin, pulp, puppet), and subscription-manager on every registered host (`/etc/rhsm`, `/etc/pki/consumer`, `entitlement`, `product`). Check the report's *Where* column and add your own paths |
| issued by a CA matching `cert_renew_exclude_issuer` (`O=Katello`, `Puppet CA`) | Satellite's or Puppet's own CA |
| a CA certificate in a chain file | a CA has no CSR from you: put the CA's new certificate in the chain file |
| a self-signed certificate (unless `cert_renew_self_signed: true`) | it does not come from a PKI. With the setting on, it gets a CSR, so the PKI issues a real certificate in its place (a STIG wants PKI certificates); `cert_renew_issuer` does not apply to it |
| a CA's own certificate (a root: key usage *Certificate Sign*) | a CA has no CSR from you, even with `cert_renew_self_signed` |
| a self-signed certificate only for `localhost` | a PKI issues no certificate for it, even with `cert_renew_self_signed` |
| a package's placeholder: postfix's `postfix.pem` (subject `O=SomeOrganization`, `check_certs_placeholder_subject`) | not a real certificate: a PKI issues nothing for that subject, even with `cert_renew_self_signed`. A found one is not even checked |

| What | Why |
|---|---|
| in the trust store (`check_certs_trust_dirs`, `/etc/pki/ca-trust/`) | a certificate the host trusts, not its own: its CA renews it. Put the CA's new certificate in the anchors folder and run `update-ca-trust`. Your server's own certificate belongs in `/etc/pki/tls/certs` (its key in `/etc/pki/tls/private`) |
| a keystore's trusted entry (`trustedCertEntry`), or a certificate in a PKCS12 without its key | one the program trusts, not its own |
| not matching `cert_renew_only` / `cert_renew_issuer` | your filters, e.g. DoD PKI only: `cert_renew_issuer: 'O=U\.S\. Government'` |

A certificate whose CSR cannot be made fails the job with the reason, for example a subject openssl cannot write back, or a kind of name it cannot copy. The other certificates are still done.

**Earlier than 30 days:** the CSR comes when the certificate report flags the certificate. For a longer PKI round trip, set `check_certs_warn_days: 60`, as a survey on the workflow, so the report, the tickets and the CSRs use the same number.

**Attachment refused:** some instances only allow some file extensions or types (`File extension not allowed: csr`, `Invalid file type: text/plain`). A refused CSR is attached again as `<name>.txt` (`cert_renew_attach_retry_txt`, on), and the work note names that file. Refused again, the CSR goes in the work note instead, and the job fails and says so: copy it from the note, and ask for `.csr` (or `.txt`) attachments to be allowed on the instance (`glide.attachment.extensions`). To name every CSR `.txt` from the start: `cert_renew_attach_ext: txt`. A run again finds the CSR under either name and attaches nothing more. The ServiceNow account needs to add attachments to incidents (the `itil` role does).

**Try it first:**

- Run with **Check** (`--check`): it lists what it would make and attach, and writes nothing on the hosts or in ServiceNow.
- Without the ServiceNow API credential, the job prints each CSR so you can copy it by hand.

**All settings** of this job, with their defaults: check_certs (`cert_skip_hosts`, `cert_skip_groups`, `cert_renew_*`).

## POA&M status (playbooks/poam_status.yml)

Reads `poam/poam.csv` (see poam/README.md for the columns and how to export from eMASS):

| Finding | Means |
|---|---|
| `POA&M 1001 [CAT II]: ... - OVERDUE by 12 days` | open item past its scheduled completion date (critical) |
| `POA&M 1002 ...: due in 20 days` | due within `poam_due_soon_days` (30) |
| `... scheduled completion date "TBD" is missing or not a date` | fix the date in the list |
| `open CAT I finding V-123456 (...) on 2 asset(s) in collection Production has no POA&M item` | STIG Manager cross-check: an open CAT I/II finding that no POA&M item covers |

It also lists "maybe ready to close": items whose STIG findings are no longer open in STIG Manager.

**The STIG Manager cross-check** is optional. It needs a Keycloak client that may use the client-credentials grant, with the scope `stig-manager:collection:read`, and a grant in the collections it reads. **Your realm owner creates that client**; nothing here changes the realm. Then create the *STIG Manager API* credential (token URL `https://<keycloak>/realms/<realm>/protocol/openid-connect/token`, client ID, secret), and set `poam_stigman_api: https://<stigman>/api`.

**All settings** of this job, with their defaults: poam.

## ServiceNow tickets (playbooks/servicenow_tickets.yml)

A workflow step after a check. For each finding at or above `servicenow_min_severity`:

- **no open ticket** for that problem on that host › it opens an incident: short description `[site-health] host: summary`; description with the host, check, finding, the `look:` command, and ACT's analysis if ACT ran; urgency/impact from the severity; assignment group, category and configuration item from the settings.
- **an open ticket exists** › it adds a work note `Still failing at ...` (`servicenow_note_repeats`).
- **a host that was targeted but never reported** › a ticket too (not reached, login failed). The check job stays green for a host it could not reach (`site_unreachable_fails`); this ticket is where it shows.
- **a problem that cleared** (its check ran again and passed) › a work note on its ticket, or it resolves it with `servicenow_resolve_cleared: true`. A finding still there, but below `servicenow_min_severity`, has not cleared: its ticket stays open (a certificate renewal ticket at warning, for one).

Tickets are matched by a correlation ID (`site-health:<host>:<check>:<finding>`, hashed), never by text, so the same problem always updates the same ticket. Without the ServiceNow API credential it only prints what it would open. If no check results reach it at all (the check step died before checking any host), it fails, so a broken schedule shows red instead of green.

**All settings** of this job, with their defaults: servicenow.

## ServiceNow - test ticket (playbooks/servicenow_test_ticket.yml)

Proves the ServiceNow setup end to end, with the same calls and settings as the tickets step. Six steps, each PASS, WARN or FAIL with what usually causes a failure:

| Step | What it does | FAIL usually means |
|---|---|---|
| 1 connection | logs in and reads one incident | certificate (CA), DNS, firewall or proxy; HTTP 401 = user or password; 403 = no `itil` role; 404 = a path after the host name in the Instance URL |
| 2 settings | `servicenow_assignment_group` and `servicenow_caller` exist | the name differs from ServiceNow's |
| 3 open | opens a low-priority incident: `[site-automation test] ... safe to close` | the account may not create incidents (`itil`), or ServiceNow refused a value (it says which) |
| 4 read back | reads it back: number, state, group, caller; prints a link | the account may not read it; WARN if the group did not stick |

| Step | What it does | FAIL usually means |
|---|---|---|
| 5 work note | adds a work note | the account may not update incidents |
| 6 resolve | resolves it (`servicenow_test_resolve: false` leaves it open) | WARN only: your instance needs another resolved state or close code (`servicenow_resolved_state`, `servicenow_close_code`) |

As *Check* (a dry run) steps 1 and 2 run, and 3 to 6 are only printed. Set up, run and verify it: SERVICENOW_SETUP.md.

## ServiceNow health (playbooks/servicenow_health.yml)

From the controller: does the instance answer its REST API, and how fast (`servicenow_slow_seconds`)? Is every MID Server **Up** and **validated**? Hourly is a good schedule. Alert with an AAP **notification** (email) on failure, not with a ticket: a broken ServiceNow cannot take tickets. The MID Server **hosts** (the Linux service, its keystore) are covered by the health check: see `inventories/example/group_vars/servicenow_mid_hosts.yml`.

**All settings** of this job, with their defaults: servicenow.

## Patch hosts (playbooks/patch_hosts.yml)

Installs updates with dnf (`patch_security_only` for security fixes only; `patch_exclude` for packages never updated here). Then:

1. **One host at a time** (`patch_serial`, in the template's Variables - in `all.yml` it does nothing). The rollout **stops at the first host that fails**.
2. Refuses with less than `patch_min_free_mb` free in `/var`.
3. **No automatic restarts** unless `automatic_restarts: true` (the survey or the template's Variables). Without it, a host that needs a reboot is listed ("reboot it by hand, or run again with automatic_restarts: true"). With it, the job reboots only when `needs-restarting -r` says so (`patch_reboot: when_needed`), and **never** hosts in `patch_never_reboot_groups` (`aap_hosts`). A host without `needs-restarting` (install `dnf-utils`), or where it fails, counts as needing a reboot after updates were installed - the job says so.
4. After the update (and reboot), **every service that was running before must be running again**. If one is not, the host fails and the rollout stops.
5. Never patches hosts in `no_patch` (vendor appliances), `aap`, `aap_hosts`, `sat`, `netapp` or `netapp_console_hosts`. The AAP host is **always** protected: even a list you write yourself (`patch_never_patch_groups`) cannot remove `aap` or `aap_hosts`. It also refuses a host that is the machine running the job, but only when the connection is `local` or the name is `localhost`; a normal SSH host that happens to be the controller is stopped by the group rule above, not by this one.

Run it as **Job type: Check** first: it lists what would be updated and changes nothing (DRY_RUNS.md).

**All settings** of this job, with their defaults: patch.

## Apply approved ACT fix (playbooks/act_fix_approved.yml)

The step after an approval in the *Fix with approval (ACT)* workflow. It runs **exactly** the commands ACT proposed and a person approved, then re-runs the checks that found the problem. It refuses to run on its own, and skips diagnose-only hosts (`aap_hosts`, `netapp_console_hosts`): there a person applies the fix by hand. See ADDING_ACT.md.

**All settings** of this job, with their defaults: site_act.

## VMware jobs and reports (playbooks/vm_*.yml)

Restart, shut down, snapshot, notes, change VLAN; the secure boot, datastore, snapshot and alarms reports; the snapshot cleanup; ACT's analysis of the alarms. They work through vCenter: setup, every report's sections and troubleshooting are in VMWARE.md.

**All settings** of this job, with their defaults: vmware_vm.

## Account lockout (playbooks/ad_lockout.yml)

On demand, for the service desk: give it the account (`jdoe`, `CORP\jdoe`, a UPN), or nothing for every account locked out now. Every DC reads its own bad password count and its Security log; the report ranks the computers the failed sign-ins came from and says what each one usually is. Setup, the audit policy it needs, the survey and the unlock are in ACTIVE_DIRECTORY.md.

| You see | Means | Usually |
|---|---|---|
| `kept trying after the account was locked out` | that computer still has the old password | it locks out again right after an unlock: fix it first |
| `tries about every 5 minutes` | a steady pace: a saved password retrying | a service, a scheduled task, a mapped drive or an application on that computer |
| `is a mail server` | the sign-ins came through Exchange (ActiveSync, Outlook) | a phone with the old password |
| `The DC did not record which computer` | a server signed in for the user | NPS / RADIUS (Wi-Fi, VPN), Exchange / OWA, a web application |
| `has a DISCONNECTED session` (with `ad_lockout_check_sources`) | a Remote Desktop session left open | sign it out (`logoff <id>` on that computer) |
| `No failed sign-in ... the DCs may not audit` | no 4740 / 4771 / 4776 events | the DCs' audit policy, or the Security log rolled over |

**All settings** of this job, with their defaults: ad_lockout.

## Horizon 8 (VDI) (playbooks/horizon_*.yml)

The API test, the health and capacity reports, the broken desktops check and its fix after an approval, user help for the service desk, the disconnected session cleanup, and the golden image push after an approval. They read the Connection Server's REST API; the account, the templates, the two workflows and every message are in HORIZON.md.

**All settings** of these jobs, with their defaults: horizon.

## VMware pick lists (playbooks/vm_pick_refresh.yml, vm_snapshot_delete_picked.yml, vm_reboot_picked.yml)

Choose from a list instead of typing VM names: a scheduled refresh writes what may be done right now

- the snapshots the cleanup could delete, the VMs that may be restarted, or only the hung ones (no heartbeat, or VMware Tools not running) - as the choices of a multiple-select survey question. The pick jobs find every pick again in vCenter, live, skip what is gone or no longer allowed (with the reason), and in their workflow plan first, wait for an approval, then do exactly the plan. Never the AAP server or a protected VM; never more than `vm_pick_max` (20) at once. Setup and limits: VMWARE.md, "Pick lists".

**All settings** of these jobs (`vm_pick_*`): vmware_vm.

## Windows hosts

Windows has its own playbooks, checks and troubleshooting areas (for example `updates` and `security` in *Windows troubleshoot*). They are described, with their settings, in WINDOWS.md. Settings for them are read the same way as any other: VARIABLES.md.

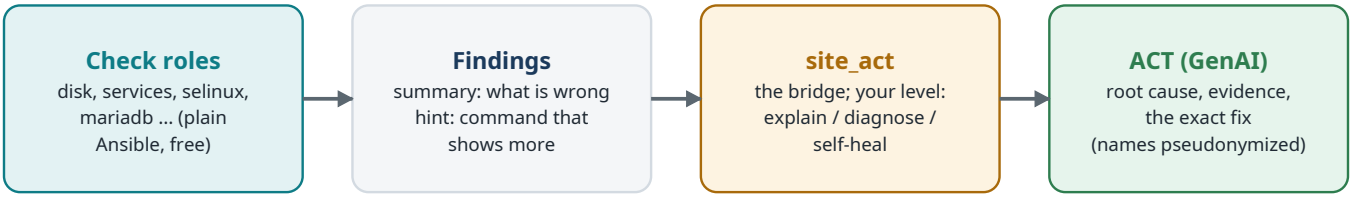
## Not built yet (and why)

| Idea | Why not yet |
|---|---|
| SCAP scan › import into STIG Manager | Needs the same Keycloak client as the POA&M cross-check, plus a decision on SCC vs OpenSCAP content. STIG Manager's own importer (stigman-watcher) is the supported path |
| AAP self-health (job failure rate, execution environment and token age) | Needs an AAP API credential; AAP's own dashboards cover most of it today |
| Container runbooks (image updates with rollback, drift) | Deferred on purpose; *Service watch* covers container outages |
| MariaDB backups, security baseline, password rotation | Out of scope for now: health only, as asked |

# 9. Adding ACT (GenAI) later

The runbooks work without ACT. Add it when the checks are running well and you want the next step done for you: **why** it broke and **what** fixes it. This page shows how the runbooks and ACT fit together, how to turn it on per template, and how to write your own check so it works with ACT automatically.

## How they fit: checks find WHAT, ACT finds WHY




Report, ServiceNow tickets and the approval workflow read the same findings - with or without ACT.

*Checks find WHAT is wrong; ACT finds WHY. A new check works with ACT automatically through its findings' hints.*

- The **checks** decide *whether* something is wrong. They are plain Ansible: the same answer every time, and they are free. The model is only called for hosts with findings.
- Every finding carries a **hint**: the read-only command a person would run next. ACT gets those commands' output as evidence. A new check you write is therefore usable by ACT with no extra work.
- `roles/site_act` is the only place that talks to ACT. Every runbook uses it the same way. Turning ACT on is a variable, not a code change.
- Host names, IP addresses and user names are **pseudonymized** before anything reaches the model (ACT 0.6.18 and newer). Its replies are translated back on the host.

## The three levels

| Level | ACT does | Changes to hosts | Approval step? |
|---|---|---|---|
| `explain` | reads the evidence (the hint commands' output, collected by Ansible) and writes the root cause and a suggested fix. **It runs no commands itself** | none | no |
| `diagnose` | investigates by running **read-only** commands itself, then proposes exact fix commands. Every change is refused and waits for a person | only after approval | yes: the job fails with `NEEDS APPROVAL` › approval step › *Apply approved ACT fix* |
| `self-heal` | like `diagnose`, but may **run fixes that match** `site_act_allow` by itself (for example `systemctl restart chronyd`). Anything else still waits for approval. The checks then run again to prove the fix worked | only fixes you listed | only for the other fixes |

Start with `explain`. Move a runbook to `diagnose` when ACT's explanations have been right for a while. Use `self-heal` only for fixes you would let a junior admin do unasked: start a stopped service, restart a hung one.

**Never fixed automatically, at any level:** hosts in `site_act_diagnose_only_groups` (default: `aap`, `aap_hosts`, `sat`, `idm`, `netapp`, `netapp_console_hosts`; `aap` and `aap_hosts` are always kept, whatever list you write). There `self-heal` becomes `diagnose`, and the apply step skips them: it prints the approved command for a person to run by hand.

## Turn it on

### 1. The ACT model key credential

Create the credential type `ACT model key` (`aap/credential_types/act_model_key.yml`) and one credential with your key: the GenAI key for GenAI.mil, or the AskSage key for Ask Sage (SETUP_AAP.md, steps 2-3). For the GenAI beta proxy, also create `ACT GenAI beta key` (`act_genai_beta_key.yml`). Attach the credential(s) to the **Health check** and **Troubleshoot** templates (and *Service watch - check*), next to `Linux ssh (sudo)`. This is the only place a key lives. The apply steps do not call the model: they run the approved commands themselves, so they need no key.

### 2. Pick the model provider

ACT can talk to three providers. Pick one with `site_act_provider`. **GenAI.mil (`genai`) is the default and the one to use: Ask Sage is retired for now.** Its settings stay below, unused, in case it comes back.

| site_act_provider | Provider | Key (credential) | Also set |
|---|---|---|---|
| `genai` (default) | GenAI.mil | *ACT model key*, **GenAI API key** field | nothing (optional: `site_act_model`) |
| `asksage` | Ask Sage (**retired for now**: do not pick it) | *ACT model key*, **AskSage API key** field | `site_act_url`: your organization's Ask Sage API URL. The default is `https://api.genai.army.mil/server/openai/v1/chat/completions` |
| `genai-beta` | GenAI.mil beta proxy (preview models) | *ACT GenAI beta key* (`aap/credential_types/act_genai_beta_key.yml`) | nothing |

The settings:

| Setting | What | Default |
|---|---|---|
| `site_act_provider` | `genai`, `asksage` or `genai-beta` | `genai` |
| `site_act_models` | the model **for each provider**, so the model follows the provider you pick, e.g. `{genai: gemini-3.8-flash, asksage: gpt-5.6-sol-gov}` | ACT's default for the provider |
| `site_act_urls` | the API URL for each provider, e.g. `{asksage: https://<host>/server/openai/v1/chat/completions}` | ACT's default for the provider |
| `site_act_model`, `site_act_url` | one model / URL for this run, whatever the provider (a survey answer) | empty: use the two above |

| Setting | What | Default |
|---|---|---|
| site_act_ca | a CA bundle file **on the hosts**, if the provider's certificate comes from a CA the hosts do not trust yet (for example a DoD CA not in /etc/pki/ca-trust) | the host's normal trust store |

**Where to set them:**

- **For everyone**: playbooks/group_vars/all.yml in this project (the file has these lines ready as comments). A setting there beats the *inventory's* Variables box in AAP, so put it in one of the two places, not both. Which place wins in every case: VARIABLES.md.

```yaml
site_act_provider: genai                # the default provider
site_act_models:                        # the model follows the provider you pick
  genai: gemini-3.8-flash
  asksage: gpt-5.6-sol-gov
site_act_urls:                          # only if your Ask Sage is not ACT's default URL
  asksage: https://<your ask sage host>/server/openai/v1/chat/completions
```

(Ask Sage is retired for now: keep site_act_provider: genai.)

- **For one group**: the group's settings file (playbooks/group_vars/<group>.yml), or the group's **Variables** box in AAP. For example site_act_provider: asksage there, and only that group's hosts use Ask Sage (with its model from site_act_models).
- **For one template**: its **Variables** box (*Extra variables*), for example site_act_provider: asksage. It beats everything above, for every run of that template.
- **Per launch**: the survey questions in step 3 (a survey answer beats everything too).

The API **key** is never a variable: it comes from the *ACT model key* credential on the template.

These settings apply to every runbook that uses ACT, including *Service watch* and *Apply approved ACT fix*. If the key for the chosen provider is missing, ACT does not run: the job says ACT did not run: this job has no API key for provider asksage. Attach ..., and the findings are still reported.

**Network:** ACT runs **on each host**, not in AAP. So each host must be able to reach the provider's URL over HTTPS. If they go through a proxy, add it: site_act_env: {HTTPS_PROXY: "http://proxy.yoursite.mil:8080"}.

**Models served only on** /v1/messages (ACT 0.6.19 and newer). Some gateways offer a model only through the Anthropic endpoint and answer HTTP 400 for it on .../v1/chat/completions. ACT tries the other endpoint by itself, with nothing to set. To test your models and set the switches on purpose, follow Both endpoint formats and :probe below.

**How long ACT may take.** site_act_model_timeout (default **600** seconds) is the time the model may take for one answer, retries after a provider error included (ACT's GENAI_TIMEOUT; ACT's own default, 90, is too short for a long analysis or a busy provider). site_act_timeout (default **1200**) is the whole ACT run, several answers. They apply to every ACT job: the runbooks with use_act, the VMware alarms and capacity analyses, *Service watch*. A model turn exceeded the 600s total timeout after a transient provider failure means the provider was failing and retrying for that long: run it again; if it repeats, raise both in all.yml (site_act_model_timeout: 900, site_act_timeout: 1800).

**Streaming.** Every ACT job has the model stream its answer (ACT_STREAM=1): data keeps flowing while it writes, so a firewall or proxy that cuts quiet connections (network error reaching API: connection reset

by peer, often on the longer answers only) no longer does. Where a gateway cannot stream, ACT notices and sends normal requests by itself. To turn it off: site_act_env: {ACT_STREAM: "0"}.

**"ACT gave no analysis: the model answered as a chat".** Now and then a model loses the thread and answers like a chat window ("I am ready. What task or command would you like me to assist you with?") instead of analysing the findings; whatever it tried to run then is no fix. The report says so (ACT status no_analysis) and nothing is passed on for approval. Run the job again; if it repeats with one model, switch that template to another (site_act_models).

Anything else ACT should know about your site goes in the same places (for example playbooks/group_vars/all.yml):

```yaml
site_act_extra_instructions: >-
  RHEL 9 servers under DISA STIG. fapolicyd and SELinux are enforcing. Never suggest disabling
    them.
```

**All settings**, with their defaults: site_act.

### 3. Survey questions

On **Health check** and **Troubleshoot** › **Survey** › **Create survey question**:

| Question | Variable | Type | Choices | Default |
|---|---|---|---|---|
| Ask ACT (GenAI) about the findings? | use_act | Multiple Choice (single select) | no, yes | no |
| What may ACT do? | site_act_level | Multiple Choice (single select) | explain, diagnose, self-heal | explain |
| Which model provider? (optional) | site_act_provider | Multiple Choice (single select) | genai, genai-beta (leave asksage out while Ask Sage is retired) | the one in your settings |
| Which model? (optional, blank = the provider's model from site_act_models) | site_act_model | Text, not required | | blank |

Leave out the last two if everyone uses the same provider. Your setting (section 2) then applies.

### 4. Try explain by hand

Launch **Troubleshoot** on a host with a known problem: ts_area = service, ts_service = the unit, use_act = yes, site_act_level = explain. In the output, read ACT | what ACT says. With ACT on, the host's ServiceNow ticket includes this analysis too.

### 5. diagnose with the approval workflow

Build *Fix with approval (ACT)* (WORKFLOWS_AND_SCHEDULES.md). What happens:

1. The health check finds, for example, required service chronyd.service is inactive.
2. ACT (diagnose) looks at the unit, its log and the config, and proposes systemctl start chronyd. The command is refused, so the job fails: NEEDS APPROVAL - ... ACT proposes: systemctl start chronyd.
3. The workflow stops at the approval step. The approver reads ACT's report and approves.

4. **Apply approved ACT fix** runs **exactly** `systemctl start chronyd` itself: no model and no key, so nothing can be reworded between the approval and the fix. A last check refuses a few commands even after approval (reboot, `mkfs`, `usermod` ...; see APPROVED_COMMANDS.md).
5. The same checks run again, without ACT, and the job is green only if the problem is gone. The tickets step notes the ticket as cleared.

### 6. self-heal for the fixes you trust

playbooks/group_vars/all.yml (or the settings file of one group)

```
site_act_allow:
  - 'systemctl (start|restart) (chronyd|rsyslog|crond)'
```

Each entry is a regular expression that must match the **whole** command. Keep them narrow: name the services, never `.*`. Then pick `site_act_level` = `self-heal`. After ACT applies an allowed fix, the checks with findings run again. The report shows `cleared: ...` or `still there: ...` and reflects the state **after** the fix.

## Both endpoint formats and :probe (ACT 0.6.19 and newer)

**What this is for.** A model gateway can offer a model on two web addresses: the *OpenAI* one (the provider URL, ending in `/v1/chat/completions`) and the *Anthropic* one (ending in `/v1/messages`). Some models answer on only one of them. If ACT prints `HTTP 400 ... bad request` for a model that the gateway lists, this is the usual cause. ACT can now speak both, and `:probe` tests which one each model needs.

**Two facts that shape the steps:**

- `:probe` is a command you type at the ACT prompt. **It cannot run inside an AAP job** (jobs are not interactive). So you run it **once, by hand, on one lab host**, and then put the result into settings the jobs read.
- In a job, ACT already works out the format by itself (one refused request, then it switches), so the jobs may need no change at all. The settings below only save that one wasted request per run, or point at an Anthropic address that is not the default.

**Not verified against a real gateway.** These steps were written from ACT's own documentation and tests. Nobody has run them against your gateway yet; treat the first run as the test.

### Step 1: check which ACT the project carries

`vendor/act/VERSION` must say `0.6.19` or newer. This release carries **0.6.25** (0.6.25: a model answer cut off on its way back is asked again instead of ending ACT; 0.6.24 changes only ACT's own example playbook `podman_app_health.yml`: see CHANGELOG.md). It sends the key only to an `https://` provider URL (an `http://` URL is refused unless `site_act_env: {ACT_ALLOW_HTTP: "1"}`), and more dangerous commands always need a person (see APPROVED_COMMANDS.md). New in 0.6.22 and 0.6.23, all automatic:

- **A bigger output limit for thinking models (0.6.23).** Gemini 2.5 and later, GPT-5 and the o-series think before they answer, and the thinking counts against the output limit. At ACT's old limit (4096 tokens) the thinking could use it all and the answer came back **empty** - on GenAI.mil every Gemini model did that in `:probe`. These models now get 16384 (others keep 4096). It is a ceiling, not a charge: you pay for the tokens used. To force one value: `site_act_env: {ACT_MAX_TOKENS: "8000"}`.
- **Temperature per model.** Gemini 3 models (and GPT-5 / o-series reasoning models) get **no** temperature, so they use their own default (1.0): Google's Gemini 3 developer guide says to keep it at 1.0 and that lower values can make the model loop. Other models keep 0.2. To force a value for every model: `site_act_env: {ACT_TEMPERATURE: "0.2"}`.

- **Fewer empty answers.** A reply cut off at the output limit (the model's thinking used it all) is asked again once with a higher limit; an empty reply is asked again once with a strict JSON schema of ACT's actions; a reply the gateway's content filter blocked is reported as that.
- **Structured output.** When a model refuses ACT's function-calling format, ACT asks for a strict JSON schema instead of "any JSON object", so the gateway itself guarantees a valid answer.
- **HTTP 429 (too many requests):** ACT waits as long as the gateway asks (`Retry-After`). A spent credit quota still stops the run (it would not help to wait).
- **Clearer errors.** A model alias the gateway retired (GenAI.mil retires aliases 60 days after deprecation) is reported as that, not as a missing endpoint or a permission problem; a locked or wrong key says to enter a new one.
- **Interactive only:** replies stream in (a counter shows progress) and **Esc** cancels a reply that is still coming. Jobs do not stream (nothing to watch, and some proxies break streaming).
- **Tool-result turns** (command results sent back the way function-calling models expect): only for models `:probe` confirmed - see step 5. If it is older, update it first: README section

"Updating ACT" (`scripts/update-act.sh`).

### Step 2: put ACT on one lab Linux host

Any host that can reach the gateway will do; it does not have to be one of your managed hosts. Copy the file `vendor/act/act` there (WinSCP, or `scp` in PowerShell), then on the host:

```shell
chmod 700 act
```

### Step 3: give it your key without writing the key anywhere

Type this on the host. It asks for the key without showing it, and keeps it only in that terminal session (use `GENAI_BETA_KEY` or `ASKSAGE_KEY` for the other providers):

```shell
read -rs GENAI_KEY; export GENAI_KEY
```

Paste the key, press Enter, and do not `echo` it. If the provider URL is not ACT's default, also `export GENAI_URL='https://<your gateway>/v1/chat/completions'`.

### Step 4: run :probe

```shell
./act
```

At the ACT prompt:

```
:probe all
```

`:probe all` tries every model the key lists, on both endpoints, with a small request and then ACT's full request. For each it shows what worked and, for a refusal, **the gateway's own reason**. `:probe <model name>` tests just one; `:probe model1 model2` (spaces or commas) tests several. Write down, for each model, which endpoint worked (`openai` or `anthropic`). Leave ACT with an empty line (or Ctrl-D). From 0.6.23 `full OK` means the model gave a usable answer; `full empty (...)` or `full no action (...)` says what came back instead (for example `finish_reason=length`: the model ran out of output room), and a test that needed

more room says `OK (needed a higher output limit: N)` - ACT remembers that limit for the model.

From 0.6.22 each model also gets these lines (under its `basic` / `full` lines):

| Line | Meaning | What to do |
|---|---|---|
| `stream OK` / `stream not supported (...)` | whether replies can stream in (interactive use only) | nothing |
| `structured output OK (strict)` / `OK (non-strict)` / `not supported (...)` | whether the gateway enforces ACT's JSON schema for this model | nothing: ACT uses the best one that works |
| `tool results OK` / `tool results not supported (...)` | whether this model accepts command results as tool turns | **write it down**: step 5 |
| `temperature: model default` / `temperature: 0.2` | what ACT sends | nothing (to force a value: `ACT_TEMPERATURE`) |
| `output limit 16384 (thinking model)` / `output limit 4096` | the most the model may write per answer, thinking included (0.6.23) | nothing |
| `full empty (finish_reason=length ...)` | even the higher limit was not enough for that test | tell the maintainer; or `ACT_MAX_TOKENS` higher |

The result is remembered for the session. If a setup file already exists for your user on that host (`~/.config/act/config.json`), ACT also saves the format there (only the format table, never the key).

**Windows.** ACT for Windows (`vendor/act-windows/act.ps1`) has the same `:probe` command. Run it the same way on a lab Windows host, in PowerShell, and enter the key with `:setup`. Its result goes into the Windows jobs' settings as above. (Not run by hand for this page.)

### Step 5: tell the jobs

A job does not read the lab host's file (it runs as another account, on another host), so put the result in settings, in the same places as the other ACT settings (section 2):

| Your probe result | Setting in playbooks/group_vars/all.yml (or a group file) |
|---|---|
| All the models you use work on the OpenAI endpoint | nothing to set |
| Some models work only on the Anthropic endpoint | nothing to set (ACT learns it during the run); optional, to skip the one refused request per run when **every** model you use is Anthropic-only: `site_act_env: {ACT_API_FORMAT: anthropic}` |
| The Anthropic endpoint is at a different address than the OpenAI one with `/messages` in place of `/chat/completions` | `site_act_env: {GENAI_ANTHROPIC_URL: "https://<host>/.../v1/messages"}` (`ASKSAGE_ANTHROPIC_URL` or `GENAI_BETA_ANTHROPIC_URL` for the other providers) |
| A model you want is listed for the key but works on neither | it is not available to your key; ask the gateway's owner. Choose another model in `site_act_models` |
| `tool results OK` for the model your jobs use (0.6.22) | optional: `site_act_env: {ACT_TOOL_RESULTS: tool}`. Jobs then send command results as tool turns, which function-calling models handle best on long runs. If a model refuses them anyway, ACT switches that model back by itself |

| Your probe result | Setting in playbooks/group_vars/all.yml (or a group file) |
|---|---|
| `tool results not supported` | nothing to set (the default sends results as ordinary messages, as before) |

`ACT_API_FORMAT: anthropic` applies to **every** model ACT uses in the job. If you use both kinds, leave it out.

### Many hosts at once: site_act_concurrency

Every host in one job uses the same key, and GenAI.mil's default quota is **60 requests and 200,000 tokens a minute** per key (and a daily credit budget). A Health check with `use_act` on 20 hosts with findings would otherwise run ACT on as many hosts at once as the job's forks allow, and the gateway answers HTTP 429. `site_act_concurrency` (default **3**) caps how many hosts run ACT at the same time; the rest wait their turn. `0` = no cap. ACT itself also waits when the gateway sends a 429 with `Retry-After`.

### How to verify

1. **The probe result:** in step 4 every model you plan to use shows one endpoint that worked.
2. **A job:** launch **Troubleshoot** with `use_act` = `yes`, `site_act_level` = `explain`, and the model in `site_act_model` (or set through `site_act_models`) on a host with a known finding. In the output, task `ACT | what ACT says` holds an analysis, not an `HTTP 400` message.
3. **The old symptom is gone:** search the job output for `400`. A line saying ACT switched to the other endpoint is fine; a final `HTTP 400` for the model is not (then the probe said neither endpoint works: step 5, last row).

If a step fails, the message after `HTTP 400` is the gateway's reason: copy it exactly when you ask its owner for help, but remove the key and host names first.

## ACT for vCenter alarms (it runs on the AAP side)

`VM - alarms ACT analysis` (VMWARE.md, "Alarms") uses ACT differently from the health checks: ACT runs **inside the job's execution environment on the AAP node**, not on a managed host, and only reads the evidence the job collected from vCenter (it runs no command). So the ESXi hosts need nothing; the **AAP node** needs HTTPS to the model's URL, and the job checks that first. It uses the same settings as everything else here: `site_act_provider`, `site_act_models`, `site_act_url`, `site_act_env` (a proxy), `site_act_ca` (a CA file **in the execution environment**, e.g. one from the project: `"{{ playbook_dir }}/files/ca/model-ca.pem"`), and the **ACT model key** credential on its template.

The **log analysis** (LOG_ANALYSIS.md: `Log analysis` for a group of Linux or Windows servers or a vCenter appliance, `VM - vCenter log analysis` for vCenter and its ESXi hosts) works the same way: the job reads the logs, groups the lines into patterns on the AAP side, and ACT explains the top ones in evidence mode - one call per job, whatever the number of hosts.

## Write your own check (it works with the report, tickets and ACT automatically)

A check is a role named `check_<name>` that **adds findings** to `site_findings`. That is the whole contract. As an example, here is a check that `/tmp` is not world-writable without the sticky bit.

1. Create `roles/check_tmp/defaults/main.yml`, with its settings and a comment saying what it checks:

```yaml
---
# TMP: /tmp and /var/tmp must have the sticky bit (mode 1777).
check_tmp_paths: [/tmp, /var/tmp]
```

2. Create `roles/check_tmp/tasks/main.yml`. Collect with read-only commands, and always use `changed_when: false` and `check_mode: false`: read commands then run in a dry run too. Turn what you saw into findings, and add them:

```yaml
---
- name: tmp | modes
  ansible.builtin.command:
    argv: "{{ ['stat', '-c', '%a %n'] + check_tmp_paths }}"
  register: _tmp
  changed_when: false
  failed_when: false
  check_mode: false

- name: tmp | findings
  ansible.builtin.set_fact:
    _tmp_findings: >-
      {%- set f = [] -%}
      {%- for line in _tmp.stdout_lines if line.split()[0] != '1777' -%}
      {%-   set path = line.split()[1] -%}
      {%-   set _ = f.append({'check': 'tmp', 'id': 'tmp:' ~ path, 'severity': 'warning',
                              'summary': path ~ ' has mode ' ~ line.split()[0] ~ ' (expected
                                1777)',
                              'hint': 'stat ' ~ path ~ '; findmnt ' ~ path}) -%}
      {%- endfor -%}
      {{ f }}

- name: tmp | add them to this host's findings
  ansible.builtin.set_fact:
    site_findings: "{{ site_findings | default([]) + _tmp_findings }}"
```

3. Let the health check know it: in `playbooks/health_check.yml`, add `tmp` to `_health_sets` (to `weekly` and `all`), and add `tmp` to the survey choices of the Health check template.
4. Test it on one host: `ansible-playbook playbooks/health_check.yml -l <host> -e health_checks=tmp`. Then break the condition on purpose (`chmod 777 /tmp` on a test host) and check that the finding appears. A check that never fires proves nothing.

That is all. The report prints it, set_stats publishes it, ServiceNow tickets it (the `id` keeps it on one ticket), and with `use_act=yes` ACT receives the summary and runs the `hint` as evidence.

**Rules for a good finding:**

- `summary`: one line a person understands, with the number and the limit (`/var is 96% full (critical at 95%)`).
- `id`: stable across runs, with no numbers that change (`disk:/var`, not `disk:/var:96`). Otherwise every run opens a new ticket.
- `hint`: **read-only**, runs in under a minute (`timeout 60 ...`), and shows the cause, not just the symptom. It runs as root when ACT is on (Ansible runs it in `explain` mode and hands ACT the output), so never put a command that changes something there.
- `severity`: `critical` = act now (outage, security); `warning` = act soon.

## Use ACT in a playbook of your own

Any playbook that fills `site_findings` can hand them to ACT the same way:

```yaml
- name: My check
  hosts: my_hosts
  become: true
  gather_facts: false
  tasks:
    - ansible.builtin.include_role: {name: site_findings, tasks_from: start.yml}
    - ansible.builtin.include_role: {name: site_findings, tasks_from: run_check.yml}
      vars: {_site_check: tmp}                              # runs roles/check_tmp
    - ansible.builtin.include_role: {name: site_act}
      when: use_act | default(false) | bool and site_findings | length > 0
    - ansible.builtin.include_role: {name: site_findings, tasks_from: report.yml}
```

`report.yml` goes last. It prints, publishes (for tickets and the approval workflow), and fails the host when there are findings or a fix waits for approval.

## What ACT is told, and what it cannot do

- Its task: the findings (one line each), "find WHY, confirm it with evidence, give the exact fix; several findings may share one cause", plus your `site_act_extra_instructions`.
- `explain`: it runs nothing. Ansible ran the hint commands.
- `diagnose`: only commands ACT can prove are read-only run on their own. Everything else is refused and becomes a proposal.
- `self-heal`: in addition, only commands matching `site_act_allow`.
- Apply step: no model at all. The playbook runs only the approved commands, character for character, in order, and stops at the first one that fails. Run as *Check*, it only shows what it would run (a dry run: see WORKFLOWS_AND_SCHEDULES.md, "Rehearse it").
- Every run is recorded in the AAP job output. The ticket carries ACT's analysis. For the service-watch style incident log, see SERVICE_WATCH_DEMO.md.

# 10. Windows servers

The Windows runbooks work the way the Linux ones do. Checks find what is wrong and say it in plain words, with a command to look further. The same report, the same ServiceNow tickets and POA&M, and the same workflows and schedules follow. Only the checks themselves are different: each one is a PowerShell script that runs on the server over WinRM.

**Nothing extra to install on AAP or on the servers.** The playbooks use only Ansible's built-in `script` module, so no Ansible collection is needed. The execution environment needs the Python library for WinRM (`pywinrm`); step 2 shows how to check. The servers need WinRM, which you already use: over HTTPS (5986), or over HTTP (5985) with Kerberos or NTLM encrypting each message (step 1). The check scripts run in Windows PowerShell 5.1 and PowerShell 7, and also in Constrained Language Mode (see step 5).

| Template | Playbook | What it does |
|---|---|---|
| Windows connection test | `playbooks/win_connection_test.yml` | run it first: can AAP reach the servers, with the right account? |
| Windows health check | `playbooks/win_health_check.yml` | the checks below; daily and weekly sets |
| Windows troubleshoot | `playbooks/win_troubleshoot.yml` | on demand: an admin's first PowerShell commands for one kind of problem |
| Windows certificate report | `playbooks/win_cert_report.yml` | every certificate on every server, soonest expiry first |
| Windows certificate renewal - CSRs | `playbooks/win_cert_renew_csr.yml` | for each of the server's own certificates that expires: a new key and a CSR (`certreq`) on the server, the CSR on the certificate's ServiceNow ticket |
| Windows certificate install - check / approved | `playbooks/win_cert_install_check.yml`, `win_cert_install_approved.yml` | the issued certificate from the catalog request: `certreq -accept`, the old certificate's bindings moved to it, after an approval (CERT_INSTALL.md) |
| Windows certificate restore | `playbooks/win_cert_restore.yml` | the bindings back on the old certificate |
| Windows patch | `playbooks/win_patch.yml` | install what Software Center offers (ConfigMgr); restart only with `automatic_restarts: yes`; prove the server is back |
| ACT for Windows - install | `playbooks/win_act_install.yml` | put ACT on the servers in `C:\ProgramData\act`, locked down and unblocked; keep it current |
| ServiceNow tickets, POA&M status | the ones you have | unchanged: they take the Windows results the same way |

**The checks** (`roles/win_check_<name>`):

| Set | Check | Finds |
|---|---|---|
| daily | `disk` | volumes too full |
| daily | `services` | required services not running; automatic services that stopped with an error; services that keep crashing |
| daily | `performance` | CPU, memory or page file too busy; names the busiest processes |
| daily | `time` | Windows Time not running, not synchronized, synced too long ago, clock off |
| daily | `network` | no gateway or DNS; names that do not resolve; ports that do not answer; a broken domain trust |
| daily | `eventlog` | unexpected shutdowns, crashes, disk errors, a cleared Security log, many errors, full logs |
| weekly | `security` | Defender real-time protection off or old signatures; endpoint services down; firewall off; SMBv1; RDP without NLA; UAC off; TLS 1.0/1.1 |
| weekly | `audit` | audit policy settings the STIG asks for; event log sizes; the log forwarding agent |
| weekly | `accounts` | Guest enabled; local passwords old or never expiring; inactive local accounts; unexpected local Administrators |
| weekly | `certs` | certificates that expire or expired, including the WinRM listener AAP connects through |
| weekly | `patching` | a restart pending for installed updates; days since the last update |

Everything is read-only. The one write is one Application event log entry per finding (source `site-health`), so a SIEM that collects event logs sees the findings too.

## Step 1. WinRM on the servers

You already use it. To see it on a server:

```
winrm enumerate winrm/config/listener
```

- **A listener with** `Transport = HTTPS`, `Port = 5986` and a `CertificateThumbprint`: the connection is TLS. The `certs` check warns before that listener's certificate expires: when it expires, AAP can no longer connect.
- **Only** `Transport = HTTP`, `Port = 5985` (often set by a GPO): that works too. Kerberos and NTLM encrypt every message themselves; `ansible_winrm_message_encryption: always` (step 4) makes the connection fail rather than ever send in clear. Never use `basic` sign-in over HTTP.

The firewall must let that port in from the AAP execution nodes.

## Step 2. The execution environment

The job runs inside an execution environment (EE), and that EE needs these Python libraries:

- `pywinrm`, always;
- `requests-ntlm`, for NTLM (the usual choice);
- or `requests-kerberos` plus a Kerberos client (`kinit`), for Kerberos.

AAP's *Default execution environment* has them. Your own EE (for example `platforms-ee`) may not. **The Windows connection test tells you**: its first part runs inside the EE and prints `YES` or `no` for each library. If `pywinrm` is missing, you have two options:

- use AAP's Default execution environment for the Windows templates; or
- add the libraries to your EE's definition and rebuild it:

```yaml
dependencies:
  python:
    - pywinrm
    - requests-ntlm
```

- or build the all-in-one EE: it has these, Kerberos for a FIPS host, and the VMware, NetApp, database and Cisco collections too (EXECUTION_ENVIRONMENT.md).

## Step 3. The credential

**Automation Execution › Infrastructure › Credentials › Create credential**:

- **Name**: `Windows (WinRM)`; **Credential type**: *Machine*.
- **Username**: `YOURDOMAIN\svc_aap` for NTLM. For Kerberos, use `svc_aap@YOURDOMAIN.MIL`, with the domain in capitals. On a **FIPS** system (STIG) NTLM does not work at all - use Kerberos (see *If something goes wrong*).
- **Password**: the account's password.
- **Privilege escalation**: leave it empty. An administrator's WinRM session is already elevated.

The account must be in the **local Administrators group** on every server, usually through a domain group added by GPO. The checks need that to read the Security log, the audit policy, local accounts and certificate stores.

## Step 4. The Windows inventory

Keep the Windows servers in their **own inventory**. Go to **Inventories › Create inventory**, name it `Windows servers`, and paste this in its **Variables** box:

```yaml
ansible_connection: winrm
ansible_port: 5986
ansible_winrm_scheme: https
ansible_winrm_transport: ntlm                  # or kerberos
ansible_winrm_server_cert_validation: validate  # needs your CA bundle in the EE, see below
ansible_winrm_ca_trust_path: /etc/pki/ca-trust/source/anchors/your-ca.pem   # path inside the EE
```

Servers with **only an HTTP listener (5985)**, step 1, take these lines instead (no certificate lines: Kerberos or NTLM encrypts the messages):

```yaml
ansible_connection: winrm
ansible_port: 5985
ansible_winrm_scheme: http
ansible_winrm_transport: kerberos              # or ntlm (not on a FIPS system)
ansible_winrm_message_encryption: always       # encrypted by Kerberos / NTLM, or no connection
```

- **Certificate validation** (HTTPS). `validate` means the EE checks the server's WinRM certificate against the CAs it trusts. EEs usually do not include your organisation's root CAs, so `validate` fails with `CERTIFICATE_VERIFY_FAILED` until you add your CA bundle to the EE and point `ansible_winrm_ca_trust_path` at it (the path inside the EE, as shown above). **Use this setting for real servers.** `ignore` keeps the traffic encrypted but does not check who the server is, so anyone who can intercept the connection could pose as it: use it **only** for a short test on a lab network, then switch back.
- **Hosts.** Add them by full name (`web01.yoursite.mil`); Kerberos needs the full name.
- **Groups.** Use whatever helps: by site, by role (`win_dc`, `win_web`...). The settings below can be set per group. `inventories/example-windows/hosts.yml` shows an example.

Linux and Windows do not mix: the Linux templates skip Windows servers, and the Windows templates skip Linux servers. So a template pointed at the wrong inventory changes nothing and opens no tickets.

## Step 5. Test the connection

Create a job template, then launch it with **Limit** set to one server:

| Field | Value |
|---|---|
| Name | `Windows connection test` |
| Inventory | `Windows servers` |
| Project | `site-automation` |
| Playbook | `playbooks/win_connection_test.yml` |
| Execution environment | one with `pywinrm` (step 2) |
| Credentials | `Windows (WinRM)` |
| Limit | empty, **Prompt on launch** ticked |

A good result ends with:

```
Connected: WEB01 (Microsoft Windows Server 2022 Standard, build 20348), domain yoursite.mil
PowerShell 5.1.20348.2849, FullLanguage
Logged on as yoursite\svc_aap; local administrator: yes
OK: the Windows runbooks can run here.
```

`ConstrainedLanguage` instead of `FullLanguage` means WDAC or AppLocker restricts PowerShell on that server. The check scripts are written and tested for that mode; only two optional parts are skipped in it (TLS endpoints and the search for missing updates). Whether Ansible itself can run on such a server depends on your ansible-core version and the policy: if this test got through and printed its result, it can.

## Step 6. The job templates

For all of these, use: **Inventory** `Windows servers`, **Project** `site-automation`, the EE from step 2, **Credentials** `Windows (WinRM)`, **Limit** empty with **Prompt on launch** ticked. Leave **Variables** empty: with no `target`, they run on every server in the inventory.

### Windows health check

| Field | Value |
|---|---|
| Playbook | `playbooks/win_health_check.yml` |
| Survey: `health_checks` | *Multiple Choice (multiple select)*: `daily`, `weekly`, `all`, `disk`, `services`, `performance`, `time`, `network`, `eventlog`, `security`, `audit`, `accounts`, `certs`, `patching`. Default `daily` |

### Windows troubleshoot

| Field | Value |
|---|---|
| Playbook | `playbooks/win_troubleshoot.yml` |
| Survey: `ts_area` | `overview`, `disk`, `performance`, `service`, `network`, `login`, `time`, `logs`, `updates`, `security`. Default `overview` |
| Survey: `ts_service` | *Text*, not required. "Service name (for area = service), e.g. W3SVC" |
| Survey: `ts_target` | *Text*, not required. "host:port it cannot reach (for area = network)" |
| Survey: `ts_hours` | *Integer*, default 2. "How many hours of event logs to read" |

Each command's output appears under a `### what it shows` line, followed by the matching health checks. Findings do not fail this job.

**All settings** of this job, with their defaults: win_troubleshoot.

### Windows certificate report

| Field | Value |
|---|---|
| Playbook | `playbooks/win_cert_report.yml` |
| Survey | none |

It reads **every machine store** (`Cert:\LocalMachine\*`), not only Personal, Web Hosting and Remote Desktop. *Automatically select the certificate store* puts a `.cer` file in **Trusted Root**, **Intermediate Certification Authorities** or **Other People**, so a certificate imported that way is found too:

- **With its private key** (the server's own certificate, in an unusual store): checked like the ones in Personal: a warning within 30 days, critical within 7, a ticket.
- **Without** (a certificate the server trusts): in the table as `LocalMachine\Root (Trusted Root Certification Authorities) (trust store)` when it expires within `win_check_certs_notice_days` (60) or expired in the last `win_check_certs_expired_report_days` (30). Never a finding or a ticket: its CA renews it. The older Windows roots that expired long ago are left out.
- **In use** (an https binding, the WinRM listener): always checked, whatever its store.

- **Remote Desktop: left out.** Windows (or your GPO's autoenrollment) renews these itself, so they are not in the report, the findings or the renewal: the **Remote Desktop** store (the self-signed one RDP makes), the certificate RDP uses, and one whose only purpose is *Remote Desktop Authentication*. One also used by an https binding or WinRM is still checked. `win_check_certs_remote_desktop: true` checks them like the others (weekly health check too).
- **Never read:** `Disallowed` (the untrusted certificates), `AuthRoot` (the roots Windows Update keeps), `REQUEST` (pending requests: a CSR's new key) and `SMS` (the MECM client's own certificates, which it renews itself): `win_check_certs_trust_skip_stores`.

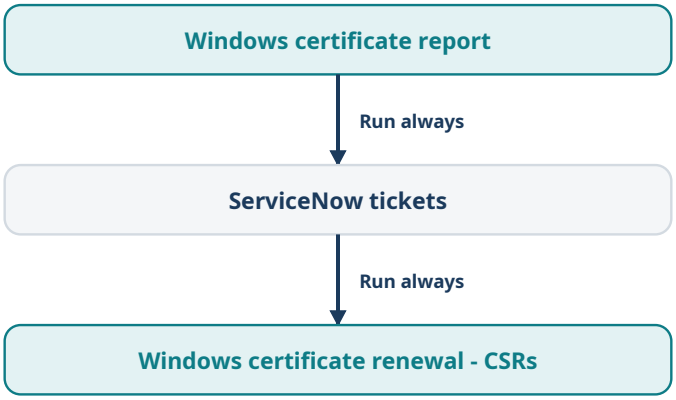
`win_check_certs_trust: false` in the template's Variables reads only `win_check_certs_stores`. The weekly health check reads only those stores. To list what a server has, on the server: `Get-ChildItem Cert:\LocalMachine -Recurse | Where-Object NotAfter -lt (Get-Date).AddDays(60) | Format-List PSParentPath, Subject, NotAfter, HasPrivateKey`.

### Windows certificate renewal - CSRs

With `cert_renew_servicenow: catalog` the CSRs are ordered from your PKI's catalog item instead of going on incidents: CERT_RENEWAL_CATALOG.md. Then the issued certificate is installed from that request, after an approval - `certreq -accept` and each binding of the old certificate moved: *Windows certificate install* (CERT_INSTALL.md).

| Field | Value |
|---|---|
| Playbook | `playbooks/win_cert_renew_csr.yml` |
| Credentials | the Windows machine credential (an administrator: `certreq -machine` writes to the machine key store), `ServiceNow API` |
| Variables | `servicenow_assignment_group: <your PKI or Windows team>`: who gets a ticket this job opens (a certificate with no ticket yet) |
| Job type | try it as **Check** first: it makes and sends nothing |

The Windows twin of the Linux renewal (RUNBOOKS.md, "Certificate renewal"). Put it in the weekly workflow after the tickets step, so each CSR lands on the ticket the report opened:




For each certificate the report flags (within `win_check_certs_warn_days`, 30 days, or expired) that is the server's own: in Personal or Web Hosting, or in any machine store with its private key. Remote Desktop's certificates are not in the report, so they get no CSR (`win_check_certs_remote_desktop: true` puts them in both).

1. **On the server**, `certreq -new -machine` makes a new key in the machine key store and a CSR. The CSR has the same subject, names (subjectAltName: DNS, IP, e-mail, URL) and purposes (EKU) as the old certificate. The key is RSA 2048 (`win_cert_renew_key_bits`); a bigger old key keeps its size, and an EC certificate gets a new key on the same curve. The key **cannot be exported** (`win_cert_renew_exportable: false`) and never leaves the server. Until the issued certificate is installed, it waits as a pending request (certlm.msc > *Certificate Enrollment Requests*). The CSR, with its `.inf` and a `.json` saying which certificate it renews, goes in `C:\ProgramData\site-automation\renew` (`win_cert_renew_dir`, administrators and SYSTEM only).
2. **On the ticket**, the CSR is attached (as for Linux: a refused `.csr` is tried again as `.txt`), with a work note saying what to do. Step 3 of that note is the Windows one: `certreq -accept -machine <issued>.cer` on the server, as an administrator. That pairs the certificate with its key and puts it in Personal. Then use it where the old one is used, and the note says how:
   - the https binding (IIS Manager, or `netsh http update sslcert`)
   - the WinRM HTTPS listener AAP connects through (`Set-Item WSMan:\localhost\Listener\...\CertificateThumbprint`)
   - Remote Desktop (with `win_check_certs_remote_desktop: true`)
3. **A run again** finds the pending request and reuses its CSR, so nothing changes on the ticket. Once the new certificate is installed, the old one is passed over: *a newer certificate for the same subject is in the store already: use it, then remove the old one*.

**Never renewed here** (each listed in the job output with the reason):

| What | Why |
|---|---|
| a trust store's certificate (Trusted Root, Intermediate CAs, Other People ... without its key) | one the server trusts, not its own: its CA renews it |
| a CA certificate | a CA has no CSR from you |
| a self-signed certificate (unless `win_cert_renew_self_signed: true`) | it does not come from a PKI; with the setting on, the PKI issues a real one in its place (never one only for `localhost`) |
| Remote Desktop's (its store, the one RDP uses, one only for Remote Desktop), unless `win_check_certs_remote_desktop: true` | Windows or the GPO renews them; not in the report |
| a certificate seen only on a port (`win_check_certs_endpoints`) | its store is not known: the one in its store gets the CSR |
| a host in `cert_renew_skip_hosts` / `cert_renew_skip_groups` | renewed by hand: still in the report |
| issued by `win_cert_renew_exclude_issuer`, not by `win_cert_renew_issuer`, or not matching `win_cert_renew_only` | your filters |
| names (subjectAltName) with an otherName (a UPN, a GUID) or a directory name | certreq cannot copy them from here: the job fails for that one certificate and says to make it by hand (certlm.msc > the certificate > *All Tasks* > *Advanced Operations*) |

Every setting: win_check_certs (`win_cert_renew_*`).

### Active Directory health report

For the domain controllers (group `win_dc`): one report for the whole domain - replication, dcdiag, FSMO, SYSVOL, DNS, time, and what changed in the last day. It needs an account with administrator rights on the DCs, in a credential of its own: ACTIVE_DIRECTORY.md.

| Field | Value |
|---|---|
| Playbook | `playbooks/ad_health_report.yml` |
| Credentials | `Windows DCs (WinRM)` |
| Variables | `report_email_to: ad-team@yoursite.mil` |

## Step 7. Workflows and schedules

These are the same as for Linux, with the Windows templates. The full list, with every step and link, is in WORKFLOWS_AND_SCHEDULES.md (workflows 12 to 17):

| Workflow | Steps | Schedule |
|---|---|---|
| Windows daily health | Windows health check (`daily`) › ServiceNow tickets (**Run always**) | every day 06:15 |
| Windows weekly compliance | Windows health check (`weekly`) › tickets; Windows certificate report › its own tickets box (both **Run always**) | Monday 07:15 |
| Windows patch with a dry run first | Windows health check (`disk`, `services`) › Windows patch as **Check** › *Approval* › Windows patch › Windows health check (`daily`) › tickets | monthly, inside the servers' ConfigMgr maintenance window |

The ServiceNow tickets template runs on AAP itself, so it stays as it is.

## Start small: the pilot order

Everything Windows here is new, so try it on **one** server that does not matter much, in this order, and widen only when each step looks right:

1. **Windows connection test**: the account, WinRM and PowerShell work.
2. **Windows health check**: the findings make sense for that server.
3. **Windows patch as Check** (the dry run): it lists what Software Center would install.
4. **Windows patch for real**, inside that server's ConfigMgr maintenance window.
5. **ACT for Windows - install** on the same server (tell your security team first, see below).

Every step can be a dry run first: DRY_RUNS.md.

## Patching through Software Center

`playbooks/win_patch.yml` does, on each server, what an admin does in Software Center: install every software update your ConfigMgr admins deployed to it ("Install all"), required and available alike. AAP decides nothing about *which* updates: ConfigMgr does, as it does today. Applications listed in Software Center are not installed; only software updates are. Then:

1. **No automatic restarts** unless you ask for them with `automatic_restarts: yes` (the survey, or the template's Variables). Without it, the job installs the updates and lists the servers that need a restart: `A RESTART IS NEEDED to finish the updates ... automatic restarts are off`. A person restarts them (the post-check's `patching` finding keeps them visible). With it, the job restarts a server only when the updates or the ConfigMgr client say so, and never one in `win_patch_never_reboot_groups`.
2. **Prove the server is back** (after a restart, and also when none was needed). A new boot time, and every automatic service that ran before runs again. Delayed services get `win_patch_services_grace_min` (5 minutes) to start.
3. **Look again.** After a restart, servicing-stack and cumulative updates often reveal more updates. There is a second round (`win_patch_max_rounds: 2`), only when it restarted.

**Safety.**

- One server at a time (`win_patch_serial`, in the template's Variables - in `all.yml` it does nothing), and the rollout stops at the first server that fails.
- By default it only patches the group `win_patch_hosts`. Set `target:` in the template's Variables (for example `target: all`) and pick the servers with the Limit at launch.
- Servers in `win_patch_never_patch_groups` (default `no_patch`) are skipped.
- **Maintenance windows still apply.** Outside the server's ConfigMgr maintenance window, ConfigMgr holds the updates. The job then stops with `ConfigMgr will not install these now ... (waiting for a maintenance window)` instead of waiting forever. Run it inside the window, or ask the ConfigMgr admin for a window that covers the patch time.
- **A dry run first:** run the template as *Check*. It rescans and lists what it would install, and changes nothing (DRY_RUNS.md). The *Windows patch with a dry run first* workflow puts that list in front of the approver.

**Needs:** the ConfigMgr client (Software Center) on the servers, and full PowerShell (not Constrained Language Mode) for the install call. Restarts use `shutdown.exe`, and AAP watches the WinRM port go down and come back. No Ansible collection is needed.

**Scheduling.** With `target: all` and no Limit, a scheduled run patches every server in the Windows inventory, one after another. Choose that on purpose. Otherwise, make one schedule per group (for example the test servers the week before the others), each with its own Limit.

**The job template:**

| Field | Value |
|---|---|
| Name | `Windows patch` |
| Playbook | `playbooks/win_patch.yml` |
| Credentials | `Windows (WinRM)` |
| Job type | Run, with **Prompt on launch** ticked (Check = the dry run) |
| Variables | `target: all` (or a group), and pick servers with the Limit |
| Survey: `automatic_restarts` | "Restart servers automatically when the updates need it?" Choices `no`, `yes`. Default `no` |

If a server fails:

| What you see | What to do |
|---|---|
| `has no working ConfigMgr client` | the SMS Agent Host service (CcmExec) is missing or stopped |
| `waiting for a maintenance window` | run inside the window, or ask the ConfigMgr admin |
| `these updates failed: ... (error 0x...)` | `C:\Windows\CCM\Logs\UpdatesDeployment.log` and `WUAHandler.log` on the server |
| `still not done after 120 minutes` | Software Center on the server; `C:\Windows\CCM\Logs\UpdatesHandler.log` |
| `did not accept a logon after the restart` | look at the server's console |
| `... ran before patching and do not run now` | the service did not come back: `Get-Service <name>`, System log |

**All settings** of this job, with their defaults: win_patch.

## ACT on the servers (C:\ProgramData\act)

`playbooks/win_act_install.yml` puts ACT for Windows (`vendor/act-windows/act.ps1`) on the servers in `C:\ProgramData\act` (setting `win_act_dir`). Run it again after you update this repository, and every server gets the new version.

**The job template** *ACT for Windows - install*: playbook `playbooks/win_act_install.yml`; your Windows inventory and its **Machine (Windows)** credential; the execution environment your Windows jobs use; **Prompt on launch** ticked for **Limit** and **Job type**. Launch it as **Check** with a Limit of one server first. To roll out group by group with an approval: workflow 15 in WORKFLOWS_AND_SCHEDULES.md.

- **Locked down.** Ordinary users may create folders and files in `C:\ProgramData` (and in `C:\temp`). Someone could then place, or swap, `act.ps1` before an administrator runs it, and their code would run with administrator rights. So the job:
- takes ownership of `C:\ProgramData\act`;
- lets only Administrators and SYSTEM in (nothing inherited from `C:\ProgramData`);
- checks the file's SHA256 against the one in the repository, every run, and replaces it if it differs.
- **Unblocked.** The file is written on the server itself, so Windows never marks it as "downloaded from the internet", and the job runs `Unblock-File` on it anyway. Every run also checks it is not blocked, and fixes it if it is.
- **Using it on a server:** an administrator opens PowerShell and runs `powershell -ExecutionPolicy Bypass -File C:\ProgramData\act\act.ps1`. The first time, `:setup` asks for their own GenAI or Ask Sage key and keeps it in their profile. No key is put on the server for everyone.
- **Dry run:** run as *Check* and it says what it would install or fix. To remove it: `win_act_state: absent`.
- **Tell your security team before the first run.** The installer is a large PowerShell script that writes a `.ps1` file, and some endpoint-protection products (Defender for Endpoint, Trellix) flag that pattern. Let them know it is coming from AAP, and try one server first.
- **Versions and `:probe`.** ACT for Windows has the same endpoint formats and `:probe` command as the Linux one (see ADDING_ACT.md); the copy in `vendor/act-windows/` is refreshed when the repository's

vendored ACT is. On Windows, an unattended run never approves the danger tier of commands: only a person at the prompt can. See APPROVED_COMMANDS.md.

- ACT itself needs full PowerShell. Where WDAC or AppLocker enforce Constrained Language Mode, ACT's own notes (sign it with your code-signing certificate) apply.

**All settings**, with their defaults: win_act.

## Settings: what each check looks at, and how to change it

Every setting is in `roles/win_check_<name>/defaults/main.yml`, with a comment saying what it does. To change one:

- **for all Windows servers**, use the `Windows servers` inventory's **Variables** box;
- **for a group**, use the group's **Variables** box, or a settings file named after the group (for example `playbooks/group_vars/win_web.yml` for a group called `win_web`);
- **for one server**, use the host's **Variables** box.

The settings you will most likely want:

network: what every Windows server must resolve and reach

```
win_check_network_dns_names: [dc01.yoursite.mil]
win_check_network_tcp:
  - {name: LDAPS, host: dc01.yoursite.mil, port: 636}
  - {name: SIEM, host: siem.yoursite.mil, port: 9997}

# security: your endpoint protection's services (for example Trellix / McAfee)
win_check_security_av_services: [masvc, mfemms]

# audit: your log forwarding agent
win_check_audit_forwarder_services: [SplunkForwarder]

# accounts: who may be a local administrator (wildcards allowed)
win_check_accounts_admins_allowed: ['*\Administrator', 'YOURDOMAIN\Domain Admins',
  'YOURDOMAIN\svc_aap']

# services: more services that must run, on a group (for example IIS servers)
win_check_services_required_extra: [W3SVC, WAS]
```

A few defaults worth knowing:

- **disk**: warning at 85% used, critical at 95%. Per volume: `win_check_disk_overrides: {'D:': {warn: 92, crit: 98}}`.
- **services**: `EventLog`, `RpcSs`, `Dnscache`, `W32Time`, `WinRM` and `mpssvc` (the firewall) must run. Automatic services that stop cleanly by design are not reported.
- **security**: Defender is checked only when it is the active antivirus (`win_check_security_av: auto`). With another product, list its services instead.
- **certs**: warning 30 days before expiry, critical 7 days before. Old expired certificates that nothing uses are listed in the report, not raised as findings.
- **patching**: warning when the last update is 35 days old, critical at 60 days.

**All settings**, with their defaults: win_check_accounts, win_check_audit, win_check_certs, win_check_disk, win_check_eventlog, win_check_network, win_check_patching, win_check_performance, win_check_security, win_check_services, win_check_time.

## If something goes wrong

| What you see | What to do |
|---|---|
| `winrm or requests is not installed` / connection test: `no winrm` | the EE has no pywinrm: step 2 |
| `CERTIFICATE_VERIFY_FAILED` | the CA bundle (step 4); `ansible_winrm_server_cert_validation: ignore` only as a short lab test |
| `the specified credentials were rejected by the server` | username format (`DOMAIN\user` for NTLM), password, or the account is locked |
| `Connection refused` / `timed out` on 5986 | the firewall, or no HTTPS listener (step 1): a server with only HTTP takes port 5985 (step 4) |
| `kerberos: ... kinit` or `Server not found in Kerberos database` | no Kerberos in the EE, or a short host name: use NTLM, or full names |
| `ntlm: ... [digital envelope routines] unsupported` | the system is in FIPS mode: NTLM needs MD4 and RC4, which FIPS forbids. Use Kerberos (`svc_aap@YOURDOMAIN.MIL`, `ansible_winrm_transport: kerberos`) |
| `KDC has no support for encryption type` (Kerberos, FIPS) | the EE's Kerberos follows the host's FIPS policy and offers only AES with SHA-2, while Active Directory's keys are AES with SHA-1. Build your Windows EE with the file `/etc/krb5.conf.d/crypto-policies` removed and a file in `/etc/krb5.conf.d/` holding `[libdefaults] permitted_enctypes = aes256-sha2 aes128-sha2 aes256-sha1 aes128-sha1` (no RC4, no DES); check it inside the EE with `kinit svc_aap@YOURDOMAIN.MIL` - it asks for the password when Kerberos is fixed |
| `local administrator: NO` | add the account to the local Administrators group |
| `the <name> check could not run: ...` | the text after it is PowerShell's error. The other checks still ran |
| `skipping: no hosts matched` | the template's inventory is not `Windows servers`, or the Limit matches no server there |
| `... is not digitally signed` or `running scripts is disabled on this system` | a GPO sets the PowerShell execution policy for the machine (for example AllSigned), which overrides what Ansible asks for. Set it to RemoteSigned by GPO for these servers, or sign the scripts with your code-signing certificate |

## Not in this release (next)

- ACT in the Windows jobs: `use_act` on the Windows health check and troubleshoot (explain, propose fixes for approval), like on Linux. ACT is already on the servers (above).
- Service watch for Windows services and IIS application pools.
- Checks for IIS and SQL Server. (Domain controllers have their own report: ACTIVE_DIRECTORY.md.)

# 11. VMware jobs (vCenter): restart, snapshots, VLAN, reports, alarms and ACT

Small jobs for virtual machines, done through vCenter (they replace the vCenter runbooks of System Center Orchestrator):

| Job template | Playbook | What it does |
|---|---|---|
| VM - restart | `playbooks/vm_reboot.yml` | restarts VMs. `guest` (default): the operating system restarts itself through VMware Tools. `hard`: a reset, like the reset button |
| VM - shut down | `playbooks/vm_shutdown.yml` | shuts VMs down and waits until they are off. `guest` (default) through VMware Tools; `hard`: power off at once |
| VM - snapshot | `playbooks/vm_snapshot.yml` | takes a snapshot **without memory** (the disks only). The AAP job number and user go into its description |
| VM - delete snapshot | `playbooks/vm_snapshot_delete.yml` | deletes one snapshot by its exact name, or every snapshot of the VM. Lists what the VM has first |
| VM - notes | `playbooks/vm_notes.yml` | edits the VM's Notes in vCenter: append a line (with date and user), replace, or clear. Prints the old notes |
| VM - change VLAN | `playbooks/vm_vlan.yml` | moves one network adapter to another port group (VLAN). The VM's other adapters are not touched |
| VM - secure boot report | `playbooks/vm_secure_boot_report.yml` | read-only: every VM's firmware and secure boot. Lists BIOS VMs and EFI VMs with secure boot off separately |
| VM - datastore report | `playbooks/vm_datastore_report.yml` | read-only: every datastore's capacity, free space, used and provisioned %; the ones at 85 / 90 % used listed apart |
| VM - snapshot report | `playbooks/vm_snapshot_report.yml` | read-only: every snapshot of every VM - name, when taken and by whom, age, size, folder - oldest first; the ones 5 days or older listed apart |
| VM - snapshot cleanup | `playbooks/vm_snapshot_cleanup.yml` | deletes the snapshots 5 days or older - never "keep" ones, excluded VMs, datacenters or clusters, the AAP server or ones over 1 TB; or only the patching snapshots (`vm_snapshot_cleanup_name_words`) - then reports what it deleted. Best run in the workflow below, after an approval |

| Job template | Playbook | What it does |
|---|---|---|
| VM - refresh the pick lists | `playbooks/vm_pick_refresh.yml` | on a schedule, read-only in vCenter: writes what may be done right now - the snapshots that may be deleted, the VMs that may be restarted (or only the hung ones) - as the choices of a question in the survey of the templates below ("Pick lists") |
| VM - delete picked snapshots | `playbooks/vm_snapshot_delete_picked.yml` | deletes the snapshots picked in the survey, each checked again live; in its workflow: plan, approval, then exactly the plan |
| VM - restart picked VMs | `playbooks/vm_reboot_picked.yml` | restarts the VMs picked in the survey (`guest` or `hard`), each checked again live - never the AAP server; in its workflow: plan, approval, then exactly the plan |
| VM - health report | `playbooks/vm_health_report.yml` | read-only: every datacenter, cluster and host of every vCenter - vSphere HA and DRS working or not (HA agents, failover capacity, DRS recommendations and faults, vCLS retreat mode), hosts not connected or in maintenance, hardware sensors not green, hosts that need a restart, CPU and memory used, configuration issues and alarms - coloured, with what to do for each problem |
| VM - alarms report | `playbooks/vm_alarm_report.yml` | read-only: the triggered alarms on vCenter, datacenters, clusters and hosts; the alarms that fired in the last 24 hours (also the ones that cleared since); hosts not connected; the last 24 hours' failed logins, VM shutdowns / HA restarts and other errors and warnings; configuration issues - coloured by severity |
| VM - alarms ACT analysis | `playbooks/vm_alarm_act_analysis.yml` | read-only: the same problems, with ACT's likely cause, fix and confidence for each. ACT runs on the AAP side, never on the hosts |
| VM - outage watch | `playbooks/vm_outage_watch.yml` | every 5 to 15 minutes, read-only: is something down or hung right now - a host not responding, an HA agent isolated, a datastore not accessible, all paths down, an HA restart or failure, a VM waiting on a question, a hung VM (no heartbeat AND no answer on the network). One line when clean; a report, a ServiceNow incident and a red job when not. Restarts nothing (phase 1). VM_OUTAGE_WATCH.md |

| Job template | Playbook | What it does |
|---|---|---|
| VM - vCenter log analysis | playbooks/vm_log_analysis.yml | read-only: vCenter's own log and each ESXi host's (hostd, vmkernel, vpxa), read through vCenter's API - no SSH - grouped into patterns across hosts, with ACT's likely cause, fix and confidence for each (LOG_ANALYSIS.md) |
| VM - capacity planning | playbooks/vm_capacity_report.yml | read-only: overall and per cluster - CPU, memory (also with a host down), vCPUs per core, VMs that still fit, hosts recommended, datastores - with growth from vCenter's own history and the days until each limit; optionally GenAI's estimate next to it |
| VM - deploy from ServiceNow ticket | playbooks/vm_deploy_from_ticket.yml | builds the VM - or VMs, one per name - a ServiceNow ticket asks for, from a template - checked against your rules, placed by site, name rule or free room, the result on the ticket. Best run in its workflow (propose, approval, build) |
| VM - deploy settings finder | playbooks/vm_deploy_inventory.yml | read-only: every vCenter's datacenters, clusters, templates, datastores, folders and networks, by the names the deploy's settings use - with the lines to paste into the deploy template's Variables |
| VM - pick up deploy tickets | playbooks/vm_deploy_poll.yml | on a schedule: finds new VM request tickets in ServiceNow and starts the deploy workflow for each. Builds nothing itself; the approval stays |
| VM - VMs created report | playbooks/vm_created_report.yml | read-only: how many VMs were created in the last 7 days and each one - when, by whom, how (cloned, deployed from a template, new), the ticket when the AAP deploy built it, where, its size - by person and by day, and the ones created and deleted since. Schedule it weekly (e.g. Fridays at noon) |
| ESXi - security settings | playbooks/esxi_security.yml | on every ESXi host: SSH off, the shell timeouts at 600 s, lockdown mode normal. Changes only what differs, so it can run every night and puts back whatever drifted |
| VMware - certificate report | playbooks/vmware_cert_report.yml | read-only: every certificate of vCenter and its ESXi hosts, soonest expiry first, and for each one due what the renewal would do. Never ServiceNow, never a CSR |

| Job template | Playbook | What it does |
|---|---|---|
| VMware - certificate renewal | `playbooks/vmware_cert_renew.yml` | the certificates of vCenter and every ESXi host: each one that expires within 30 days gets a ServiceNow incident, and a CSR - made where its key stays, by vCenter or the host - attached to it with what to do next. The days can be raised to try it on certificates years from expiring |

They run on the AAP side and talk to vCenter. They never log in to the VMs.

**Never the AAP server.** Restart, shut down and VLAN change refuse the AAP server's VM. A job would cut off the system running it, and AAP must never be restarted from AAP. The VM is recognised by its name, its guest host name or its IP address, matched against the hosts in your inventory groups `aap` and `aap_hosts`. If the template's inventory has no such group and you have not named the AAP VM (below), these three jobs refuse to run at all. Snapshots, notes and the report are allowed on every VM, since they do not stop it.

## Before you start

### 1. The execution environment must have the VMware collection

The jobs use Red Hat's certified collection `vmware.vmware` (2.11 or later) and its Python libraries (pyVmomi and the vSphere Automation SDK). Find out whether your execution environment (EE) has them. On the AAP server, as the user AAP runs as:

```
podman images | grep -i ee-
podman run --rm <the EE image your templates use> ansible-galaxy collection list vmware.vmware
podman run --rm <the EE image your templates use> python3 -c "import pyVmomi, vmware.vapi;
  print('ok')"
```

- **Both answer:** nothing to do.
- **Not there:** build an EE that has it, with `ansible-builder`. Start from your AAP version's `ee-minimal` image, and pull the collection from your Private Automation Hub when the network is air-gapped. ansible-builder reads the collection's Python requirements by itself:

execution-environment.yml

```
version: 3
images:
  base_image:
    name: registry.redhat.io/ansible-automation-platform-27/ee-minimal-rhel9:latest
dependencies:
  galaxy:
    collections:
      - name: vmware.vmware
```

Then in AAP: **Execution Environments > Add** (the image you pushed). Pick it on the VM templates. The all-in-one EE has it, with everything else: EXECUTION_ENVIRONMENT.md.

Do **not** add a `collections/requirements.yml` to this repository for it. AAP would then try to download from Galaxy on every project sync, and on an air-gapped network every sync would fail.

vCenter 7 or 8 is needed: the jobs also use vCenter's REST API.

### 2. A vCenter account for AAP

A service account with a vCenter role that has only what the jobs need:

| Job | vCenter privileges |
|---|---|
| restart | Virtual machine > Interaction > Reset |
| shut down | Virtual machine > Interaction > Power off |
| snapshot / delete snapshot | Virtual machine > Snapshot management > Create snapshot / Remove snapshot |
| notes | Virtual machine > Change configuration > Set annotation |
| change VLAN | Virtual machine > Change configuration > Modify device settings, and Network > Assign network on the port groups |
| ESXi security settings | Host > Configuration > Security profile and firewall (the SSH service), Advanced settings (the timeouts), Change settings (lockdown mode and its exception users). If one is missing, the job's error names it (e.g. `lacks the privilege Host.Config.Settings`) |
| certificate renewal: an ESXi host's CSR | Certificates > Manage certificates, on the hosts. If it is missing, the job's error names it |
| certificate renewal: vCenter's CSR | Certificate Management (`CertificateManagement.Manage` and `CertificateManagement.Administer`), in a role given as a **Global Permission** - or the account in the single sign-on group **CAAdmins** (Administration > Single Sign On > Users and Groups > Groups > CAAdmins > Add members), as the vSphere Client's own CSR needs; administrator@vsphere.local has it |
| all (to read), and the alarms report | Read-only, which every vCenter role includes |

Give it the role on the folders or clusters of the VMs these jobs may touch, and nowhere else.

### 3. The credential

AAP: **Credentials > Add**, type **VMware vCenter**: vCenter Host (its name, e.g. `vcsa01.yoursite.mil`), Username, Password. It hands the jobs `VMWARE_HOST`, `VMWARE_USER` and `VMWARE_PASSWORD`. Nothing about vCenter goes in Git.

**vCenter's certificate:** the jobs check it. If the EE does not trust your CA, a job fails with `certificate verify failed`. The right fix is to add your CA to the EE. Until then, `vmware_validate_certs: false` turns the check off, in the template's Variables (not recommended).

**Several vCenters:** the reports (alarms, datastores, snapshots and their cleanup, capacity planning) and the ESXi security settings read **every vCenter in one job, into one report**: keep one VMware vCenter credential and list the vCenters in `vmware_vcenters` (see "Several vCenters, one report" below). The jobs that act on VMs by name (restart, shut down, snapshot, notes, VLAN, secure boot) still use the credential's vCenter only: for a VM in another vCenter, use a second credential with that vCenter's address, or tick **Prompt on launch** for Credentials and pick it at launch.

### 5. Several vCenters, one report

With two or more vCenters (for example in Enhanced Linked Mode, as one tree in the vSphere Client), list them all in `playbooks/group_vars/all.yml` (for every job) or in a template's Variables:

```yaml
vmware_vcenters: [vc01.yoursite.mil, vc02.yoursite.mil]
```

- **One credential** is enough: the jobs log in to every vCenter with its user name and password. In Enhanced Linked Mode the vCenters share single sign-on, so the same account works on all of them. Give its role as a **Global Permission** (vSphere Client: **Administration > Access Control > Global Permissions**), or on each vCenter's root with **Propagate to children**.
- The credential's own vCenter is always read. Listing it again (or by another name, its IP) is fine: each vCenter is read once.
- A vCenter on another port: `vc03.yoursite.mil:8443`.
- **The report:** one email for all. The **Datacenter** column says which vCenter (`vc02 / DC1`); the footer lists the vCenters read. A cluster or datastore name that exists in two places is shown with its vCenter and datacenter, e.g. `PROD (vc02 / DC1)`, and never added together.
- **A vCenter that cannot be read** (down, a firewall, a refused login) does not stop the others: the report covers the others, lists it in red on top ("vCenters not read") and its title ends with "- 1 vCenter(s) not read". The job stays **green**: the report completed (`vmware_vcenter_fail_unread: true` makes it fail, e.g. for a workflow's failure path).
- **Snapshot cleanup:** each snapshot is deleted on its own vCenter. A VM's ID repeats in every vCenter (both have a `vm-42`), so the report hands the cleanup the vCenter with each snapshot, and the cleanup deletes only that one.
- The ESXi security settings set the hosts of every vCenter listed.

### 4. Name the AAP VM (and vCenter's)

In your `playbooks/group_vars/all.yml`:

```yaml
vmware_protected_vms: [AAP01, VCSA01]      # the exact VM names in vCenter
```

This covers the case where the AAP VM's name differs from its host name and VMware Tools is not running, so neither the guest host name nor the IP is known. More groups to refuse: `vmware_protected_groups: [aap, aap_hosts, my_group]` (aap and aap_hosts are always refused anyway).

## Which VMs a job acts on

You type them. Every job (except the report) asks for **VM names**, one per line: one VM, or several. These are the names as vCenter shows them in its inventory. They are exact, upper/lower case included, with no wildcards.

The job runs on the AAP server and asks vCenter for each name. It does **not** use the template's inventory hosts or a Limit to choose VMs; it uses the inventory only to recognise the AAP server. So a VM does not have to be in any AAP inventory to be restarted or snapshotted.

- Names that do not exist, or exist twice in vCenter, stop the job before anything is done.
- With several VMs, every check is done for all of them first. If one is refused (the AAP server, no VMware Tools for a clean restart), nothing is done to any of them.
- Several VMs are handled in the order given. Restarts are requested one right after the other, without waiting for each to come back up, so restart the members of a cluster (two database nodes, for example)

in separate runs. A shutdown waits for each VM to be off before the next.

## The templates

Make one job template per job:

- **Inventory:** your Linux inventory, the one with the group `aap`. The jobs need it for the AAP check; they run nothing on its hosts.
- **Credentials:** the VMware vCenter credential.
- **Playbook:** one of the files in the table at the top.
- **Execution environment:** the one with `vmware.vmware`.
- **Limit:** leave it **empty** and do **not** prompt for it. These jobs run on the AAP side (`localhost`); a Limit, or a workflow's Limit, would make them skip without doing anything.
- **Job type:** tick **Prompt on launch** if you want dry runs. As **Check**, every job says what it would do and changes nothing.

Surveys (**Survey > Add**, then switch the survey **on**):

| Template | Question (variable) | Type | Default / choices |
|---|---|---|---|
| all but the report | VM names, one per line (`vm_names`) | Textarea, required | |
| restart, shut down | How (`vm_power_mode`) | Multiple choice | `guest`, `hard`; default `guest` |
| snapshot | Snapshot name (`vm_snapshot_name`) | Text, optional | empty = `aap-<date>-<time>` |
| snapshot | Why / change number (`vm_snapshot_description`) | Text | |
| delete snapshot | Snapshot name (`vm_snapshot_name`) | Text | |
| delete snapshot | Delete ALL snapshots (`vm_snapshot_delete_all`) | Multiple choice | `false`, `true`; default `false` |
| notes | Text (`vm_notes`) | Textarea | |
| notes | How (`vm_notes_mode`) | Multiple choice | `append`, `replace`, `clear`; default `append` |
| change VLAN | Network adapter number (`vm_nic`) | Integer | `1` |
| change VLAN | Port group (`vm_portgroup`) | Text, required | its name as vCenter shows it |
| any (optional) | Email the result to (`report_email_to`) | Text, not required | empty = no email ("Email the report" below) |

The secure boot and datastore reports need no survey. Schedule them weekly with `report_email_to` in their Variables. They stay green when they find VMs or datastores to fix - the report and email say which; `vm_secure_boot_fail: true` / `vm_datastore_fail: true` mark the job failed instead, for a notification on failure (the *ServiceNow tickets* job does not take these reports' results). `vm_secure_boot_folder` limits the secure boot report to one vCenter folder; `vmware_datacenter` limits either report to one datacenter; `vm_datastore_warn_pct` / `vm_datastore_crit_pct` (85 / 90) set the datastore thresholds.

# Snapshots: the report, the cleanup, and the approval workflow

## The idea, in one picture

```
1. SNAPSHOT REPORT          looks at every VM and makes a list of its snapshots.
        |                   It changes nothing. It emails you the list.
        v
2. APPROVAL                 AAP stops and waits. A person reads the list and
        |                   clicks Approve (go ahead) or Deny (stop).
        v  (only on Approve)
3. SNAPSHOT CLEANUP         deletes the old snapshots that were ON THAT LIST -
                            nothing else - and emails what it deleted.
```

Think of it like cleaning out a fridge: first someone writes down everything old in it (the report), then a grown-up says "yes, throw those out" (the approval), then the helper throws out **only what is on the paper** (the cleanup). Anything someone put a "keep" label on stays.

## What the report shows

For every snapshot of every VM: the VM, the snapshot's name, when it was taken, **who took it**, how many days old it is, how big it is (MB, GB or TB), the VM's folder and the description. Oldest first. **Every snapshot is in that list, whatever its age** - even one taken an hour ago. Then two short lists on top:

- **To delete:** snapshots older than 5 days (`vm_snapshot_max_age_days`).
- **Held back:** old snapshots the cleanup will never delete, each with the reason.

The email's heading and subject always say the limit, e.g. `VMware snapshot report (older than 2 days): 4 to delete (310 GB); 23 snapshot(s) on 17 VM(s)`.

**To use another number of days than 5:**

| Where | Change |
|---|---|
| In the workflow | the survey answer when you launch it - or for good: the workflow > **Survey** tab > the question > **Default** 2. The report and the cleanup both use it: after the approval, snapshots older than 2 days are deleted. |
| The report template on its own | Templates > `VM - snapshot report` > Edit > **Variables**: `vm_snapshot_max_age_days: 2` |

In the workflow, the survey answer wins over the template's Variables.

"Who took it" comes from vCenter's history. vCenter forgets old history (often after 30 days), so for an old snapshot it may say **unknown**. Snapshots taken by the "VM - snapshot" job also have the AAP user in their description.

### What the cleanup never deletes

| Never deleted | Why | Change it with (in playbooks/group_vars/all.yml) |
|---|---|---|
| A snapshot whose name or description says **keep** or **do not delete** | someone wants it kept | `vm_snapshot_keep_regex` |
| Any snapshot of the VMs on your do-not-touch list | domain controllers, vCenter, ... | `vm_snapshot_cleanup_exclude_vms: [DC*, VCSA*]` (* = anything) |
| Any snapshot of a VM in a datacenter on your list | a datacenter (DR site, a lab) whose snapshots someone else looks after | `vm_snapshot_cleanup_exclude_datacenters: [DRSITE]` (the datacenter's name in vCenter) |
| Any snapshot of a VM running in a cluster on your list | a cluster (SQL, management) whose snapshots are done by hand | `vm_snapshot_cleanup_exclude_clusters: [SQL*, MGMT*]` (a VM on a host in no cluster is not matched) |
| Any snapshot of the VMs in `vmware_protected_vms` | the same protected VMs as for restart | `vmware_protected_vms` |
| Any snapshot of the AAP server | AAP must never be put at risk by AAP | (always) |
| With words in `vm_snapshot_cleanup_name_words`: a snapshot whose **name** contains none of them | only the patching snapshots are cleaned up (any case; `Patch` also matches `Patching`, `Patches` - and `dispatch`). Empty (the default) = any name | `vm_snapshot_cleanup_name_words: [Patching, Patches, Patch]` |
| A snapshot bigger than **1 TB**, or whose size is unknown | merging a huge snapshot can take hours and slow the VM: do it by hand, at a quiet time | `vm_snapshot_cleanup_max_size_gb: 1024` |
| More than 50 snapshots in one run | a typo in the days would otherwise delete everything | `vm_snapshot_cleanup_max: 50` |
| Anything that was not on the approved list | the approver only said yes to that list | (always, in the workflow) |

### Set it up (once)

**Step 0 - what you need first** (see "Before you start" above): the VMware vCenter credential, the execution environment with `vmware.vmware`, and the mail relay in `all.yml` ("Email the report").

**Step 1 - your do-not-touch list.** In VS Code, in `playbooks/group_vars/all.yml`, add (with your VM names; `*` matches anything) - then Commit, Sync Changes, and sync the project in AAP:

```yaml
vm_snapshot_cleanup_exclude_vms: [DC*, VCSA*, AAP*]
```

**Step 2 - the report template.** Templates > Create template > Create job template:

- Name: `VM - snapshot report`
- Inventory: your inventory with the `aap` group
- Project: site-automation. Playbook: `playbooks/vm_snapshot_report.yml`
- Execution environment: the one with `vmware.vmware`
- Credentials: your **VMware vCenter** credential
- Leave Limit empty. No survey. Save.

**Step 3 - the cleanup template.** The same again:

- Name: `VM - snapshot cleanup`
- Playbook: `playbooks/vm_snapshot_cleanup.yml`
- Tick **Prompt on launch** next to **Job type** (this lets you do a dry run - below).
- No survey. Save.

**Step 4 - the workflow.** Templates > Create template > **Create workflow job template**:

- Name: `VM snapshot cleanup (with approval)`, your organization. Save.
- **Survey** tab > Create survey question - two questions, then switch the survey **on**:

| Question | Answer variable name | Answer type | Default |
|---|---|---|---|
| Delete snapshots older than how many days? | `vm_snapshot_max_age_days` | Integer, required, minimum 1 | `5` |
| Email the reports to | `report_email_to` | Text | your team's address |

The workflow hands these answers to both jobs, so the report and the cleanup use the same days.

- **Visualizer** (the boxes):

1. Click **Add step**. Node type: **Job template**. Pick `VM - snapshot report`. Save.
2. Hover over that box > **Add step and link** (the plus sign). Node type: **Approval**. Name: `Delete the snapshots listed in the report?`. Timeout: e.g. `1 day` (no answer by then = no). Run type: **Run on success**. Save.
3. Hover over the approval box > **Add step and link**. Node type: **Job template**. Pick `VM - snapshot cleanup`. Run type: **Run on success** (after an approval, that means: approved). Save. **Never Run always here:** the cleanup would then also run when the approval is denied or times out - and it still gets the report's list, so it would delete. (Check the link's label in the Visualizer: green *Run on success*, not blue *Run always*.)
4. Click **Save** at the top of the Visualizer.

**Step 5 - who may approve.** On the workflow: **User Access** (or Team Access) > Add > pick the people > role **Approve**. They need no other rights. Optional: **Notifications** tab > turn on the approval notification so they get an email when something waits for them.

**Step 6 - when it runs.** On the workflow: **Schedules** > Create schedule (e.g. every Monday 7:00). Or launch it by hand.

### Run it

1. Launch the workflow (or the schedule does). Answer the survey (or keep 5 days).

2. The report runs and **emails the list**: what will be deleted, what is held back, and why.
3. AAP waits at the approval. Open **Automation Execution > Administration > Workflow Approvals** (or the workflow job's view) > the approval > **Approve** or **Deny**.
4. On **Approve**: the cleanup deletes the snapshots from that list that are still there, and still that old, and **emails what it deleted** (and how much space came back). On **Deny** or timeout: nothing is deleted.

The cleanup's email can be sent only when something was actually deleted: `vm_snapshot_cleanup_email_only_if_deleted: true` (in the cleanup template's Variables).

### Dry run: try it, delete nothing

- **The safest first try: say no.** Launch the workflow, read the report email, then **Deny** the approval. The report only reads, so nothing changed anywhere.
- **See exactly what the cleanup would delete, without deleting:** in the workflow's Visualizer, click the `VM - snapshot cleanup` box > **Edit** (pencil) > Job type **Check** > Save, and save the Visualizer. Run the workflow and **Approve**: the cleanup job's output lists `WOULD DELETE ...` for each snapshot and deletes nothing (and sends no email). Set the box back to **Run** afterwards. (This needs the Step 3 tick "Prompt on launch" for Job type.)
- **Without the workflow:** launch `VM - snapshot cleanup` on its own. With no approved list and no `vm_snapshot_cleanup_confirm: true` it only lists what it would delete - it never deletes.

### Without the workflow

To clean up by hand, make a separate template (not the one in the workflow) on `playbooks/vm_snapshot_cleanup.yml` with a survey: `vm_snapshot_max_age_days` (Integer, 5) and `vm_snapshot_cleanup_confirm` ("Delete them?", Multiple choice `false` / `true`, default `false`). With `false` it only lists; with `true` it deletes - with every rule above still in place.

#### Only the patching snapshots, without the approval

Until the workflow's approval works, the cleanup can run on its own, on a schedule, and delete **only** the snapshots whose name contains `Patching`, `Patches` or `Patch` - every other snapshot is left alone and listed as held back, with the reason `its name has none of: Patching, Patches, Patch`.

1. In `playbooks/group_vars/all.yml` (so the snapshot report lists the same ones as "To delete"):

```yaml
vm_snapshot_cleanup_name_words: [Patching, Patches, Patch]
```

Any case matches, anywhere in the name: `Pre-Patching 10/14`, `PATCHES oct`, `patch-web01`. The name only, not the description. `Patch` alone already matches the other two; keep all three if you like the list to say what you mean. A phrase works too: `[Patch Tuesday]`.

2. Make a separate template (not the one in the workflow): `VM - snapshot cleanup (patching)`, playbook `playbooks/vm_snapshot_cleanup.yml`, credential `VMware vCenter`, **Variables**:

```yaml
vm_snapshot_cleanup_confirm: true
```

3. **First, a dry run:** launch it with Job type **Check** (tick "Prompt on launch" for Job type). The output and email list `WOULD DELETE ...` for each patching snapshot and `HELD BACK ...` for the others - nothing is deleted. Check the list; then launch it for real, and schedule it (e.g. daily, after the patching window).

Still in force: the age (`vm_snapshot_max_age_days`, 5 - a patching snapshot taken yesterday waits; `0` = any age), the keep words, the do-not-touch VMs, the AAP server, the 1 TB limit and at most 50 in one run. The email's subject says it: `VMware snapshot cleanup: deleted 4 snapshot(s) older than 5 days whose name has Patching, Patches or Patch (120 GB)`. When the workflow works, the setting can stay: the report then lists only the patching snapshots for approval - or set it back to `[]`.

## Pick lists: choose what to restart, which snapshots to delete

The other jobs ask for VM names typed in. Here whoever launches the job **picks from a list** in the survey - the snapshots that may be deleted right now, or the VMs that may be restarted (or only the ones that look hung) - and ticks the ones to do.

### Why the list needs a job of its own

AAP cannot fill a survey from vCenter when you click Launch: a survey's choices are fixed text, saved with the template. And inside a workflow nothing can ask a question halfway: an **approval step takes no input** (only Approve or Deny), and the surveys of the job templates inside a workflow are **never asked** - only the workflow's own survey, at launch. So the picking happens at launch, from a multiple-choice question whose choices a scheduled job keeps current:

```
VM - refresh the pick lists     every 30 minutes: reads vCenter, writes what may be done NOW as
  the
       (schedule)               choices of the question in the survey of the templates you name
       .
       .   later, a person launches:
       v
workflow survey                 ticks the snapshots (or VMs) in the list
       |
1. PLAN  (the pick job)         finds each pick again in vCenter, live, and lists exactly what
  will be
       |                        done - the ones that are gone or no longer allowed are skipped,
         with
       |                        the reason. Changes nothing.
2. APPROVAL                     a person reads the plan and clicks Approve or Deny
       |  (only on Approve)
3. DO IT (the same pick job)    does exactly the plan - checked live once more - nothing else
```

Each choice is readable text that ends with the item's IDs in brackets, e.g.

```
DC0_C0_RP0_VM1: "Pre-Patching-Oct" (12.4 GB, 6.2 days old, taken 2026-10-14 08:12 UTC)
  [vc01.yoursite.mil/vm-42/7]
web01 (no heartbeat, up 3.2 days, DC1 / PROD-CL01, guest web01.yoursite.mil 192.0.2.20)
  [vc01.yoursite.mil/vm-42]
```

The brackets are the vCenter, the VM's ID and (for a snapshot) the snapshot's ID: two snapshots with the same name, or two VMs with the same name in different vCenters, are never mixed up.

### What may be picked

| List | Offered | Never offered (counted in the refresh's output, with the reason) |
|---|---|---|
| Snapshots | every snapshot the **snapshot cleanup could delete**: the same holds (table "What the cleanup never deletes" above), at least `vm_pick_snapshot_min_age_days` old (0 = any age) | a name or description that says keep, the excluded VMs, datacenters and clusters, `vmware_protected_vms`, the AAP server, bigger than 1 TB. **Not** the name words: they only narrow the automatic cleanup |
| VMs | every VM that is **powered on**, among `vm_pick_reboot_vms` / `vm_pick_reboot_clusters` / `vm_pick_reboot_folders` (empty = all) | templates, VMs that are off, the AAP server and the protected VMs (the same rules as every restart), `vm_pick_reboot_exclude_vms` |
| Hung VMs (`vm_pick_reboot_only_hung: true`) | powered on, VMware Tools **installed**, but **no heartbeat** (red in vCenter) or Tools **not running** - and up for `vm_pick_hung_min_minutes` (15) at least | a VM that is still starting (it has neither yet), a VM without VMware Tools (vCenter cannot tell if it is hung) |

A list has at most `vm_pick_max_choices` (200) choices; the refresh's output says how many more there were. With nothing to pick, the list has one line `(nothing to pick: ... - list of <time>)`, which the pick jobs ignore. The question's description ends with when the list was made, e.g. `[pick list of 2026-10-20 10:00 UTC: 12 snapshot(s) that may be deleted]`.

### Set it up (once)

**Step 1 - the pick job templates.** Templates > Create template > Create job template, twice:

| Name | Playbook | Credentials | Job type |
|---|---|---|---|
| `VM - delete picked snapshots` | `playbooks/vm_snapshot_delete_picked.yml` | VMware vCenter | tick Prompt on launch (dry runs) |
| `VM - restart picked VMs` | `playbooks/vm_reboot_picked.yml` | VMware vCenter | tick Prompt on launch (dry runs) |

Inventory: your inventory with the `aap` group. Execution environment: the one with `vmware.vmware`. Leave Limit empty. **Do not tick Prompt on launch for Variables** (the step after the approval must get its plan from the step before, not from whoever launches). No survey.

**Step 2 - the workflows**, one per list. Templates > Create template > Create workflow job template:

- `VM - delete picked snapshots (workflow)`. **Survey** tab > Create survey question:

| Question | Answer variable name | Answer type | Required | Choices |
|---|---|---|---|---|
| Snapshots to delete | `vm_snapshot_picks` | **Multiple choice (multiple select)** | yes | type one placeholder, e.g. `(nothing to pick yet)` - the refresh replaces it |

- `VM - restart picked VMs (workflow)`. **Survey**:

| Question | Answer variable name | Answer type | Required | Choices / default |
|---|---|---|---|---|
| VMs to restart | `vm_reboot_picks` | **Multiple choice (multiple select)** | yes | a placeholder, as above |
| How | `vm_power_mode` | Multiple choice | yes | `guest`, `hard`; default `guest` (`hard` for the hung list) |

Switch each survey **on**.

- **Visualizer** of each: (1) **Add step**: Job template `VM - delete picked snapshots` (or `restart picked VMs`). (2) On it, **Add step and link**: **Approval**, e.g. `Do exactly this plan?`, timeout e.g. 4 hours, Run type **Run on success**. (3) On the approval, **Add step and link**: the **same** job template again, Run type **Run on success** - never Run always. Save.

The first run of the job plans (inside a workflow it never deletes or restarts before an approval, whatever its Variables say); its plan goes to the approval and on to the second run (job artifact `vm_snapshot_pick_plan` / `vm_reboot_pick_plan`), which does exactly that. When nothing that was picked can be done any more, the plan step fails and the workflow stops there.

**Step 3 - who may launch and approve.** On each workflow: User Access > the service desk > role **Execute** (launch) and the approvers > role **Approve**.

**Step 4 - the refresh.** One more job template:

- Name: `VM - refresh the pick lists`. Playbook: `playbooks/vm_pick_refresh.yml`.
- Credentials: **VMware vCenter** and **Red Hat Ansible Automation Platform** (its AAP user needs the **Admin** role on the two workflows - to write their surveys).
- **Variables** (`aap/extra_vars/vm_pick_refresh.yml` has every setting):

```yaml
vm_pick_snapshot_templates: ["VM - delete picked snapshots (workflow)"]
vm_pick_reboot_templates: ["VM - restart picked VMs (workflow)"]
# vm_pick_reboot_only_hung: true        # the restart list: only the hung VMs
# vm_pick_reboot_clusters: [PROD-*]     # ... of these clusters
```

A job template and a workflow with the same name: `{template: "<name or ID>", kind: workflow}`.

- **Schedules** > every 30 minutes (or what suits you). Run it once by hand first: its output lists every choice it wrote, and what it held back and why.

To have both a list of every VM and a list of only the hung ones, make two refresh templates (one with `vm_pick_reboot_only_hung: true`) writing into two workflows.

### Run it

1. Launch the workflow. The survey shows the list as it was at the last refresh (its time is in the question's description). Tick what to do; pick **How** for a restart.
2. The plan step lists `WOULD DELETE ...` / `WOULD RESTART ...` for each pick that can still be done, and `SKIPPED ... - <reason>` for the others: deleted or renamed since the list was made, held back now, powered off, no longer hung, a guest restart of a VM without VMware Tools running.
3. The approver reads that plan (the plan step's output) and approves or denies.
4. On **Approve**: the same checks again, then exactly the plan - `DELETED ...` / `RESTARTED ...`.

Never more than `vm_pick_max` (20) in one run: more picks are refused, nothing done. As **Check** (a dry run), no step changes anything.

**Without a workflow:** launch `VM - delete picked snapshots` or `VM - restart picked VMs` on its own with a survey of its own (the same question, and name that template in the refresh's settings). It only lists the plan, unless `vm_pick_confirm: true` (a survey question "Do it now?", `false` / `true`) - then it does the picks at once, with every check above. Inside a workflow `vm_pick_confirm` is never used.

### Limits

- **The list is as fresh as the last refresh.** Every pick is checked again, live, before anything is done - a pick that is gone or no longer allowed is skipped, never guessed - but something new since the refresh is not in the list until the next one.
- **The approval is all or nothing.** The approver approves the whole plan or none of it. To leave one out: Deny, launch again and pick fewer.
- AAP checks a survey answer against the choices: if the refresh replaced the list while someone had the survey open, AAP may refuse their launch - open the survey again.
- The heartbeat comes from VMware Tools. A VM without VMware Tools is never in the hung list.
- Tested against vcsim and a fake AAP (`tests/vmware/run_vmware_test.sh`, `tests/test_aap_survey.py`), not a real AAP survey: run the refresh once by hand and look at the workflow's survey before you schedule it.

## Health: every datacenter, cluster and host

The read-only **VM - health report** answers "is the virtual infrastructure healthy?" for every vCenter (the credential's and `vmware_vcenters`), every datacenter, every cluster and every host - in one email, coloured by severity, with what to do for each problem. The alarms themselves, failed logins and the day's events are in the **alarms report** below: run both every morning (workflow 18 in WORKFLOWS_AND_SCHEDULES.md).

### What the report shows

| Section | What is in it |
|---|---|
| Needs attention | every problem found, most severe first: severity, the object (vCenter, datacenter, cluster or host), the problem, **what to do**, and where it is (datacenter / cluster) |
| Datacenters | each datacenter with everything in it: clusters, hosts, hosts running VMs, in maintenance, not connected, its problems |
| Clusters | each cluster: hosts running VMs, **vSphere HA** (on or OFF, what admission control reserves, host agents not OK), **DRS** (on or OFF, automation level, score, recommendations waiting), **vCLS** (normal or RETREAT MODE), CPU and memory used, alarms, problems |
| Hosts | every host, by datacenter and cluster (standalone hosts too): connection, maintenance mode, the **HA agent** (primary, secondary, isolated, partitioned ...), **hardware** (sensors and hardware status not green), CPU and memory used, days up, ESXi version and build, alarms |
| vCenter | each vCenter's version and the alarms on vCenter itself |

On top: the counts in coloured boxes (hosts not connected, HA problems, DRS problems, hosts with hardware faults, triggered alarms). The subject says it in one line, e.g. `[AAP] VMware health report: 2`

`critical, 3 warning(s) - 3 datacenter(s), 7 cluster(s), 26 host(s).`

| Colour | What |
|---|---|
| **Red (critical)** | a host not connected to vCenter (or powered off); a host's HA agent isolated, partitioned, unreachable or in error; vCLS retreat mode on a cluster with DRS (DRS stops); a red hardware sensor; a red alarm; CPU or memory from 95 % |
| **Amber (warning)** | HA or DRS off on a cluster of 2 or more hosts; HA host monitoring off; HA failover capacity below what admission control reserves; DRS recommendations waiting, or DRS faults; retreat mode without DRS; a yellow sensor; a host that needs a restart; a configuration issue (SSH on, ...); a yellow alarm; CPU from 85 %, memory from 90 % |
| **Blue (info)** | a host in maintenance mode or standby; HA admission control off; DRS in manual mode - worth knowing, not broken |

A one-host cluster without HA or DRS is fine, and so is a host in maintenance mode whose HA agent is not configured (that is what maintenance mode does).

### Set up the report

1. **Templates > Create template > Create job template:**
   - Name: `VM - health report`
   - Inventory: any (the job runs on the AAP side)
   - Project: site-automation. Playbook: `playbooks/vm_health_report.yml`
   - Execution environment: your VMware one
   - Credentials: your **VMware vCenter** credential. A read-only vCenter account is enough.
   - Variables: `report_email_to: vmteam@yoursite.mil` (and `vmware_vcenters: [...]` for more vCenters, if not already in `all.yml`)
   - Save, then **Launch** it once and read the email.
2. **Schedules > Create schedule:** e.g. every day at 06:00 - or put it in front of the alarms report in workflow 18.

| Setting (template Variables) | Default | What it does |
|---|---|---|
| `vm_health_require_ha` | `true` | a cluster of 2 or more hosts without vSphere HA is a warning; `false` if that is on purpose |
| `vm_health_require_drs` | `true` | the same for DRS; `false` where DRS is not licensed (e.g. vSphere Standard) |
| `vm_health_cpu_warn_pct` / `vm_health_cpu_crit_pct` | `85` / `95` | CPU used (a host, or a cluster's hosts running VMs): amber from, red from |
| `vm_health_mem_warn_pct` / `vm_health_mem_crit_pct` | `90` / `95` | the same for memory |
| `vm_health_sensors` | `true` | read the hosts' hardware sensors |
| `vm_health_fail` | `false` | `true` = the job shows **failed** on a critical problem, for a workflow |
| `vmware_datacenter` | (all) | one datacenter only |

**vCLS retreat mode** is read from vCenter's advanced settings (`config.vcls.clusters.<cluster id>.enabled = false`). If the account may not read them, the report says so in blue, and retreat mode still shows as vCenter's own alarm on the cluster.

## VMs created: the weekly report

The read-only **VM - VMs created report** answers "how many VMs were created this past week?" for every vCenter (the credential's and `vmware_vcenters`): the count up front, then each VM.

| Section | What is in it |
|---|---|
| On top | VMs created, of them by AAP from tickets, deleted since; vCPU, memory and disk added |
| VMs created, oldest first | when (UTC), the VM, **by whom**, **how** (deployed from template X, cloned from Y, created new), the **ticket** when the AAP deploy built it, cluster, folder, vCPU, memory, disk, operating system, power |
| By whom | each person (account) and how many VMs - and how many of them came from tickets |
| By day | how many VMs each day |
| Created and deleted since | VMs created in the week that are gone now (counted on top too), and when they were deleted |

The subject says it in one line, e.g. `[AAP] VMware: 5 VMs created in the last 7 days (3 by AAP from tickets, 1 deleted since)`.

- **When:** the VM's own creation date (vSphere 6.7 and later), else its creation event. **By whom and how:** vCenter's events. vCenter keeps events for a limited time (often 30 days): a week is well inside that. A VM whose event is gone still counts, by its date, with "By" empty.
- **The window** is the last `vm_created_days` (7) days up to the moment the job starts. Run every Friday at noon, each report covers about Friday noon to Friday noon (a job that starts a minute late moves the window by that minute).
- **The ticket** comes from the notes the deploy writes on each VM - their first line, *"INC0012345 - built 2026-10-20"*, and *"Template: ..."* under it (VMs built by older releases: *"Deployed from ... for INC... by ..."*); a VM built by hand has none. Templates are not counted.

### Set up the report, every Friday at noon

1. **Templates > Create template > Create job template:**
   - Name: `VM - VMs created report`
   - Inventory: any (the job runs on the AAP side)
   - Project: site-automation. Playbook: `playbooks/vm_created_report.yml`
   - Execution environment: your VMware one
   - Credentials: your **VMware vCenter** credential. A read-only vCenter account is enough (it reads the VMs and the events).
   - Variables: `report_email_to: vmteam@yoursite.mil` (and `vmware_vcenters: [...]` for more vCenters, if not already in `all.yml`)
   - Save, then **Launch** it once and read the email.

2. **Schedules tab > Create schedule:**
   - Name: `Weekly - VMs created`
   - Start date/time: the next Friday, **12:00**, and **Time zone: your site's** (not UTC - so 12:00 is your noon)
   - Repeat frequency: **Week**, every **1** week, on **Friday**
   - Save. The schedule's next run shows on the template's Schedules tab.

| Setting (template Variables) | Default | What it does |
|---|---|---|
| `vm_created_days` | `7` | the window: the last this many days (e.g. `14` for a report every other Friday, `30` for a monthly one) |
| `vmware_datacenter` | (all) | one datacenter only |

## Alarms: the report, and ACT's analysis

Two read-only jobs. The **report** shows what is wrong in vCenter now and what happened in the last 24 hours. The **ACT analysis** has ACT (the AI assistant) work out the likely cause and the fix of each of those problems, and say how sure it is. Neither changes anything.

### What the report shows

| Section | What is in it | Colour |
|---|---|---|
| Triggered alarms | the alarms vCenter shows now on vCenter itself, the datacenters, clusters and hosts: severity, object, alarm, since when, acknowledged by whom | red = critical, amber = warning |
| Alarms that fired in the last 24 h | every alarm that turned yellow or red in the window, one row per alarm and object: how many times it went red and yellow, first and last change, and how it is now - still on, or cleared since (an alarm that comes and goes is worth a look too) | red / amber = still on, blue = cleared |
| Host connection | hosts not connected to vCenter now, and hosts that lost the connection in the last 24 h and came back | red / amber |
| Failed logins | wrong user name or password: one row per user, source address and server, with the count and the first and last time | amber; red from 10 failures |
| Virtual machine events | guest shutdowns and restarts, power-offs, resets, HA restarts and failures, and who did it | blue = someone did it, amber = HA or nobody, red = a failover or power-on failed |
| Other errors and warnings | every other error and warning event; the same event on the same object counted once | red = error, amber = warning |
| Configuration issues | what vCenter shows as configuration issues (SSH left on, HA problems, ...) | amber |
| All hosts | every host: connection, maintenance mode, ESXi version and build, hardware, its alarms and its errors / warnings | red / amber / blue row |

On top: the counts in coloured boxes. The subject says the most important ones, e.g. `[AAP] VMware alarms report: 1 critical, 2 warning alarm(s), 47 failed login(s) (events: last 24 h)`.

### Set up the report

1. **Templates > Create template > Create job template:**
   - Name: `VM - alarms report`
   - Inventory: any (e.g. your VMware one: the job runs on the AAP side)
   - Project: site-automation. Playbook: `playbooks/vm_alarm_report.yml`
   - Execution environment: your VMware one (this job needs only pyVmomi, which it has)
   - Credentials: your **VMware vCenter** credential. A read-only vCenter account is enough.
   - Variables: `report_email_to: vmteam@yoursite.mil`
   - Save.
2. **Schedules > Create schedule:** e.g. every day at 06:00.

| Setting (template Variables) | Default | What it does |
|---|---|---|
| `vm_alarm_hours` | `24` | the events of the last this-many hours (168 for a weekly report; 0 = alarms and hosts only) |
| `vm_alarm_types` | `[vcenter, datacenter, cluster, host]` | whose alarms; add `vm`, `datastore` or `network` for theirs too |
| `vm_alarm_login_critical` | `10` | this many failed logins of one user from one place, or more, is red |
| `vm_alarm_fail` | `false` | `true` = the job shows **failed** on a critical alarm or a host not connected, for a workflow |
| `vmware_datacenter` | (all) | one datacenter only |

### The ACT analysis

**What you get:** an email "VMware alarms ACT analysis". First "What to do first", then one row per problem (P1, P2, ...): severity, object, the problem, the **likely cause**, the evidence, the **fix** and a **confidence**. The confidence is ACT's own estimate, from 0 to 100: green 80 and up (the evidence shows it), amber 50-79 (likely), red under 50 (a guess: check first). These are likely causes, not certain ones: check before you change anything. ACT runs no command and changes nothing.

**Where ACT runs, and what about the hosts?** ACT runs inside this job's execution environment, on the AAP server (or the execution node that runs the job). Not on the ESXi hosts, not on the VMs. So:

- The **ESXi hosts need nothing**: no Python, no network path to ACT.
- **Python** is in every execution environment (Ansible itself runs on it; ACT needs 3.8 or later).
- The **AAP node must reach the model's address over HTTPS** (port 443): `https://api.genai.mil` for GenAI.mil, or your `site_act_url`. The job checks this first and says plainly when it cannot:
- *"cannot reach the model ... Open HTTPS ... or set a proxy"*: ask for a firewall rule from the AAP node to that address, or set your proxy in `playbooks/group_vars/all.yml`: `site_act_env: {HTTPS_PROXY: "http://proxy.yoursite.mil:8080", NO_PROXY: "localhost,.yoursite.mil"}`
- *"Its certificate is not trusted"*: the execution environment does not trust the model's certificate authority. Put that CA certificate (public, `.pem`) at `playbooks/files/ca/model-ca.pem`, add `site_act_ca: "{{ playbook_dir }}/files/ca/model-ca.pem"` to `all.yml`, and add the line `playbooks/files/ca/` to `.site-local` (so an update keeps it).

**Privacy:** before anything leaves, ACT replaces host, cluster, datacenter and VM names, IP addresses and user names with placeholders, and puts the real names back into its answer.

**Set it up:**

1. **The key.** If your other ACT jobs have an **ACT model key** credential, use it. If not: create the credential type from `aap/credential_types/act_model_key.yml`, then a credential of that type with your GenAI.mil (or Ask Sage) key (SECRETS.md).
2. **Templates > Create template > Create job template:**
   - Name: `VM - alarms ACT analysis`

- Inventory, project, execution environment: as for the report. Playbook: `playbooks/vm_alarm_act_analysis.yml`
- Credentials: **VMware vCenter** and **ACT model key** (both)
- Tick **Prompt on launch** next to **Job type** (for the dry run)
- Variables: `report_email_to: vmteam@yoursite.mil`
- Save. The provider and model are your usual ACT settings (`site_act_provider`, `site_act_models` in `all.yml`); the default is GenAI.mil.

3. **First run, as a dry run:** Launch, Job type **Check**. It reads vCenter, lists the problems it would give ACT, and calls nothing. Then launch it normally.

**Both in one go:** a workflow with `VM - alarms report`, then `VM - alarms ACT analysis` linked with **Always**. Each sends its own email. Or schedule just the ACT analysis: its email lists every problem too.

**Green or red:** nothing wrong = ACT is not called and no email is sent (`vm_alarm_act_email_if_none: true` sends one anyway). The job is **red** when ACT could not run (no key, no network, an ACT error): the email still lists the problems, and says why.

| Setting | Default | What it does |
|---|---|---|
| `vm_alarm_act_max_items` | `40` | give ACT at most this many problems, the most severe first (the rest are listed as not analyzed) |
| `vm_alarm_act_config_issues` | `true` | configuration issues too |
| `vm_alarm_act_check_url` | `true` | check that the AAP node can reach the model first |
| `vm_alarm_act_model_timeout` | `site_act_model_timeout` (600) | seconds the model may take for its answer (a `GENAI_TIMEOUT` in `site_act_env` wins) |
| `site_act_timeout` | `1200` | seconds the whole ACT run may take (keep it above the model's) |

## Outage watch: what is down or hung right now

`playbooks/vm_outage_watch.yml`, every 5 to 15 minutes, in a workflow before the ServiceNow tickets job. It reads only what means an outage: hosts not responding, HA agents isolated, datastores not accessible, all paths down and device loss, HA restarts and failures, VMs waiting on a question, and hung VMs. A VM is hung only when it has no guest heartbeat AND does not answer a ping (or `vm_watch_probe_port`) from the AAP node, in every check of the run. A clean run is one line, with no email. Watch only: it restarts nothing, and lists the hung VMs a later phase would restart, so the false alarms can be counted first. Everything - what it reads, the settings, the workflow, the phase 2 plan - is in VM_OUTAGE_WATCH.md.

## Capacity planning: overall and per cluster

`playbooks/vm_capacity_report.yml` answers "how much room is left, and for how long?" - for all clusters together and for each cluster. It only reads. The history comes from **vCenter's own daily statistics** (it keeps a year of them at the default statistics level), so it works from the first run: nothing has to be collected first.

### What the report shows

| Section | What it shows |
|---|---|
| **Overall** | all clusters together: CPU and memory capacity, used, used with a host down in each cluster, growth per month and the days until the target; hosts usable and **hosts recommended** (now, and within 12 months); vCPUs per core and room for more VMs; shared storage with its growth and runway |
| **Clusters: CPU and memory** | per cluster: hosts usable, CPU and memory used, **memory used if the biggest host fails** (N+1: the cluster must still run everything), vCPUs per core, VMs and their average size, memory growth, **memory runway** (days and date until the target with a host down), **room for more VMs** of the average size (and what limits it), **hosts recommended**, VMs added / removed in the last 30 days |
| **Clusters: storage** | per cluster, its shared datastores together: capacity, free, used, provisioned (thin disks at full size), growth, days until 85 % and until full |
| **Datastores: runway** | each shared datastore, the one that fills first on top: growth, days (and date) until 85 %, 90 % and full, and how well a straight line fits its history |
| **Math vs GenAI** (with `vm_capacity_act: true`) | for every item, the job's runway (the math) next to ACT's own estimate (GenAI), whether they agree, ACT's recommendation and reasoning, and its confidence |
| **Hosts per cluster** | every host: state, cores, CPU and memory used, VMs - an unbalanced cluster shows here |

**Hosts recommended** is a count of hosts of the cluster's average size (cores, memory) that keeps memory and CPU at their targets (80 %) with one host down - for example `1 now (memory 59% on average then); 2 within 12 months`. "Within 12 months" applies the current growth. It is a sizing guide in hosts, not a list of models.

**How the runway is worked out:** a straight line through the last 30 days of vCenter's daily values (memory consumed, CPU used, datastore space used), continued until it reaches the limit. "Not growing" means the line is flat or falling. The **trend fit** ($R^2$) says how straight the history is: poor means jumps (VMs moved, snapshots deleted, a datastore grown), and the runway is a rough guide then.

### Math vs GenAI

With `vm_capacity_act: true`, ACT (the AI assistant, GenAI.mil or Ask Sage) reads the same numbers and makes **its own estimate** for the overall picture, each cluster and each datastore: when it needs action (in days), what to do, why, and how sure it is. The report puts the two side by side:

| Math (straight line) | GenAI (ACT) | Agreement |
|---|---|---|
| 102 days (90 % full) | 55 days | GenAI 47 days sooner |
| 0 days (memory) | 0 days | agree |

They **agree** when they are within 30 days or 25 % of each other. Where they differ, read ACT's reasoning (a recent jump, a limit that comes first, a poor fit), and plan for the sooner one. ACT states needs in hosts, cores, GB and TB, and timing in days or months. It runs on the AAP side and runs no command (as in "The

ACT analysis" above: the same key, URL, proxy and CA settings).

### Set it up

1. **Templates > Create template > Create job template:**
   - Name: `VM - capacity planning`
   - Inventory: any (e.g. your VMware one). Project: site-automation. Playbook: `playbooks/vm_capacity_report.yml`
   - Execution environment: your VMware one. Credentials: **VMware vCenter** (read-only is enough)
   - Tick **Prompt on launch** next to **Variables** (to try one cluster)
   - Variables: `report_email_to: vmteam@yoursite.mil`
   - Save. Launch it once and read the report.
2. **Per cluster:** the same report for one or a few clusters - Variables `vm_capacity_clusters: [PROD-CL01]` (`*` works: `[PROD-*]`). A second template with that variable gives each cluster its own report.
3. **GenAI's estimate (optional):** add the **ACT model key** credential to the template, and `vm_capacity_act: true` to its Variables. Launch.
4. **Schedule: Schedules > Create schedule**, e.g. every Monday at 07:00 (your time zone).

| Setting | Default | What it does |
|---|---|---|
| `vm_capacity_clusters` | `[]` (all) | only these clusters |
| `vm_capacity_failover_hosts` | `1` | hosts down to plan for (N+1); a larger HA admission control failover level wins |
| `vm_capacity_mem_target_pct` / `_cpu_target_pct` | `80` / `80` | memory / CPU used at most this, with those hosts down |
| `vm_capacity_vcpu_per_core_max` | `4` | vCPUs per physical core (amber from here) |
| `vm_capacity_storage_warn_pct` / `_crit_pct` | `85` / `90` | datastore used % (the storage runway is to 90 %) |
| `vm_capacity_runway_warn_days` / `_crit_days` | `90` / `30` | a runway shorter than this is amber / red |
| `vm_capacity_horizon_months` | `12` | "hosts recommended within N months" |
| `vm_capacity_trend_days` / `vm_capacity_history_days` | `30` / `90` | the days the trend is fitted on / read |
| `vm_capacity_include_local_datastores` | `false` | count a host's own datastores too |
| `vm_capacity_act` | `false` | GenAI's estimate next to the math |
| `vm_capacity_fail` | `false` | `true` = the job shows failed when a cluster or datastore is critical |

## Deploy a VM from a ServiceNow ticket

Four templates make up the VM deploy from a ServiceNow ticket - a change, its task, an incident or a catalog task:

| Job template | Playbook | What it does |
|---|---|---|
| VM - deploy from ServiceNow ticket | `playbooks/vm_deploy_from_ticket.yml` | reads the ticket, checks the request against your rules, builds each VM from a template, writes the result on the ticket. Check = a dry run |
| VM - deploy from ServiceNow ticket (workflow) | - (a workflow) | the deploy as *propose*, a person's approval, then *build* - one launch |
| VM - pick up deploy tickets | `playbooks/vm_deploy_poll.yml` | on a schedule: starts the workflow for each new VM request ticket. Builds nothing itself |
| VM - deploy settings finder | `playbooks/vm_deploy_inventory.yml` | read-only: your clusters, templates, datastores and folders, with the lines to paste |

**Its complete guide is its own document:** VM_DEPLOY_GUIDE.md, also a PDF - `docs/pdf/Site-Automation-VM-Deploy-Guide.pdf`. It has the setup in AAP step by step (the execution environment, the vCenter role and its privileges, the credentials, the job template and its survey, the approval workflow, the scheduled pickup, the server's service and its AAP inventory, launching from AAP's API), how a run works, what requesters write, where each VM goes (sites, name rules, automatic placement, empty templates for MECM or kickstart), ready-made configurations, every message and what to do, where the logic lives - and every `vm_deploy_*` setting with its default, an example and when to set it.

## Trust stores: vCenter's trusted roots and the ESXi hosts' trusted CAs

The *VMware - trust store report* (`playbooks/vmware_trust_report.yml`, a *VMware vCenter* credential, no inventory) reads vCenter's trusted root store (the same list as Administration > Certificates > Trusted Root Certificates; the account needs *Certificate Management* > *Manage* for the API call) and each ESXi host's trusted CA certificates (`vmware_trust_esxi: false`: vCenter only), and says which hold a compromised or dropped certificate authority - the same list as the trust check on the Linux hosts. Read-only; a hit fails the job; emailed as the other VMware reports. With `check_trust_remove: true` the chain is deleted from vCenter's trusted roots and each ESXi host's trusted CA list is replaced without it (Check mode shows what would go). TRUST_STORES.md.

## ESXi security settings (every night): SSH off, timeouts, lockdown

One job, `playbooks/esxi_security.yml`, sets these on every ESXi host, through vCenter:

| What | Setting | Default |
|---|---|---|
| **SSH** off: the service stopped, and set to start manually | `esxi_security_ssh` | `disabled` (`enabled`, or `""` = leave it) |
| **ESXi Shell** off (the host's local shell, a separate service from SSH): the same | `esxi_security_shell` | `disabled` (`enabled`, or `""` = leave it) |

| What | Setting | Default |
|---|---|---|
| **Timeouts** at 600 s (10 minutes): SSH and the ESXi Shell stop by themselves 600 s after someone starts them (`UserVars.ESXiShellTimeOut`), and an idle SSH or shell session is logged out after 600 s (`UserVars.ESXiShellInteractiveTimeOut`) | `esxi_security_settings` | those two, at `600` |
| **Lockdown mode normal**: the host is managed only through vCenter; the DCUI (the host's console) still works | `esxi_security_lockdown` | `normal` (`disabled`, or `""` = leave it). **strict is refused**, and a host already in strict is left alone |

It changes **only what differs**: a host that already matches is not touched. So it is safe to run every night: it puts back whatever someone changed (SSH turned on, a timeout changed, lockdown turned off) and the email says which host and what. A dry run (Check) changes nothing and lists what it would change. Hosts not connected to vCenter are skipped and listed.

**Another timeout?** `esxi_security_settings` is a list of advanced settings, name: value. Add or remove lines, e.g. the DCUI and Host Client idle timeouts:

YAML

```yaml
esxi_security_settings:
  UserVars.ESXiShellTimeOut: 600
  UserVars.ESXiShellInteractiveTimeOut: 600
  UserVars.DcuiTimeOut: 600
  UserVars.HostClientSessionTimeout: 600
```

### Before the first real run: who logs in to the hosts directly?

Lockdown mode **blocks direct logins** to the hosts: SSH, the Host Client (`https://<host>/ui`), and any tool that connects to a host instead of vCenter. Most often that is the **vulnerability scanner** with ESXi credentials, and sometimes a monitoring or backup tool. Ask those teams first. For each such account, add it to the host's **exception users**: the job adds them **before** it turns lockdown on, and never removes one.

YAML

```yaml
esxi_security_lockdown_exception_users: [svc_scanner, svc_monitor]
```

Exception users must be **local accounts on the host**, or **Active Directory accounts with their own permission on the host**. A `vsphere.local` account or an AD group does not work. Your vCenter service account for AAP is not affected: lockdown keeps vCenter's access.

### Set it up

1. **The vCenter account** needs the privileges in "Before you start", step 2 (the ESXi security settings row), on the hosts or their cluster.
2. **Templates > Create template > Create job template:**
   - Name: `ESXi - security settings`
   - Inventory: any (e.g. your VMware one). Project: site-automation. Playbook: `playbooks/esxi_security.yml`
   - Execution environment: your VMware one. Credentials: your **VMware vCenter** credential
   - Tick **Prompt on launch** next to **Job type** (for the dry run) and next to **Variables** (to try it on one host)

- Variables: `report_email_to: vmteam@yoursite.mil` and, if you have them, `esxi_security_lockdown_exception_users: [...]`
- Save.

3. **Dry run:** Launch, Job type **Check**. Read the report: every host and what it would change.
4. **One host for real:** Launch, Job type **Run**, Variables `esxi_security_hosts: [esx01.yoursite.mil]`. Check the host in the vSphere Client: **Configure > System > Services** (SSH and ESXi Shell stopped, policy Start and stop manually), **Advanced System Settings** (the two timeouts), **Security Profile > Lockdown Mode** (Normal). Have the scanner team run a scan of it.
5. **All hosts:** Launch again without `esxi_security_hosts`.
6. **Every night at midnight:** the template's **Schedules** tab > **Create schedule**:
   - Name: `Nightly 00:00`
   - Start date: tomorrow, start time **00:00**
   - **Time zone: yours** (e.g. `America/New_York`). AAP does not assume your local time: with the wrong zone, "midnight" runs at another hour.
   - Repeat: **Daily**, every 1 day. No end. Save.

The nightly email comes only when a host was changed or failed (an empty night sends nothing); `esxi_security_email_only_if_changed: false` emails every night.

### A host that needs SSH (a support case)

The nightly run would turn SSH off again. Exempt the host while the case is open, in the template's Variables:

```yaml
esxi_security_exclude_hosts: [esx07.yoursite.mil]
```

Then turn SSH on in the vSphere Client (and lockdown off if the vendor needs the Host Client). Remove the line when the case is closed: the next night sets the host back. To **undo** on purpose for every host: `esxi_security_lockdown: disabled`, and `esxi_security_ssh: ""` to leave SSH as it is.

| Setting | Default | What it does |
|---|---|---|
| `esxi_security_hosts` | `[]` (all) | only these hosts (names, `*` and `?` allowed; the short name works) |
| `esxi_security_clusters` | `[]` (all) | only the hosts of these clusters |
| `esxi_security_exclude_hosts` | `[]` | never these hosts |
| `vmware_datacenter` | (all) | one datacenter only |

## Certificates of vCenter and the ESXi hosts: incidents and CSRs (weekly)

With `cert_renew_servicenow: catalog` there is no incident: each CSR is ordered from your PKI's catalog item, and that request is the CSR's record (no second CSR while it is open): CERT_RENEWAL_CATALOG.md.

Two jobs, with the same settings:

| Template | Playbook | Credentials | What it does |
|---|---|---|---|
| `VMware - certificate report` | `playbooks/vmware_cert_report.yml` | VMware vCenter | **the report only**: every certificate, soonest expiry first, and for each one due what the renewal would do. Never ServiceNow, never a CSR - even with the ServiceNow credential attached. Printed and emailed |
| `VMware - certificate renewal` | `playbooks/vmware_cert_renew.yml` | VMware vCenter, ServiceNow API | the same report, **and** an incident and a CSR for each certificate due (below) |

**No inventory of hosts.** Both run on the AAP side and talk to vCenter only: each host's certificate is read through vCenter, and its CSR is made through vCenter's API - no SSH, no ESXi account. vCenter's own certificate is read on its address, port 443. The template needs an inventory picked (any one); leave **Limit** empty - pick hosts with `vmware_cert_only`.

The renewal, `playbooks/vmware_cert_renew.yml` (template `VMware - certificate renewal`):

1. It reads **vCenter's certificate** - its Machine SSL certificate, the one its web pages show - and the certificate of **every ESXi host** connected to it (`vmware_cert_esxi: false`: vCenter's only). Reading changes nothing.
2. Each certificate that expires within `vmware_cert_warn_days` (30) - or has expired - is **due**, and gets a **ServiceNow incident**: critical within `vmware_cert_crit_days` (7), else warning, whatever `servicenow_min_severity` says.
3. For each due certificate, a **CSR** (certificate signing request) is made **where its key stays** and attached to the incident, with a work note: the certificate, the names in the CSR, and what to do next.
   - **vCenter** makes its own, through its API - the same as the vSphere Client's *Generate Certificate Signing Request*. The subject and names are the current certificate's. The new key stays in vCenter's certificate store.
   - **An ESXi host** makes its own - the same as the vSphere Client's *Generate CSR using FQDN*. The new key stays on the host.
4. While a certificate is due, each run adds a note to its incident. Once it is renewed, the next run notes the incident as cleared (`servicenow_resolve_cleared: true` resolves it).

**Nothing in use is changed.** A CSR only makes a new key that waits: vCenter and the hosts keep their certificates, no service restarts. Installing the issued certificate is a later step, by a person, as the work note says (that step does restart vCenter's services before 8.0 Update 2; a host may need a reconnect). It is emailed (`report_email_to`), and a dry run (Check) makes no CSR and sends nothing.

**One incident per certificate.** 15 hosts due: 15 incidents, each with its own CSR and steps, all for `servicenow_assignment_group`. A run again adds a work note to the same incident - never a second incident or a second CSR for the same certificate. A renewed certificate's incident is noted as cleared (resolved with `servicenow_resolve_cleared: true`); when it comes due again years later, that is a new incident.

| The certificate | Incident | CSR | The work note says (step 3) |
|---|---|---|---|
| vCenter's, signed by your PKI (or self-signed) | yes | made by vCenter | Administration > Certificate Management > Machine SSL Certificate > Import and Replace Certificate > *Replace with external CA certificate where CSR is generated from vCenter Server (private key embedded)*: the issued certificate, and the CA chain. A snapshot first; vCenter HA off |
| An ESXi host's, signed by your PKI (or self-signed) | yes | made by the host | ESXi 8.0 Update 3 and later: the CA chain in vCenter's Trusted Root Certificates, `vpxd.certmgmt.mode` = `custom`, then the host > Configure > System > Certificate > Manage with External CA > Import and Replace > *Replace with external CA certificate where CSR is generated by ESXi (Private key embedded)* |
| Signed by vCenter's own CA (VMCA - how vCenter gives hosts their certificates unless told otherwise) | yes: it says to renew it in vCenter (its **Renew**) | none (`vmware_cert_vmca_csr: true`: one, to replace it with a certificate of your PKI) | - |

**One CSR per renewal.** vCenter and each host keep **one** waiting key: a second CSR replaces it, and the certificate the PKI issues for the first one no longer fits. So the job makes a CSR once. Before it makes one, it looks at the certificate's incident: a CSR attached, or in a work note (when ServiceNow refused the attachment) - and then it makes none. For the same reason, **make no CSR in the vSphere Client** while the PKI signs this one. A CSR lost? Close the incident: the next run opens a new one, with a new CSR. Without the ServiceNow credential nothing would record the CSR, so none is made (`vmware_cert_csr_without_servicenow: true` makes them and prints them).

### Try it on certificates that are years from expiring

Every certificate you have is far from expiring, and you cannot add one that is not. Raise `vmware_cert_warn_days` for a run: with `200`, every certificate that expires within 200 days is due (`3650`: within 10 years). It is one number for both: the report's amber, and the renewal's incident and CSR. Each incident says the number it was opened with (`Due: it expires within vmware_cert_warn_days (200 days)`).

0. **The report of every host:** the `VMware - certificate report` template with `vmware_cert_warn_days: 200` - every certificate, and what the renewal would do for each one due. Nothing is sent anywhere but the email.
1. **Dry run.** Launch with Job type **Check** and these Variables:

```yaml
vmware_cert_warn_days: 200
vmware_cert_only: [esx01.yoursite.mil]       # one host; e.g. [esx01, esx02] or ["esx0*"]; your
  vCenter by its name
```

The output says what it would do: `would open: esx01.yoursite.mil: ESXi host certificate ...` and `would make the CSR, and would open a ticket and attach it`.

2. **The incident only.** The same Variables plus `vmware_cert_make_csr: false`, Job type **Run**. One incident, for that host.
3. **With the CSR.** Remove `vmware_cert_make_csr: false` and run again. The host makes the CSR; it is attached to the same incident with the work note. Run it once more: no second CSR (`CSR on the ticket already`). Pick a host whose certificate is not being renewed right now: its CSR replaces any CSR the host already had waiting.
4. **vCenter.** The same with `vmware_cert_only: [vc01.yoursite.mil]`. vCenter's CSR needs the account's certificate rights ("Before you start", step 2). One vCenter first.
5. **Clean up.** Run without those lines (back to 30 days) and with `servicenow_resolve_cleared: true`: the test incidents are noted and resolved. Leave none open - an open test incident would be used again when that certificate really comes due.

### Set it up

1. **The vCenter account**: the certificate rows in "Before you start", step 2.
2. **Templates > Create template > Create job template** - the report first:
   - Name: `VMware - certificate report`. Playbook: `playbooks/vmware_cert_report.yml`. Inventory: any. Execution environment: your VMware one. Credentials: your **VMware vCenter** credential only
   - Variables: `report_email_to: vmteam@yoursite.mil`. Tick **Prompt on launch** next to **Variables**. Save.
   - Schedule it weekly if you like (Monday 07:00): the list to hand to whoever renews certificates.

Then the renewal - **Create job template** again:

- Name: `VMware - certificate renewal`
- Inventory: any (e.g. your VMware one). Project: site-automation. Playbook: `playbooks/vmware_cert_renew.yml`
- Execution environment: your VMware one. Credentials: your **VMware vCenter** credential and your **ServiceNow API** credential
- Tick **Prompt on launch** next to **Job type** and next to **Variables** (to try it)
- Variables: `servicenow_assignment_group: YOUR GROUP NAME` (who gets the incidents) and `report_email_to: vmteam@yoursite.mil` - the rest: aap/extra_vars/vmware_cert_renew.yml
- Save.
- Optional - the days asked at launch instead of typed into Variables: the template's **Survey** tab > **Create question**, three times, then switch the survey **on**:

| Question | Answer variable name | Answer type | Default | Required |
|---|---|---|---|---|
| Days to expiry (raise it to try the job) | `vmware_cert_warn_days` | Integer | `30` | yes |

| Question | Answer variable name | Answer type | Default | Required |
|---|---|---|---|---|
| Only these vCenters or hosts (empty = all) | `vmware_cert_only` | Text | (empty) | no |
| Make the CSRs | `vmware_cert_make_csr` | Multiple choice (single select): `true`, `false` | `true` | yes |

A schedule runs with the defaults (30, all, true).

3. **Try it**, as above.
4. **Every week:** the template's **Schedules** tab > **Create schedule**, e.g. every Monday at 07:00 in your time zone. 30 days gives four runs before a certificate expires; the incident is open from the first.

| Setting | Default | What it does |
|---|---|---|
| `vmware_cert_warn_days` | `30` | a certificate that expires within this many days is due. Raise it to try the job |
| `vmware_cert_crit_days` | `7` | within this many days (or expired) the incident is critical |
| `vmware_cert_only` | `[]` (all) | only these vCenters and hosts (names, `*` and `?` allowed; the short name works) |
| `vmware_cert_make_csr` | `true` | `false` = the incidents only |
| `vmware_cert_esxi` | `true` | `false` = vCenter's certificate only |
| `vmware_cert_vmca_csr` | `false` | `true` = a CSR also for a certificate signed by VMCA |
| `vmware_cert_vmca_issuers` | `[]` | more issuers to take as VMCA (`*` and `?` allowed) |
| `vmware_cert_csr_fields` | `{}` | subject fields for vCenter's CSR (`organization`, `organization_unit`, `locality`, `state_or_province`, `country`, `key_size`) over the current certificate's |
| `vmware_cert_csr_without_servicenow` | `false` | `true` = make the CSRs without the ServiceNow credential, and print them |

The attachment's name and type are the Linux renewal's settings: `cert_renew_attach_ext` (`csr`; `txt` for an instance that refuses `.csr`) and `cert_renew_attach_retry_txt`.

### Whoever gets the incident

1. Submit the attached CSR to the PKI for a server certificate (the same names).
2. Attach the issued certificate (Base64 `.cer` / `.crt`) and its chain to the incident.
3. Install it as the work note says. The next run reads the new certificate and notes the incident as cleared.

## Email the report

Every VMware job can email its result: the secure boot report its whole list, the other jobs what they did. Email is off until there is an address to send to.

**Step 1 - ask your mail admins** for:

- the mail relay's name and port (e.g. `smtp.yoursite.mil`, port 25);
- whether it uses encryption: STARTTLS (usual on 25 and 587), SSL (port 465), or none;
- whether it needs a login, or instead accepts mail from allowed servers. Then the **AAP server's IP address** must be on its allowed list: the email leaves from the AAP server (with separate AAP execution nodes, from the node that runs the job - list those);
- which sender address to use (e.g. `aap-noreply@yoursite.mil`).

**Step 2 - the relay, in your** `playbooks/group_vars/all.yml` (VS Code, then Commit and Sync Changes; AAP picks it up at the next project sync):

```yaml
report_email_smtp_host: smtp.yoursite.mil      # the relay from step 1
report_email_from: aap-noreply@yoursite.mil    # the sender from step 1
```

Add these lines only when step 1 says so:

| The relay | Add to all.yml |
|---|---|
| port 25 or 587 with STARTTLS | nothing: that is the default |
| a port other than 25 | `report_email_smtp_port: 587` (its port) |
| SSL on port 465 | `report_email_smtp_port: 465` and `report_email_security: ssl` |
| no encryption at all | `report_email_security: none` (then no login can be used) |
| its certificate is from your own CA (DoD) and the job says `certificate verify failed` | put the CA certificate (`.pem`, public) at `playbooks/files/ca/smtp-ca.pem` in the repository, `report_email_ca_path: "{{ playbook_dir }}/files/ca/smtp-ca.pem"`, and the line `playbooks/files/ca/` in `.site-local` (so an update keeps it) |

**Step 3 - only if the relay needs a login:** a credential, so the password stays out of Git.

1. AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**. Name it `SMTP relay`. Paste the `inputs:` part of `aap/credential_types/smtp_relay.yml` into **Input configuration** and the `injectors:` part into **Injector configuration**. Save.
2. **Credentials > Create credential**: type **SMTP relay**, the user name and password. Save.
3. On each VMware template: **Edit > Credentials**, add it next to the vCenter credential.

**Step 4 - who gets it.** One of:

- every VMware job, always: `report_email_to: [vmteam@yoursite.mil]` in `all.yml`;
- one template: its **Variables** box, `report_email_to: vmteam@yoursite.mil, ops@yoursite.mil`;
- whoever launches it: a survey question, variable `report_email_to`, type **Text**, not required. Left empty = no email.

Several addresses: separate them with commas. Copies: `report_email_cc`, the same way.

**Step 5 - test it** with the secure boot report: it only reads, so it is safe to run any time. Launch it with `report_email_to` set. In the job output, the task **Email | result** says `emailed "[AAP] VMware secure boot report: ..." to vmteam@yoursite.mil`, and the email arrives.

The other reports (health checks, certificate reports, ...) email the same way: EMAIL_REPORTS.md lists them, and explains when a report job is green or red.

**What arrives:** a formatted email, subject `[AAP] VMware secure boot report: 29 VM(s) to fix` (or `[AAP] VM restart: web01`, `[AAP] VM snapshot before-patch: web01`, ...). The secure boot report has number boxes (VMs checked, secure boot on / off, BIOS) and a table per problem: each VM with its folder, ESXi host, guest host name, IP, power state and guest OS - so VMs with the same name can be told apart. At the end: the AAP job, who started it, when, and which vCenter. The email also carries a plain-text copy. A dry run (Check) sends nothing; it says `DRY RUN: would email ...`. To change the look of the email: EMAIL_REPORTS.md.

The same report is in the job's **Details > Artifacts** (`vm_report`, and for the secure boot report `vm_secure_boot` with every VM's details), which is also what the next step of a workflow receives.

**If it fails**, the job fails (so nobody misses it) and says why:

| The job says | Fix |
|---|---|
| `set report_email_smtp_host ... and report_email_from` | step 2 is missing (or not synced to AAP yet) |
| `could not send the email through ...: [Errno 111] Connection refused` / `timed out` | wrong name or port, or a firewall between the AAP server and the relay |
| `does not offer STARTTLS` | the relay uses SSL (port 465) or no encryption: see step 2 |
| `TLS with the mail relay ... failed: ... certificate verify failed` | your own CA: the last row of step 2 |
| `refused the login` | the user name or password in the SMTP relay credential |
| `could not send ...: 530 ... authentication required` | the relay needs a login (step 3), or the AAP server's IP on its allowed list |
| `refused every recipient` / `(refused: x@...)` | the relay does not accept that address (wrong address, or outside mail not allowed) |
| `the password would cross the network unencrypted` | a login with `report_email_security: none`: use starttls or ssl |

AAP's own **Notifications** (the template's Notifications tab) only say that a job succeeded or failed. This sends the report itself.

## What you see

- **restart:** `Restart requested for (operating system restart through VMware Tools): web01`. The job does not wait for the server to come back up; check its services afterwards, or run the health check.
- **shut down:** `Shut down (...): web01.`, once the VM is off (up to `vm_power_timeout`, 600 s). VMs that were already off are listed as such.
- **snapshot:** `Took snapshot "before-patch" (no memory) of: web01`.
- **delete snapshot:** `web01: deleted before-patch`. If the name is not there: `no snapshot named "x" (it has: ...)`.
- **notes:** the notes `BEFORE` and `AFTER`, for each VM.
- **change VLAN:** `web01: Network adapter 2 moved VLAN200`, or `already on VLAN200`.

- **secure boot report:** a count, then one line per VM to fix, with what to do.
- **datastore report, snapshot report, alarms report:** the report as text (the same tables as the email), with the counts in the task name, e.g. `VMware alarms report: 1 critical, 2 warning alarm(s), 47 failed login(s) (events: last 24 h)`.
- **alarms ACT analysis:** `VMware alarms ACT analysis: likely causes and fixes for 12 problem(s)`, then one row per problem with ACT's cause, fix and confidence.

**Refusals, and what to do:**

- `VMware Tools is not running`: a `guest` restart or shutdown cannot work. Fix Tools, or use `hard` if losing unsaved data is acceptable.
- `No VM is named ...`: names are exact (case included), with no wildcards.
- `2 VMs are named ...`: rename one, or set `vmware_datacenter` if they are in different datacenters.
- `Refused: ... is the AAP server or a protected VM`: by design. Do it in vCenter yourself.

## Troubleshooting

Find the message in the job's output (the red task near the end), or in the email, then do what the last column says.

### Every VMware job

| You see | It means | Do |
|---|---|---|
| `No vCenter to talk to: attach a credential of type "VMware vCenter"` | the template has no vCenter credential | Template › Edit › Credentials › add your **VMware vCenter** credential |
| `vCenter ... refused the login` | the message says which: no domain in the user name (a local vCenter account is `name@vsphere.local`), a wrong user name or password, a locked account or an expired password - or that vCenter does not know the account (the jobs log in to every vCenter of `vmware_vcenters` with it; a `vsphere.local` account works only on the vCenters of its own single sign-on domain, in linked mode) | fix the credential (type **VMware vCenter**); log in to the vSphere Client of **that** vCenter with the same account to test it. The deploy stops when any vCenter refuses: it checks the new name is free in all of them |
| `vCenter ... accepted the login, but the account has no permission in that vCenter` (before: `NoPermission ... System.View ... group-d1`) | the account logs in (single sign-on) but has no role anywhere in that vCenter, so vCenter refuses the session | give it its role there: a **Global Permission** (Administration > Access Control > Global Permissions, Propagate to children) covers every vCenter of its single sign-on domain. Read-only is enough on a vCenter the job only reads - the deploy reads every vCenter of `vmware_vcenters` to check the new name |
| `Could not connect to vCenter ...: ... CERTIFICATE_VERIFY_FAILED` | the execution environment does not trust vCenter's certificate | add vCenter's CA to the execution environment; to test only, `vmware_validate_certs: false` in the template's Variables |
| `Could not connect to vCenter ...: timed out` or `Connection refused` | the AAP node cannot reach vCenter on port 443 | a firewall rule from the AAP node (or execution node) to vCenter, port 443 |
| `pyVmomi is not installed in the execution environment`, or `couldn't resolve module/action 'vmware.vmware...'` | the template uses an execution environment without the VMware parts | pick your VMware execution environment on the template ("Before you start", step 1) |
| A VM, host or datastore you see in vSphere is missing from a report | the vCenter account cannot see it | give its role on that folder or cluster, with **Propagate to children** ("Before you start", step 2) |
| A whole vCenter's datacenters and clusters are missing (the vSphere Client shows two or more vCenters) | the jobs read the credential's vCenter, plus only those in `vmware_vcenters` | `vmware_vcenters: [vc01..., vc02...]` in `all.yml` ("Before you start", step 5) |

| You see | It means | Do |
|---|---|---|
| Red **vCenters not read** on top, `- 1 vCenter(s) not read` in the title (the job failed at `every vCenter was read` only with `vmware_vcenter_fail_unread: true`) | that vCenter could not be reached, or refused the login; the rest of the report covers the others | the reason is in the row: `refused the login` = the account is not known there (in linked mode: give it a Global Permission); `timed out` / `Connection refused` = a firewall rule to that vCenter, port 443 |
| `Could not find imported module support code for ... site_vmware_... Looked for ... ansible.module_utils.site_vcenters` | the folder `roles/vmware_vm/module_utils/` is missing: an update copied the changed files but not that new folder | copy `roles/vmware_vm/module_utils/` from the release into your repository, commit, sync the project |
| `the same vCenter as ...: read once` in the artifacts | a vCenter listed twice (a short name and its FQDN, or its IP) | nothing: it is read once |
| `Refused: ... is the AAP server or a protected VM` | by design | do it in vCenter yourself |

## Datastore and snapshot reports, snapshot cleanup

| You see | It means | Do |
|---|---|---|
| The datastore heading still says 80% | `vm_datastore_warn_pct: 80` is set somewhere (it wins over the default 85) | remove it from `all.yml` and the template's Variables, or set the number you want |
| `vm_snapshot_max_age_days must be a number of days` | a survey or variable answer like `three` | a number: `1`, `2`, `3` |
| The snapshot report says 3 days although you set 2 (or the default is now 5) | in the workflow, the **survey answer** wins over the template's Variables and the default - a survey made from an earlier release still has Default `3` | change the survey's answer or its Default (section "Snapshots") |
| `Refused: N snapshots are older than ... - more than vm_snapshot_cleanup_max` | more than 50 at once (often a wrong number of days) | check the days; for a big cleanup on purpose, raise `vm_snapshot_cleanup_max` |
| `NOT DELETED ...: no longer there, or no longer old enough / now kept` | the snapshot changed between the report and the approval | nothing: it is safe. Run the workflow again |
| `Taken by` says `unknown` | vCenter no longer keeps the event (often after 30 days) | nothing: the snapshot's description may name the AAP user |
| `HELD BACK ...: its name has none of: Patching, Patches, Patch` | `vm_snapshot_cleanup_name_words` is set: only those names are cleaned up | nothing, if that is what you want; `[]` = any name |
| A patching snapshot was not deleted | it is younger than `vm_snapshot_max_age_days` (it waits), its name or description says keep, or the word is only in its **description** (only the name counts) | read its line in the job output: `HELD BACK` gives the reason; a young one is not listed as old at all |

### Pick lists

| You see | It means | Do |
|---|---|---|
| `NOT UPDATED ...: its survey has no question whose variable is vm_snapshot_picks` | the workflow's survey has no such question, or its Answer variable name differs | add the question (section "Pick lists", Step 2), or set `vm_pick_snapshot_question` / `vm_pick_reboot_question` to its variable name |
| `NOT UPDATED ...: the question ... is a text question` | the question is not multiple choice | make it **Multiple choice (multiple select)** |
| `NOT UPDATED ...: no job template or workflow job template ... that the AAP user can see` | a typo in the name, or the AAP credential's user has no role on it | the name as AAP shows it; the **Admin** role on that template for the user |
| `2 templates are named ...` | a job template and a workflow share the name | `{template: "<its ID>", kind: workflow}` |
| `WARNING: the survey of ... is turned off` | the list was written, but nobody is asked | the workflow > Survey > switch it on |
| `SKIPPED ... - not a choice of the pick list` | a line that does not end with `[vCenter/VM...]` (typed by hand, or from an old list) | pick from the list |
| `SKIPPED ... - VMware Tools is not running in it` | a guest restart needs VMware Tools | pick it again with How = `hard` |
| `Refused: the approved plan is for a hard restart, this run says vm_power_mode guest` | the step after the approval got another answer than the plan | nothing was done: launch the workflow again |
| `None of the picks can be ... now` | everything picked is gone, held back or not allowed now (the reasons are listed) | nothing: the workflow stopped before its approval |

### ESXi security settings

| You see | It means | Do |
|---|---|---|
| `... the vCenter account lacks the privilege Host.Config.... on this host` | the vCenter role misses that privilege | add it to the role ("Before you start", step 2), on the hosts or their cluster |
| `lockdown strict is refused` | `esxi_security_lockdown: strict` | use `normal` |
| `No host matches (hosts: ...)` | a typo in `esxi_security_hosts` / `_clusters`, or `vmware_datacenter` | the names as the vSphere Client shows them |
| `X: the host has no such setting` | that advanced setting does not exist on this ESXi version | check the name (case matters), or remove it from `esxi_security_settings` |
| `lockdown exception users: ... - lockdown left disabled, so they are not locked out` | the exception users could not be added, so lockdown was not turned on (on purpose) | fix the cause (often a privilege, or an account the host does not know) and run again |
| `lockdown is strict: left as it is` | someone set strict on that host | nothing, or set it to normal in vCenter if strict was a mistake |
| A host is listed as **skipped** | it is not connected to vCenter | reconnect it; the next run sets it |
| The scanner (or another tool) cannot log in to a host any more | lockdown blocks direct logins | add its account to `esxi_security_lockdown_exception_users` and run again, or exempt the host |
| SSH is off again on a host you opened for support | the nightly run did its job | `esxi_security_exclude_hosts` while the case is open |
| The ESXi Shell stays on, and the report does not mention it | `esxi_security_shell: ""` is set somewhere (leave it as it is) | remove it: the default is `disabled` |
| Every night's email lists the same host as changed | something turns the setting back every day (a host profile, a script, a person) | find what it is: two tools fighting over one setting |

## Certificate renewal

| You see | It means | Do |
|---|---|---|
| `vCenter X refused to make the CSR (HTTP 403 ...)` | the account may read vCenter but not make its certificates | give it Certificate Management as a Global Permission, or add it to CAAdmins ("Before you start", step 2); or make that one CSR in the vSphere Client and attach it to the incident by hand |
| `vCenter X refused the login to its API (HTTP 401)` | the credential's user name (`name@domain`) or password | fix the VMware vCenter credential |
| `the vCenter account lacks the privilege Certificate.Manage` (an ESXi host's CSR) | the role misses Certificates > Manage certificates | add it to the role, on the hosts |
| `vCenter X has no certificate API` | vCenter older than 6.7 Update 2 | make the CSR in the vSphere Client and attach it to the incident by hand |
| `vCenter X did not make the CSR (HTTP 400: ...)` naming a field | vCenter wants a subject field the certificate does not have | set it in `vmware_cert_csr_fields` (the PKI sets the subject by its own rules anyway) |
| `the attachment was refused ... as a .txt file too` | the instance does not take `.csr` or `.txt` attachments | the CSR is in the incident's work note, to copy from there; the next run makes no other. To have them attached: allow `.csr` or `.txt` on the instance, or `cert_renew_attach_ext` |
| `no ServiceNow API credential: the CSR is made only with its ticket` | the template has no ServiceNow credential | attach it; or `vmware_cert_csr_without_servicenow: true` to have the CSRs printed |
| `vm_cert_warn_days: not a setting of this job` | a setting named like the other VMware jobs' (`vm_...`): it would be ignored, and the job would run at 30 days | the settings start with `vmware_cert_`: `vmware_cert_warn_days`, `vmware_cert_only` |
| The report says `within vmware_cert_warn_days (30)` though you set another number | a survey question for `vmware_cert_warn_days` (its answer, default 30, wins over the Variables box), or the name misspelt | change the survey's default, or the name |
| `vmware_cert_only (...) matches none of the vCenters and hosts read` | a typo, or a host of another vCenter | the names as the vSphere Client shows them; add the vCenter to `vmware_vcenters` |
| A host under **Not checked**, `not connected` | it is not connected to vCenter | reconnect it; the next run reads it |
| `CSR on the ticket already`, but the PKI lost the CSR | the incident has its CSR | close the incident: the next run opens a new one with a new CSR |

| You see | It means | Do |
|---|---|---|
| An incident says to renew in vCenter (VMCA), but your hosts use your PKI's certificates | that host still has the certificate vCenter gave it | `vmware_cert_vmca_csr: true` makes a CSR for it, to replace it with one of your PKI |
| Test incidents still open | they were opened with the days raised | run with the real days and `servicenow_resolve_cleared: true`, or close them |

### Deploy from a ServiceNow ticket

Every message of the deploy, its workflow and the scheduled pickup, with what to do: VM_DEPLOY_GUIDE.md, *When something goes wrong*.

### Capacity planning

| You see | It means | Do |
|---|---|---|
| `not enough history` / `No trend: vCenter has fewer than 7 daily samples` | a new vCenter, or its statistics level or daily interval was changed | wait a week; check vCenter > Configure > General > Statistics: the 1-day interval enabled, level 1 or more |
| `vCenter's daily statistics interval is turned off: no history` | the 1-day interval is disabled | enable it (same place); usage now is still shown |
| A runway that jumps from run to run | the trend fit is poor: VMs were moved, snapshots deleted, a datastore grown | read the Trend fit column; use `vm_capacity_trend_days: 60` for a longer, steadier line |
| `cannot lose a host` | the cluster has as many hosts down (or in maintenance) as it plans to survive | bring hosts back; or set `vm_capacity_failover_hosts: 0` for a cluster that needs no failover |
| `No cluster matches ...` | a typo in `vm_capacity_clusters` | the cluster's name as the vSphere Client shows it |
| `ACT did not run: ...` (the report), `ACT did not estimate: ...` (the job's last line) | as for the alarms analysis (key, network, certificate, a reply cut off) | the ACT rows above |
| Math and GenAI differ a lot | ACT weighed something the line does not (a jump, a limit that comes first) | read ACT's reasoning; plan for the sooner of the two |

### VMs created report

| You see | It means | Do |
|---|---|---|
| *By* is unknown | vCenter no longer has the VM's creation event (it keeps them for a limited time), or the VM was created on a host directly | nothing: the VM still counts, by its creation date |
| *Ticket* is - for a VM the deploy built | its notes in vCenter were edited, so they no longer have the ticket's line (*"<ticket> - built <date>"*) | nothing for the count; the ticket's own notes say which VM it built |
| a VM restored from a backup is listed | a restore creates a new VM in vCenter | nothing: to vCenter it is new |
| `vm_created_days must be a number of days` | the setting is not a number | `vm_created_days: 7` |

### Health report

| You see | It means | Do |
|---|---|---|
| `vSphere HA is off` / `DRS is off` on a cluster that should not have them | your design | `vm_health_require_ha: false` / `vm_health_require_drs: false` on the template |
| `vCLS in retreat mode` | someone set `config.vcls.clusters.<id>.enabled` to `false` in vCenter's advanced settings (often for maintenance) and it was not set back | vCenter > Configure > Settings > Advanced Settings: set it to `true` when the work is done; DRS starts again |
| `vSphere HA agent: network isolated` / `network partitioned` | the host's management network cannot reach the other hosts (or its isolation address) | the host's management VLAN, uplinks and switch ports; then **Reconfigure for vSphere HA** on the host |
| `HA can survive 0 host failure(s) now` | the cluster is too full to restart a failed host's VMs | add capacity or move VMs: see the capacity planning report |
| HA agent ? on every host | vCenter did not say (some versions or a limited account) | the HA state shows in the vSphere Client: cluster > Monitor > vSphere HA |
| `Could not read vCLS retreat mode` (blue) | the account may not read vCenter's advanced settings | nothing needed: retreat mode still raises vCenter's own alarm on the cluster |
| `vCenter ... has no datacenter named ...` | a typo in `vmware_datacenter` | the exact name, as the vSphere Client shows it |
| `Reading the health of datacenters, clusters and hosts from vCenter ... failed: ...` | something this job did not expect from your vCenter | send that line to the people who maintain this repository |

### Alarms report

| You see | It means | Do |
|---|---|---|
| `types: unknown object type(s) ...` | a typo in `vm_alarm_types` | use `vcenter`, `datacenter`, `cluster`, `host`, `datastore`, `vm`, `network` |
| `vCenter ... has no datacenter named ...` | a typo in `vmware_datacenter` | the exact name, as the vSphere Client shows it |
| An alarm the vSphere Client shows is missing | it is on a VM, datastore or network (not read by default), or in another datacenter | add its type to `vm_alarm_types` (e.g. `[vcenter, datacenter, cluster, host, vm]`); check `vmware_datacenter` |
| No events at all, or fewer than in the vSphere Client | `vm_alarm_hours: 0`, or the account sees only part of the inventory (vCenter shows an account only the events of what it can see) | give the role at the top of the vCenter inventory, with Propagate to children |
| `Only the newest 5000 events were read` | a busy vCenter | lower `vm_alarm_hours`, or raise `vm_alarm_max_events` |
| `Only the newest 2000 alarm changes were read` | an alarm that flaps (on, off, on ...) all day | find it in *Alarms that fired* (the highest counts) and fix its cause; or raise `vm_alarm_max_alarm_changes` |
| `Only the first 200 are shown` | a long table | the job's **Artifacts** (`vm_alarms`) have all of them; or raise `vm_alarm_max_rows` |
| The job shows **failed** with `critical alarm(s) ... host(s) not connected` | `vm_alarm_fail: true` is set | expected (for a workflow); remove it for a report only |
| `Reading alarms and events from vCenter ... failed: ...` | something this job did not expect from your vCenter | send that line to the people who maintain this repository |

## Alarms ACT analysis

| You see | It means | Do |
|---|---|---|
| `nothing to analyze`, and no email | nothing is wrong: no alarm, no problem in the window | nothing. `vm_alarm_act_email_if_none: true` sends an email anyway |
| `A dry run (Check): ACT was not called` | the job ran as Check | launch it with Job type Run |
| `This job has no API key for provider genai: attach ...` | no **ACT model key** credential on the template, or its GenAI field is empty | add the credential (with the key for your provider) to the template |
| `This AAP node cannot reach the model at ... (... timed out / Connection refused / Name or service not known ...)` | a firewall, DNS or a missing proxy between the AAP node and the model | ask for HTTPS (443) from the AAP node to that address, or set your proxy: `site_act_env: {HTTPS_PROXY: "http://proxy.yoursite.mil:8080"}` in `all.yml` |
| `... Its certificate is not trusted: set site_act_ca ...` | the execution environment does not trust the model's certificate authority | put that CA certificate at `playbooks/files/ca/model-ca.pem`, `site_act_ca: "{{ playbook_dir }}/files/ca/model-ca.pem"` in `all.yml`, and `playbooks/files/ca/` in `.site-local` |
| `ACT failed: ...` with `401` or `Unauthorized` | the key is wrong or expired | a new key in the credential |
| `ACT failed: ...` with `429` or `Too Many Requests` | the key's quota per minute | run it less often, or lower `vm_alarm_act_max_items` |
| `model turn exceeded the 600s total timeout` (older releases: `90s`, `300s`), often `after a transient provider failure` | the model (or the provider, retrying after an error) took longer than ACT allows for one answer: `site_act_model_timeout`, 600 s | run it again (a provider hiccup passes); `site_act_model_timeout: 900` in `all.yml` (keep `site_act_timeout` above it); fewer items (`vm_alarm_act_max_items: 20`); or a faster model |
| `ACT failed: ... the reply was cut off after N bytes ... (after 4 attempts)` (before 0.17.1: `ACT failed: crash: IncompleteRead: IncompleteRead(N bytes read)`) | the model's answer was cut off on its way back: the gateway or a proxy closed the connection partway through it. Since 0.17.1 (ACT 0.6.25) ACT asks again, so this means every attempt was cut | run it again (a dropped connection passes); if it keeps failing at about the same size, the answer is too long for something on the way: fewer items (`vm_capacity_act_max_items: 10`, `vm_alarm_act_max_items: 10`); tell the network team the size and how long it took |
| `ACT produced no result (it timed out after 1200s ...)` | a slow model, or a lot of evidence | `site_act_timeout: 1800`, or `vm_alarm_act_max_items: 20` |
| `ACT's answer is below as text` | the model did not use the table format | the answer is still in the email. Run it again, or try another model (`site_act_models`) |
| A row says `ACT gave no answer for this one` | the model skipped that problem | run it again; fewer problems (`vm_alarm_act_max_items`) help a small model |

| You see | It means | Do |
|---|---|---|
| The job is red, but the email came | ACT could not run; the email lists the problems and says why | fix what the email says, then run it again |

**Test the way to the model by hand**, from the AAP server (the execution environment uses the same network):

```shell
curl -sS -o /dev/null -w '%{http_code}\n' https://api.genai.mil/v1/chat/completions
```

Any number (401, 404, 405) means the way is open. `Could not resolve host`, a timeout, or `SSL certificate problem` is the problem the job reports.

## Limits

- **Change VLAN between a standard and a distributed switch.** Moving an adapter from a standard switch port group to a distributed one, or back, fails inside the `vmware.vmware` collection (2.11); the job says so and changes nothing. Do that one in vCenter. Moves between port groups of the same kind of switch work.
- The guest keeps its IP settings after a VLAN change. A new VLAN usually needs new ones.
- `vm_names` are exact VM names. To act on many VMs, list them.
- The pick lists: see "Pick lists" > Limits.
- Tested against vcsim, VMware's vCenter simulator (`tests/vmware/run_vmware_test.sh`), not against a real vCenter. Run each job once as **Check**, then on a test VM, before you use it on real servers. The simulator raises no alarms: the alarms report's alarm handling is tested with recorded data (`tests/test_filters.py`, `tests/test_vmware_modules.py`). The ACT analysis is tested with the real ACT against a scripted model (`tests/test_alarm_act_contract.py`), not a real one.

# 12. VM outage watch: what is down or hung right now

A quiet job that asks vCenter one question, every few minutes: **is something down or hung right now?** When nothing is, it prints one line and stops: no email, no web page, no ticket. When something is, it reports it, hands it to the ServiceNow tickets job, and fails, so a workflow can react.

It is **not** the alarms report or the health report. Those are the morning overview and stay on their daily schedule. The watch reads only what means an outage, so it can run every 5 to 15 minutes without anyone noticing it.

**Phase 1 - watch only (this release).** The watch restarts nothing. For every VM it finds hung, it says what a later phase *would* do, so you can check each one by hand and count the false alarms before any action is turned on. Phase 2 (the response workflow: guest restart, approval, reset) is described at the end; it is not built yet.

| Job template | Playbook | Credential |
|---|---|---|
| VM - outage watch | `playbooks/vm_outage_watch.yml` | VMware vCenter (read-only is enough); no Limit |

## What it looks at

Every vCenter of the job (the credential's, and `vmware_vcenters`), or one datacenter (`vmware_datacenter`).

| Condition | Severity |
|---|---|
| An ESXi host not responding or disconnected, and not in maintenance mode | critical |
| A host's vSphere HA agent isolated, partitioned from the primary, host down, or in error | critical |
| A host's HA agent unreachable, or it failed to uninitialize | warning |
| A datastore not accessible (and not in maintenance mode) | critical |
| All paths down, a permanent device loss, or a VMFS heartbeat timeout in the window, **not cleared** since | critical |
| The same, cleared since (the clear event came later, on the same host and device) | warning |
| HA could not restart a VM, or found a host failed or isolated (events of the window) | critical |
| HA restarted or reset a VM, or started a failover (events of the window) | warning |
| A VM waiting for an answer in vCenter (a question: "moved or copied?", a locked file) | critical |

| Condition | Severity |
|---|---|
| A VM **hung**: no guest heartbeat AND no answer on the network, in every check of the run | critical |
| A VM with no guest heartbeat that answers on the network, or could not be probed (one signal only) | warning (`vm_watch_one_signal_severity`) |
| A vCenter that could not be read | critical |

"The window" is the last `vm_watch_window_minutes` (15): set it to the schedule's interval, so every event is seen once.

### When is a VM hung?

Only when **two independent signals agree**, in **every** check of the run:

1. **vCenter's signal.** The VM is powered on, has VMware Tools installed, has been up at least `vm_watch_min_up_minutes` (15; a VM still starting has no heartbeat yet), and its guest heartbeat is red - or Tools stopped answering and the heartbeat is gray.
2. **The network.** The AAP node probes the VM: a ping, and a TCP connect to `vm_watch_probe_port` when you set one (22 or 3389, say). It tries the IP addresses VMware Tools last reported, then the guest's host name and the VM's name in DNS (Tools that stopped often leave no IP). A refused connection counts as an answer: the guest's network stack is alive.

Only the VMs that fail the first signal are probed. The rest of the estate is never pinged.

Then the watch waits `vm_watch_confirm_seconds` (120) and reads those VMs again, for `vm_watch_confirm_checks` (2) checks in all. This happens in the same run, because an AAP execution environment keeps nothing between runs. What one signal alone means:

- **No heartbeat, but it answers the network.** VMware Tools died in a guest that still runs (common after a Tools upgrade, or under memory pressure). This is a warning and a ticket, never "hung".
- **No heartbeat, but it could not be probed.** There was no address, or no ping in the execution environment. This is also a warning, never "hung".
- **The heartbeat came back in a later check** (or the VM restarted, or someone powered it off). It is listed under "Not judged this run".
- **A vCenter task was running on it** (snapshot, consolidation, migration, clone, power operation). It is skipped this run and listed with the task's name. A VM stuns or loses its network for a moment during those.

A VM without VMware Tools is never judged on its heartbeat: vCenter has none to give.

## What you see

**Nothing down** - the job is green, one line:

```
No outage: 24 host(s), 31 datastore(s), 812 powered-on VM(s) checked (vCenter vc01.yoursite.mil).
```

Warnings below `vm_watch_report_on` are listed under that line, each on its own line, and do not make a report.

**Something down** - the report (in the job output; emailed with `report_email_to`, on the report website with `report_web_dest`), then the job fails:

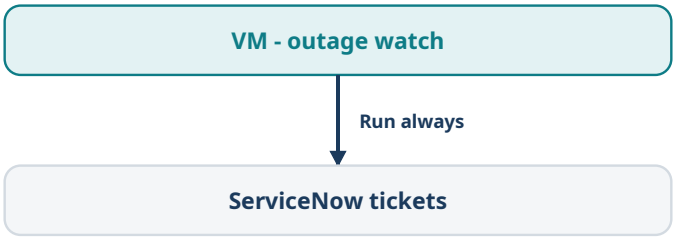
- **Down or hung now.** Each condition with its severity, where it is, and what to do.
- **Would be restarted in phase 2 (nothing was done).** The hung VMs a response would restart, with the signals and when they were checked. Check each by hand now. Was it really hung? Count the ones that were not.
- **Protected: ticket only.** Hung VMs that would never be restarted, with the reason: the AAP server, `vmware_protected_vms` / `vmware_protected_groups`, `vm_watch_protect`, or over `vm_watch_max_actions` (that many hung VMs at once means a host, the storage or the network, not the VMs).
- **Not judged this run.** Busy VMs, and VMs that recovered during the run.
- **How it judged.** The probe used.

Job artifacts, for the next step of a workflow:

- `vm_watch`: the conditions, the counts, `would_restart`, `protected`, `over_limit`, `not_judged`.
- `site_health` and `site_health_targets`: what the ServiceNow tickets job reads. There is one entry per vCenter, the check is `vm_watch`, and each condition is one finding.

## Set it up

1. **The job template** "VM - outage watch": playbook `playbooks/vm_outage_watch.yml`, the VMware vCenter credential, the inventory that has your `aap` group (the AAP server is found by it), no Limit.
2. **Try it** once by hand. A clean estate is one green line; a red run shows the report.
3. **The workflow** "VM - outage watch (workflow)":




*Always*, not *On failure*. The tickets job opens one incident per condition, adds a work note while it lasts, and notes (or with `servicenow_resolve_cleared: true` resolves) the incident once a clean run no longer finds it. That only works when the clean runs reach it too. The incidents carry the vCenter's name and the object's, e.g. `[site-health] vc01.yoursite.mil: VM app01 is hung: ...`. `servicenow_min_severity: critical` keeps the one-signal warnings out of ServiceNow.

4. **The schedule.** Put it on the workflow, every 5 to 15 minutes, with **Concurrent jobs off** on the workflow so that a slow run never overlaps the next. Set `vm_watch_window_minutes` to the interval. Leave the daily health and alarms reports as they are.
5. **Email** (optional): `report_email_to` on the job template. Only runs that find something at `vm_watch_report_on` (critical) send it.

## Settings

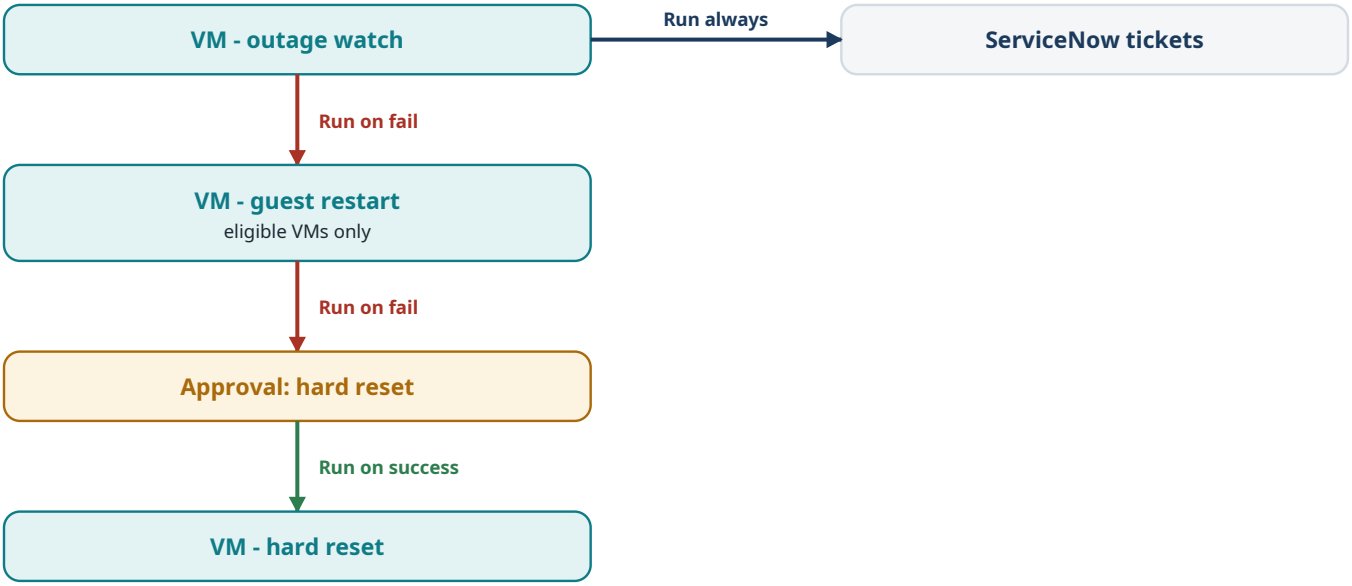
| Setting | Default | What |
|---|---|---|
| `vm_watch_window_minutes` | 15 | how far back HA and storage events are read; the schedule's interval |
| `vm_watch_min_up_minutes` | 15 | a VM up for less than this is not judged |
| `vm_watch_confirm_checks` | 2 | checks of a suspect in one run (1 = once, no waiting) |
| `vm_watch_confirm_seconds` | 120 | seconds between them |
| `vm_watch_probe` | true | false = no probe: no VM is then hung, only one-signal warnings |
| `vm_watch_probe_port` | 0 | also a TCP connect to this port (0 = ping only) |
| `vm_watch_probe_timeout` | 3 | seconds per address |
| `vm_watch_max_suspects` | 50 | suspects probed and confirmed per run; the rest are "not probed" |
| `vm_watch_one_signal_severity` | warning | `info` keeps one-signal VMs out of the tickets |
| `vm_watch_skip_vms` | [] | VM names never judged (`*` and `?`) |
| `vm_watch_skip_folders` | [] | VM folders never judged, e.g. `/DC1/vm/Lab*` |
| `vm_watch_protect` | [] | VMs a response would never restart (on top of `vmware_protected_vms` and the AAP server) |
| `vm_watch_max_actions` | 5 | the most VMs one run would restart; beyond it, ticket only |
| `vm_watch_fail_on` | [critical] | severities that make the job red; `[]` = always green |
| `vm_watch_report_on` | [critical] | severities that make a report (email, web page); `[critical, warning]` for the warnings too |

Also used: `vmware_vcenters`, `vmware_datacenter`, `vmware_validate_certs`, `vmware_protected_vms`, `vmware_protected_groups`, `report_email_to`, `report_web_dest`, and the `servicenow_*` settings of the tickets job.

**The probe and the execution environment.** The ping runs inside the AAP execution environment. Without a `ping` program there (or without the right to send ICMP), every suspect is "could not be probed" and nothing is ever called hung. The job says so ("unavailable"). Set `vm_watch_probe_port` to a port your VMs listen on (22 for Linux, 3389 for Windows, or one each, in two templates split by folder). The TCP connect needs nothing from the execution environment. The AAP node must be allowed to reach the VMs on that port, or ping them, through the firewalls in between.

## Phase 2 - the response (not built yet)

Phase 2 will be built only once phase 1 has run long enough to show its false alarms. It will be conservative by default:




- **Only the VMs in "Would be restarted in phase 2".** Confirmed hung (both signals, every check), not protected, at most `vm_watch_max_actions`. Everything else stays a ticket.
- **A guest restart first, always.** It goes through VMware Tools, so the operating system shuts down cleanly. A wrongly judged, healthy guest reboots cleanly instead of losing what was in memory. When Tools is dead, the step says "not possible" and goes to the approval.
- **A hard reset only after a person approves it.** The approval node carries which VMs, how long each has been dark, and the diagnosis. A fully automatic reset would be a separate, opt-in mode for labs and stateless desktop pools.
- **Every action written down.** It goes in the VM's Notes in vCenter (the existing notes job: date, job, why) and in the incident's work notes.
- **Never a host.** No host reboot, agent restart or evacuation: those stay manual.

## Event-driven, later

When an Aria Operations alert or an SNMP / mail receiver is available, an Event-Driven Ansible rulebook can start the same workflow from the vCenter alarm. The watch job would still be the workflow's first step, so the response always works from a fresh look, never only from the event. Nothing in the jobs would change.

## Troubleshooting

| You see | What it means |
|---|---|
| Every suspect "could not be probed (unavailable)" | no `ping` in the execution environment, or ICMP not allowed: set `vm_watch_probe_port` |
| "could not be probed (no address)" | VMware Tools reported no IP, and neither the guest host name nor the VM name is in DNS |
| A VM that answers is reported hung | the firewall between the AAP node and the VM drops the ping: use `vm_watch_probe_port` with a port it allows |
| The same one-signal warning every run | VMware Tools is stopped in that guest: restart or update Tools, or add the VM to `vm_watch_skip_vms` |
| An all-paths-down condition stays after the storage came back | the clear event names another device, or came after the window: it ages out after `vm_watch_window_minutes` |
| "does not say which VM is the AAP server" | run the template with the inventory that has the `aap` group, or list the AAP VM in `vmware_protected_vms` |

# 13. Log analysis: the patterns in a group of servers' logs, explained by ACT

Two jobs read logs and have ACT (GenAI) explain what they show. Both are read-only.

| Job template | Playbook | What it reads |
|---|---|---|
| Log analysis | `playbooks/log_analysis.yml` | the hosts of the job's target or Limit: Linux (the journal, units, podman containers, text logs), Windows over WinRM (event logs, providers, event IDs, text logs), a vCenter appliance over SSH |
| VM - vCenter log analysis | `playbooks/vm_log_analysis.yml` | vCenter's own log (vpxd) and each ESXi host's (hostd, vmkernel, vpxa), through vCenter's API: no SSH, nothing to switch on at the hosts |

## What it does

1. **Read.** Each host's **warning, error and critical** lines of the window: the last `log_analysis_hours` (24), or a fixed window such as the outage, `log_analysis_since` .. `log_analysis_until`. The journal and the event logs say how bad a line is; a text log's line is read when it matches `log_analysis_match` (error, fail, fatal, denied, refused, timeout ...). At most `log_analysis_max_lines` (500) lines per log, the newest.
2. **Group, on the AAP side.** Each line becomes a pattern: the line with its numbers, times, IP addresses, IDs and quoted values taken out. The same pattern on five hosts is **one** row, "on 5 hosts, 212 lines, first 07:02, last 07:58". Patterns are ranked: critical, then error, then warning; within a level, the most hosts and then the most lines first.
3. **Explain.** ACT gets the top `log_analysis_max_items` (30) patterns, with up to three example lines each, and answers for each: the **likely cause**, the **evidence**, the **fix**, and a **confidence** from 0 to 100. It is told to say when a pattern is harmless noise and to name the patterns that share a cause. ACT runs in **evidence mode**: on the AAP side, reading what the job collected, running **no command** on any host. Before anything leaves, ACT replaces the host names (and `log_analysis_names_extra`) with placeholders.
4. **Report.** One report, every run, in the same look as the other reports: printed in the job output, kept as the job artifact `log_analysis` (every pattern, ACT's answer merged in), **emailed** (`report_email_to`, and the "Also email the report to" box: EMAIL_REPORTS.md) and **published** on the report website (`report_web_dest`: WEB_REPORTS.md). It has:
   - the hosts **not read**, in red, with why (unreachable, login refused, a log it may not read);
   - your question, if you asked one, and ACT's "What to do first";
   - **Likely causes and fixes**: one row per pattern ACT analyzed, with the confidence coloured (green 80+, amber 50-79, red under 50);
   - **Patterns**, always - also without ACT: level, hosts, log, lines, first and last seen, an example;
   - the hosts read, with the logs each one gave;
   - **What was read**: the window, the profiles per host, how many lines the ignore list left out.

A run that finds nothing is not emailed unless `log_analysis_email_if_clean: true` (the website gets every run). The job is **green** when it reports; **red** when ACT was wanted and could not run (no key, no network: the report still lists the patterns and says why), when no host at all could be read, and - with `log_analysis_fail: true`, for a workflow - when a critical or error pattern was found. A host that cannot be reached is listed, and does not fail the job.

## What it reads, per kind of host

`log_analysis_source` says what a host is: `auto` (Windows when its connection is WinRM or PSRP, else Linux), `linux`, `windows` or `vcsa`. Set it per group in the inventory.

| Kind | Read | How |
|---|---|---|
| Linux | the journal at `log_analysis_priority` (`err`) or worse; the units in `log_analysis_units` and the syslog identifiers in `log_analysis_identifiers` at `log_analysis_units_priority` (`warning`); the containers in `log_analysis_containers` (`podman logs --since`); the text logs in `log_analysis_files` (+ `_extra`, + the profiles') that changed in the window | journalctl, podman, find and grep, as root (`log_analysis_become`) - read-only. A journal line two reads return is counted once |
| Windows | the event logs in `log_analysis_win_logs` (+ `_extra`): critical, error and (`log_analysis_win_level: 3`) warning events; the providers in `log_analysis_win_providers` at every level up to information; the event IDs in `log_analysis_win_event_ids` whatever their level; the text logs in `log_analysis_win_files` (+ `_extra`) | `files/log_collect.ps1` over WinRM - no collection needed. PowerShell 5.1 and 7, also in Constrained Language Mode |
| vCenter appliance (`vcsa`) | as Linux, plus the `vcsa` profile: vpxd, vpxd-svcs, vmon, SSO (STS), envoy, rhttpproxy, vsphere-ui, applmgmt, vpostgres, vmdird, certificate management, EAM, content library | over SSH as root. **The appliance's login shell must be bash** for Ansible: on the appliance, `chsh -s /bin/bash root`. Its Machine credential: root, or an account that may sudo |
| vCenter and ESXi through the API | vCenter's `log_analysis_vcenter_logs` (`vpxd`); each host's `log_analysis_esxi_logs` (`hostd`, `vmkernel`, `vpxa`; also e.g. `fdm`, `vobd`) - the last `log_analysis_read_lines` (20000) lines of each, then the window; a line ESXi or vCenter marks warning, error or critical, or one that matches `log_analysis_match` | `playbooks/vm_log_analysis.yml`, vCenter's DiagnosticManager (what the vSphere Client's "Export System Logs" reads). The **VMware vCenter** credential's account needs **Global > Diagnostics** besides read-only. A host not connected to vCenter is listed as not read |

## The settings

Every setting can be set for all hosts (`all.yml`), per group or host in the inventory, on the job template, or as a **survey** answer. A list may be typed as `a, b` or one per line.

| Setting | Default | What |
|---|---|---|
| log_analysis_hours | 24 | the window: the last this-many hours |
| log_analysis_since, log_analysis_until | empty | a fixed window, UTC, e.g. 2026-10-23 08:00 and 2026-10-23 10:30 ("what happened during the outage"). Since alone = until now |
| log_analysis_question | empty | what you want to know, in your words: ACT answers it first ("users report slow logins since 09:00"). A good survey box |
| log_analysis_profiles | [] | one word switches on a product's logs (below) |
| log_analysis_profiles_extra | {} | your own profiles, or a shipped one changed: {stigman: {files: [/opt/stigman/logs/*.log], units: [stigman-api]}} |
| log_analysis_source | auto | auto, linux, windows, vcsa |
| log_analysis_priority | err | the journal's lines this bad or worse (crit, err, warning ...) |
| log_analysis_journal | true | false = no general journal read (units, containers and text logs only) |
| log_analysis_units, log_analysis_identifiers | [] | units (sssd, dirsrv@*) and syslog identifiers (sudo) read at log_analysis_units_priority (warning) - lower than the rest |
| log_analysis_containers | [] | podman containers: name for root's, user/name for a user's (rootless) |
| log_analysis_files, log_analysis_files_extra | [] | Linux text logs (globs). _extra adds to the base list and to the profiles' - it never replaces them |
| log_analysis_win_logs, log_analysis_win_logs_extra | [System, Application], [] | event log channels, e.g. Microsoft-Windows-TaskScheduler/Operational, Directory Service, DNS Server |
| log_analysis_win_level | 3 | 1 critical, 2 + error, 3 + warning |
| log_analysis_win_providers | [] | providers read at every level up to information (Service Control Manager); log_analysis_win_providers_only: true = only these |
| log_analysis_win_event_ids | [] | IDs always read: "System:7031", or a number for every log read plus Security (4625, 4740) |
| log_analysis_win_files, log_analysis_win_files_extra | [] | Windows text logs (wildcards), e.g. 'D:\Apps*\logs*.log' |
| log_analysis_match, log_analysis_match_extra | error, fail, fatal ... / [] | the words a text log's line (or a container's) is read for; _extra adds words (SEVERE) |

| Setting | Default | What |
|---|---|---|
| `log_analysis_ignore`, `log_analysis_ignore_extra` | two harmless Windows events / `[]` | lines left out before they are grouped (regular expressions, matched against "program line"): known noise. `_extra` adds yours |
| `log_analysis_names_extra` | `[]` | more names ACT replaces with placeholders (host names always are) |
| `log_analysis_act` | `true` | `false` = the patterns only, no ACT, no key needed |
| `log_analysis_max_items` | `30` | ACT gets at most this many patterns, the most severe first; the rest are in the patterns table |
| `log_analysis_max_lines`, `log_analysis_max_line_chars` | `500`, `400` | per log: the newest lines kept; a longer line is cut |
| `log_analysis_max_chars` | `60000` | the evidence ACT reads is cut here |
| `log_analysis_min_count` | `1` | a pattern seen fewer times is left out |
| `log_analysis_email_if_clean` | `false` | `true` = email also when nothing was found |
| `log_analysis_fail` | `false` | `true` = the job is red when a critical or error pattern was found |
| `log_analysis_esxi_hosts` | `[]` | `vm_log_analysis.yml`: only these hosts (`esx0*`) |
| `log_analysis_vcenter_logs`, `log_analysis_esxi_logs`, `log_analysis_read_lines` | `[vpxd]`, `[hostd, vmkernel, vpxa]`, `20000` | `vm_log_analysis.yml`: which logs, and how far back each is read |

ACT's own settings are the usual ones (ADDING_ACT.md): `site_act_provider` (GenAI.mil by default), `site_act_models`, `site_act_env` (a proxy), `site_act_ca`; `log_analysis_act_model_timeout` (600 s) for one answer.

## Profiles

A profile is a product's standard log locations and units, switched on with one word. They ship in `roles/log_analysis/vars/main.yml`; yours go in `log_analysis_profiles_extra` (an entry of the same name replaces the shipped one). What your site moved elsewhere goes in the `_extra` lists.

| Profile | Reads |
|---|---|
| `vcsa` | the vCenter appliance's logs (on by itself with `log_analysis_source: vcsa`) |
| `mid_server` | a ServiceNow MID Server on Windows: `agent0.log*` and `wrapper.log*` under `C:\ServiceNow\*\agent\logs` and `D:\...`; its service stopping or crashing (System 7023, 7024, 7031, 7034); lines with SEVERE |
| `iis` | WAS / W3SVC / worker process events; the HTTP.sys error logs (`HTTPERR`) |
| `sql_server` | the MSSQLSERVER and SQL Agent events; `ERRORLOG*`; severity 16-25, deadlocks, failed logins |
| `nginx`, `httpd` | their error logs and units |
| `mariadb` | MariaDB and MySQL error logs, `mariadb` and `mysqld` units |

| Profile | Reads |
|---|---|
| `podman` | the podman units and the `podman` / `conmon` identifiers |
| `sssd` | the SSSD logs and unit |
| `idm` | an IdM (FreeIPA) server: Directory Server errors, the KDC, kadmin, httpd, named, custodia, the CA |
| `auditd` | `audit.log`'s failed results (`res=failed`, `success=no`) and the unit |
| `aap` | an AAP node installed from RPMs: `/var/log/tower/*.log`, the controller, receptor, gateway, nginx and redis units. A containerized AAP: name its containers in `log_analysis_containers` (`user/automation-controller-task` ...) |

The **MID Servers run on Windows**: put them in a group of the Windows inventory with

```yaml
log_analysis_profiles: [mid_server]
# installed elsewhere than C:\ServiceNow or D:\ServiceNow:
log_analysis_win_files_extra: ['E:\MID\*\agent\logs\agent0.log*']
```

## Set it up in AAP

1. **The key.** The **ACT model key** credential your other ACT jobs use (or create it: `aap/credential_types/act_model_key.yml`). Not needed with `log_analysis_act: false`.
2. **Log analysis** (`playbooks/log_analysis.yml`) - one template per inventory:
   - Linux servers: the Linux inventory and its **Machine** credential (an account that may sudo).
   - Windows servers: the Windows inventory and its **Machine (Windows)** credential (WinRM).
   - The vCenter appliance: a host in a group with `log_analysis_source: vcsa`, its Machine credential root (or a sudo account), the appliance's shell bash (above).
   - Credentials: the Machine credential **and** ACT model key. **Prompt on launch** for the **Limit**, and a **survey** with `target` (the group), `log_analysis_question` (text, not required), `log_analysis_since` / `log_analysis_until` (text, not required) and, if you like, `log_analysis_profiles` (multiple choice).
   - Variables: `report_email_to: ops@yoursite.mil` (or empty, with `report_web_dest` only).
3. **VM - vCenter log analysis** (`playbooks/vm_log_analysis.yml`): inventory, project and execution environment as the other VM templates; credentials **VMware vCenter** (Global > Diagnostics) and **ACT model key**; no Limit. Several vCenters: `vmware_vcenters`, as the other VM jobs.
4. **First run as a dry run:** Job type **Check**. Everything is read and reported; ACT is not called.

## Sizes and the GenAI quota

ACT is called **once per job**, whatever the number of hosts: the grouping does the shrinking (20 hosts x 500 lines become a few dozen patterns), and the evidence is capped at `log_analysis_max_chars` (60000 characters, roughly 15000 tokens) - well inside GenAI.mil's default quota of 200,000 tokens a minute per key. The lines themselves travel from each host to the AAP node: at most `log_analysis_max_lines` per log, `log_analysis_max_line_chars` each. Raise `log_analysis_max_items` for more patterns explained, or lower it (and set `log_analysis_min_count: 3`) when a noisy group crowds the list; quiet known noise for good with `log_analysis_ignore_extra`.

# 14. NetApp ONTAP health report

One job, `playbooks/ontap_health_report.yml`, reads every NetApp ONTAP cluster you list and emails one report: what is broken first, then capacity, protection and services. Each row is coloured: red = critical, amber = warning, green = OK, blue = for your information.

**It only reads.** It sends GET requests to each cluster's REST API (the same API System Manager uses) and nothing else: the code refuses any other kind of request. It runs on the AAP side, like the VMware jobs: no inventory of the clusters is needed, no SSH, no Limit.

## What the report shows

At the top: the title with the number of critical and warning findings, then boxes with the main counts (nodes down, health alerts, hardware, full volumes and aggregates, SnapMirror, network, certificates). Then, in this order:

| Section | What it shows | Colours |
|---|---|---|
| **Clusters** | each cluster: ONTAP version, system health, nodes up, space used on its aggregates, NTP servers. A cluster that could not be read is listed in red with the reason | health, nodes, space |
| **Nodes** | state, uptime (days hours:minutes), model, version, whether the partner could take over (storage failover), service processor, NVRAM battery, the node's clock against the AAP server's | green UP / red DOWN, and each status |
| **System health** | ONTAP's health monitors (`system health subsystem show`) that are not OK | amber degraded |
| **Health alerts** | what the health monitors raised (`system health alert show`): node, severity, alert, resource, probable cause, corrective action | red critical / major, amber minor |
| **Error events (last 24 h)** | ONTAP's event log (EMS): emergency, alert and error events, the same event on the same node counted once | red alert / emergency, amber error |
| **Hardware: controllers and shelves** | each controller and disk shelf: power supplies, fans, temperature, failed parts (FRUs) | red failed |
| **Disks** | broken disks; disks rebuilding, copying or in maintenance; unassigned disks | red / amber / blue |
| **Spare disks** | spare disks per node (a failed disk is rebuilt onto a spare) | red none, amber few |
| **Network interfaces (LIFs)** | LIFs down, not on their home port, or disabled | red / amber / blue |
| **Ports down** | enabled Ethernet and FC ports that are not up | red in use, blue no cable / not used |

| Section | What it shows | Colours |
|---|---|---|
| **Port errors (CRC)** | ports with receive errors or CRC errors since the node booted (CRC counters: ONTAP 9.11+) | amber from 1, red from 10,000 |
| **Aggregates** | every data aggregate, fullest first: node, home node, size, used, available, used % | amber 85 %, red 90 % |
| **Aggregates not on their home node** | after a takeover, aggregates still on the partner | amber |
| **Volumes** | volumes 85 % full or more, and volumes offline or restricted, with their aggregate, state, type, size, available, used % | amber 85 %, red 90 %, red offline |
| **Inodes (files)** | volumes using 85 % or more of their inodes (no new file can be created when they run out) | amber 85 %, red 90 % |
| **Snapshot space** | the 25 volumes whose snapshots hold the most: type, volume size, snapshot count, reserve, snapshot used, % of reserve, % of volume | amber from 100 % of the reserve (spilling into the volume) |
| **SnapMirror** | relationships that are unhealthy, lagging, broken off, paused or uninitialized, and transfers running now: mirror state, status, healthy, lag, last transfer, progress, progress updated, policy, reason | red unhealthy / 48 h lag, amber 24 h lag, blue transferring |
| **Cluster and SVM peers** | cluster peers not available, SVM peers not peered | red / amber |
| **SVMs not running** | stopped storage VMs (a stopped DR destination is normal: blue) | red |
| **CIFS (SMB) servers** | each CIFS server: enabled or not, and whether its domain controllers answer | red stopped / no DC, amber a DC down |
| **LUNs** | LUNs not mapped to any host (forgotten?) and LUNs not online | amber / red |
| **Certificates expiring** | certificates expiring within 60 days or expired | amber 60 days, red 14 days |

A section with nothing to report says so in one line ("Every LIF is up and at home."). Long tables show the problems first and are cut at 50 rows (`ontap_report_max_rows`); the job's artifacts (`ontap_health`) have every finding.

## Before you start

### 1. A read-only account on every cluster

The report needs an ONTAP account with the built-in **readonly** role, for the **http** application (the REST API). On each cluster, as an administrator (`cluster1` is the cluster's admin SVM, its name):

```
security login create -vserver cluster1 -user-or-group-name svc_aap_ro -application http
  -authentication-method password -role readonly
```

Use the **same user name and password on every cluster**: the job has one credential for all. (An Active Directory account works too, with `-authentication-method domain`, if the cluster is set up for AD logins.)

### 2. The way from AAP to the clusters

The AAP server (or the execution node that runs the job) must reach each cluster's **management address** on HTTPS, port 443. Test from the AAP server (it asks for the password):

```
curl -sk -u svc_aap_ro "https://cluster1.yoursite.mil/api/cluster?fields=version"
```

A short JSON answer with the version means the account, the network and the API are fine.

That `curl` is not the job. The job runs **in the execution environment** (a container on the AAP execution node), checks the certificate (`-k` above skips that), and uses the EE's OpenSSL, DNS and proxy settings. When the curl works and the report still says **UNREACHABLE**, read the reason on the cluster's row: it says which of these it is and what to set. To run the same test the way the job does, on the execution node:

```
podman run --rm -it <your EE image> curl -s -u svc_aap_ro
  "https://cluster1.yoursite.mil/api/cluster?fields=version"
```

### 3. The cluster's certificate

ONTAP's management certificate is often self-signed, or from your own CA. The job checks it by default. Either:

- put the CA certificate (public, `.pem`) at `playbooks/files/ca/ontap-ca.pem`, add `ontap_ca_path: "{{ playbook_dir }}/files/ca/ontap-ca.pem"` to `playbooks/group_vars/all.yml`, and the line `playbooks/files/ca/` to `.site-local` (so an update keeps it); or
- set `ontap_validate_certs: false` (the password still travels encrypted; the job just does not check who answers).

### 4. The credential

1. **Automation Execution > Infrastructure > Credential Types > Create credential type**: name `NetApp ONTAP`; paste the `inputs` block of `aap/credential_types/netapp_ontap.yml` into **Input configuration** and the `injectors` block into **Injector configuration**. Save.
2. **Credentials > Create credential**: type **NetApp ONTAP**, name e.g. `NetApp ONTAP read-only`, the user name and password from step 1. Save.

## Set it up

1. **Templates > Create template > Create job template:**
   - Name: `NetApp ONTAP health report`
   - Inventory: any (e.g. your VMware one: the job runs on the AAP side)
   - Project: site-automation. Playbook: `playbooks/ontap_health_report.yml`
   - Execution environment: any with Python 3 (no NetApp collection is needed)
   - Credentials: **NetApp ONTAP read-only**
   - Tick **Prompt on launch** next to **Variables** (to try one cluster first)

- Variables:

```yaml
ontap_clusters: [cluster1.yoursite.mil, cluster2.yoursite.mil]
report_email_to: storage-team@yoursite.mil
```

- Leave Limit empty. Save.

2. **First run: one cluster.** Launch with Variables `ontap_clusters: [cluster1.yoursite.mil]`. Read the report in the job's output. Lines that start with `cluster1: could not read ...` or `... reduced detail ...` under a section say what this cluster's ONTAP version or the account's role did not allow: see Troubleshooting.
3. **All clusters**, then a schedule: **Schedules > Create schedule**, e.g. every day at 06:00 (set **your** time zone).

A dry run (Check) reads and prints the same report, and sends no email.

**SnapMirror** relationships are read on their **destination** cluster: list the destination clusters too, or the relationships into them are not checked.

## Settings

Set them in the template's Variables, or in `playbooks/group_vars/all.yml` for every run.

| Setting | Default | What it does |
|---|---|---|
| `ontap_clusters` | `[]` | the clusters' management addresses |
| `ontap_validate_certs` / `ontap_ca_path` | `true` / (none) | check the clusters' certificates; the CA file to check them with |
| `ontap_volume_warn_pct` / `_crit_pct` | `85` / `90` | volume used % for amber / red |
| `ontap_aggr_warn_pct` / `_crit_pct` | `85` / `90` | aggregate used % |
| `ontap_inode_warn_pct` / `_crit_pct` | `85` / `90` | inodes used % |
| `ontap_snapshot_reserve_warn_pct` / `_crit_pct` | `100` / `150` | snapshot space as % of the snapshot reserve |
| `ontap_snapmirror_lag_warn_hours` / `_crit_hours` | `24` / `48` | time since the last good SnapMirror transfer |
| `ontap_cert_warn_days` / `_crit_days` | `60` / `14` | days before a certificate expires |
| `ontap_time_drift_warn_seconds` / `_crit_seconds` | `60` / `300` | a node's clock against the AAP server's |
| `ontap_port_errors_warn` / `_crit` | `1` / `10000` | receive or CRC errors on a port, since boot |
| `ontap_min_spares` | `1` | fewer spare disks than this on a node is amber (0 is red) |
| `ontap_ems_hours` / `ontap_ems_max` | `24` / `500` | the event log window, and the most events read per cluster |
| `ontap_report_max_rows` | `50` | rows per table |
| `ontap_snapshot_top` | `25` | volumes in the snapshot space table |

| Setting | Default | What it does |
|---|---|---|
| `ontap_report_fail` | `false` | `true` = the job shows **failed** on a critical finding or a cluster not read (for a workflow) |
| `ontap_timeout` | `30` | seconds for one request (no answer: asked once more). The job output lists every request that took over 10 seconds, to pick a number |
| `ontap_tls_legacy` | `false` | connect to older ONTAP whose TLS the EE refuses (see Troubleshooting) |

**All settings** of this job, with their defaults: ontap_report.

## Troubleshooting

| You see | It means | Do |
|---|---|---|
| `No ONTAP account: attach a credential of type "NetApp ONTAP"` | the template has no NetApp ONTAP credential | add it (Before you start, step 4) |
| `ontap_clusters is empty` | no cluster listed | `ontap_clusters: [...]` in the template's Variables |
| A cluster **UNREACHABLE** with `timed out` or `Connection refused` | the AAP node cannot reach the cluster on port 443 (each request is asked twice) | a firewall rule from the AAP execution node to the cluster's management address; test with the `podman run ... curl` above |
| **UNREACHABLE** ... `it went through the proxy ...` | the EE has `https_proxy` set, and the cluster is not in `no_proxy` | add the clusters (or `.yoursite.mil`) to `no_proxy` in the EE or the job's environment |
| A table says `no answer within 30 s, asked twice` and the rest of the cluster is there | that one table is slow on a big cluster | `ontap_timeout: 120` |
| **UNREACHABLE** with `UNSAFE_LEGACY_RENEGOTIATION_DISABLED`, `KEY_TOO_SMALL`, `MD_TOO_WEAK`, `UNSUPPORTED_PROTOCOL` or `HANDSHAKE_FAILURE` | an older ONTAP whose TLS the EE (OpenSSL 3) refuses; curl on a host with another crypto policy still connects | update ONTAP, or `ontap_tls_legacy: true` for those clusters' template |
| **UNREACHABLE** with `Hostname mismatch` | the cluster is listed by a name (or IP) that is not on its certificate | list it by the name on its certificate |
| **UNREACHABLE** with `Name or service not known` | the name does not resolve on the AAP node | use the name DNS knows, or the IP address |
| **UNREACHABLE** with `CERTIFICATE_VERIFY_FAILED` | the cluster's certificate is not trusted | Before you start, step 3 |
| `the ONTAP account was refused (HTTP 401)` | wrong user or password, the account does not exist on **that** cluster, it is locked, or it lacks the `http` application | create it the same on every cluster (step 1); unlock it (`security login unlock`) |
| `cluster1: could not read alerts - HTTP 403 ... the account's role may not allow this` | the role does not allow that command | use the built-in `readonly` role (step 1) |
| `... reduced detail - ONTAP refused a field ...` | this ONTAP version does not know a field; the job asked again with fewer | nothing: the section is still shown, a column may say `?`. Send the line to the people who maintain this repository |
| `CRC counters need ONTAP 9.11 or later` | older ONTAP has no CRC counter in its REST API | nothing: receive errors are still shown |
| SnapMirror is empty, but you have relationships | they are read on the destination cluster | add the destination cluster to `ontap_clusters` |

| You see | It means | Do |
|---|---|---|
| An aggregate you know is missing | root aggregates are not returned by ONTAP's REST API | by design: root aggregates are not listed |
| A volume shows ? for size | it is offline: ONTAP reports no space for it | it is listed in red as offline |
| The email is too long | many findings | lower `ontap_report_max_rows` / `ontap_snapshot_top`; the artifacts keep everything |
| `could not read ems - ... only the first 500 records were read` | many error events | raise `ontap_ems_max`, or lower `ontap_ems_hours` |

## Limits

- The fields the job reads were checked against NetApp's ONTAP REST API reference for ONTAP 9.10 to 9.16. It was tested against a fake ONTAP API (`tests/netapp/`), not a real cluster: run it on one cluster first, and send any `could not read` or `reduced detail` line you see.
- The CLI commands it reads through REST (system health, storage failover, SnapMirror progress) are ONTAP `show` commands at admin level, which the `readonly` role may run.
- It reports; it changes nothing. Fixes are done in System Manager or the ONTAP CLI.

# 15. Network device certificates: report and renewal

The TLS certificate each **switch, firewall, router, printer, iLO / iDRAC, UPS, storage array** ... presents, read from the controller with a TLS hello - **no login to the devices, nothing changed on them**. Two jobs:

| Template | Playbook | Does |
|---|---|---|
| Network device certificate report | `playbooks/net_cert_report.yml` | every certificate, soonest expiry first, and for each one due what the renewal would do. Read-only: never ServiceNow, never a CSR |
| Network device certificate renewal | `playbooks/net_cert_renew.yml` | a ServiceNow incident per certificate due (critical within 7 days, else warning), with the device, port, subject, names, issuer and how to renew it. Optionally, on Cisco IOS / IOS-XE, a CSR made on the device and attached |

Both can also write every certificate to the certificate inventory (CSV / SQL Server, CERT_INVENTORY.md).

## Which devices

**An inventory group** (the default, and the safest: only what you list). Put the devices in the group `net_cert_devices` (`net_cert_group`) of the inventory the template uses:

```yaml
net_cert_devices:
  hosts:
    sw01.example.mil:                        # connected to by name (or ansible_host)
    pr01.example.mil: {ansible_host: 192.0.2.30}
    ilo-esx01.example.mil: {net_cert_ports: [443, 17990]}
    rtr01.example.mil: {ansible_network_os: cisco.ios.ios, ansible_connection:
      ansible.netcommon.network_cli}
  vars:
    net_cert_ports: [443]                    # the ports read (default net_cert_ports: [443])
```

Per host: `net_cert_ports`, `net_cert_address` (what is connected to; default `ansible_host`, else the name), `net_cert_servername` (the name sent in the TLS hello; default the host's name). The jobs never log in to these hosts - a device needs no credential to be read.

**Subnets** - finds devices nobody listed. `net_cert_targets: subnets` (or `both`, with the group):

```yaml
net_cert_targets: both
net_cert_subnets: ['192.0.2.0/24', '198.51.100.0/25']
net_cert_ports: [443, 8443]
net_cert_exclude: ['192.0.2.1', 'ups*']        # never connected to: addresses, CIDR ranges, names
```

Every address of each subnet, each port. A swept address that answers is named by its DNS name (PTR) in the report. A port that does not answer is only counted. **Ask your network and security teams first:** a sweep is a port scan, and an intrusion detection system may alert on it. `net_cert_max_addresses` (4096) refuses a bigger sweep - a `/16` by mistake would take hours. Try the group first; sweep to find what it misses.

## What is due

`net_cert_warn_days` (30): due - an incident (warning); `net_cert_crit_days` (7) or expired: critical; `net_cert_notice_days` (60): blue in the report, no incident. `net_cert_only: ['sw01*']` limits a run to some devices - to try the renewal on one. `net_cert_unanswered_finding: true` opens an incident for an inventory device whose port gives no certificate (off: listed under *No certificate read*).

Old devices speak old TLS (1.0, weak ciphers): the jobs accept it - only the certificate is read, nothing is sent. An expired, self-signed or untrusted certificate is read like any other.

## The renewal

With `cert_renew_servicenow: catalog` there is no incident: a CSR made on a Cisco device (`net_cert_cisco_csr`) is ordered from your PKI's catalog item, and the other devices are in the report only: CERT_RENEWAL_CATALOG.md.

**The incident only** (the default, `net_cert_cisco_csr: false`): one incident per certificate due, for `servicenow_assignment_group`, with the steps: make the CSR on the device (its web interface or CLI - the key stays there), have the PKI sign it, import the issued certificate and its chain the way the vendor says. A run again notes the same incident; once the certificate is renewed the incident is noted as cleared (`servicenow_resolve_cleared: true` resolves it). A renewed certificate that comes due years later is a new incident.

**Cisco IOS / IOS-XE CSRs** (`net_cert_cisco_csr: true`) - for a due certificate of an inventory host with `ansible_network_os: cisco.ios.ios`, the job logs in (network_cli, the template's Machine credential for the devices) and, on the device:

1. `crypto key generate rsa general-keys label SA-RENEW-<expiry> modulus 2048` - a new key, **not exportable**: it never leaves the device (`net_cert_cisco_key_bits`; the label: `net_cert_cisco_trustpoint_prefix`);
2. `crypto pki trustpoint SA-RENEW-<expiry>` - `enrollment terminal pem`, the old certificate's subject and DNS names (`subject-name`, `subject-alt-name`), `rsakeypair` the new key;
3. `crypto pki enroll SA-RENEW-<expiry>` - the CSR, printed, attached to the incident with what to do next: `crypto pki authenticate` (the CA), `crypto pki import ... certificate` (the issued one), then `ip http secure-trustpoint` and restart HTTPS. **Then** `write memory` - the new key and trustpoint are in the running configuration only, and a reload before that loses the key (the issued certificate would not fit).

Nothing in use changes: the device keeps its certificate until someone imports the new one. A CSR is made once

- its incident is the record; a run again makes none while the incident has one. Other vendors (and NX-OS, ASA):

the incident only. **Try it first on one lab or spare device** (`net_cert_only: ['rtr-lab*']`), and in Check: the CLI's prompts differ between IOS releases. `subject-alt-name` needs IOS-XE 16 or later; on older IOS the job says it did not work (the incident stays, without a CSR).

## Templates

| Field | Report | Renewal |
|---|---|---|
| Inventory | the one with `net_cert_devices` | the same |
| Execution environment | any (the minimal one); the all-in-one EE for the Cisco CSRs (`cisco.ios`) and SQL Server | |
| Credentials | none | `ServiceNow API`; with `net_cert_cisco_csr`, a Machine credential that logs in to the Cisco devices |
| Variables | (optional) `net_cert_targets`, `net_cert_subnets`, `report_email_to` | `servicenow_assignment_group: <your network team>` |
| Limit | empty, **no** Prompt on launch (it runs on the controller) | the same |
| Job type | Run | **Check** first: it opens nothing, makes no CSR |

The Variables of each: `aap/extra_vars/net_cert_report.yml`, `net_cert_renew.yml`.

**A workflow** (daily): `[Network device certificate renewal]` alone - it reads, opens the incidents and emails the report (`report_email_to`). The report template is for a look on demand, or a weekly email.

## Troubleshooting

| You see | Means / do |
|---|---|
| `The inventory group net_cert_devices ... has no hosts` | put the devices in that group of the template's inventory, or sweep (`net_cert_targets: subnets`) |
| *No certificate read*: `refused` | nothing listens on that port: the device's `net_cert_ports` |
| `no answer (timeout)` | a firewall between the AAP node and the device, or a slow device: `net_cert_timeout` |
| `not TLS, or a TLS this controller cannot speak` | a plain-text port, or a device so old even TLS 1.0 fails |
| `the sweep would be more than 4096 addresses` | smaller subnets, or `net_cert_max_addresses` |
| `no CSR came back from the device` | the job could not log in to the Cisco device: its credential, `ansible_network_os`, `ansible_connection` |
| `crypto pki enroll printed no CSR` | the device's prompts differ: run the three commands above by hand once, and tell us what it printed |

# 16. Trust stores: who still trusts a compromised or dropped CA

When a certificate authority is compromised, the browsers block it within hours. Everything else keeps trusting it until *its own* trust store is updated - and a server has many: the system's bundle, every Java's `cacerts`, every Python environment's `certifi`, Node's built-in roots, the image inside every container, the bundle an application ships. A Go program does not even read a store for this: it carries its verification in the binary and has to be rebuilt.

The **trust check** reads all of them on every Linux host, and vCenter's and the ESXi hosts' trust stores through vCenter, and says which still hold a certificate on its list. Read-only - until you switch the removal on (below).

- the check `trust`, in the health check's **weekly** set (`playbooks/health_check.yml`: `health_checks: [trust]` runs it alone)
- the **Trust store report** (`playbooks/trust_report.yml`): the same check on every Linux host, one table of every store read, emailed (`report_email_to`). The hosts that hold one fail, so a workflow can open tickets.
- the **VMware trust store report** (`playbooks/vmware_trust_report.yml`): vCenter's trusted root store and each ESXi host's trusted CAs, from the controller with a *VMware vCenter* credential - its own template, so it runs on its own.

## The list

It ships with the check (`roles/check_trust/vars/main.yml`): **October 2026, the eMudhra (emSign) compromise** - eight intermediate CAs whose private keys were weak enough to compute from the certificates themselves (six for TLS, two for code signing), each issued twice: by eMudhra's own root and cross-signed by Sectigo's *AAA Certificate Services*. All sixteen are revoked. The roots:

| Certificate | Severity | Why |
|---|---|---|
| the 8 intermediates (16 certificates, `EM DV / OV / EV TLS CA - G3B / G3C`, `EM OV CS CA - G3B / G3C`) | **critical** | anything that trusts one accepts certificates anyone can sign |
| `emSign ECC Root CA - G3` | warning | it cross-signs the issuer of the compromised intermediates; removed by Mozilla (NSS 3.131.1), Chrome, certifi 2026.10.19, webpki-roots 1.0.10, curl's bundle. A store that still has it accepts the compromised chains |
| `emSign Root TLS CA - G3`, `emSign Root CS CA - G3` | warning | the direct issuers (Microsoft's root store only) |

| Certificate | Severity | Why |
|---|---|---|
| `AAA Certificate Services` | notice | Sectigo's root that cross-signed all eight. It is in nearly every trust store and many unrelated certificates chain to it. Red Hat's advisory says to blocklist it: decide first what breaks. *Notice* = in the report, no finding, no ticket - `check_trust_severity: {"AAA Certificate Services": warning}` once you decide |

Every certificate is matched by its SHA-256 fingerprint **and by its public key**: a certificate not on the list but carrying one of the compromised keys - a cross-sign nobody listed - is a hit too (shown as *the same key, as ...*).

**Your own list.** `check_trust_extra` adds certificates (a retired internal root, the next incident):

```yaml
check_trust_extra:
  - {name: "Old Corp Root CA", sha256: "AB:CD:...", kind: root, severity: warning, why: "retired 2026-01"}
```

`openssl x509 -in the.pem -noout -fingerprint -sha256` gives the fingerprint (with or without colons). A name that is already on the shipped list takes your entry's severity and words.

## What it reads, on a Linux host

| Store | Where | Fix |
|---|---|---|
| the system bundles | `/etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem` and the OpenSSL trust bundle (RHEL), `/etc/ssl/certs/ca-certificates.crt` (Debian, Ubuntu, Alpine), SUSE's - the same file under two names once | update `ca-certificates`, or blocklist the certificate: copy its PEM into `/etc/pki/ca-trust/source/blocklist/` and run `update-ca-trust` |
| p11-kit's live view | `trust extract`: the anchors NSS and GnuTLS clients see, and the blocklist | a blocklisted certificate is reported as *blocked*, no finding; still in a bundle too = `update-ca-trust` was not run (the finding says so) |
| Java | every `cacerts` / `jssecacerts` under `/usr/lib/jvm`, `/opt`, `/usr/local` ... and the ones running java programs use, with the JDK's version | update the JDK, or `keytool -delete`; a `cacerts` that links to `/etc/pki/java/cacerts` follows the system trust |
| Python | every `certifi/cacert.pem` (requests, urllib3, pip carry their own roots) under `/usr/lib`, `/opt`, `/home` ..., with the certifi version | `pip install -U 'certifi>=2026.10.19'` in each environment (the finding lists them) |
| Node | its built-in roots (`tls.rootCertificates`) and version, per node binary found | update Node to its line's fixed version (22.23.4, 24.22.1, 26.12.1), or run it with `--use-openssl-ca` |
| podman containers | the bundle inside every running container, root's and each logged-in user's (`podman exec ... cat`) | pull or rebuild the image, restart the container |
| certificate files | every `*.pem`, `*.crt`, `*.cer`, `*.der`, `cacert*`, `ca-bundle*` under `/etc`, `/opt`, `/srv`, `/usr/local`, `/var/lib`, `/root`, `/home` - only the files with a hit are listed; container image layers and the copies `update-ca-trust` extracts are skipped | an application's own bundle: update the application, or take the certificate out |
| Java keystores with other names | every `*.jks`, `*.keystore`, `keystore*`, `*truststore*` under the same folders (JKS natively, no password needed to read the trusted entries); `*.p12` / `*.pfx` with keytool and the default password | `keytool -delete` the entry |
| NSS databases | `/etc/pki/nssdb` (the system's: curl and other NSS clients on older RHEL), root's and each user's `~/.pki/nssdb` (Chrome, Evolution), read with `certutil -L` by nickname | `certutil -D -d sql:<dir> -n <nickname>` |
| Go programs | the running ones, and the executables under `/usr/local/bin`, `/opt`, `/srv`: the Go version each was built with (from the binary's build information) | rebuild with Go 1.27.3 / 1.26.10 or later (CVE-2026-111943: they reject every chain with one of the compromised keys); a program built with an older line has no fixed version to go to |

| Store | Where | Fix |
|---|---|---|
| NSS, ca-certificates | the packages' versions (`nss` below 3.131.1 is a finding; `check_trust_ca_certificates_fixed` compares `ca-certificates` once the errata is out) | update the packages |

Not read: Firefox and Thunderbird profiles (add the profile folder to `check_trust_nss_dirs` to read one), Rust programs built with `webpki-roots` (the roots are compiled in: rebuild with 1.0.10), Docker containers (podman only; the image layers under `/var/lib/docker` are skipped on purpose), keystores that need a password other than `changeit`. What could not be read is listed under *Not read*.

**The vCenter appliance itself** is a Linux (Photon) host with its own bundle, a bundled JRE and Python: put it in the Linux inventory (root over SSH, the shell enabled) and the same check reads it. vCenter's *trust store* is another thing: the VMware report below.

The folders searched, the limits and the fixed versions are settings: check_trust. Each search stays on the host's own filesystems (no NFS) and stops after `check_trust_timeout` seconds; a bigger server gets a longer timeout.

## The findings

One finding per *kind* of store on a host, so a host with four certifi bundles gets one finding that names all four, and a host with eighteen Go programs built before the fix gets one that names them:

| Finding | Severity | Means |
|---|---|---|
| `the system trust (tls-ca-bundle.pem and 1 more, ca-certificates 2025.2.80-7.el9) trusts emSign ECC Root CA - G3 - ...` | the worst certificate it holds | the system bundles still have it: every OpenSSL, Go, Python and curl client on the host accepts the compromised chains. Update `ca-certificates` or blocklist |
| `/opt/app/certs/bundle.pem (certificate file) trusts EM DV TLS CA - G3B - its private key is computable ...` | **critical** | a compromised intermediate itself, in an application's bundle |
| `2 Python certifi bundles (...) trusts emSign ECC Root CA - G3` | warning | `pip install -U certifi` in each environment |
| `3 Go programs built with a Go before the fix: ...` | warning (`check_trust_software_severity`) | rebuild them, or get the vendor's build made with a fixed Go |
| `Node.js 22.23.1 (/usr/bin/node) is below 22.23.4` | warning | update Node |
| `NSS 3.126.0 is below 3.131.1` | warning | update the `nss` package |

| Finding | Severity | Means |
|---|---|---|
| `no system CA bundle was read on this host` | warning | none of `check_trust_bundles` exists: name the host's bundle there |

The email is laid out in the order to act, then the proof:

1. **Remove now** - the stores that hold a compromised intermediate (red): what to do, per store
2. **Remove next** - the stores that still hold the distrusted root (amber), the ones the removal can clear: the system trust, Java, certifi, an application's file, vCenter, ESXi
3. **Then update** - what cannot be edited: Node's built-in roots, a container's image, Go programs, NSS, below the version that fixed it
4. **Listed only** - notice-level, nothing to do until you decide (AAA Certificate Services)
5. *Removed* - when the removal ran: what was done, each backup
6. **Everything read** - every store and program on every host with what it holds, worst first: also the ones that hold nothing (green) and the ones with only a notice-level certificate (blue). The count is how many certificates the store holds, not a problem count.
7. *Not read* - what could not be opened

## Windows

The **Windows trust store report** (`playbooks/win_trust_report.yml`, template *Windows - trust store report*) and the check `trust` in the Windows health check's **weekly** set read, with the same list:

| Store | Where | Fix |
|---|---|---|
| the machine certificate stores | Trusted Root (`Root`), Intermediate CAs (`CA`), the roots Windows Update keeps (`AuthRoot`), Trusted Publishers, Trusted People; the connecting account's Root and CA | put the certificate in the **Disallowed** store - that is Windows' blocklist, it wins over every other store, also over the roots Windows Update re-adds - and delete it from the store it is in (`check_trust_remove` does both; an AuthRoot entry is left in place, Disallowed covers it) |
| the Disallowed stores | a certificate there is *blocked*: listed, no finding | |
| Java | every `cacerts` under Program Files (Java, Adoptium, Corretto, Zulu, Microsoft, Red Hat): JKS natively, PKCS12 with the JDK's keytool | update the JDK, or `keytool -delete` |
| Python | every `certifi\cacert.pem` under the Python installs and Anaconda | `pip install -U certifi` in each |

Expect the two Microsoft-only emSign roots (`emSign Root TLS CA - G3`, `emSign Root CS CA - G3`, warning) on most Windows servers: Microsoft's root program ships them. They are the amber rows there until Microsoft moves them to its disallowed list, or you do with the removal. The Windows report has the same sections and the same `check_trust_extra`, `check_trust_severity` and `check_trust_remove`.

## VMware

The **VMware trust store report** (`playbooks/vmware_trust_report.yml`, template *VMware - trust store report*, a *VMware vCenter* credential, no inventory) reads, from the controller:

- **vCenter's trusted root store** (Administration > Certificates > Trusted Root Certificates: the REST API `certificate-management/vcenter/trusted-root-chains`, vCenter 6.7 Update 2 and later). The account needs *Certificate Management* > *Manage* for this call; without it the row says so.
- **each ESXi host's trusted CA certificates** (the host's certificate manager, through vCenter).

A hit fails the job; one table, worst first, emailed as the other VMware reports. `vmware_trust_esxi: false` reads vCenter only. Several vCenters: `vmware_vcenters`, as the other VMware reports. The same `check_trust_extra` and `check_trust_severity` apply; `check_trust_remove: true` deletes (above).

## Removing them

The report only reads. `check_trust_remove: true` on the *Trust store report* takes the listed certificates out of the stores that hold them, then reads every host again, so the report and the findings show what is left. **Run it as Check first** (Job type *Check*): the same plan, per store, nothing changed. Off by default; it never runs from the health check.

| Store | What happens | Undo |
|---|---|---|
| the system trust | the certificate is **blocklisted**: its PEM goes to `/etc/pki/ca-trust/source/blocklist/` (SUSE: `/etc/pki/trust/blacklist/`) and `update-ca-trust` runs; Debian and Ubuntu: its line in `/etc/ca-certificates.conf` is switched off and `update-ca-certificates` runs. The package's bundle is never edited (an update would put it back; the blocklist survives updates) | delete the file from the blocklist folder (or the `!` in the conf), run the update command again |
| Java `cacerts` | the entry is deleted (JKS natively, with the keystore's integrity digest rewritten; PKCS12 with `keytool`, password `check_trust_keystore_password`). A `cacerts` that follows the system trust (`/etc/pki/java/cacerts`, `/etc/ssl/certs/java/`) is left to the blocklist. A running JVM keeps the old trust until it restarts: restart the Java services that use the keystore | copy the backup back |
| Python `certifi` | the certificate's block (and its comment lines) is cut out of `cacert.pem`. One a package owns is left alone (update the package); one that holds nothing else is left alone (cut, requests would have no roots: `pip install -U certifi` there) | copy the backup back |
| an application's file | a bundle loses the block; a file that **is** the certificate (a DER or a single PEM) is moved to the backup folder | copy the backup back |
| vCenter, ESXi | `check_trust_remove: true` on the *VMware trust store report*: the chain is deleted from vCenter's trusted root store; each ESXi host's trusted CA list is replaced by the same list without it (its CRLs kept). The account needs *Certificate Management* > *Manage* | add the certificate again in vCenter |

| Store | What happens | Undo |
|---|---|---|
| containers, Node, Go programs, NSS | nothing: pull or rebuild the image, update Node, rebuild the Go program, update `nss`. They stay in the report | |

Every file changed is copied first to `check_trust_remove_backup_dir` (`/var/backups/site-automation/trust/<date-time>/`, the path with `__` for `/`); the report's *Removed* table names the backup of each. Only certificates at `check_trust_remove_severities` go (critical and warning): *AAA Certificate Services* at notice is never touched unless you put `notice` there or raise its severity - and that one you decide on first, because much chains to it. `check_trust_remove_kinds` narrows it to, say, `[system]`.

```yaml
check_trust_remove: true                 # the switch (Check mode first) - on either report
check_trust_remove_kinds: [system, java, python, file]
```

After a run, the hosts that still hold something (Node, a Go program, a container) still fail the job: the finding says what to update. A host where everything went passes.

## Set it up

| Field | Value |
|---|---|
| Job template | *Trust store report*, `playbooks/trust_report.yml` (the Linux hosts) |
| Credentials | `Linux ssh (sudo)` |
| Variables | `aap/extra_vars/trust_report.yml`; `report_email_to` as the other reports |
| Job template | *VMware - trust store report*, `playbooks/vmware_trust_report.yml` (vCenter and the ESXi hosts; `aap/extra_vars/vmware_trust_report.yml`) |
| Credentials | `VMware vCenter` |
| Job template | *Windows - trust store report*, `playbooks/win_trust_report.yml` (the Windows servers; `aap/extra_vars/win_trust_report.yml`); the check `trust` is in the Windows health check's weekly set too |
| Credentials | `Machine (Windows)` |
| Schedule | weekly with the health check's weekly set (it is in it), and on demand when a CA incident is announced: add the certificates to `check_trust_extra`, run the report |

In a workflow: *Trust store report* > *ServiceNow tickets* (Run always), like the certificate report. The tickets are per finding, so one per host for the system trust, one per host for Python, and so on.

## When the fixes land

An updated `ca-certificates` drops `emSign ECC Root CA - G3`; the extracted bundles, Java's `/etc/pki/java/cacerts` and p11-kit's view follow (`update-ca-trust` runs in the package's update). The report then shows the system rows green. The certifi, Node, Java (own `cacerts`), container and Go rows clear one by one as each is updated - the report is the checklist. `AAA Certificate Services` stays, at notice, until you decide.

# 17. STIG drift: has a server moved away from its checklist?

A server is STIG'd, reviewed and its checklist accepted in STIG Manager. Then a package update, a hand edit or another team's change quietly undoes a setting, and nobody knows until the next assessment. The **STIG drift report** finds that on the next scheduled run.

- Each Linux host is **scanned** against the DISA STIG with OpenSCAP: `oscap` and the SCAP Security Guide's STIG profile for its RHEL version (scap-security-guide 0.1.82: RHEL 9 **V2R9**, RHEL 10 **V1R2**, RHEL 8 too). The scan's HTML report, results and summary are **kept on the host**.
- The result is **compared with the host's checklist in STIG Manager** - read-only: the job never writes to STIG Manager - and with the **host's last scan**.
- What moved is a finding; the hosts with one fail, so a workflow can open tickets. The report is emailed (`report_email_to`) and published (`report_web_dest`) like every other report.

Linux only. VMware has its own STIG tooling (the appliance), and Windows has no free SCAP content.

## What it reports

| | When | Finding |
|---|---|---|
| **Drifted** | the scan fails a rule that STIG Manager has as **Not a Finding** (or **Not Applicable**) | yes: CAT I **critical**, CAT II and III **warning** (`check_stig_severity`) |
| **New fail** | the scan fails a rule that **passed in the host's last scan** (and STIG Manager does not already have it Open) | yes, the same severities |
| Fixed, still Open | the scan passes a rule STIG Manager still has **Open** - the checklist is behind | no: report only. Update the review (and close its POA&M item) |
| No review | the scan fails a rule STIG Manager has no review for | no: report only |
| Not compared | the host's asset was not found in STIG Manager, or matched two | `check_stig_stigman_missing_severity` (warning; or `none`) |
| Not scanned | no `oscap` or no STIG content on the host, or the scan failed | warning |

A rule is matched with STIG Manager by its **STIG rule ID** (`SV-257985`), whatever the revision (`SV-257985r1069364_rule` in the scan, `SV-257985r1045000_rule` in an older checklist): a new STIG release does not break the comparison. Several SCAP checks of one STIG rule count once - the worst result. Only the reviews with a status in `check_stig_stigman_statuses` count as the checklist (saved, submitted, accepted - a rejected review does not); `[accepted]` is stricter.

The first scan of a host has nothing of its own to compare with: without STIG Manager it is just kept. From the second scan on, a new fail is reported even without STIG Manager.

## On the host

`check_stig_dir` (`/var/lib/site-automation/stig`, root-only):

```
20261024T184350Z_report.html       oscap's HTML report (open it in a browser: every rule, why it
  failed, the fix)
20261024T184350Z_results.xml.gz    the XCCDF results (STIG Manager and eMASS can import them)
20261024T184350Z_summary.json      one result per STIG rule: what the next scan is compared with
latest-report.html, latest-summary.json
```

The last `check_stig_keep` (10) scans are kept. A dry run (Job type **Check**) scans and compares but keeps nothing. A scan takes from a few seconds to a few minutes and only reads; it runs as root (`become`).

## STIG Manager: the login and finding each host's asset

The job logs in as a **Keycloak client** with the client credentials grant - your **autostig** client, or another one. Give it:

- the scope `stig-manager:collection:read` (or `stig-manager:collection`);
- a grant in each collection to read, for its service account user (`service-account-autostig`): **Full** sees every asset in the collection; **Restricted** sees only the assets and STIGs assigned to it, so a host whose asset it was not given is reported as not found. With only the read scope, the token cannot write whatever the grant.

In AAP, attach a **STIG Manager API** credential (`aap/credential_types/stigman_api.yml`: the token URL, the client ID, the secret) to the template, and set `check_stig_stigman_api`:

YAML

```yaml
check_stig_stigman_api: https://stigman.example.mil/api
# from the command line instead of the credential (keep the secret in a vault):
# check_stig_stigman_token_url:
  https://keycloak.example.mil/realms/stigman/protocol/openid-connect/token
# check_stig_stigman_client_id: autostig
# check_stig_stigman_client_secret: "{{ vault_stigman_secret }}"
```

Only GET requests reach STIG Manager (and one POST to Keycloak for the token). Behind nginx with `ssl_verify_client optional` the API answers the token without a client certificate. A private CA: `check_stig_stigman_ca_path`.

**The right asset in the right collection.** Set each host's collection where your hosts are grouped - group_vars, host_vars, or the inventory's variables in AAP:

inventory group rhel_servers

```
check_stig_stigman_collection: RHEL Servers
```

The job then looks for the host's asset only there: an asset whose **name or FQDN** is the host's inventory name, then its short name (`web01` for `web01.example.mil`), case-blind, assigned the host's STIG (`check_stig_stigman_benchmarks`: `RHEL_9_STIG` for RHEL 9 ...). An asset named differently: `check_stig_stigman_asset: WEB01-PROD` for that host. Without a collection, every collection the client can read is searched (or only `check_stig_stigman_collections`); a host that matches assets in two of them is listed - *2 assets match: web01 in RHEL Servers, web01 in Lab* - and not compared until its collection is set.

The report's every-host table shows what was matched - *RHEL Servers / web01 (V2R7)*, the collection, the asset and the checklist's STIG revision - beside the scan's (*V2R9*).

## Setting it up

1. On the hosts: `openscap-scanner` and `scap-security-guide`, from your RHEL repositories (Satellite). Or `check_stig_install: true` installs them on the first run.
2. In STIG Manager: the client and its grants (above).
3. In AAP: the job template (docs/SETUP_AAP.md, "STIG drift report"): `playbooks/stig_drift_report.yml`, the machine credential and the STIG Manager API credential; `aap/extra_vars/stig_drift_report.yml` has its settings.
4. Run it once as **Check** with a Limit of one host, then for real, then schedule it weekly (and in the patching workflow after the patch job).

## Settings

Every one is in `roles/check_stig/defaults/main.yml` and docs/VARIABLES_REFERENCE.md. The ones you are most likely to set:

| Setting | Default | |
|---|---|---|
| `check_stig_stigman_api` | empty | STIG Manager's API; empty = compared with the last scan only |
| `check_stig_stigman_collection` | empty | the host's collection (per group or host) |
| `check_stig_stigman_asset` | the inventory name | the host's asset name, when it differs |
| `check_stig_profile` | `stig` | `stig_gui` for a host with a desktop, or a full profile id |
| `check_stig_tailoring` / `_src` | empty | your site's tailoring file (on the host / in your repository) |
| `check_stig_datastream` / `_src` | the host's SSG | another datastream - newer content, or a DISA SCAP benchmark |
| `check_stig_severity` | CAT I critical, II and III warning | the finding's severity by CAT |
| `check_stig_stigman_statuses` | saved, submitted, accepted | the reviews that count as the checklist |
| `check_stig_dir`, `check_stig_keep` | `/var/lib/site-automation/stig`, 10 | where the scans are kept, and how many |

## Not in this version

- Nothing is written to STIG Manager - by design. To import a scan, the results file is on the host.
- No remediation. A drifted rule's fix is in the HTML report; fixing it automatically, behind an approval, is a possible later step.
- In no health check shortcut (daily, weekly, all): a scan takes minutes. The health check runs it when asked by name (`health_checks: [stig]`); its own template is the usual way.

# 18. Network device configurations: backup and load

Two jobs for **Cisco IOS / IOS-XE** and **Palo Alto Networks PAN-OS** devices:

| Template | Playbook | Does |
|---|---|---|
| Network config backup | `playbooks/net_config_backup.yml` | every device's running configuration, kept in a **private git repository** (one commit when any changed - its history is the record of every change) and / or on a **file share** (the latest and dated copies). The report shows what changed, with the diff; optionally a ServiceNow incident for each change. Read-only on the devices |
| Network config load | `playbooks/net_config_load.yml` | **lines** sent to the devices (IOS configuration lines; PAN-OS changes at an xpath), or a device **restored** to a backup - after an approved change request, and always tried in **Check** first: what would change, nothing sent |

**The backups hold the devices' secrets** - password hashes, SNMP communities, VPN and API keys - so that a restore puts back exactly what was there. Keep them in a **private, internal** repository or share that only the devices' admins can read - never in this (public) repository. What the jobs *show* - the report, the email, an incident, the note on a change - has the secret values replaced by `<hidden>`; the job output never shows a configuration (every task that has one runs with `no_log`).

## Which devices

An inventory group, `net_config_devices` (`net_config_group`), in the template's inventory:

YAML

```yaml
net_config_devices:
  hosts:
    rtr01.example.mil:
      ansible_network_os: cisco.ios.ios
      ansible_connection: ansible.netcommon.network_cli
    sw01.example.mil: {ansible_host: 192.0.2.21, ansible_network_os: cisco.ios.ios,
      ansible_connection: ansible.netcommon.network_cli}
    fw01.example.mil: {ansible_network_os: paloaltonetworks.panos}
```

- **Cisco IOS / IOS-XE**: over its CLI (network_cli, SSH), logged in with the template's **Machine** credential (the user needs privilege 15 - or `ansible_become: true`, `ansible_become_method: enable` and the enable password in the Machine credential). The backup runs `show running-config` (`net_config_ios_command`).
- **Palo Alto PAN-OS**: over its **XML API**, from the controller (HTTPS, port 443: `net_config_panos_port`), with the **Palo Alto PAN-OS API** credential (`aap/credential_types/panos_api.yml`): an API key, or a username and password. The firewall's certificate is checked (`net_config_panos_validate_certs`, `net_config_panos_ca_path` for your CA). The backup is the running configuration as *Export named configuration snapshot* gives it. No collection is needed - the job's own module talks to the API.

- The platform comes from `ansible_network_os`, or set `net_config_platform: ios` / `panos` on the host. `net_config_address` connects to another address than `ansible_host` (PAN-OS).
- `net_config_only: ['rtr-lab*']` limits a run to some devices - to try a job on one first.
- The template's **Limit** works too, with `localhost` in it (`rtr-lab01,localhost`): the job keeps and reports its work on localhost. A Limit without it is refused before anything runs.

## The backup

**git** (`net_config_store: git`, the default) - set `net_config_git_repo` to the repository (`ssh://git@git.example.mil/netops/configs.git`, or `https://...`), with the **Network config git repository** credential (`aap/credential_types/net_config_git.yml`): an SSH **deploy key with write access**, or for https a username and an access token. Each run clones `net_config_git_branch` (`main`; made when the repository is new and empty), compares each device with its file - `configs/<device>.cfg` (IOS) or `.xml` (PAN-OS) in `net_config_git_path` - and commits the new and changed ones in **one commit** ("Backup 2026-10-21 06:00 UTC: 2 changed (rtr01..., fw01...)"), then pushes. Nothing changed: no commit. The git server's SSH host key is learned on the first connection (`net_config_git_host_key_checking: accept-new`), or give it (`net_config_git_known_hosts`, `net_config_git_host_key_checking: yes`). In an execution environment ssh's own `known_hosts` starts empty every job, so `accept-new` learns the key every time and checks nothing: give the key (`ssh-keyscan -t ed25519 git.example.mil`, compared with what the git server's admin says) with `yes` - the only real check. An https token goes to git as a header in a git config file in the job's work folder (mode 0600, deleted at the end), never on a command line or in the environment. `git log -p configs/rtr01.example.mil.cfg` is then every change of rtr01, with its date.

**A file share** (`net_config_store: share`, or `both`) - `net_config_share_dir` on the AAP node (only kept if AAP exposes that path to jobs) or on `net_config_share_host` (a Linux host, or a Windows file server; its credential on the template): a folder per device with the latest (`rtr01.example.mil.cfg`) and, each time it changed, a dated copy (`rtr01.example.mil-20261021T060000Z.cfg`); the oldest beyond `net_config_share_keep` (30) go. With `both`, the report and the incidents compare with git.

IOS prints lines that change on every read although nothing changed (*Current configuration : N bytes*, *Last configuration change at ...*, `ntp clock-period`): they are left out, so an untouched router is never "changed".

**ServiceNow** (`net_config_tickets: true`, the **ServiceNow API** credential, `servicenow_assignment_group`): an incident (warning) when a device's configuration changed - one per change, with the diff (secrets hidden), "if the change was not planned (no change request), find out who made it" - and one when a device could not be backed up. The next good backup notes them as cleared (`servicenow_resolve_cleared: true` resolves them). `net_config_ticket_changes` / `net_config_ticket_failures` switch either off.

A device that could not be backed up (unreachable, the login, the API) is listed under *Not backed up* with the reason; its last good backup stays as it was, and the job fails (red) so the schedule shows it (`net_config_fail_on_failed: false` keeps it green).

## The load

Always run it in **Check** first: it reads each device and shows what would change - the lines it would send, or the restore's diff - and sends nothing. A Check needs no change request.

A real run needs the **change request** it belongs to - `net_config_change: CHG0012345`, in state **Implement** (`net_config_change_states`) - and the **ServiceNow API** credential: the job reads the change first and stops when it is in another state, and afterwards writes a work note on it (`net_config_change_note`): each device, what was sent or restored, the result. At most `net_config_max_devices` (10) devices at once - narrow it with

`net_config_only` or the template's Limit.

**Lines** (`net_config_load_mode: lines`):

YAML

```
net_config_change: CHG0012345
net_config_only: [rtr-lab01*, fw-lab01*]
# IOS: configuration lines, as typed in configuration mode - only the ones not there yet are sent
net_config_lines:
  - ntp server 192.0.2.123
  - logging host 192.0.2.50
net_config_parents: []          # e.g. ["interface GigabitEthernet1"] for lines under one
# PAN-OS: changes at an xpath - set (adds or replaces the element's children), edit (replaces it),
  delete
net_config_panos_entries:
  - xpath: "/config/devices/entry[@name='localhost.localdomain']/deviceconfig/system"
    element: "<login-banner>Authorized use only</login-banner>"
```

- IOS: then `write memory` (`net_config_save: false` leaves only the running configuration - a reload undoes it).
- PAN-OS: the changes go into the candidate configuration, are **validated**, then **committed** (the commit's description: the change number and the AAP job). A change that does not validate is reverted - nothing is committed. The firewall must have **no uncommitted changes** first: someone's half-done work in the web interface would be committed with ours - the job refuses (`net_config_panos_allow_pending: true` overrides).

**A restore** (`net_config_load_mode: restore`): each device put back to its backup - git: the newest, or `net_config_restore_ref` (a commit, tag or branch: `git log configs/rtr01.example.mil.cfg` shows which); share: the latest, or a dated copy (`net_config_restore_ref: 20261021T060000Z`).

- IOS: the backup is copied to the device's flash by **SCP** (`net_config_ios_file`, `flash:site-automation-restore.cfg`; the device needs `ip scp server enable`, and the user privilege 15), then `configure replace flash:site-automation-restore.cfg force` - IOS works out the difference itself and leaves what is the same untouched; the copy is deleted, the result read back, then `write memory`. On IOS-XE the file system may be `bootflash:` - set `net_config_ios_file`.
- PAN-OS: the backup is imported as a named configuration, loaded as the candidate, validated and committed.
- A device already as its backup: unchanged, nothing sent.

## Templates

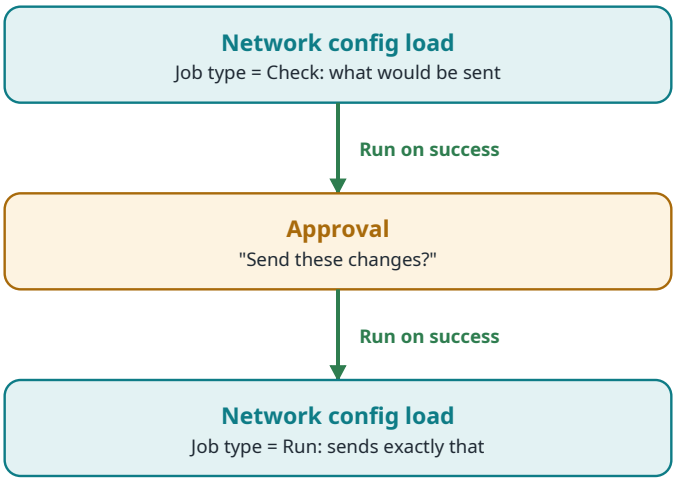
| Field | Backup | Load |
|---|---|---|
| Inventory | the one with `net_config_devices` | the same |
| Execution environment | the all-in-one EE (`cisco.ios`, `ansible.netcommon`) | the same (with `scp` for IOS restores) |
| Credentials | Machine (the Cisco devices), Palo Alto PAN-OS API, Network config git repository; ServiceNow API with `net_config_tickets` | Machine, Palo Alto PAN-OS API, ServiceNow API; Network config git repository for a restore from git |
| Variables | `net_config_git_repo` (or `net_config_store: share` and `net_config_share_dir`), `report_email_to` | `net_config_git_repo`; `net_config_change`, the lines or the restore at launch (a survey, or the API's `extra_vars`) |
| Limit | empty | empty, or the devices (with `net_config_only`) |
| Job type | Run | **Check** first, then Run |

The Variables of each: `aap/extra_vars/net_config_backup.yml`, `net_config_load.yml`.

**A schedule** (daily): *Network config backup*.

### A load, with an approval

The approver sees the dry run's report - exactly what will be sent - before anything is.




1. On the template *Network config load*, tick **Prompt on launch** for **Job type**.
2. **Create workflow job template** `Network config load (workflow)`. Tick **Prompt on launch** for its **Variables**: each launch, paste the change there - every step gets the workflow's Variables:

```yaml
net_config_change: CHG0012345
net_config_load_mode: lines
net_config_only: [rtr-lab01*]
net_config_lines:
  - ntp server 192.0.2.123
```

(A survey cannot carry the lines: they are a list.)

3. **Add step** *Network config load*, Job type **Check**. On it, **+** › **Run on success** › **Approval**: name `Send these changes?`, timeout e.g. 4 hours, description `Open the Check job: its report lists each device and the lines or the diff.`
4. On the approval, **+** › **Run on success** › *Network config load*, Job type **Run**. **Save**.

The Check step needs no change request in state Implement; the Run step does, and stops if the change moved on meanwhile.

## Troubleshooting

| You see | Means / do |
|---|---|
| `which platform? Set ansible_network_os ...` | the host has no `ansible_network_os` (or `net_config_platform`): `cisco.ios.ios` or `paloaltonetworks.panos` |
| `Could not clone ...` | the git credential (the deploy key on the server, with write access; or the username and token), the URL, the server's host key (`net_config_git_known_hosts`) |
| `Unable to connect to port 22` / `command timeout` | the AAP node cannot reach the device's SSH, or the login is slow: the Machine credential, `ansible_command_timeout` |
| PAN-OS `Invalid Credential` / `HTTP 403` | the API key or the user (its admin role must allow the XML API: export for the backup, configuration and commit for the load) |
| `the firewall has uncommitted changes` | someone's changes wait in the candidate: commit or revert them first (the web interface: *Commit* / *Revert Changes*) |
| `validation failed - nothing committed, the candidate reverted` | the change refers to something that is not there (a zone, an object): fix the entries |
| `entry 2 of 3 (set /config/...): ... - nothing committed, the candidate reverted` | the firewall refused that entry (a wrong xpath or element); the entries before it were taken back too: fix that entry |
| `the copy to the device failed (scp ...)` | `ip scp server enable` on the device, the user's privilege 15, the file system (`net_config_ios_file`) |
| `configure replace failed` | IOS refused the backup (a command this release does not know - a backup from another model or release?); nothing was replaced |
| `still different: N line(s)` after a restore | IOS added defaults of its own (a newer release): look at the diff in the next backup |
| `change CHG... is Scheduled - a load needs it in Implement` | move the change to Implement, or (your process permitting) `net_config_change_states` |

# 19. The certificate inventory: CSV, JSON and SQL Server, for a dashboard

Every certificate report can also write **every certificate it found**, with the same fields from each, for a dashboard (Power BI, Excel, Grafana ...). Turn it on in the template's Variables (or `all.yml`):

| Report | Template | Its rows in the inventory (source) |
|---|---|---|
| Linux | *Certificate report* (`playbooks/cert_report.yml`) | `linux`: files, ports, keystores, the trust store |
| Windows | *Windows certificate report* (`playbooks/win_cert_report.yml`) | `windows`: every machine store, bindings, TLS endpoints |
| VMware | *VMware - certificate report* (`playbooks/vmware_cert_report.yml`) | `vmware`: vCenter's Machine SSL certificate, each ESXi host's |
| Network devices | *Network device certificate report* (`playbooks/net_cert_report.yml`) | `net`: switches, firewalls, printers, iLO / iDRAC ... (NETWORK_CERTS.md) |

Two ways, each on by itself - **files** (CSV and JSON) and a **SQL Server table**. A report with neither writes nothing (the default). A Check run (dry run) writes nothing either.

## What each certificate has

| Field | Example | What it is |
|---|---|---|
| `id` | `9c1f...` (64 hex) | the same host, place and certificate keep their id from run to run (the SQL table's key) |
| `source` | `linux` | linux, windows, vmware, net |
| `host` | `web01.example.mil` | the server or device |
| `location` | `/etc/pki/tls/certs/web.crt`, `LocalMachine\My, used by https 0.0.0.0:443`, `https://192.0.2.10:443` | where it is |
| `subject`, `issuer` | `CN=web01.example.mil,O=Example,C=US` | |
| `sans` | `DNS:web01.example.mil; IP:192.0.2.10` | its names (a list in the JSON) |
| `not_after`, `days` | `2027-01-31`, `103` | when it expires; days left when it was read (negative: expired) |
| `status` | `warning` | expired, critical, warning, notice, ok - the report's own days (warn, crit, notice) |
| `tier` | `ca-signed` | **root** (a self-signed CA that signs certificates, or one a host trusts), **intermediate** (a CA signed by another), **ca-signed** (issued by a CA), **self-signed** |
| `trust_store` | `false` | true: a certificate the host trusts (Trusted Root, the trust store), not its own |
| `in_use` | `true` | bound to a service (an https binding, WinRM, a port, vCenter, a device); empty: not known |
| `public_key`, `sig_alg` | `RSA 2048`, `sha256WithRSAEncryption` | |
| `serial`, `sha1`, `sha256` | | the serial number; the fingerprints (sha1 is Windows' thumbprint) |
| `scanned_at`, `job` | `2026-10-20T06:00:00Z`, `4711` | when, and the AAP job that read it |

The same certificate in two places (a file and the port it is served on) is two rows - one per place - with the same `sha256`: count `DISTINCT sha256` for certificates, rows for places.

## Files: CSV and JSON

```yaml
cert_inventory_files: true
cert_inventory_dir: /var/lib/site-automation/cert-inventory
cert_inventory_host: files01.example.mil      # optional - see below
cert_inventory_history: false                  # true: also a dated copy each run
```

Each report writes `<source>-certificates.csv` and `.json` (`linux-certificates.csv` ...), replaced each run.

**Where they go.** A job runs in a container that is gone when it ends, so the files must land somewhere that stays:

- `cert_inventory_host` (recommended): a host in the job's inventory - a Linux host with the share mounted, or a Windows file server (then `cert_inventory_dir` is a Windows path, e.g. `D:\Shares\Certificates`). The job template must carry a credential that logs in to it. A Windows file server needs the `ansible.windows` collection in the execution environment (the all-in-one EE has it).
- **The AAP node itself**, `cert_inventory_host` empty: only when an administrator exposes the folder to jobs - **Settings > Job settings > Paths to expose to isolated jobs**: `/var/lib/site-automation/cert-inventory:/var/lib/site-automation/cert-inventory:rw`, and the folder exists, writable, on every execution node. Otherwise the job writes into its container and the files are lost - the job's last line says so.

**Power BI:** *Get data* > *Folder* on the share, *Combine & Transform*: one table of every source.

## SQL Server

```yaml
cert_inventory_sql: true
cert_inventory_sql_server: sql01.example.mil
cert_inventory_sql_database: Certificates
cert_inventory_sql_table: dbo.cert_inventory      # schema.table
cert_inventory_sql_create: true                   # the first time: make the table and its view
```

1. **The credential.** Create the credential type *SQL Server (certificate inventory)* (`aap/credential_types/cert_inventory_sql.yml`), then one credential: a SQL login, or `DOMAIN\account` for Windows authentication. Attach it to each certificate report template that writes the table.
2. **The account** needs `SELECT, INSERT, UPDATE` on the table - and, once, `CREATE TABLE` and `CREATE VIEW` for `cert_inventory_sql_create: true` (or have the DBA run the table's script: `roles/cert_inventory/templates/create.sql.j2`, with `{{ _ci_table }}` replaced by the table name).
3. **SQL Server 2016 or later** (the rows go in as one JSON parameter, `OPENJSON` - no certificate's text is ever part of the SQL). The execution environment needs `pymssql` and `community.general` (the all-in-one EE has both).

**The table:** a row per host, place and certificate, with every field above and `first_seen`, `last_seen` and `present`. Each run adds the new rows and updates the ones it saw again; on the hosts it read, a row it did not see again (the certificate was replaced or removed) gets `present = 0` - kept, for the history. A host the run did not read (unreachable, left out) keeps its rows as they were. **The view** `<table>_current` is the rows present now - what the dashboard shows.

**Power BI:** *Get data* > *SQL Server*, the database, the view `dbo.cert_inventory_current` (Import, refreshed after the reports run). Useful measures: certificates by `tier` and `status`; expiring in 30 / 60 / 90 days (`days`);

self-signed in use (`tier = 'self-signed' AND in_use = 1`); weak keys (`public_key = 'RSA 1024'`) or signatures (`sig_alg LIKE 'sha1%'`); trend (`first_seen` by month).

## Schedule

Run the four reports weekly (the Certificate watch workflows run them daily - WORKFLOWS_AND_SCHEDULES.md), with the same `cert_inventory_*` settings - in `all.yml` they apply to every report at once.

## Troubleshooting

| You see | Means / do |
|---|---|
| `... on localhost (AAP node: only kept if AAP exposes that path to jobs ...)` and no files on the share | set `cert_inventory_host`, or expose the folder (above) |
| `cert_inventory_host '...' is not in this job's inventory` | add the file server to the inventory, with a credential on the template |
| `cert_inventory_sql: true needs cert_inventory_sql_server ...` | a setting, or the SQL Server credential, is missing |
| `pymssql` / `mssql_script` not found | the execution environment lacks them: the all-in-one EE (EXECUTION_ENVIRONMENT.md) |
| `Invalid object name 'dbo.cert_inventory'` | the table is not there: `cert_inventory_sql_create: true` once, or the DBA's script |
| `Login failed` | the credential's login, or the database's permissions |

# 20. Certificate renewal through the PKI's ServiceNow catalog item

The certificate renewals - **Certificate renewal - CSRs** (Linux), **Windows certificate renewal**, **VMware - certificate renewal** and **Network device certificate renewal** - make a new key and a CSR for each certificate that expires soon. By default each CSR goes on a ServiceNow **incident**, and a person submits it to the PKI.

With `cert_renew_servicenow: catalog` there is no incident: the job **orders your PKI's catalog item** (for example *DoD PKI Certificate Request*) for each certificate, with the CSR in the item's *Please add CSR* question - exactly what a person would do on the portal.

| | incident (the default) | catalog |
|---|---|---|
| Per certificate | an incident, the CSR attached, a work note with the next steps | a request (REQ / RITM) on the PKI's catalog item, the CSR in its CSR question |
| Who sends it to the PKI | a person, by hand | the job |
| A run again | finds the incident; no second CSR | finds the request by the CSR on it; no second order |
| Once the certificate is replaced | the incident is noted as cleared (or resolved) | nothing to close: the PKI closes the request |

## What a run does

For each certificate that expires within the warning days (30 by default):

1. **The CSR** is made where the key stays, as always: on the Linux or Windows server, on vCenter or the ESXi host, on the Cisco device. The key never leaves it.
2. **Already ordered?** The job looks for a request carrying this CSR. The CSR's name in the order says which certificate it is for and when that certificate expires: `web01.example.mil-1a2b3c4d-exp20261114.csr`.
   - found **open, in progress, or Closed Complete** (the PKI issued it - install it): not ordered again.
   - found only **Closed Incomplete or Closed Skipped** (rejected, cancelled - `cert_renew_catalog_reorder_states`): ordered again. On vCenter, an ESXi host or a Cisco device that also means a new CSR.
   - not found: ordered.
3. **The order**: one request per certificate. Your answers (`cert_renew_catalog_variables`), the CSR in the CSR question, and - if you name a question for it (`cert_renew_catalog_names_variable`) - the certificate's names (subjectAltName).
4. **Checked**: the job reads the requested item (RITM) back and makes sure the CSR is on it. If ServiceNow did not move the CSR into the request, the job attaches it to the RITM (the paperclip) - the PKI sees it either way, and the next run finds the order by it.

The job output lists every certificate:

```
ORDERED RITM0012345 (REQ0012344) | web01.example.mil | web01.example.mil-1a2b3c4d-exp20261114.csr
  | the CSR is in the request (its question csr_file)
ORDERED ALREADY RITM0012001 (Work in Progress) | app01.example.mil |
  app01.example.mil-77aa1c2e-exp20261102.csr
ORDERED ALREADY RITM0011876 (Closed Complete) | db01.example.mil | ... | the PKI issued it:
  install it
ORDERED RITM0012350 (REQ0012349) (again: RITM0011990 closed Closed Incomplete) | ...
FAILED  | web02.example.mil | ... | the order was refused: HTTP 400: Mandatory Variables are
  required - ...
```

A certificate that could not be ordered fails the job (the others are still ordered), so a schedule shows red.

**Not ordered here:** a network device without a CSR made by the job (anything but Cisco IOS / IOS-XE with `net_cert_cisco_csr`): it is in the report; make its CSR on the device and order it by hand. A VMCA-signed vCenter or host certificate: renew it in vCenter, as before.

## Before you start

- **The item's sys_id.** Open the item on the portal: the address ends in `?id=sc_cat_item&sys_id=a3034a7325933210361adc6bd7ce8c93`. That sys_id is the surest way to name it.
- **The account that orders.** The renewal orders as the account in the *ServiceNow API* credential. That account must be allowed to order the item (the item's *Available for* / user criteria) and to read the requests it made. Ask the PKI team whether they accept requests from a service account, and who should be *Requested For*.
- **Same instance.** The job orders on the instance in the credential. If the PKI's catalog is on another ServiceNow, give the renewal templates a *ServiceNow API* credential for that one.

## Step 1 - the item's questions (their internal names)

The settings need each question's **internal name** (e.g. `requested_for`), not its label ("Requested For"). Two ways to see them:

**In your browser, no job needed.** Logged in to ServiceNow, open:

```
https://<your-instance>/api/sn_sc/servicecatalog/items/<the item's sys_id>
```

It shows the item as JSON (Firefox lays it out as a tree). Under `variables`, each question has:

| Field | Means |
|---|---|
| `name` | its internal name - what the settings use |
| `label` | what the form shows |
| `mandatory` | `true`: it needs an answer |
| `type` | 33 attachment (the CSR's), 31 requested for, 8 reference, 5 select box, 6 single line text, 2 multi line text, 11 label (nothing to fill in) |
| `reference` | for a reference: the table (`sys_user`, `cmn_location` ...) |
| `choices` | for a select box: each `value` and `label` |

This is what *your* account sees. The job sees what the credential's account sees - step 3 checks that.

With catalog admin rights there is also the list:
`https://<your-instance>/item_option_new_list.do?sysparm_query=cat_item=<the item's sys_id>`
(columns Name, Type, Mandatory).

**Or run the catalog test** (step 3): it prints the same, one line per question.

## Step 2 - the settings

In `playbooks/group_vars/all.yml` (every renewal job reads them). An example for a form like the DoD PKI one - **the internal names are examples**: use the ones step 1 shows.

YAML

```
cert_renew_servicenow: catalog
cert_renew_catalog_item: a3034a7325933210361adc6bd7ce8c93     # or its name: "DoD PKI Certificate
  Request"
cert_renew_catalog_variables:
  requested_for: svc_aap_sn            # a reference: a user name, a name, or a sys_id
  title_rank: CTR
  email: sysadmins@yoursite.mil
  dsn: "314-555-0100"
  directorate: 0f1e2d3c4b5a69788796a5b4c3d2e1f0   # a reference to a table of your own: its sys_id
  room_number: "010"
  contact_method: Email                # a select box: its label or its value
  location: "Building 100"             # a reference: the location's name, or its sys_id
cert_renew_catalog_csr_variable: ""    # empty = the item's only attachment question
cert_renew_catalog_names_variable: additional_sans   # the "List any additional SANs" box gets the
  certificate's names
```

- **A reference** (Requested For, Location ...): the record's **name** works - the job looks it up and sends its sys_id (a user: user name or full name). For a table of your own the job can only look up a `name` column: if it says "no record named", give the **sys_id** (open the record, or the reference field's lookup, and copy it).
- **A select box**: its label (`Email`) or its value (`email`).
- **In an answer**, `{host}`, `{subject}`, `{names}`, `{expires}`, `{issuer}` and `{where}` become the certificate's - for a free-text question such as a justification: `justification: "Renewal of {subject} on {host}, expires {expires}"`.
- **Not every question needs an answer** - only the mandatory ones (and the CSR's, which the job fills in).

| Setting | Default | What it does |
|---|---|---|
| `cert_renew_servicenow` | `incident` | `catalog` = order the item, no incident |
| `cert_renew_catalog_item` | `""` | the item: its name as the catalog shows it, or its sys_id |
| `cert_renew_catalog_variables` | `{}` | the answers, by the questions' internal names |
| `cert_renew_catalog_csr_variable` | `""` | the attachment question for the CSR (empty = the only one) |
| `cert_renew_catalog_names_variable` | `""` | a question that gets the certificate's names |

| Setting | Default | What it does |
|---|---|---|
| `cert_renew_catalog_reorder_states` | `["4", "7"]` | a request closed this way (Closed Incomplete, Closed Skipped) is ordered again |
| `cert_renew_catalog_upload_table` | `ZZ_YYsc_cart_item` | where the CSR is uploaded before the order (ServiceNow moves it to the request) |
| `servicenow_catalog_test_order` | `false` | the catalog test also orders once, with a test CSR |

## Step 3 - the catalog test

A job template: **ServiceNow - catalog test**, playbook `playbooks/servicenow_catalog_test.yml`, inventory any, credential **ServiceNow API** (the account that will order). Variables: aap/extra_vars/servicenow_catalog_test.yml. Launch it:

```
PASS 1 item: DoD PKI Certificate Request (sys_id a3034a73...), 11 question(s), read as svc_aap_sn
  at https://...
     2 questions: requested_for | Requested For | requested for (31) | MANDATORY | a sys_user
       record: its name or sys_id
                 contact_method | Preferred method of contact | select box (5) | MANDATORY |
                   choices: email = Email, phone = Phone
                 csr_file | Please add CSR | attachment (33) | MANDATORY | the CSR goes here
                 ...
PASS 3 settings: the order would send these answers (references as sys_ids), the CSR in csr_file
  ...
SKIP 4 order: servicenow_catalog_test_order is false
```

Steps 1-3 only read. A FAIL says what to change: the item not found (or not orderable by that account), a question name that is not on the item, a mandatory question left empty, a choice it does not have, a name with no record.

**One real test order** (optional): `servicenow_catalog_test_order: true`. It orders the item once, exactly as the renewal does, with a test CSR (`CN=site-automation-test.invalid`, named `site-automation-TEST-...csr`). **The PKI gets a real request** - tell them first, then cancel it. Run again, it finds that order and makes no second one (until it is cancelled: Closed Skipped).

## Step 4 - the renewals

- Set `cert_renew_servicenow: catalog` (step 2). Nothing else changes on the renewal templates: the same playbooks, the same *ServiceNow API* credential.
- **Try it first** with Job type **Check** and a **Limit** of one host: the output says `would order it`, or `ordered already: RITM...`, and nothing is sent.
- **The workflows:** the certificate workflows run *Certificate report -> ServiceNow tickets -> Certificate renewal*. The *ServiceNow tickets* step opens incidents for the report's findings: with no incidents wanted, take that step out (link the report straight to the renewal, **Run always**).
- **The VMware and network renewals** make no incident either: the request is the CSR's record (vCenter, a host and a Cisco device keep one pending key - a second CSR would replace it).

## After the PKI issues the certificate

The request shows **Closed Complete** (each run says `the PKI issued it: install it`).

**Installed by the job, after an approval:** *Certificate install - check* (on a schedule) finds the certificate on the request - whatever the PKI attached or pasted, as long as its public key is the CSR's - and says what installing it would do; after the approval, *Certificate install - approved* puts it in place, reloads the service, checks the port, and puts the old one back if anything is wrong. Linux and Windows: CERT_INSTALL.md.

**By hand** - a keystore, a service that needs a restart, vCenter, ESXi, Cisco - where the key is:

| Where | How |
|---|---|
| Linux | the new key is in `cert_renew_dir` next to the CSR: RUNBOOKS.md, "Certificate renewal: the CSRs" |
| Windows | `certreq -accept -machine <issued.cer>` on the server: WINDOWS.md, "Windows certificate renewal - CSRs" |
| vCenter, ESXi | the vSphere Client's *Import and Replace*: VMWARE.md, "Certificates of vCenter and the ESXi hosts" |
| Cisco | `crypto pki import <trustpoint> certificate`: NETWORK_CERTS.md, "The renewal" |

Once the new certificate is in place, it has another expiry date: nothing more is ordered until it is due.

## When it goes wrong

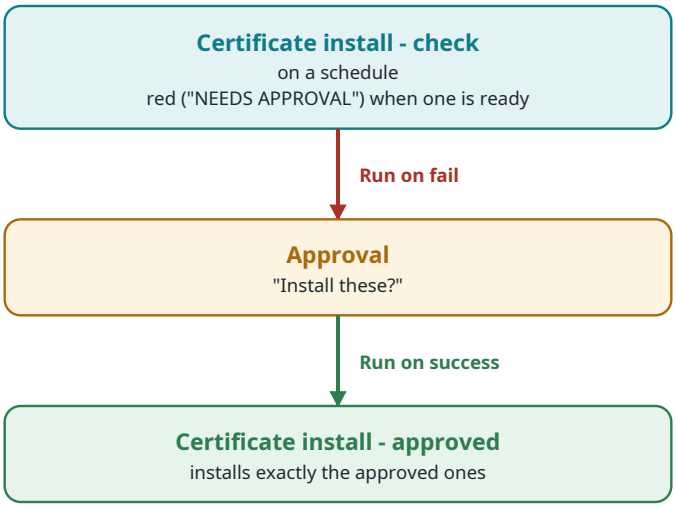
| The job says | Means | Do |
|---|---|---|
| `no catalog item named "..." that the API account can see` | the name differs, or the account may not order it | use the sys_id from the portal address; ask the catalog owner to make it available to the account |
| `the item has no question X (its questions: ...)` | an internal name that is not on the item | take the names from step 1 |
| `X (Label) is mandatory: give it a value` | a mandatory question without an answer | add it to `cert_renew_catalog_variables` |
| `'Fax' is not one of its choices (email = Email, ...)` | a select box answer it does not have | one of the listed values or labels |
| `X: no record named 'Building 9' in cmn_location` / `2 records named` | a reference by name: none, or several | the sys_id |
| `the order was refused: HTTP 400: ...` | ServiceNow's reason (a mandatory answer, a value it does not take, a rule of the item) | fix what it says; the catalog test shows the same |
| `ordered (REQ...), but its requested item could not be read` | the account cannot read the RITM it ordered | let it read its own requests; check the request by hand - the next run would order again |
| `ordered (RITM...), but the CSR is not on it` | ServiceNow refused the attachment on the RITM | attach the CSR by hand (the job output names it); allow `.csr` / `.txt` attachments |
| `the CSR could not be uploaded` | the instance refused the file (an extension it does not allow) | `cert_renew_attach_ext: txt` |
| The PKI says the CSR question is empty | the instance did not take the uploaded file for the question | the CSR is attached to the request (the paperclip): point them there; tell us the instance's version |

## Limits

- **Tested against a stand-in ServiceNow** (`tests/servicenow/snmock.py`): the Service Catalog API calls are the documented ones (`/api/sn_sc/servicecatalog/items`, `order_now`, the Attachment API), but how an instance handles an attachment question filled through the API differs between releases and customizations. Run the catalog test with one test order first.
- **A catalog item, not a record producer**: the job uses `order_now`. A record producer (it makes some other record, not a request) is not supported yet.
- **One request per certificate.** The form's paperclip takes more CSRs, but one request each keeps each certificate's order findable, and the PKI can fulfil them one by one.

# 21. Installing the issued certificate: from the request, with an approval

The certificate renewal makes a new key and a CSR where the old certificate is, and - with `cert_renew_servicenow: catalog` - orders it from your PKI's catalog item (CERT_RENEWAL_CATALOG.md). The PKI admin then puts the certificate from DISA on **that same request**. The certificate install takes it from there and puts it in place of the old one - after a person approved what it will do.




The same for Windows: *Windows certificate install - check* › approval › *Windows certificate install - approved*.

## What it does

### The check (on a schedule - reads only)

1. **What waits.** On each host, the renewals still waiting for their certificate: the key, CSR and info file in `cert_renew_dir` (Linux), the pending request in `win_cert_renew_dir` (Windows).
2. **The request.** On the AAP side - the hosts never get the ServiceNow credential - the request carrying each CSR (by the CSR's name in the order, as the renewal finds its orders), and what the PKI put there:
   - files attached to the RITM, to one of its attachment questions, to its REQ, or to its catalog task (SCTASK - where the PKI team often works the request);
   - a certificate pasted into a work note or comment of the RITM or its task;
   - each one PEM (.cer, .crt, .pem), DER, PKCS7 (.p7b, text or binary) or a zip of them.

**The certificate counts only when its public key is the CSR's.** Anything else there - someone else's certificate, a form - is listed and never used. The CA certificates with it are its chain.

3. **The plan**, on the host: what installing it would do - or why a person does it (below).
4. **Ready ones go to the approval:** the job lists them, hands the plan on, and ends red - `NEEDS APPROVAL: ...` - so the workflow takes its *Run on fail* path. Nothing ready: it stays green.

The job output says, per certificate:

```
READY | web01.example.mil | CN=web01.example.mil,... | RITM0012345 | install in
  /etc/pki/tls/certs/web01.crt, /etc/pki/tls/private/web01.key | reload httpd.service | then port
  443 must serve it | the new one: serial 4A3F..., expires 2027-11-28 09:35 UTC
WAITING | app01.example.mil | CN=app01.example.mil,... | RITM0012346 | RITM0012346 (Work in
  Progress): no certificate for this key on it yet
HELD | db01.example.mil | CN=db01.example.mil,... | RITM0012350 | the old key of
  /etc/pki/tls/certs/db01.crt is in several files (...): name the one in use in
  cert_install_key_files
BY HAND | jboss01.example.mil | CN=jboss01... | RITM0012351 | in a keystore (/opt/app/keystore.jks
  (alias server)): its program needs a restart ... | steps: ...
```

### The approved install (after the approval)

For each approved certificate, the plan is **made again first**: the same certificate, the same files (Linux) or bindings (Windows), the same services - or it is refused ("changed since the approval"). Then:

**Linux**

1. Each file it replaces is copied aside - `cert_renew_dir/backup/<date>-<certificate>/` (0700, the copies 0600: the old key is among them).
2. The new files are written **in the old ones' layout** - text or binary (DER), with the chain if the old file had one (the PKI's chain; a root only if the old file had one), with the key if the old file held it - and with the old file's owner, mode and SELinux label. The old key's file gets the new key.
3. The service's configuration is tested (`apachectl configtest`, `nginx -t`, `haproxy -c`, `postfix check` ...).
4. The service is **reloaded** - never restarted.
5. The port must then serve the new certificate (`cert_install_verify_wait` seconds).

Any step fails: **the copies go back** and the service is reloaded again; the job says what failed and ends red. Installed: the renewal's key and CSR move to `cert_renew_dir/installed/`, and a work note on the request says what was installed where.

**Windows**

1. Its chain's intermediate CAs go into *Intermediate Certification Authorities* - **never a root** (a root is trusted by policy, not by this job: with the DoD roots in place, as on every DoD server, nothing is needed).
2. `certreq -accept -machine` puts it together with its pending key in *Personal*.
3. The new key gets the old key's extra permissions (read) - a service account that used the old one - and, for Remote Desktop, NETWORK SERVICE.
4. Each binding of the old certificate moves to the new one, and is read back:
   - **https** in HTTP.sys (IIS, WSUS, any `netsh http` binding - IP:port and hostname:port), the same application and store;
   - **Remote Desktop** (RDP-Tcp);
   - **the WinRM HTTPS listener**.
5. With full PowerShell, each https port is asked which certificate it serves (in Constrained Language Mode the bindings are read back only).

Anything fails: the bindings go back to the old certificate. **The old certificate stays in Personal**, so putting it back is only a rebinding.

## Never automatic - a person installs it (BY HAND)

The job lists these with the steps, and never touches them:

| Linux | Windows |
|---|---|
| a keystore (Java: its program needs a restart) | SQL Server uses it (a restart) |
| a certificate seen only on a port (its file not known) | a Central Certificate Store binding (a file on the share) |
| a service that cannot reload (only a restart) | the WinRM HTTPS listener **AAP connects through** (it would cut the job) |
| the port is served by a container or a user's program | a binding kind not in `win_cert_install_rebind` |
| a path in `cert_renew_install_notes` (something else deploys it - STIG Manager's comes from an AAP credential) | |
| a service in `cert_install_never_reload` | |
| **an AAP host** (groups `aap`, `aap_hosts` - its installer manages its certificates) | **an AAP host** |

## Held for a person (HELD)

Something to look at first; the next run checks again.

- the certificate is not for the key on the host, not valid yet, expired, or valid fewer than `cert_install_min_days` (30) more days;
- it lacks names (subjectAltName) the old one had (`cert_install_fewer_names: true` installs it anyway);
- its chain does not build up to a root - attach the chain (.p7b) to the request (or `cert_install_trust_check: false`);
- the file no longer holds the certificate the CSR was made for (replaced since);
- the old key not found, or found in several files - name it: `cert_install_key_files`;
- the old key is also another certificate's key (replacing it would break that one);
- no service found that uses the file - name it: `cert_install_services`;
- Windows: the pending request (the new key) is gone from the server - the next renewal makes a new CSR.

## Which service, which key (Linux)

Found by itself:

- **the service** - the configuration of httpd, nginx, haproxy, postfix, dovecot, slapd or cockpit that names the certificate's file (`cert_install_known_services` to change the list), and the program listening on the port the certificate report saw it on;
- **the old key** - the file the same configuration names (`SSLCertificateKeyFile`, `ssl_certificate_key` ...), else the one file with the old certificate's key in the certificate's folder, a `private/` next to it, and the usual folders (`/etc/pki/tls/private` ... - `cert_install_key_dirs` adds yours).

For the rest, in `all.yml`:

```yaml
cert_install_services:                    # a file no configuration names -> the service to reload
  - {path: '/opt/app/tls/*.crt', service: app}
cert_install_key_files:                   # when the old key cannot be found, or is in several
  files
  /etc/pki/tls/certs/db01.crt: /etc/pki/tls/private/db01.key
cert_install_never_reload: ['slapd*']     # services the job never reloads: their certificates are
  installed by a person
```

## Setting it up

1. **The renewal orders from the catalog** (`cert_renew_servicenow: catalog`) - CERT_RENEWAL_CATALOG.md.
2. **The PKI puts the certificate on the request.** Ask them to attach DISA's files to the RITM or to its catalog task (or paste the certificate into a work note there) - the .cer and the chain (.p7b) as DISA sends them. The API account must be able to read the request's and its tasks' attachments and journal (and, for the note after the install, write a work note on the RITM).
3. **The job templates** (project: this repository, the same inventory as the renewal):

| Template | Playbook | Credentials | Options |
|---|---|---|---|
| Certificate install - check | `playbooks/cert_install_check.yml` | Machine, ServiceNow API | Job type **Check** allowed |
| Certificate install - approved | `playbooks/cert_install_approved.yml` | Machine, ServiceNow API | Variables **not** prompted on launch; no Limit |
| Certificate restore | `playbooks/cert_restore.yml` | Machine | Limit and Variables prompted on launch |
| Windows certificate install - check | `playbooks/win_cert_install_check.yml` | Machine (Windows), ServiceNow API | |
| Windows certificate install - approved | `playbooks/win_cert_install_approved.yml` | Machine (Windows), ServiceNow API | Variables **not** prompted on launch; no Limit |
| Windows certificate restore | `playbooks/win_cert_restore.yml` | Machine (Windows) | Limit and Variables prompted on launch |

The settings each one reads: `aap/extra_vars/<playbook>.yml`.

4. **The workflow** (Templates › Add › Add workflow template), one for Linux, one for Windows:

| Step | Node | Link |
|---|---|---|
| 1 | *Certificate install - check* | (start) |
| 2 | *Approval* - name `Install the certificates the check listed?`, **Timeout** e.g. 8 hours. Description: `Open the check job: its READY lines say what each one changes.` | **Run on fail**, from 1 |
| 3 | *Certificate install - approved* | **Run on success**, from 2 |

The approved job **must have the check job's inventory and no Limit**: a host of the plan it does not run on makes it stop before anything is installed.

5. **The schedule:** the workflow daily (e.g. 07:00, after the certificate watch), or a few times a day during working hours.
6. **Try it first:** the check with Job type **Check** and a Limit of one host - the same READY lines, nothing sent. The approved job launched as **Check** from the workflow says `WOULD INSTALL` and changes nothing.

## Put the old certificate back

**Linux** - *Certificate restore*, with a Limit:

- Job type **Check**, no variables: it lists the backups on the host (`backup <date>-<certificate> | subject | made ... | files`).
- Then `cert_restore_only: web01` (the newest backup of each certificate whose name or subject matches) or `cert_restore_backup: <a backup's folder name>`: the files go back with their owner, mode and SELinux label, the configuration is tested, the service reloaded, and the port must serve the old certificate again.

**Windows** - *Windows certificate restore*: the same, with `win_cert_restore_only` / `win_cert_restore_backup` - each binding the install moved goes back to the old certificate (it stayed in Personal).

After a restore the renewal **waits again**: the next install check finds its certificate and asks for the approval again. Leave it waiting while you look at what went wrong (deny the approval), or let the old certificate run out and renew again.

## When it goes wrong

| The job says | Means | Do |
|---|---|---|
| `WAITING ... no certificate for this key on it yet` | the PKI has not put it there, or put another one | look at the request: `others` lists certificates there for another key |
| `NOT ORDERED ... no catalog request carries this CSR` | the CSR went another way (by hand, an incident) | install it by hand, or order with the catalog |
| `CLOSED ... its request was closed Closed Incomplete` | the PKI rejected it | the renewal orders it again |
| `HELD ... its chain does not build up to a root` | the chain did not come with it, or its root is not trusted on the host | attach the chain (.p7b) to the request; or `cert_install_trust_check: false` |
| `HELD ... no service found that uses ...` | no known configuration names the file, no port known | `cert_install_services` |
| `ROLLED BACK ... the configuration test of httpd.service failed` | the new files do not fit the configuration | the job output has the test's message; the old files are back |
| `ROLLED BACK ... after the reload, 127.0.0.1:443 serves ...` | the service still serves something else after the reload | the configuration may name another file - the old files are back |
| `REFUSED ... changed since the approval` | between the check and the approval, a file, a binding or a service changed | the next check plans it again |
| `The approved plan has certificates for X, which this job does not run on` | the approved template's inventory or Limit | the check job's inventory, no Limit |
| `FAILED ... certreq -accept -machine did not put it in Personal` (Windows) | its chain is not trusted on the server (0x800b0109), or the pending request is not this one | the DoD roots in Trusted Root; nothing else was changed |
| `ipport=0.0.0.0:443: the port was not asked (... does not answer)` (Windows) | nothing listens on that port now (the site is stopped), or Constrained Language | the binding was read back and is on the new certificate; browse to the site to see it |
| `the work note was refused` | the API account may not write to the request | nothing else: the job output says what was installed |

## Limits

- **Tested** against a stand-in ServiceNow (`tests/servicenow/snmock.py`), a test CA and a stand-in web server (`tests/certs/run_cert_install_test.sh`); the Windows script with stand-ins in Windows PowerShell 5.1 and 7, normal and Constrained Language (`tests/windows/test_win_checks.ps1`), and for real on a Windows Server runner: the renewal's CSR, `certreq -accept`, an HTTP.sys binding moved and served, the restore (`.github/workflows/check.yml`, *Windows certificate install, for real*).
- **Not automatic:** vCenter, ESXi and Cisco certificates (install them as VMWARE.md and NETWORK_CERTS.md say); a service that needs a restart.

- **One host at a time per certificate:** a certificate found on several hosts (a wildcard) has a CSR - and a key - per host, so each host installs its own.

# 22. Emailed reports: the formatted email and its look

Jobs can email what they found or did, as a **formatted (HTML) report** with a plain-text copy for mail clients that do not show HTML. One email per job run, whatever the number of hosts.

## Which jobs email

| Template | Playbook | The email |
|---|---|---|
| Health check (Linux) | `health_check.yml` | findings of every host (critical first), hosts that did not report, healthy hosts |
| Windows health check | `win_health_check.yml` | the same, for Windows servers |
| Database health | `database_health.yml` | the same, for the database check |
| Certificate report (Linux / Windows) | `cert_report.yml`, `win_cert_report.yml` | findings, plus every certificate, soonest expiry first: red expired, amber within 30 days, blue within 60 |
| POA&M status | `poam_status.yml` | overdue, due-soon and missing items |
| ServiceNow health | `servicenow_health.yml` | the instance and the MID Servers |
| Troubleshoot (Linux / Windows) | `troubleshoot.yml`, `win_troubleshoot.yml` | what the checks found |
| Apply approved ACT fix | `act_fix_approved.yml` | the checks after the fix |
| VM secure boot report | `vm_secure_boot_report.yml` | VMs with secure boot off, BIOS VMs (folder, host, guest, IP) |
| VM datastore report | `vm_datastore_report.yml` | datastores 85 / 90 % used or more, then every datastore, fullest first |
| VM snapshot report | `vm_snapshot_report.yml` | snapshots older than N days, kept ones, then every snapshot with size and age |
| VM snapshot cleanup | `vm_snapshot_cleanup.yml` | what was deleted (and the space freed), what could not be |
| VM alarms report | `vm_alarm_report.yml` | triggered alarms, host connection, failed logins, VM events, other errors and warnings, configuration issues, every host - each row coloured by severity |
| VM alarms ACT analysis | `vm_alarm_act_analysis.yml` | per problem: ACT's likely cause, evidence, fix and confidence (coloured); what to do first |
| Active Directory health report | `ad_health_report.yml` | domains, domain controllers, FSMO, replication, dcdiag, DNS, time, events, clients in no site, certificates, trusts, sites, then the day's changes (privileged groups, accounts, GPOs, schema and configuration) - problems first, coloured (ACTIVE_DIRECTORY.md) |

| Template | Playbook | The email |
|---|---|---|
| NetApp ONTAP health report | `ontap_health_report.yml` | clusters, nodes, health alerts, events, hardware, disks, network, aggregates, volumes, inodes, snapshots, SnapMirror, SVMs, CIFS, LUNs, certificates - problems first, coloured (NETAPP.md) |
| Horizon health report, capacity report, desktops check, session cleanup, image push plan | `horizon_*.yml` | Connection Servers, gateways, vCenter, AD, event database, license, pools; each pool's use; the broken desktops; the sessions logged off; the push plan (HORIZON.md) |
| VM capacity planning | `vm_capacity_report.yml` | overall and per cluster: CPU, memory with a host down, vCPUs per core, VMs that fit, hosts recommended, datastores with runways; with GenAI, its estimate next to the math |
| VMware certificate report | `vmware_cert_report.yml` | every certificate of vCenter and its hosts, soonest expiry first (expired red, due amber), and for each one due what the renewal would do; the hosts not checked |
| VMware certificate renewal | `vmware_cert_renew.yml` | the certificates due (expired red, the rest amber): the ticket and the CSR of each, or why none; every certificate of vCenter and its hosts, soonest expiry first; the hosts not checked |
| ESXi security settings | `esxi_security.yml` | the hosts it changed (what), the ones that failed (why), every host's settings before the run. By default only when a host was changed or failed |
| VM deploy from a ServiceNow ticket | `vm_deploy_from_ticket.yml` | the VM (IP or MAC, where it went), the request with where each value came from; in the workflow's first step, where it would go - for the approver |
| VM deploy settings finder | `vm_deploy_inventory.yml` | the clusters, templates, datastores and folders by the names the deploy uses, what to look at first, and the lines to paste into the deploy template's Variables |

| Template | Playbook | The email |
|---|---|---|
| VM request pickup | `vm_deploy_poll.yml` | the tickets picked up and their workflow jobs, the ones not started (why), the ones skipped. Only when it started something or something failed |
| VM restart, shut down, snapshot, delete snapshot, notes, change VLAN | `vm_*.yml` | what was done to which VM |

## Turning it on

1. **The mail relay, once**, in your `playbooks/group_vars/all.yml` (then Commit, Sync Changes, and sync the project in AAP). A relay that accepts the AAP server by a whitelist needs nothing more:

```yaml
report_email_smtp_host: smtp.yoursite.mil
report_email_from: aap-noreply@yoursite.mil
report_email_security: none          # only if the relay does not offer STARTTLS
```

Encryption, your own CA and a relay login: VMWARE.md, "Email the report".

2. **Who gets each report: on each template.** Templates > the template > Edit > **Variables**:

```yaml
report_email_to: linux-team@yoursite.mil, ops@yoursite.mil
```

Or a survey question with the variable `report_email_to` (Text, not required) - but what is typed there **replaces** the list. To let people **add** someone for one run, use the "Also email the report to" question instead (next section). Putting `report_email_to` in `all.yml` instead makes **every** job in the table above email those people, each time it runs - usually more than you want.

Not in an AAP inventory **group's** Variables: the email is sent by the AAP side of the job, which never gets a group's variables. The template's Variables, a survey, the inventory's own Variables and `all.yml` all work.

| Setting | Default | What it does |
|---|---|---|
| `report_email_to` / `report_email_cc` | (none) | who gets it; a list, or text with commas |
| `report_email_html` | `true` | send the formatted version too; `false` = plain text only |
| `report_email_theme` | `standard` | the look: `standard`, `command` or `portal` ("The look", below) |
| `report_logo_file` | (none) | your logo in the header, e.g. `files/report_logo.png` |
| `report_email_html_template` | `report.html.j2` | your own look instead (below) |
| `report_email_subject_prefix` | `[AAP]` | put before every subject |

The subject is the report's title with its counts, e.g. `[AAP] Health check: 3 finding(s) on 2 of 48 host(s)`. Every email ends with the AAP job number, the template, who started it and when. A dry run

(Check) sends nothing. A job without `report_email_to` says `No email: report_email_to is not set`.

**All settings**, with their defaults: site_email.

## Also email the report to (asked at launch)

Whoever launches a report template can send that run's report to more people: a box **Also email the report to** in the template's survey. What is typed there is **added** to the template's own `report_email_to` - never in its place - for that run only. A blank box changes nothing, so schedules and runs nobody fills in work as before. A template without `report_email_to` emails only the people typed in the box (and nothing when it is blank).

**Add the question to every report template at once** with the job template **AAP - report email question** (`playbooks/aap_email_question.yml`, nothing but the **Red Hat Ansible Automation Platform** credential, no inventory hosts needed - localhost). Its AAP user needs the **Admin** role on the report templates and their workflows. It finds them through AAP's API **by their playbook**, so your own template names work, and:

- adds the question at the end of each one's survey; a template without a survey gets one, with this question alone, turned on;
- adds it to every **workflow** that runs one of them too - AAP never asks a job its own survey inside a workflow, only the workflow's, and the answer then reaches every report step of it (in Certificate watch, the people typed get one email per step that emails);
- leaves a survey that is **turned off** off (turning it on would also ask its other questions, and their defaults would reach every run): the job lists those - turn the survey on by hand if you want it there;
- leaves a question that is already there as it is, so you can reword it in AAP and run the job again;
- points out a survey that asks `report_email_to` itself (that one replaces the list).

Run it with **Check** first: it then only reads and lists, per template, what it would do. Run it again after you add report templates. `aap_email_question_state: absent` takes the question out again (a survey left with no question is removed and turned off). Settings: aap_email_question

- e.g. `aap_email_question_project` (only your project's templates), `aap_email_question_exclude`, `aap_email_question_text` (the question's wording).

**What people can type:** addresses separated by commas, semicolons, spaces or one per line (a column pasted from Excel works), or pasted from Outlook (`Doe, Jane <jane.doe@yoursite.mil>; Roe, Rick <rick.roe@yoursite.mil>`). Each address gets one email, even when it is in `report_email_to` too. Text that is no address is listed in the job (`Email | not sent to these`) and the report still goes to everyone else.

**Only your own domains** - so a report never goes outside by a typo - in `all.yml`:

```yaml
report_email_also_domains: [yoursite.mil]     # a subdomain too (mail.yoursite.mil); empty = any
  domain
```

An address in another domain is listed in the job, with why, and gets no email. The usual recipients in `report_email_to` are not checked against it.

**The same rules as the usual recipients.** A job that emails only when something happened - ESXi security settings (only when a host was changed or failed), the snapshot cleanup with `vm_snapshot_cleanup_email_only_if_deleted`, the VM request pickup - sends the people typed in the box nothing either when nothing happened; the job's output says why (`No email: ...`). And the mail relay must

be set up (`report_email_smtp_host`, `report_email_from` in `all.yml`): without it, a report launched with an address in the box fails at its email step - the setup job warns when it is not set.

By hand instead: Templates > the template > **Survey** > Add question - question *Also email the report to*, answer variable name `report_email_also`, answer type **Textarea** (a one-line Text box drops the line breaks of a pasted list), **Required** off, no default; then turn the survey on. For a workflow, the same on the workflow.

## Green or red: when a report finds problems

AAP shows a job as **successful (green)** or **failed (red)**, nothing in between. What a report job does when it finds problems:

| Job | Default | Change it with (template Variables) |
|---|---|---|
| VM secure boot report, VM datastore report, VM snapshot report | **green**: the report and email say what to fix | `vm_secure_boot_fail: true` / `vm_datastore_fail: true` = red |
| VM snapshot cleanup | green; **red** when a deletion failed | - |
| VM alarms report | **green**: the report and email say what is wrong | `vm_alarm_fail: true` = red on a critical alarm or a host not connected |
| VM alarms ACT analysis | green; **red** when ACT could not run (no key, no network): the email still lists the problems | - |
| VM capacity planning | **green**; red when GenAI was asked for and could not run | `vm_capacity_fail: true` = red when a cluster or datastore is critical |
| ESXi security settings | green; **red** when a host could not be set (the email says why) | - |
| NetApp ONTAP health report | **green**: the report and email say what is wrong | `ontap_report_fail: true` = red on a critical finding or a cluster not read |
| Horizon health and capacity reports | **green**: the report and email say what is wrong | `horizon_health_fail: true` = red on a critical finding or a full pool |
| Health checks, certificate reports, database health, POA&M, ServiceNow health | **red** when there are findings: AAP workflows start tickets (and ACT) from that | `site_fail_on: []` = green, report only |

So for a template whose job is to email a report to people, set `site_fail_on: []` and it stays green; the email is the alert. Keep the default on templates that feed a ticket workflow.

Red also means one of these, whatever the setting: a host could not be reached, or the job itself broke (a wrong setting, a missing credential). Those always need a look.

## What a report looks like

A title (also the subject) in a coloured band - green, amber, red or blue - then summary boxes with numbers, then sections: each a heading, a line of advice, and a table (or a list). In tables that have a severity, each row is tinted (light red, amber or blue) and the severity cell is filled in its colour, so the problems stand out at a glance. Every value from a host or vCenter is escaped, so a VM name cannot add markup to the email.

## The look: standard, command, portal or glass, your logo, the last runs

`report_email_theme` picks one of four looks, for every report job at once when it is in `all.yml`:

- **standard** (the default): the header band in the report's colour, number boxes under it - as described above.
- **command**: a dark header with your logo, a small line of text above the title (`report_email_header_text`, default `Site automation report`) and the status as a label beside it; under it **the job's last runs** as a strip of bars, oldest first, this run last and outlined. A bar's colour is how that run went (green, amber, red; grey = cancelled); its height is the report's first plain number (the VMs checked, the snapshots deleted), against the highest of the strip. Then the numbers in one row, then the sections.
- **portal**: white cards on a pale mint background with a dark green accent, like the AAP portal pages. A header card with your logo, `report_email_header_text`, the AAP job and the status as a pill (Successful, Warning, Critical); the numbers as tiles - a tile that is amber or red gets an outline in that colour; a card with **the job's last runs** (the same bars, rounded) and a line `This run Critical, Last run Successful 2 days ago`; then one card per section. New Outlook, Outlook on the web and phones show the rounded cards and shadows; classic desktop Outlook shows the same layout with square corners.
- **glass**: see-through, frosted cards on a soft blue-green background. A header card with your logo (at most 80 px tall), `report_email_header_text`, the title and the status with what it asks of the reader (*Needs attention*, *Action required*); the numbers as tiles with an icon - a warning or critical tile tinted in its colour; **the job's last runs** with a colour key, beside *This run* and *Last run*; then one card per section, its table on a near-solid white panel. Only the frame is see-through: every text stays on a surface solid enough to read as on paper (the lightest text is at least 4.5:1 against what is behind it). The inbox shows the subtitle and the warning and critical numbers as the line under the subject; with the report website (`report_web_dest`) the email ends with an **Open the full report** button. On a phone the tiles go two by two and wide tables scroll sideways. Mail programs that draw transparency, rounded corners and shadows show the glass; the others (classic desktop Outlook) show the same layout with solid white cards and square corners. The web page (`report_web_theme: glass`) adds what a browser draws and a mail program does not: the blur behind the cards, line icons, and a plain print.

playbooks/group_vars/all.yml

```
report_email_theme: command                 # or portal, or glass
report_web_theme: glass                     # the web page's look (docs/WEB_REPORTS.md)
report_logo_file: files/report_logo.png     # = playbooks/files/report_logo.png
```

**The logo** (every look): a PNG (best, with a transparent background), JPEG or GIF in your repository. Put it in `playbooks/files/` and set `report_logo_file` (a path from `playbooks/`, or a full path). It is attached to each email and shown from there - Outlook blocks pictures from links - so keep it small: about 160 px tall, under 100 KB. It is shown `report_logo_height` (84) pixels tall; in the portal look's header card at most 56, in the glass look's at most 80. Your logo belongs in your own repository only. If you update with `scripts/update-from-release`, add the line `playbooks/files/report_logo.png` to `.site-local`, or the

update deletes it.

**The last runs come from AAP** - a single email has no memory, AAP does. At the end of each run the report asks AAP for its template's earlier jobs (the last `report_history_runs` - 14 - with this one). For that, attach a credential of type **Red Hat Ansible Automation Platform** to the template: the one the pick lists use works (VMWARE.md, "Pick lists"). Its user must be able to see the template's jobs: give it **Organization Auditor** on the template's organization (AAP 2.5 and later: **Access Management > Users >** the user **> Roles > Assign roles**, resource type **Organization**, your organization, **Organization Auditor**). That is read-only and covers every template in the organization, also new ones. A role on each template works too (JobTemplate Execute includes seeing its jobs, but also lets the user start it). Without it the email goes out as usual, without the strip, and the job says why: `No strip of past runs in the email: ...`. Each run keeps its report's status and number in AAP as the job artifact `site_report_run`; runs from before 0.20.0 have none, so their bar shows only how the job ended (green or red), half height. `report_history: false` turns the strip off.

The plain-text copy of the email has the strip as one line: `Last 14 runs, oldest first: ok ok warning ... critical (this run)`.

## Also on a web page

Every report can also be written as a web page - the same report, in the portal look - on a Windows server (over WinRM), into a folder or onto a web server, with a `latest.html` per template and an index of every report; the job prints the page's address, and the email links to it. With or without the email: WEB_REPORTS.md. Report templates whose execution environment cannot reach that server keep their report for one job that writes the pages for all of them (`report_web_copy`; WEB_REPORTS.md, "The copy job").

## Changing the look

The look is one file, `roles/site_email/templates/report.html.j2` (Jinja and HTML, styles inline because Outlook ignores most style sheets). To change it without touching the repository's file:

1. Copy it (or `report_command.html.j2` / `report_portal.html.j2` / `report_glass.html.j2`, the other looks) to `playbooks/files/email/report.html.j2` in your repository and edit the copy: colours, fonts, a banner line your site requires.
2. In `all.yml`: `report_email_html_template: "{{ playbook_dir }}/files/email/report.html.j2"`.
3. Add the line `playbooks/files/email/` to the file `.site-local` at the top of your repository (create it if it is not there). Without it, the next update from a release **deletes** your copy: the release does not have that file.

Check your copy after an update that changes the report layout (the CHANGELOG says so when that happens).

The **title** of a report (the coloured band, also the subject) is written by the playbook, not the look file: for the VMware reports, `_vm_report_subject` in `roles/vmware_vm/tasks/<report>.yml`. An edit there is undone by the next update, unless that file is in `.site-local` - and then you no longer get the release's fixes to it either.

## Emailing a report from another playbook

Describe the report as data, then include the role. For example, at the end of a play that runs on localhost:

YAML

```
- name: Email the report
  ansible.builtin.include_role:
    name: site_email
  vars:
    _report:
      title: "Certificate report: 3 expire within 30 days"   # also the subject
      status: warning                  # ok | warning | critical | info: the colour of the band
      subtitle: "212 certificates on 48 servers"           # optional
      summary:                         # optional: number boxes
        - {label: Certificates, value: 212}
        - {label: Expire within 30 days, value: 3, status: warning}
      sections:
        - title: Expire within 30 days
          status: warning
          text: "Renew them before the date shown."        # optional
          columns: [Server, Subject, Expires]
          rows: "{{ my_rows }}"          # a list of lists, one per row, in the order of columns
        - title: Notes
          lines: [one line, another line]                  # a bullet list instead of a table
      footer: "Source: playbooks/cert_report.yml"           # optional
  when: report_email_to | default([]) | length > 0 or report_email_also | default([]) | length > 0
```

A section has `columns` and `rows` (a table; an empty list shows "None."), or `lines`, or just `text`. To colour a table, add `row_status` (one status per row: `critical`, `warning`, `info`, `ok` or `''`; tints the row) and `cell_status` (per row, one status per cell; fills that cell), e.g. `row_status: [critical]` and `cell_status: [[critical, '', '']]` for a red row with a red first cell. In a play that runs on many hosts, build the rows from `hostvars` and add `run_once: true` to the include, so one email goes out, not one per host.

## Troubleshooting

The job's output says what happened to the email, in the task `Email | result` (or `No email`).

| You see | It means | Do |
|---|---|---|
| `No email: report_email_to is not set` | no recipient for this template | add `report_email_to` to the template's Variables (or its survey) |
| `Email \| not sent to these, from "Also email the report to"` | text in the box that is no address, or an address outside `report_email_also_domains` | the rest got the email; send it again with the address corrected |
| `Email question \| ... FAILED: the AAP user may not change this job template` | the AAP credential's user has no Admin role on that template | give it the Admin role on the template (or its organization's job templates), or leave it out: `aap_email_question_exclude` |
| `To email the report, set report_email_smtp_host ...` on a report someone launched with an address in the box | the mail relay is not set up, so no report can email | step 1 of "Turning it on", in `all.yml` - or leave the box empty |
| `... its survey is turned off, and was left off` | the template's survey is switched off | turn it on (Survey > the switch) if you want the question asked there; check its other questions' defaults first |

| You see | It means | Do |
|---|---|---|
| `To email the report, set report_email_smtp_host ... and report_email_from ...` | the relay is not set up | step 1 of "Turning it on", in `all.yml` |
| `the mail relay ... does not offer STARTTLS` | the relay has no encryption on that port | `report_email_security: none` (or `ssl` with port 465) |
| `TLS with the mail relay ... failed: ... certificate verify failed` | the execution environment does not trust the relay's certificate | `report_email_ca_path` to the CA file (VMWARE.md, "Email the report"); list `playbooks/files/ca/` in `.site-local` |
| `could not send the email through ...: ... timed out` or `Connection refused` | the AAP node cannot reach the relay | a firewall rule from the AAP node (and every execution node) to the relay's port |
| `the mail relay ... refused every recipient` | the relay does not accept mail from the AAP node, or for those addresses | ask the mail admins to add the AAP node's address to the relay's allowed list |
| `emailed "..." to ...` but nothing arrives | the relay accepted it; something after it held it | look in junk / quarantine; ask the mail admins to trace it by the subject; check that `report_email_from` is an address they allow |
| `DRY RUN: would email ...` | the job ran as Check | nothing: a dry run never sends |
| The email shows plain text only | `report_email_html: false`, or a mail gateway removed the HTML part | ask the mail admins whether HTML mail is stripped; the text copy has the same tables |
| `No logo in the email: ... is not there, or not a PNG, JPEG or GIF` | `report_logo_file` points at nothing (a path from `playbooks/`, e.g. `files/report_logo.png`) | check the path and that the file is committed and synced |
| `No strip of past runs in the email: no AAP connection` | the template has no AAP credential | attach the "Red Hat Ansible Automation Platform" credential ("The look", above) |
| `No strip of past runs in the email: AAP refused the list of this template's jobs (HTTP 403)` | the credential's user may not see the template | give that user Organization Auditor on the template's organization ("The look", above) |
| `No strip of past runs in the email: AAP refused the login (HTTP 401)` | the credential's user name or password is wrong | fix the credential |
| The job is red, but the email came | a check report found problems (`site_fail_on`) | by design for ticket workflows; `site_fail_on: []` on email-only templates ("Green or red") |

# 23. The report website: every report as a web page

Every report job (EMAIL_REPORTS.md, "Which jobs email") can also write its report as a **web page**: the same report as the email, in the portal look (or `report_web_theme`: standard, command, glass), with your logo. Each run gets its own page, each template a `latest.html` that is always its newest run, and an `index.html` lists every report's last run, worst first. The report's data is written as JSON beside each page, so a web developer can build their own site on it.

It works **without email**: set `report_web_dest` and leave `report_email_to` empty. With both set, the email carries a link to its page ("Open this report in a browser").

Each report job writes its own pages. When some report templates run in an execution environment that cannot reach the web server (a Windows server needs WinRM, and with NTLM blocked Kerberos, in the execution environment), let **the copy job** write them instead: one template, in the execution environment your Windows jobs use, writes every report job's pages ("The copy job", below).

The job prints the page's address, and keeps it as the job artifact `site_report_url`:

```
TASK [site_email : Email | the report's address]
ok: [localhost] => {
    "msg": "Report:
      https://reports.yoursite.mil/linux-certificate-trust-report/2026-10-23_0605_job-907.html"
}
```

## What it writes

```
index.html                                  every report's last run, worst first (status, name,
  title, when)
index.json                                  the same as data
linux-certificate-trust-report/
  2026-10-23_0605_job-907.html              this run's page - the address the job prints
  2026-10-23_0605_job-907.json              this run's data
  latest.html                               always the newest run (a link that never changes)
  latest.json
  2026-10-22_0600_job-901.html              earlier runs stay
  ...
vmware-snapshot-cleanup/
  ...
```

- **The folder** is the AAP template's name, lower case with dashes: `Linux - Certificate trust report` becomes `linux-certificate-trust-report`. Outside AAP (the command line), the play's name.
- **The file** is the time the job wrote it (UTC) and the AAP job number: listing a folder sorts the runs oldest to newest, and the number leads straight to the job in AAP.
- Renaming a template in AAP starts a new folder for its new runs (the old runs stay in the old one). The index shows each folder's last run; one that stopped can be deleted on the server.
- Every page stands alone: the logo is inside it (no separate image file), no scripts, no outside fonts or styles.
- Nothing is ever deleted. To keep, say, 90 days, a nightly job on the web server: `find /srv/reports -name '20??-*_job-*.*' -mtime +90 -delete`.

### Changing the names

Every name is a setting, with `{names}` filled in (`roles/site_email/defaults/main.yml`):

| Setting | Default |
|---|---|
| report_web_page | {template}/{date}_{time}_job-{job}.html - each run's page |
| report_web_latest | {template}/latest.html - "" for none |
| report_web_index | index.html - "" for none |
| report_web_names | {} - your own names, e.g. {site: hq} for {site}/{template}/... |

The names: {template}, {template_id}, {job} (local outside AAP), {status} (ok, warning, critical, info), {date} (2026-10-23), {time} (0605), {year}, {month}, {day}, and your own. For example a folder per site and per month:

YAML

```
report_web_names: {site: hq}
report_web_page: "{site}/{template}/{year}-{month}/{date}_{time}_job-{job}.html"
report_web_latest: "{site}/{template}/latest.html"
```

A name must end in .html and stay inside the report folder (no leading /, no ..); a value is made safe for a file name (letters, digits, ., -, _; the rest becomes -). A name that does not exist stops the job and lists the ones that do.

### The JSON (for your own site)

Each run's .json (and latest.json):

```
{
  "report": {"title": "...", "subtitle": "...", "status": "warning",
             "summary": [{"label": "Hosts", "value": 34}, {"label": "Findings", "value": 752,
               "status": "warning"}],
             "sections": [{"title": "...", "status": "...", "columns": ["Host", "..."], "rows":
               [["web01", "..."]],
                          "row_status": ["critical"], "cell_status": [["critical", ""]]},
                         {"title": "Notes", "lines": ["..."]}],
             "footer": "..."},
  "job": 907, "template": "Linux - Certificate trust report", "template_id": 71,
  "when": "2026-10-23 06:05 UTC",
  "runs": [{"status": "ok", "value": 3, "label": "Findings", "when": "...", "ago": "2 days ago",
    "current": false}],
  "page": "linux-certificate-trust-report/2026-10-23_0605_job-907.html",
  "url":
    "https://reports.yoursite.mil/linux-certificate-trust-report/2026-10-23_0605_job-907.html",
  "latest": "linux-certificate-trust-report/latest.html", "index": "index.html"
}
```

report is the layout every report uses (EMAIL_REPORTS.md, "Emailing a report from another playbook"). runs is the strip of last runs (with an AAP credential on the template). index.json: {"generated": "...", "reports": [{"key", "name", "title", "status", "when", "job", "page", "latest"}, ...]}, worst first.

## Where the files go

Three ways; the pages are the same in each.

### A Windows server, over WinRM

```yaml
report_web_host: web01.yoursite.mil       # the Windows server (a file server, or the IIS server
  itself)
report_web_dest: 'D:\reports'             # the folder on it - there already
report_web_url: https://reports.yoursite.mil
```

The job makes the pages on the AAP side, as always, and writes them on that server over WinRM - the way the Windows jobs reach your servers. Nothing to mount on the execution nodes, no WebDAV, no share. Two steps there per run: one reads the folder's `index.json` back (to add this template's line), one PowerShell script writes the run's files (each to a temporary name, then over the old one). Both work in Windows PowerShell 5.1 and 7, and in Constrained Language Mode; the pages are written as ASCII (every other character as an HTML character reference, e.g. `é`), so no byte-order mark and no code page gets in the way.

1. **The folder**: e.g. `D:\reports`, made once by hand. The job makes the folders inside it.
2. **The account**: one that may use WinRM on that server (a local administrator, or a member of **Remote Management Users**) with **Modify** on the folder. A service account for it alone is best.
3. **The login**: create the **Report website** credential type (`aap/credential_types/report_web.yml`), a credential of it with that account (`DOMAIN\user` and its password), and attach it to each report template. It is a credential type of its own, so it goes beside the template's Machine credential (a Linux job's SSH key, say). Without it, the job logs in with the template's Machine credential.
4. **The connection**, as your Windows inventory's (docs/WINDOWS.md, step 4) - the defaults:

| Setting | Default | |
|---|---|---|
| `report_web_host_connection` | `winrm` | or `psrp` |
| `report_web_host_port` | `5986` | |
| `report_web_host_scheme` | `https` | `http` with Kerberos or NTLM (they encrypt each message then) |
| `report_web_host_transport` | `ntlm` | `kerberos` (needs Kerberos in the execution environment), `credssp`, `basic` |
| `report_web_host_cert_validation` | `validate` | `ignore` only to try it; `report_web_ca_path` is the CA that signed the WinRM certificate |

The execution environments from Red Hat do not trust DoD certificate authorities: with `validate`, set `report_web_ca_path` to your CA file inside the EE (as `ansible_winrm_ca_trust_path` in docs/WINDOWS.md), or the job stops with `certificate verify failed`.

5. **The website**: IIS on that server (or any web server reading the folder) serves `D:\reports` at `report_web_url`. Only reading - nothing is uploaded to IIS.

The server does not have to be in the job's inventory: the report jobs use many inventories, and the settings above are all it needs. Every report template must then run in an execution environment with WinRM (and Kerberos, if you use it) - or let the copy job write the pages ("The copy job", below). `report_web_dest` is a folder **on** that server (`D:\reports`), not a share (`\\fileserver\reports`): over WinRM that would be a second hop, which NTLM does not allow.

### A web server that takes uploads (HTTP PUT)

`report_web_dest: https://reports.yoursite.mil/upload` - the job uploads each file with `PUT` (WebDAV), from the AAP side. Nothing to mount, nothing on the execution nodes. The job reads `index.json` back from the same address (`GET`) to add its line. nginx, for example (the web developer's part):

```
server {
    listen 443 ssl;
    server_name reports.yoursite.mil;
    # ssl_certificate ... ;
    root /srv/reports;
    charset utf-8;

    location / {                          # reading: the pages
        index index.html;
        try_files $uri $uri/ =404;
    }
    location /upload/ {                   # writing: the jobs
        alias /srv/reports/;
        dav_methods PUT MKCOL;
        create_full_put_path on;
        client_max_body_size 50m;
        limit_except GET {                # GET (reading index.json back) open; PUT needs the login
            auth_basic "report uploads";
            auth_basic_user_file /etc/nginx/reports.htpasswd;   # htpasswd -c ... aap-reports
        }
    }
}
```

`/srv/reports` must be writable by nginx (`chown nginx: /srv/reports`; with SELinux `semanage fcontext -a -t httpd_sys_rw_content_t '/srv/reports(/.*)?'` and `restorecon -R /srv/reports`). IIS works too, with the WebDAV Publishing feature and a login on the upload path; so does Apache with `mod_dav`, or any endpoint of your own that takes `PUT` (answering 409 when a folder is not there makes the job create it with `MKCOL`).

The upload's login: create the **Report website** credential type (`aap/credential_types/report_web.yml`), a credential of it (a user name and password - sent as basic authentication - or a token, sent as `Authorization: Bearer ...`), and attach it to each report template. Never put it in Variables.

YAML

```
report_web_dest: https://reports.yoursite.mil/upload
report_web_url: https://reports.yoursite.mil          # what people open; the job prints addresses
  under it
report_web_ca_path: /etc/pki/ca-trust/source/anchors/yoursite-ca.pem   # only if the EE does not
  trust the server's CA
```

The report folder itself (`/upload/` here) must exist; the job makes only the folders inside it.

### A folder: a share mounted on the execution nodes

`report_web_dest: /mnt/reports` - the job writes the files there, from the AAP side, so the folder must be visible inside the job's container:

1. Mount the share on **every execution node** that runs these jobs (the same path on each), writable by the user AAP runs jobs as (`awx`). An SMB share, e.g. in `/etc/fstab`: `//fileserver/reports /mnt/reports cifs credentials=/etc/reports.cred,uid=awx,gid=awx,file_mode=0664,dir_mode=0775,_netdev 0 0` (with SELinux add `context=system_u:object_r:container_file_t:s0`).

2. In AAP: **Settings > Job settings > Paths to expose to isolated jobs**: `/mnt/reports:/mnt/reports:rw`.
3. The web server serves the same share (IIS on the file server, or nginx with the share mounted).

A folder that is not there stops the job with "is the share mounted?" - the job never creates it, so a share that failed to mount does not fill the node's own disk. No credential is needed.

## Turning it on

For every report job, in `playbooks/group_vars/all.yml` (the address is the same for all of them):

```yaml
report_web_host: web01.yoursite.mil                    # a Windows server - or leave it out for
  an upload or a folder
report_web_dest: 'D:\reports'                          # (https://reports.yoursite.mil/upload,
  /mnt/reports)
report_web_url: https://reports.yoursite.mil
report_logo_file: files/report_logo.png                # your logo, in your repository
```

Or on one template's Variables to try it on one job first. Like the email, not in an inventory group's Variables: the page is written by the AAP side of the job, which never gets a group's variables.

| Setting | Default | |
|---|---|---|
| `report_web_dest` | `""` (off) | the folder on `report_web_host` (`D:\reports`), the upload address (`https://...`) or the folder (`/mnt/...`) |
| `report_web_host` | `""` | a Windows server to write the pages on, over WinRM; `report_web_host_*` for the connection ("A Windows server") |
| `report_web_url` | `""` | the address people open; `""` = `report_web_dest` when it is `https://` (else the job prints where the page is: `web01:D:\reports\...`) |
| `report_web_theme` | `portal` | `standard`, `command`, `portal` or `glass`, as `report_email_theme`; `glass` adds the blur behind the cards, icons and a plain print |
| `report_web_json` | `true` | the JSON beside each page |
| `report_web_ca_path` | `""` | a CA file for an `https://` upload address, or for `report_web_host`'s WinRM certificate |
| `report_web_page`, `_latest`, `_index`, `_names` | see "Changing the names" | |

A dry run (Check) writes nothing and prints the address it would have used.

## The copy job: report templates in any execution environment

With `report_web_copy: true` the report jobs **write nothing** on the web server and connect to nothing there. Each keeps its report in AAP, as part of the job (the job artifact `site_report_page`: the report, its strip of past runs, the file names and the address, compressed - a 750-row certificate report is about 12 to 40 KB). One job template, **Report website - copy** (`playbooks/report_web_copy.yml`), in an execution environment that reaches the server, then writes those pages - the same pages, at the same address. So:

- the report templates keep their own execution environments and credentials: no WinRM, no Kerberos, no Report website credential on each one;
- the copy job is the only template that knows the server. When `report_web_host` is a host of **its** inventory (by the same name), it reaches it the way your Windows jobs do - that host's connection settings (`ansible_port`, `ansible_winrm_transport` ...) and the template's Machine credential. The `report_web_host_*` settings are not needed then;
- each report job still prints the page's address, keeps it as `site_report_url`, and its email gets the button (**Open the full report**, *Open this report in a browser*) - the page is there once the copy job has run, so a link opened before that answers *not found*.

### Setting it up

1. **The settings**, in `playbooks/group_vars/all.yml` (the report jobs and the copy job both read them):

```yaml
report_web_copy: true
report_web_host: web01.yoursite.mil      # the server's name as the copy job's inventory has it
report_web_dest: 'D:\reports'
report_web_url: https://web01.yoursite.mil/reports
report_web_theme: glass                  # the pages' look - the copy job draws them
report_logo_file: files/report_logo.png
```

Hold on to `report_web_copy: true` until step 3 has passed. To try it, put it on one report template's Variables first, run that template (Run, not Check), and look at that job in AAP: **Jobs >** the job **> Details > Artifacts** has `site_report_page`.

2. **The template** *Report website - copy*: playbook `playbooks/report_web_copy.yml`; the **execution environment your Windows jobs work with**; the **inventory that has the web server** (your Windows inventory), no Limit; its **Machine credential** - an account with **Modify** on the folder; and a **Red Hat Ansible Automation Platform** credential whose user has **Organization Auditor** on the organizations of the report templates (the one of the strip of past runs works). Leave *Concurrent jobs* off: one copy at a time, so the index is never written twice at once.
3. **Launch it with Job type Check**: it lists each report job it would copy (`Would copy: job 907 ...`) and writes nothing. Then **Run**: `job 907 'Linux - Certificate trust report': https://...` for each page written, and how the server was reached: `web01.yoursite.mil (winrm, port 5985, from this job's inventory): its connection settings in this job's inventory`.
4. **A schedule**: every 15 minutes, say (*Templates* > the template > *Schedules*). Or add it as the last step of a workflow that runs reports, so their pages are there when the workflow ends - or both.

### What each run copies

The report jobs that finished since the run before (a few minutes before it, leaving out those it already copied), oldest first, whatever their template or how they ended. Where a run got to is its own job artifact `site_report_copied`; a **Check** run, and a run with `report_web_copy_jobs`, keep none, so they never move it on. The first run (no run before it) copies the report jobs of the last `report_web_copy_hours` (24). A run copies at most `report_web_copy_max_pages` (50) pages and says when there are more: the next run copies the rest.

`report_web_copy_jobs: [907, 908]` (on the template's Variables, or a survey) copies those jobs' pages again, whatever was copied before - a page deleted on the server, say.

An older run never takes the place of a newer one: the template's `latest.html` and its line in the index are only written over by a run at least as new as the one there (by its time); the older run gets its own page.

A report too big to keep (more than `report_web_copy_max_kb`, 256 KB, compressed) has its longest tables cut until it fits - the page says *This page shows the first N of M rows* - and the email always has the whole report.

| Setting | Default | |
|---|---|---|
| `report_web_copy` | `false` | `true` = the report jobs keep their report for the copy job instead of writing their pages |
| `report_web_copy_max_kb` | `256` | the kept report's greatest size, compressed |
| `report_web_copy_jobs` | `[]` | the copy job: these job IDs only |
| `report_web_copy_hours` | `24` | the copy job's first run: the report jobs of the last hours |
| `report_web_copy_max_pages` | `50` | the copy job: pages a run writes at most |

Good to know:

- The copy job sees only the jobs its AAP user may see: **Organization Auditor** on every organization that has report templates. A report template in another organization is left out, without a word.
- Per-template settings of the website (`report_web_dest`, `report_web_url` on one template's Variables) are the copy job's own: it writes everything into its own `report_web_dest`. The file names (`report_web_page`, `report_web_names`) are the report job's - they are kept with the report.
- A template with `report_web_copy: false` on its Variables writes its pages itself, as before. It then shares the index with the copy job: two jobs writing it in the same second can lose one line (it is back after that template's next run).
- `report_web_host` not a host of the copy job's inventory (or under another name there, a short name, say): the copy job reaches it with the `report_web_host_*` settings and the Report website credential (else its Machine credential), as a report job would. The job says which it did.

## Using the address in a workflow or another job

The job keeps the page's address as the artifact `site_report_url`. In a workflow, the next step gets it as a variable (`{{ site_report_url }}`) - for a Teams post or a ServiceNow work note. From AAP's API: `GET /api/controller/v2/jobs/<id>/` - `artifacts.site_report_url`. From the job output, the line `Report: https://...`.

## Troubleshooting

| The job says | Why, and what to do |
|---|---|
| `the folder /mnt/reports is not there - is the share mounted, and the path exposed to the job` | The share is not mounted on the execution node that ran the job, or not in "Paths to expose to isolated jobs" |
| `writing /mnt/reports/...: Permission denied` | The share is not writable by `awx` (the mount's `uid` / `file_mode`), or SELinux (`context=` on the mount) |
| `the folder https://.../upload is not there on the website` | The upload's own folder is missing on the server (`alias` / its directory) |
| `uploading ...: HTTP 401` or `403` ... `check the login` | Attach the Report website credential; check its user name and password or token |
| `uploading ...: HTTP 405` | The server does not take `PUT` at that address (nginx: `dav_methods PUT MKCOL;` in that location) |
| `uploading ...: HTTP 413` | The page is bigger than the server allows (nginx: `client_max_body_size`) |
| `certificate verify failed` | Set `report_web_ca_path` to the CA that signed the server's certificate |
| `could not reach report_web_host web01... (winrm, port 5986): ...` - `the specified credentials were rejected` | The login: the Report website credential (`DOMAIN\user`), or the account is locked; without that credential, the template's Machine credential is used |
| `could not reach report_web_host ...: ... certificate verify failed` | `report_web_ca_path` (the CA of the server's WinRM certificate), or `report_web_host_cert_validation: ignore` to try it |
| `could not reach report_web_host ...: ... Connection refused` or timed out | WinRM is not on that port (`report_web_host_port`, `_scheme`), or a firewall between AAP and the server |
| `the folder D:\reports is not there on WEB01 (report_web_dest)` | Make the folder on that server; the job makes only the folders inside it |
| `The report website: Access to the path 'D:\reports\...' is denied` | The account has no Modify on the folder (NTFS) |
| `winrm or requests is not installed` | The execution environment has no `pywinrm` (docs/WINDOWS.md, step 2) |
| `report_web_page '...': there is no {x}` | A name that does not exist; the message lists the ones that do |
| A report missing from the index | Two jobs finished in the same second and one line was lost; it is back after that job's next run |
| The copy job: `Nothing to copy: no report job kept its report for this job since ...` | No report job finished with `report_web_copy: true` since then: is it in `all.yml`, did the project sync? A report run with Check keeps nothing |
| The copy job: `AAP refused the list of jobs (HTTP 403)` | Its AAP credential's user needs Organization Auditor on the report templates' organizations |

| The job says | Why, and what to do |
|---|---|
| The copy job: `could not reach report_web_host web01..., a host of this job's inventory` | The template's Machine credential, and that host's connection settings in the inventory - as for your other Windows jobs |
| The copy job: `job 907 (...): its report cannot be read` | The report job's project is newer or older than the copy job's (sync both), or the artifact was cut; the other reports are still copied |
| The report job: `No web page for this run: its report is bigger than report_web_copy_max_kb` | Raise `report_web_copy_max_kb`, or let that template write its pages itself (`report_web_copy: false` on its Variables) |
| The email's button answers *not found* | The copy job has not run since that report: it runs on its schedule (or launch it) |

# 24. Secrets: where they live

## Recommendation

**Store every secret as an AAP credential, and inject it into jobs as environment variables or files through custom credential types.** Nothing secret goes into Git, inventory, job template variables, or surveys.

- AAP encrypts credential values at rest and never shows them again after you save them. People can be allowed to **use** a credential in a job without being able to **see** it.
- If your organization already runs **CyberArk, HashiCorp Vault, or Azure Key Vault**, keep the secrets there and link the AAP credential fields to it with AAP's credential lookup plugins (on the credential's form, the key icon next to a secret field). AAP fetches the value when a job starts; nothing else in this repository changes.
- Use **Ansible Vault** only for playbooks you must also run outside AAP. It is weaker: one vault password decrypts every value, and the encrypted values sit in Git forever (rotating the password does not un-publish old ciphertext).

## Where each secret lives

| Secret | AAP credential type | Reaches the playbook as |
|---|---|---|
| SSH key + sudo for the hosts | built-in **Machine** | the SSH connection (never visible to tasks) |
| WinRM login for Windows servers | built-in **Machine** (user name and password; Kerberos or NTLM) | the WinRM connection (never visible to tasks). Use a **separate** credential from the SSH one; see WINDOWS.md |
| Git access for the project | built-in **Source Control** | used only by the project sync |
| MySQL root / STIG Manager DB passwords | **STIG Manager database** (`aap/credential_types/stigman_database.yml`) | env `STIGMAN_MYSQL_ROOT_PASSWORD`, `STIGMAN_DB_PASSWORD` |
| TLS certificate, key, CA chain | **TLS certificate** (`tls_certificate.yml`) | files; env `TLS_CERT_FILE`, `TLS_KEY_FILE`, `TLS_CA_FILE` hold the paths |
| Docker Hub / quay.io login for hosts | **Container registry login (hosts)** (`registry_login.yml`) | env `REGISTRY_*`, used with `podman login --password-stdin` |
| ACT model key | **ACT model key** (`act_model_key.yml`) | env `GENAI_KEY` / `ASKSAGE_API_KEY` on the controller. **Note:** when ACT runs on a managed host, the job hands the key to that one ACT process on the task's stdin (never on a command line, so not in `ps` or the sudo log) and it is gone when the process ends. So the host's administrators and its process auditing can see it while ACT runs: use a key with a spending limit, and rotate it if a host is untrusted. `VM - alarms ACT analysis` runs ACT on the AAP node itself: there the key never reaches another server |
| ACT GenAI beta key | **ACT GenAI beta key** (`act_genai_beta_key.yml`) | env `GENAI_BETA_KEY` |
| Teams webhook URL | **Teams webhook** (`teams_webhook.yml`) | env `TEAMS_WEBHOOK_URL` |
| ServiceNow API account | **ServiceNow API** (`servicenow_api.yml`) | env `SN_HOST`, `SN_USERNAME`, `SN_PASSWORD` (basic auth on the controller; never sent to hosts) |
| MariaDB monitor account | **MariaDB monitor** (`mariadb_monitor.yml`) | env `MARIADB_MONITOR_USER` / `_PASSWORD`; the password goes to the client on stdin and reaches it only as `MYSQL_PWD` in its own environment (never a command line, never a fact) |

| Secret | AAP credential type | Reaches the playbook as |
|---|---|---|
| STIG Manager API client | **STIG Manager API** (`stigman_api.yml`) | env `STIGMAN_TOKEN_URL`, `STIGMAN_CLIENT_ID`, `STIGMAN_CLIENT_SECRET` (token request is `no_log`) |
| Java keystore password | **Keystore password** (`keystore_password.yml`) | env `KEYSTORE_PASSWORD`; the inventory names the variable (`password_env`), never the value; the check reads it on stdin and `keytool` takes it with `-storepass:env` (never a command line) |
| vCenter account (VMware jobs) | built-in **VMware vCenter** | env `VMWARE_HOST`, `VMWARE_USER`, `VMWARE_PASSWORD` on the controller (the jobs talk only to vCenter) |
| NetApp ONTAP read-only account (ONTAP health report) | **NetApp ONTAP** (`netapp_ontap.yml`) | env `ONTAP_USERNAME`, `ONTAP_PASSWORD` on the controller; sent only to the clusters' REST API over HTTPS (basic auth); the job sends GET requests only |
| Horizon service account (the `horizon_*.yml` jobs) | **Horizon API** (`horizon_api.yml`) | env `HORIZON_USERNAME`, `HORIZON_PASSWORD` (and `HORIZON_DOMAIN`) on the controller; sent only to the Connection Server's REST API over HTTPS (`POST /rest/login`), which answers a token for that job; the job logs out at the end |
| Mail relay login (emailed reports) | **SMTP relay** (`smtp_relay.yml`), only if the relay needs a login | env `SMTP_USERNAME` / `SMTP_PASSWORD`; sent only over STARTTLS or SSL (refused with `report_email_security: none`) |

Create one credential of each type **per environment** (dev / test / prod) so a test job can never use production secrets.

### Why environment variables and files, not extra variables

A credential type can inject into `extra_vars`, but then the secret is an ordinary play variable: any `debug` task, any failed task's error message, or `-vvv` output can print it. An environment variable or a file on the controller has to be read on purpose (`lookup('ansible.builtin.env', 'NAME')`), which keeps secrets out of the variable namespace unless a task asks for them. Files are the right choice for certificates and keys: AAP writes them to private temporary files that disappear when the job ends.

## Rules for every playbook in this repository

1. `no_log: true` on every task that receives, compares, or writes a secret. On a loop, also set `loop_control: label:` to something harmless, or the item is printed.

2. `diff: false` on any task that writes secret material (copy/template of a key, a `.cnf` with a password). With AAP's *Show changes* enabled, a diff prints the file's contents.
3. **Never put a secret on a command line.** Command lines are visible in `ps`, in audit logs (`execve` records), and in verbose job output. Instead:
   - pass it on **stdin**: `podman secret create NAME -`, `podman login --password-stdin`;
   - or in a **file** the program reads (`--defaults-extra-file` for MySQL clients), created with mode `0600` and removed afterwards;
   - never `mysql -pPassword` or `curl -u user:password`.
   - never through Ansible's `environment:` keyword on a Linux host with become: Ansible puts those variables on the `sudo /bin/sh -c '...'` command line, which `ps` shows and sudo writes to the host's journal / `/var/log/secure` - `no_log` does not prevent that. Send the secret on the task's `stdin:` to a small `sh` that exports it (see `roles/check_mariadb/tasks/health.yml`).
4. **On the hosts, secrets are podman secrets**, not environment lines in unit files:
   - mounted as files where the image supports it (`MYSQL_ROOT_PASSWORD_FILE`, `MYSQL_PASSWORD_FILE`), with `mode=0400` and the container user's uid;
   - injected as an environment variable only when the application cannot read a file (`STIGMAN_DB_PASSWORD`): `Secret=NAME,type=env,target=VAR`.

Podman keeps the value out of the unit file and out of `podman inspect` (checked on podman 5.8; verify on your RHEL's podman). It is still readable by root on the host and inside the container, so protect root and the container.

5. **Private keys on disk:** `0600`, owned by root; certificate and CA files `0644`.
6. **Don't register command output that can contain secrets** without `no_log`, and never `debug: var=` a registered result from such a task.
7. **Fail closed**: assert a secret is present (with `no_log`) instead of letting a task run with an empty password.
8. **Changing a secret in AAP is not the same as rotating it.** For a database password the value inside MySQL must change too (`ALTER USER`). The deploy role refuses to silently swap a deployed database password; rotation is a separate, deliberate run.

## The real trust boundary

AAP will not show a stored secret, but **any playbook it runs can print it**. So anyone who can:

- push to the branch or tag the AAP project uses,
- edit a job template (its playbook, credentials, or verbosity), or
- run a template with *Prompt on launch* for variables that change what the playbook does,

can extract every secret that template uses. Therefore:

- **protect the branch** (required reviews) and pin the project to a **tag**;
- give most people **Execute** on templates, not **Admin**; give only automation owners **Use** on credentials;
- keep *Prompt on launch* for variables off on templates that hold production credentials;
- review the **Activity Stream** (who changed which credential or template) regularly.

## Keeping secrets out of Git

- `.gitignore` in this repository blocks common key/cert/vault-password file names.

- `.pre-commit-config.yaml` runs **gitleaks** on each commit (`pre-commit install`).
- The inventory in `inventories/example/` holds placeholders only; the real inventory lives in your work Git or as an AAP inventory.
- **No real host names or addresses either.** Every name in the repository is a placeholder (`vc01.example.mil`, `smtp.yoursite.mil`), every IP address is from the ranges kept for documentation (`192.0.2.x`, `198.51.100.x`, `203.0.113.x`), and the README and the changelog talk about features, never about one site. `tests/check_public.py` checks it in CI and on each commit. Your own site's words (a site code, a vCenter's name, a team, a ticket title) go in `~/.config/site-automation/private-words`, one per line, on the machine you commit from: they are looked for too, and the file never enters the repository.
- If a secret is ever committed: rotate it first, then remove it from history. Removing it from history alone is not enough, because clones and caches keep it.

## Rotation checklist

| Secret | How |
|---|---|
| TLS certificate | update the **TLS certificate** credential, re-run `stigman_deploy.yml` (nginx restarts) |
| Registry token | update the credential; the next run logs in with it |
| STIG Manager DB password | password-rotation run: `ALTER USER` in MySQL, update the podman secret, restart the API; then update the AAP credential. The playbook for this comes with the day-2 set. Changing only the credential is refused by the next deploy (the password in MySQL would stay the old one). **Creating the database again** from nothing: launch the deploy once with `stigman_replace_secrets: true` (Prompt on launch, Variables) - never save it in the template |
| SSH key | add the new public key to the hosts, update the **Machine** credential, remove the old key |
| ACT / Teams | update the credential; nothing else |
| ServiceNow API account | change the password in ServiceNow, update the **ServiceNow API** credential, then run *ServiceNow - test ticket* (SERVICENOW_SETUP.md) to prove it works |
| MariaDB monitor account | change it in the database (`ALTER USER`), update the **MariaDB monitor** credential, run a health check with `health_checks=mariadb` (MARIADB.md) |
| WinRM login | change it in the directory, update the Windows **Machine** credential, run *Windows connection test* |

# 25. Approved commands: what ACT may run by itself

ACT is a tool that runs **on each Linux server** when a job asks it to. This page explains, in plain words, which commands ACT runs by itself, which ones wait for a person, and how you add a command to the "run without asking" list.

**Short version.** There are three kinds of command:

| Kind | Examples | What happens |
|---|---|---|
| **Reading** | `systemctl status chronyd`, `df -h`, `journalctl -u nginx`, `ss -tlnp` | ACT runs it. It never asks. **Nothing on this page limits reading.** |
| **Changing**, not on your list | `systemctl restart nginx`, `dnf -y install httpd` | Refused. The job says `NEEDS APPROVAL` and shows the exact command. A person approves it in the workflow. |
| **Changing**, on your list | the ones you wrote in `site_act_allow` | ACT runs it by itself (only at level `self-heal`). |

The list is one setting: `site_act_allow`. You add a command by adding one line to it.

## Read this first: two safety rules that you cannot switch off

1. **Some commands are never run by a job**, whatever you write. ACT itself never pre-approves a *catastrophic* command (recursive delete, disk or volume destruction, power off, reboot). And the step that applies an approved fix (the *Apply approved ACT fix* job, and the service-watch fix step) has its own last check: reboot, shutdown, `kexec`, `mkfs`, `fdisk`, `usermod`, `passwd`, recursive deletes of system folders and a few more are refused **even after a person approved them** - also with a path in front (`/sbin/reboot`) or inside `bash -c '...'` / `$( )`. Patching and restarts have their own job: RUNBOOKS.md, section "Patch hosts".
2. **AAP itself is never fixed by a job.** Hosts in the groups `aap` and `aap_hosts` are diagnose-only: ACT reads and proposes, and the apply step prints the command for a person to run by hand. Nothing you set can remove those two groups (they are built into the code).

Everything below works inside those two rules.

## How a command becomes "approved": the three ways

### Way 1: a person approves it (the workflow), nothing to configure

This is the default at level `diagnose`.

1. The health check finds a problem. ACT investigates by reading, then proposes the fix.
2. Because the fix changes something and is not on any list, ACT is **refused** and the job fails with `NEEDS APPROVAL - ... ACT proposes: systemctl start chronyd`.
3. The workflow stops at its **Approval** step. The approver reads the proposal and approves or denies.
4. *Apply approved ACT fix* runs **exactly the approved text**, no model involved, then the checks run again.

What the approver sees is what runs. To set this up, see ADDING_ACT.md and WORKFLOWS_AND_SCHEDULES.md.

Two details worth knowing:

- Only a **plain single command** can go through this path (see the character rule below). A proposal such as `cd /x && ./fix.sh` or `something | tee file` is listed in the report as a **manual fix**: a person runs it by hand. That is deliberate.
- The approval covers that command on the hosts that proposed it, once. It does not add the command to any list. Next week the same problem needs the same approval.

### Way 2: you put it on the list (site_act_allow), for fixes you trust

Use this for boring, safe, repeated fixes: start a stopped service, restart a hung one. It only takes effect at level `self-heal`. At `explain` and `diagnose`, the list is ignored.

YAML

```
site_act_allow:
  - 'systemctl restart (chronyd|rsyslog|crond)'
  - 'systemctl start (chronyd|rsyslog|crond)'
```

Where to put it, step by step, is in the next section.

### Way 3: --allow on the ACT command line (when you run ACT by hand)

If you run `act` on a server yourself, the same idea is `--allow`:

shell

```
act --allow 'systemctl restart (nginx|httpd)' "the web server is down; find out why and fix it"
```

`site_act_allow` is this, done for you: each list item becomes one `--allow` when the job calls ACT. (Full ACT reference: "Approved commands" in ACT's own README.)

At the prompt that ACT shows when you run it by hand (`run? [y]es / [n]o / [e]dit / [q]uit`) there is **no "always allow" answer**. This is on purpose: the list is the only way to make ACT stop asking, and the list is in Git, where a reviewer can see it.

## Add an approved command, step by step

You need: VS Code with this repository, and the command as you would type it on the server.

### Step 1: write the command exactly as ACT would run it

Try it by hand on a lab server first: `sudo systemctl restart chronyd`. Note the words in between: `systemctl restart chronyd` (drop `sudo`; ACT already runs as root on the host).

### Step 2: turn it into a pattern

A pattern is a *regular expression* that must match the **whole** command, from the first character to the last. `systemctl restart chronyd` on its own is already a valid pattern that matches exactly that command and nothing else. Start there. Widen it only when you have to:

| You want | Pattern | Matches | Does not match |
|---|---|---|---|
| exactly one command | `systemctl restart chronyd` | `systemctl restart chronyd` | `systemctl restart chronyd.service` |
| a short list of services | `systemctl restart (chronyd\|rsyslog\|crond)` | `systemctl restart rsyslog` | `systemctl restart sshd` |

| You want | Pattern | Matches | Does not match |
|---|---|---|---|
| restart or start | `systemctl (start\|restart) (chronyd\|rsyslog)` | `systemctl start rsyslog` | `systemctl stop rsyslog` |
| a unit name with a dot | `systemctl restart chronyd\.service` | `systemctl restart chronyd.service` | `systemctl restart chronydXservice` |
| a container | `podman restart (mysql\|nginx)` | `podman restart nginx` | `podman rm -f nginx` |

(In the YAML file, write `|` normally. The `\|` above is only how a table shows a bar.)

Rules for the pattern:

- **The whole command must match.** `systemctl restart .*` looks like "restart anything", but it can only match a plain command. It is also far too wide: it would let ACT restart `sshd` and the network. Name the services.
- **Only a plain single command counts.** The command may contain only letters, digits and these characters: `space _ . / : = @ % + , -`. Anything else is never pre-approved, however you write the pattern: no `; && ||` (chaining), no `|` (pipes), no `> <` (redirection), no `$( )` or backticks, no `*` or `?` (globs), no quotes, no `~`, no line breaks. Such a command still needs a person.
- **A dot in a pattern means "any character".** Write `\.` when the exact dot matters, as in `chronyd\.service`.
- **No file edits.** A pattern covers `run` commands only; ACT changing a file (edit or write) always needs a person.
- **Dangerous commands are never approved by a pattern**, whatever you write: ACT's danger tier (package removal such as `dnf -y remove httpd`, `usermod -aG wheel alice`, `chmod 777`, `podman rm -f mysql`, `rm -f`, ...) and reboots, power-off and disk tools always need a person (tested with ACT 0.6.21). They come back as `NEEDS APPROVAL` / manual fixes.
- **Never put these on the list** anyway: `.*` on its own, `systemctl (stop|disable|mask) .*`, `kill .*`. Those are ordinary changes a pattern *can* approve, and far too wide: `systemctl stop .*` would let ACT stop `sshd`.

### Step 3: put it in the right settings file

`site_act_allow` is a setting like any other (VARIABLES.md explains where settings live). Pick the place that matches who should get it:

| Who gets the command | Where to write it |
|---|---|
| **Every host** | `playbooks/group_vars/all.yml`: add the lines at the end of the file (copy the example below). |
| **One group** (for example `stigman`) | `playbooks/group_vars/stigman.yml` (the file name is the AAP group name) |
| **One host** | `playbooks/host_vars/<host name as in the inventory>.yml` (create the folder and file if they do not exist) |
| **One template, every run** | the job template's **Variables** box (extra variables), YAML: `site_act_allow: ['systemctl restart chronyd']` |

**A list replaces, it does not add.** If `all.yml` has two commands and `stigman.yml` has one command, the stigman hosts get **only that one**. Repeat the ones you keep. (The exact order of who wins is in VARIABLES.md.)

Example, `playbooks/group_vars/all.yml`:

```yaml
site_act_allow:
  - 'systemctl (start|restart) (chronyd|rsyslog|crond)'
```

Use single quotes. Keep the indentation. Spaces, never tabs.

### Step 4: save, commit, sync

In VS Code: save, **Commit**, **Sync Changes**. AAP picks the file up on the next run (the project syncs when *Update revision on launch* is ticked).

### Step 5: use the level self-heal

Launch the check with `use_act` = `yes` and `site_act_level` = `self-heal` (the survey question on the Health check and Troubleshoot templates; ADDING_ACT.md).

## How to verify it worked

Do all three. They take ten minutes and catch the usual mistakes.

**1. Check the pattern matches (and only what you meant).** No server needed. In PowerShell:

```
$p = 'systemctl restart (chronyd|rsyslog|crond)'      # your pattern, without the quotes' worth of
  YAML
foreach ($c in 'systemctl restart chronyd',            # should be True
               'systemctl restart sshd',               # should be False
               'systemctl restart chronyd; reboot') {  # should be False
  '{0,-40} {1}' -f $c, ($c -cmatch ('\A(?:' + $p + ')\z'))
}
```

Expected output: `True`, `False`, `False`. (Verified. The same test in Python, on a machine that has `vendor/act/act`: load it with `importlib` and call `compile_preapproved`/`preapproved_pattern`; ACT applies exactly `\A(?:your pattern)\Z`.) Add a line for every command you want to *stay* refused and check it prints `False`.

**2. Check ACT itself agrees, on a lab server.** Log in, put your key in the environment the way SECRETS.md describes, and run ACT in the same mode a job uses. It will not ask anything; it either runs what you allowed or refuses:

```shell
sudo systemctl stop chronyd                # break it on purpose (lab server only)
act --non-interactive --allow 'systemctl (start|restart) chronyd' \
    --result-file /tmp/r.json "chronyd is stopped; find out why and fix it"
echo "exit code: $?"                       # 0 = fixed, 4 = something was refused
python3 -m json.tool /tmp/r.json | less    # see commands[].approval and denied[]
```

In `/tmp/r.json`: `commands[].approval` says `pre_approved` for what your list allowed (with the `pattern` that matched) and `auto` for reads. `denied[]` lists what was refused, each with `pre_approvable`: `true` means "a pattern could approve this" (add one if you want it); `false` means a person has to do it.

**3. Check the job.** On the same lab server, launch **Health check** with `use_act` = `yes`, `site_act_level` = `self-heal`. In the output:

- `ACT | what ACT says` shows `APPLIED (pre-approved): systemctl start chronyd`;
- the report afterwards says `cleared: ...`.

Then break something that is **not** on your list and check the job says `NEEDS APPROVAL` and does not touch it. A dry run (job type *Check*) does not call ACT, so it cannot test this; see DRY_RUNS.md.

### If it does not work

| You see | Why | Fix |
|---|---|---|
| `NEEDS APPROVAL` for a command you listed | the pattern does not match the **whole** command; or a listed command contains a `;`, `\|`, quote or glob; or the level is `explain`/`diagnose`, not `self-heal` | run check 1 with the exact command from the job output; fix the pattern or the level |
| the command is in `manual fixes`, not `proposed` | it is not a plain single command | ask ACT for a simpler fix, or apply by hand |
| the setting seems ignored | a more specific place wins (the group or host file, or the template's Variables box) | see the order in VARIABLES.md; the job prints a notice when a settings file overrides the AAP Variables box |
| an invalid pattern | ACT does not run: the job shows `ACT error ... invalid --allow pattern '...'` and the reason (for instance an unbalanced bracket) | fix the YAML/regex |
| a command on an AAP host does not run | by design: AAP is diagnose-only | run it by hand from the printed line |
| the approved command was refused at apply time | the apply-time guard (reboot, shutdown, `mkfs`, `usermod`, recursive delete of a system folder, ...) | do it by hand if you really mean it |

## Reference: what the guard rails are

| Where | Rule | Can you change it? |
|---|---|---|
| ACT, every command | proven read-only commands run without asking; everything else asks | no (it is the design) |
| ACT, `--allow` / `site_act_allow` | anchored to the whole command; plain characters only; never catastrophic; never a file edit | you choose the patterns |
| ACT, `--auto` | runs reads and ordinary changes without asking, still asks for the danger tier. Not used by these jobs | not used |
| `site_act_diagnose_only_groups` | hosts where ACT never fixes; the list *you* write replaces the shipped one (`sat`, `idm`, ...), but `aap`, `aap_hosts` are always kept | you choose the groups; you cannot remove `aap`/`aap_hosts` |
| Apply-time guard (`_apply_refuse`, `roles/site_findings/vars/main.yml`) | last check before an approved command runs | edit the code to change it |
| Host log | `/var/log/site-automation-fix.log` on each host lists what the apply job ran, when, and for which AAP job and user | |

The apply-time guard applies to the **approval** path (Way 1). Commands run by **self-heal** (Way 2) are run by ACT itself, so what protects you there is your list. Keep it short and boring.

### Windows

`site_act_allow` is read by the Linux jobs. Windows servers use the approval path only (ACT proposes, a person approves); see WINDOWS.md. If you run ACT for Windows by hand, its `-Allow` option follows the same rules as Linux `--allow`:

| | Linux ACT (--allow) | Windows ACT (-Allow) |
|---|---|---|
| A pattern that names a danger-tier command (for example `dnf -y remove httpd`, `podman rm -f mysql`) | **does not** approve it | **does not** approve it |
| A danger-tier command in an unattended run | **always refused**; only a person at an interactive prompt can approve it | the same |
| Catastrophic commands | never | never |
| `git`, cmd built-ins, aliases such as `ls` and `dir` in an unattended run | follow the rules above | **still ask**, because they were never on Windows ACT's unattended list |
| Result file (`--result-file` / `-ResultFile`) | written (since 0.6.16) | written (since 0.6.16; its `tokens` field is filled since 0.6.21) |

(ACT 0.6.20 for Linux, released for a few minutes on 2026-09-30, let a pattern approve a danger-tier command; 0.6.21 and later (this project carries 0.6.22) do not. The Windows rows are taken from ACT-Windows 0.6.22 and its self-tests and were not run by hand on a Windows server.)

# 26. The MariaDB and MySQL check, step by step

There are **two ways to run the database check**, and both check **MariaDB and MySQL** (MySQL 5.7, 8.0 and 8.4, MariaDB 10.x and 11.x, and Percona), installed on the host or **in podman containers** (one, several, found automatically, rootless):

| How | Job template | Use it when |
|---|---|---|
| **Its own template** | **Database health - MariaDB / MySQL** (`playbooks/database_health.yml`; SETUP_AAP.md) | the database team wants its own job, schedule and tickets. Runs only this check, on every database server it is pointed at |
| **Part of the Health check** | **Health check** with `health_checks` = `mariadb`, `mysql` or `database` (all three are the same check; `daily` includes it) | you already run the Health check and want the databases in it |

This page shows how to turn the check on, how to give it a login, and how to see that it works. Nothing here needs you to know Ansible. It changes nothing on the database: the check only asks status questions and never reads your data.

Words used here:

- **container name** - the name podman gives a container. You read it with `podman ps -a`.
- **rootless** - the container belongs to an ordinary user (not root). Only that user can see it.
- **settings file** - a small file in `playbooks/group_vars/` that holds your choices for one group of hosts. See VARIABLES.md.

## The short version

1. On the database host, find the container name: `sudo podman ps -a` (rootless: `sudo -u THE_USER podman ps -a`).
2. Put the name in the settings file for your MariaDB group (section 3).
3. Ask the DBA for a read-only monitoring account and put it in an AAP credential (section 4).
4. Run the **Database health** job (or the *Health check* with `health_checks` = `mariadb`) and read "How to verify" below.

## 1. Which hosts the check runs on

The check runs on a host when **any** of these is true:

- the host is in one of the inventory groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` or `mysql` (any of the five works, for either engine), or
- the host has a container name set (section 2), or
- you set `check_mariadb_enabled: true` for the host or its group.

The **Database health** template does not need this: every server it runs on is checked. It runs on the groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` and `mysql` (since 0.6.0; before that only the first three). A **Limit** only narrows that list; it cannot add a server that is in none of them. For a server outside those groups, put `target: <group or host>` in the template's **Variables** box, or add the server to one of those groups. If `target` matches no host, the job stops with a message that lists the groups your inventory does have. On every other host, in the Health check, the job prints `skipped: the MariaDB/MySQL check is`

`off for <host>` and moves on. That is normal; it is not an error.

## 2. Tell it where MariaDB is

Pick **one** of the four ways. Put the lines in the settings file (section 3).

| Your situation | Setting |
|---|---|
| MariaDB or MySQL is installed on the host itself (a service) | nothing to set. The check finds the service `mariadb`, `mysqld` or `mysql` and tells the engine and version apart by itself. To name the service: `check_mariadb_service: mariadb` |
| In a podman container, with **no** database server installed on the host, or started by a unit that runs podman (`podman compose up -d`, `podman start` - such a unit shows `active (exited)` or `inactive` while the database runs) | usually nothing to set (0.7.0 and later): the check sees that and finds the container by its image. Naming it is still clearer: `check_mariadb_container: mariadb` |
| One container | `check_mariadb_container: snow-mariadb` |
| Several containers on the same host | `check_mariadb_containers: [snow-mariadb, other-db]` |
| You do not want to list them: find them | `check_mariadb_container_discover: true` |

Details that matter:

- **Discovery** looks at every container on the host, running or not, and picks the ones whose **image name** matches `check_mariadb_container_image_regex` (default `(mariadb|mysql|percona)`). If your image has another name, change the pattern, for example `'(mariadb|snowdb)'`. Discovery that finds nothing is reported as a warning `mariadb:no-container`, so a wrong pattern is not silent.
- With **several** instances on one host, each is checked on its own. Their findings start with the container name, for example `[snow-mariadb] 143 of 151 connections in use`.
- The check runs the database's own client **inside** the container (`podman exec`), so you do not need a client on the host and you do not open any network port.
- The engine (MariaDB or MySQL) and its version are read from the server, so the same settings work for both.
- A container that is named but does not exist is reported as `mariadb:down` ("MariaDB/MySQL is not running (container ... is missing)"), not skipped.

### Rootless containers

When the containers belong to a user, name that user:

```yaml
check_mariadb_container_user: appuser
```

The check then runs podman **as that user** (the job needs privilege escalation, which the templates in SETUP_AAP.md already have). If that user does not exist on the host you get the critical finding `mariadb:user`, with the hint `getent passwd <user>`. Leave the setting empty for containers that belong to root.

## 3. Where to put the settings

Open `playbooks/group_vars/mariadb.yml` (MySQL: `playbooks/group_vars/mysql.yml`) in VS Code. It already holds the lines below as comments. Remove the `#` in front of the ones you need and replace `CHANGE-ME`:

```yaml
check_mariadb_enabled: true
check_mariadb_container: snow-mariadb
# check_mariadb_container_user: appuser      # only for rootless containers
```

**A settings file applies to the group with the same name as the file.** `mariadb.yml` is for the group `mariadb`, `mysql.yml` for `mysql`. If your hosts are in a group with another name (the example inventory uses `mariadb_hosts` and `mysql_hosts`), copy the file to `playbooks/group_vars/<your group name>.yml`. The check accepts all five group names, but the *settings* are only read from the file that matches the group. (The **Database health** template runs on the `_hosts` groups only: for a group called `mariadb` or `mysql`, give it a `target`, see section 1.) One host that differs from the rest: `playbooks/host_vars/<host name>.yml` (create it). Which place wins, and how to set the same thing in AAP's own Variables boxes, is in VARIABLES.md.

Then commit, push, and run **Sync** on the project in AAP.

## 4. The login: the "MariaDB monitor" credential

| Where the database runs | What to do |
|---|---|
| MariaDB installed on the host | Usually nothing. Run with privilege escalation and the check logs in as root through the local socket (RHEL's default). |
| MySQL installed on the host | Usually the credential: MySQL's root account normally has a password. |
| In a container (either engine) | Use the credential. The official images give root a password, and without it the job reports a finding that contains `Access denied` and ends with `attach the "MariaDB monitor" credential`. |

Steps:

1. Ask the DBA to run this **inside that database** (the account needs no data access). The privileges differ by engine:

```sql
CREATE USER 'aap_monitor'@'localhost' IDENTIFIED BY '<password>';

-- MySQL 5.7 / 8.0 / 8.4:
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
-- MySQL Group Replication only, also:
GRANT SELECT ON performance_schema.* TO 'aap_monitor'@'localhost';

-- MariaDB 10.5.9 and newer:
GRANT PROCESS, REPLICA MONITOR ON *.* TO 'aap_monitor'@'localhost';
-- MariaDB before 10.5.9:
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
```

**On MariaDB 10.5.9 and newer,** `REPLICATION CLIENT` **is not enough:** it no longer covers the replication status, so the account needs `REPLICA MONITOR`. `PROCESS` lets the check see other sessions (for long queries). Without the replication privilege the check still works and reports `mariadb:replication-query` (below) with

the grant to ask for. The host is `localhost` because the client runs inside the container.

2. In AAP create a credential of type **MariaDB monitor** (the type is in `aap/credential_types/mariadb_monitor.yml`; SETUP_AAP.md shows how to load it). Type the account name and password into the credential. **Never** put the password in a settings file, the inventory or Git: SECRETS.md.
3. Attach the credential to the **Database health**, **Health check** and **Troubleshoot** job templates. Credentials attach to templates, not to groups; the check only uses it on hosts that have a database.

The password reaches the client only through its environment (`MYSQL_PWD`, and `podman exec -e MYSQL_PWD` for containers), never on a command line. If your client needs other options, for example a socket path or a TCP address, use `check_mariadb_client_args: ['--socket=/var/lib/mysql/mysql.sock']`.

## 5. What it checks, and the limits you can change

Replication is read with the statement that fits the engine: `SHOW ALL SLAVES STATUS` (MariaDB), `SHOW REPLICA STATUS` (MySQL 8.0.22 and newer, 8.4) or `SHOW SLAVE STATUS` (older MySQL). The job's status line names the engine, for example `MySQL 8.4.11 - container snow-mysql (podman, image ...): up 312.4 h, 23/151 connections, ...`.

Each limit is a setting. The defaults are in the table; change one the same way as in section 3.

| Finding | Turns on when | Setting (default) |
|---|---|---|
| `mariadb:down` | the service or container is not running, or the named container does not exist | - |
| `mariadb:connect` | it runs but the status query failed (login, hang) | - |
| `mariadb:restarted` | restarted less than this many minutes ago | `check_mariadb_restart_warn_minutes` (60) |
| `mariadb:connections` | connections in use, percent of `max_connections` | `check_mariadb_conn_warn_pct` (80), `check_mariadb_conn_crit_pct` (95) |
| `mariadb:refused` | clients were refused because the limit was reached | - |
| `mariadb:threads-running` | queries executing at this moment | `check_mariadb_threads_running_warn` (32) |
| `mariadb:long-queries` | a query ran longer than this many seconds (never the query text) | `check_mariadb_long_query_seconds` (300) |
| `mariadb:bufferpool` | percent of reads served from memory (judged after 1 million reads) | `check_mariadb_bufferpool_hit_warn` (95) |
| `mariadb:replica:<name>`, `mariadb:replica-lag:<name>` | replication stopped, or behind by seconds | `check_mariadb_replica_lag_warn` (300), `check_mariadb_replica_lag_crit` (1800) |

| Finding | Turns on when | Setting (default) |
|---|---|---|
| `mariadb:replication-query` | the replication status could not be read (usually a missing privilege; the finding names the grant) | section 4 |
| `mariadb:galera`, `mariadb:galera-size` | Galera / XtraDB Cluster node not healthy; cluster smaller than expected | `check_mariadb_galera_size` (0 = not Galera; set 3 for three nodes) |
| `mariadb:group`, `mariadb:group-size` | MySQL Group Replication: a member is not ONLINE (or none is listed); fewer ONLINE members than expected | `check_mariadb_group_size` (0 = do not check the size; set 3 for three members) |
| `mariadb:group-query` | the Group Replication members could not be read (needs `SELECT` on `performance_schema`) | section 4 |
| `mariadb:errorlog` | `[ERROR]` lines in its log this recent (days) | `check_mariadb_error_since_days` (1) |
| `mariadb:disk` | the disk holding the data directory is this full (percent) | `check_mariadb_disk_warn_pct` (85), `check_mariadb_disk_crit_pct` (95) |
| `mariadb:user` | `check_mariadb_container_user` names a user that does not exist | - |
| `mariadb:no-container` | discovery found no container with a matching image | `check_mariadb_container_image_regex` |
| **performance** (warnings): `mariadb:slow-queries`, `tmp-disk`, `full-joins`, `table-locks`, `row-locks`, `deadlocks`, `bufferpool-wait`, `log-waits`, `aborted-connects`, `table-cache` | averages since the server started, judged after it has run 2 hours: slow queries per hour, % of temporary tables on disk, joins without an index per hour, table lock waits, average row lock wait, deadlocks per hour ... | `check_mariadb_performance` (true), `check_mariadb_perf_min_uptime_hours` (2), `check_mariadb_slow_per_hour_warn` (60), `check_mariadb_tmp_disk_pct_warn` (25), `check_mariadb_full_join_per_hour_warn` (100), `check_mariadb_table_lock_wait_pct_warn` (1), `check_mariadb_row_lock_avg_ms_warn` (500), `check_mariadb_deadlocks_per_hour_warn` (1), `check_mariadb_innodb_waits_per_hour_warn` (1), `check_mariadb_aborted_connects_per_hour_warn` (60), `check_mariadb_table_open_per_hour_warn` (600) |
| **performance, now** (warnings): `mariadb:lock-waits`, `long-transactions`, `purge-lag`, `open-files` | a session waiting this many seconds for a lock; a transaction open this long (idle ones are named); InnoDB history list length; files open, % of `open_files_limit` | `check_mariadb_lock_wait_seconds` (30), `check_mariadb_long_trx_seconds` (600), `check_mariadb_history_length_warn` (1000000), `check_mariadb_open_files_warn_pct` (85) |

The finding ids keep the `mariadb:` prefix for MySQL too, so existing tickets and filters still match. What each finding means and the usual cause are in RUNBOOKS.md. Every setting with its default is in VARIABLES_REFERENCE.md. With several containers on one host, the finding id includes the container

name, for example `mariadb:snow-mariadb:connections`.

**All settings** of this job, with their defaults: check_mariadb.

## How to verify

1. **The names are right.** On the database host: `sudo podman ps -a` (rootless: `sudo -u THE_USER podman ps -a`). The name you put in the settings file is in the NAMES column.
2. **The account works.** On the host, run the client inside the container as the monitoring account (it asks for the password; type it here, not in a file):

```shell
sudo podman exec -it snow-mariadb mariadb --user=aap_monitor -p -e "SHOW GLOBAL STATUS LIKE
  'Uptime'"
```

A number back means the login and the `PROCESS` rights work. (Some images have `mysql` instead of `mariadb`.)

3. **Run the job.** In AAP launch **Database health - MariaDB / MySQL** (first as *Check*, then as *Run*; SETUP_AAP.md), or the **Health check** with `health_checks` = `mysql` (or `mariadb`, or `database`) and a Limit of the database host. In the output, find the task `MariaDB | status`. It prints one line per instance, like `MariaDB 11.8.9 - container snow-mariadb (podman, image ...): up 312.4 h, 23/151 connections, 1 running, 0 long queries, replication connections 0`. That line proves the check reached the database. Each host then prints `HEALTHY` or its findings, and the job ends with `N healthy, M with findings`.
4. **No findings** means the database is healthy by these limits. To see a finding on purpose, set a limit that is sure to trip, for example `check_mariadb_conn_warn_pct: 1`, run again, look for `mariadb:connections`, then remove the line.

## What was tested, and what was not

| Tested against real containers (throwaway rootless podman) | Tested only against a fake podman |
|---|---|
| MySQL 8.0.46, MySQL 8.4.11, MariaDB 11.8.9: the engine and version are recognised; a replication source that never connects is reported as broken; the error log is read; a login with no credential gives `mariadb:connect` with the grant to ask for; the `REPLICA MONITOR` grant and revoke on MariaDB behave as described above | MySQL 5.7, and MySQL Group Replication (`mariadb:group`, `group-size`, `group-query`). They follow the documented statements but have not met a real server. The **performance** findings (0.15.0): their queries use documented status variables and `information_schema` tables of both engines, but only the fake has answered them so far. |

The fake-podman test (`bash tests/mariadb/run_mariadb_test.sh`) covers one container, several, discovery, a stopped and a missing container, a rootless user that does not exist, MySQL 5.7, 8.0 and 8.4, replication, Group Replication, a denied grant, and that the password never appears on a command line. Nothing here has been run against **your** database: step 3 is the real proof.

## If it does not work

| You see | Cause | Fix |
|---|---|---|
| `skipped: the MariaDB/MySQL check is off for <host>` (Health check only) | the host is in none of the five groups and no container is named | section 1, or use the **Database health** template |
| `mariadb:replication-query` | the account lacks the replication privilege (MariaDB 10.5.9+: `REPLICA MONITOR`, not `REPLICATION CLIENT`) | section 4 |
| `mariadb:no-container` | discovery matched nothing, or podman is run as the wrong user | `podman ps -a` as the right user; set `check_mariadb_container_image_regex` or `check_mariadb_container_user` |
| `mariadb:user` | the rootless user name is wrong | `getent passwd <user>` on the host |
| `status query failed: ... Access denied ...` and `attach the "MariaDB monitor" credential` | container root has a password | section 4 |
| `... is not running (container ... is missing)` but it is running | the name is wrong, or the containers belong to another user | compare with the `podman ps -a` output; `check_mariadb_container_user` |
| The check runs on the wrong host, or not at all after a rename | a settings file only applies to the group named like the file | section 3 |
| `MariaDB is not running (service mariadb is inactive)` - or a login error from the host's client - but `podman ps` shows the database container up | the host's `mariadb.service` only starts the container (`podman compose up -d`); before 0.7.0 the check took it for the database | 0.7.0 and later find the container by themselves; or name it: `check_mariadb_container: mariadb` |
| **Database health** says `skipping: no hosts matched` (0.5.0), or `target '...' matches no host` | the server is in none of `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb`, `mysql` (a Limit cannot add it) | section 1: `target` in the template's Variables, or add it to one of those groups |
| Your setting has no effect | another place wins | VARIABLES.md |

# 27. Active Directory health report and account lockout

One job, `playbooks/ad_health_report.yml`, reads every domain controller (DC) in the Windows inventory and emails one report for the whole domain: what is broken first, then replication, DNS, time and SYSVOL, then **what changed** in the last 24 hours (privileged groups, accounts, Group Policy, schema and configuration). Each row is coloured: red = critical, amber = warning, green = OK, blue = for your information.

**It only reads.** Each DC runs a PowerShell script over WinRM (the same way the Windows health check does) that queries Active Directory, runs `dcdiag` tests that only read, `repadmin /showrepl`, `/queue`, `/showutdvec` and `/showbackup`, `w32tm /query` and `/stripchart`, `nltest /sc_verify`, and reads the event logs. It changes nothing: it does not run dcdiag's `DnsDynamicUpdate` test (that one writes a test record), and it does not run `repadmin /syncall`, `/replicate` or anything else that acts.

A second job, `playbooks/ad_lockout.yml`, is for the service desk: is an account locked out, and **which computer is locking it out** - see Account lockout at the end.

**No second hop.** Every DC asks only itself (`localhost`). AD is replicated, so the DC that reads the domain has every object in its own copy, and the replication metadata says on which DC a change was made. So the job works over WinRM with NTLM or Kerberos and needs no delegation or CredSSP.

## What the report shows

At the top: the title with the number of critical and warning findings, then boxes with the main counts (DCs read, replication, dcdiag, services, SYSVOL / GPO, FSMO / DNS / time, privileged changes, accounts created / deleted, GPO changes). Then, in this order:

| Section | What it shows | Colours |
|---|---|---|
| **Domains** | each domain: functional levels, schema version, DCs checked / in AD, the PDC emulator, tombstone lifetime, Recycle Bin, RID pool left, the oldest partition's last backup, the krbtgt password's age, the SYSVOL migration state | amber Recycle Bin off, krbtgt older than 180 days, FRS; red RID pool under 2 %, backup older than 30 days or the tombstone lifetime |
| **Domain controllers** | each DC: site, OS, GC / RODC, the FSMO roles it holds, LDAP answering (and synchronized), DC services not running, dcdiag's failed tests, SYSVOL, its clock, the least free space of its database, log and SYSVOL volumes, uptime. A DC in AD that is not in the inventory is listed in blue | red unreachable, LDAP down, not synchronized, a DC service stopped |
| **Operations masters (FSMO)** | who holds each role, whether that holder still exists as a DC, whether it answers on LDAP | red holder gone or not answering |

| Section | What it shows | Colours |
|---|---|---|
| **Inbound replication** | links with a problem only (`repadmin /showrepl` on each DC): partition, partner, failures, last success, last error with what the code means | amber 6 h without success, red 24 h, a failing link, or longer than the tombstone lifetime; amber 50+ requests in the queue |
| **Replication from every DC (up-to-dateness)** | how long ago each DC's latest change reached the DC that read the domain (`repadmin /showutdvec /latency`): this also catches a DC that is **not in the inventory** and stopped replicating | amber 6 h, red 24 h |
| **dcdiag** | the failed tests and what dcdiag said. `SystemLog` is skipped by default (the Events section covers it) | red Connectivity, Advertising, NetLogons, Replications, SysVolCheck, Services, KnowsOfRoleHolders, RidManager, MachineAccount; amber the others |
| **DNS** | dcdiag's DNS tests that only read (basic, forwarders, delegation, record registration), and the SRV records `_ldap` / `_kerberos` (`dc._msdcs`) and `_ldap._tcp.gc`: every DC must be registered | red record missing; amber a DC not registered, a DNS test failed |
| **Time** | each DC's source, stratum, last sync, offset: a DC against the PDC emulator, the PDC against its NTP source | red the PDC on its own clock, 60 s off; amber 5 s off, no sync for 24 h |
| **Events** | the Directory Service, DFS Replication and DNS Server logs, and the Netlogon, KDC, time and Group Policy entries of the System log: the events that matter on a DC, with what they mean (2042, 1388 / 1988 lingering objects, 2087, DFSR 2213 / 4012, LDAP signing 2887, Netlogon 5719 ...), and every other error in blue | per event |
| **Clients in no site** | client addresses in no AD subnet (`netlogon.log` NO_CLIENT_SITE), and subnets linked to no site | amber |
| **Certificates (LDAPS and Kerberos)** | the newest LDAPS and KDC certificate of each DC | amber 30 days, red 7 days; blue none |
| **Trusts** | every trust; outbound and two-way trusts verified (`nltest /sc_verify`) | red does not verify |
| **Sites** | each site, its DCs and subnets | blue no DC or no subnet |

| Section | What it shows | Colours |
|---|---|---|
| **Privileged group changes** | members added to or removed from Domain / Enterprise / Schema Admins, Administrators, Account / Server / Print / Backup Operators, Group Policy Creator Owners, Key Admins, Cert Publishers, DnsAdmins (and your own): when, which DC, **by whom** | amber (`ad_privileged_change_severity`) |
| **Privileged groups** | their direct members now | amber a group in `ad_groups_should_be_empty` with a member |
| **Users, groups and computers created or deleted** | when, where, by whom | blue |
| **Accounts enabled, disabled, locked or unlocked** | when, which DC, by whom; the accounts locked out now | blue |
| **Group Policy changes** | GPOs created, changed, deleted; OUs, domains and sites whose links changed (and what is linked there now) | blue |
| **Group Policy: AD against SYSVOL** | GPOs with no SYSVOL folder, GPOs whose version differs in AD and SYSVOL, SYSVOL folders with no GPO | amber / blue |
| **Schema and configuration changes** | schema version and new classes / attributes, sites, subnets, site links, DCs added or removed, new trusts | amber schema and trusts; blue the rest (`ad_dc_change_severity`) |
| **Security log: notable events** | the Security log cleared on a DC (red), the DSRM password set, trusts created / removed, the domain policy changed | per event |
| **Not read** | anything a script could not read, with the reason | |

**When** and **which DC** come from AD's replication metadata, which every DC keeps: no audit policy is needed for them. **By whom** comes from the Security log of the DC the change was made on (events 4720, 4722, 4725, 4726, 4728-4733, 4741, 4743, 4756, 4757, 4767), so it is filled in only when *Audit Security Group Management*, *Audit User Account Management* and *Audit Computer Account Management* are on (the DISA STIG asks for all three), and that DC is in the inventory.

The job's artifacts (`ad_health`) have every finding, for a workflow or ServiceNow.

## Before you start

### 1. The DCs in the Windows inventory

The DCs go in the group `win_dc` of your Windows inventory (the example has it: `inventories/example-windows/hosts.yml`). WinRM is set up as for every Windows server (WINDOWS.md, steps 1 to 5). Test it with *Windows connection test* and Limit `win_dc`.

Put **every** DC in the group if you can. A DC that is not there is still covered by the domain-wide sections (replication from every DC, SRV records, FSMO, the changes), but its own services, dcdiag, events, time and disks are not read, and changes made on it show no "by whom".

### 2. The account

`dcdiag` and `repadmin` need administrator rights **on the DCs**, and the local Administrators group of a DC is the domain's built-in Administrators group: there is no smaller group that lets them run. So the job needs an account in the domain's **Administrators** (or **Domain Admins**) group.

- Make a **dedicated** account for it (e.g. `svc_aap_adhealth`), used by nothing else.
- Treat it as tier 0: allow it to log on to the DCs only (*Log on to* / an authentication policy silo), and add it to **Protected Users** if your AAP uses Kerberos (Protected Users blocks NTLM).
- Its password lives only in AAP, in a **Machine** credential of its own, e.g. `Windows DCs (WinRM)` (same fields as the `Windows (WinRM)` credential of WINDOWS.md, step 3).

Reading the **Deleted Objects** container (for "deleted" in the changes) also needs these rights; with less, that part is listed under *Not read* and the rest of the report is complete.

### 3. On the DCs

- The **ActiveDirectory** PowerShell module (`RSAT-AD-PowerShell`): installed with the AD DS role, so every DC has it unless it was removed. Without it the DC's LDAP checks and the domain-wide part are skipped and the report says so (amber).
- **English** dcdiag output: the job reads dcdiag's `passed test` / `failed test` lines. On a DC with another display language, dcdiag's results are reported as *not run* (amber).
- Windows Server 2012 R2 and later; Windows PowerShell 5.1. Constrained Language Mode (WDAC, AppLocker) is fine: the scripts are tested in it.

## Set it up

1. **Templates > Create template > Create job template:**
   - Name: `Active Directory health report`
   - Inventory: `Windows servers`. Project: site-automation. Playbook: `playbooks/ad_health_report.yml`
   - Execution environment: your Windows EE (WINDOWS.md, step 2)
   - Credentials: **Windows DCs (WinRM)** (the account of step 2)
   - Tick **Prompt on launch** next to **Variables** and **Limit**
   - Variables:

```yaml
report_email_to: ad-team@yoursite.mil
```

   - Leave Limit empty (with no `target`, it reads the group `win_dc`). Save.

2. **First run: one DC.** Launch with Variables `target: dc01.yoursite.mil`. The domain-wide part is read on that DC too. Read the report in the job's output; anything under *Not read* says what that DC or account could not read (see Troubleshooting).
3. **All DCs**, then a schedule: **Schedules > Create schedule**, every day at 06:30 (set **your** time zone). Keep `ad_report_hours` (24) equal to the time between runs, so every change is in exactly one report.

A dry run (Check) reads and prints the same report, and sends no email. It takes one to three minutes per DC (mostly dcdiag); the DCs are read in parallel (the template's **Forks**).

The **PDC emulator** reads the domain-wide part when it is in the job; otherwise the first DC of the domain whose LDAP answered. The job output says which (`corp.example.mil: read by ...`).

## Settings

Set them in the template's Variables, or in `playbooks/group_vars/all.yml` (or `win_dc.yml`). Ready to paste, with every setting switched off: aap/extra_vars/ad_health_report.yml.

| Setting | Default | What it does |
|---|---|---|
| `ad_report_hours` | `24` | the changes of the last this-many hours (match the schedule) |
| `ad_event_hours` | `24` | the event log entries of the last this-many hours |
| `ad_security_events` | `true` | read the Security log for "by whom" |
| `ad_privileged_groups_extra` | `[DnsAdmins]` | more groups to watch (names or SIDs) |
| `ad_groups_should_be_empty` | `[]` | groups that must have no member, e.g. `[Schema Admins]` |
| `ad_privileged_change_severity` | `warning` | a privileged group change: `warning`, `critical` or `info` |
| `ad_schema_change_severity` / `ad_dc_change_severity` | `warning` / `info` | the schema changed; a DC added or removed |
| `ad_dcdiag_skip` | `[SystemLog]` | dcdiag tests to skip |
| `ad_dcdiag_dns` | `true` | also dcdiag's DNS tests that only read |
| `ad_trust_verify` | `true` | verify outbound and two-way trusts |
| `ad_repl_warn_hours` / `_crit_hours` | `6` / `24` | a replication link without success for this long (keep warn above your inter-site schedule) |
| `ad_repl_queue_warn` | `50` | requests in a DC's inbound replication queue |
| `ad_time_warn_seconds` / `_crit_seconds` | `5` / `60` | a DC's clock offset |
| `ad_time_sync_warn_hours` | `24` | no time sync for this long |
| `ad_disk_warn_pct` / `_crit_pct` | `15` / `5` | free % of the database, log and SYSVOL volumes |

| Setting | Default | What it does |
|---|---|---|
| ad_cert_warn_days / _crit_days | 30 / 7 | the LDAPS / KDC certificate expires |
| ad_backup_warn_days / _crit_days | 7 / 30 | the last system-state backup of a partition |
| ad_krbtgt_max_days | 180 | the krbtgt password's age (the STIG asks for 180) |
| ad_rid_warn_pct / _crit_pct | 10 / 2 | RID pool left |
| ad_no_client_site_warn | 1 | client addresses in no subnet before it is amber |
| ad_max_events / ad_changes_max | 2000 / 500 | the most event log entries read per log and DC; the most changed objects per kind |
| ad_report_max_rows | 50 | rows per table (problems first; the artifacts have all) |
| ad_report_fail | false | true = the job shows **failed** on a critical finding or a DC not read (for a workflow) |

**All settings** of this job, with their defaults: ad_report.

## Troubleshooting

| You see | It means | Do |
|---|---|---|
| A DC **UNREACHABLE** with `ntlm: ... rejected` / `timed out` | WinRM to that DC failed | *Windows connection test* with Limit that DC; WINDOWS.md, "If something goes wrong" |
| LDAP **not checked** and `ActiveDirectory PowerShell module` under *Not read* | the module was removed from that DC | `Install-WindowsFeature RSAT-AD-PowerShell` |
| dcdiag **not run**: `no test result in its output (is dcdiag in English?)` | dcdiag prints another language | run the job with an account whose display language is English, or accept it (the other sections do not need dcdiag) |
| dcdiag: `The term 'dcdiag.exe' is not recognized` | the host is not a DC | take it out of `win_dc` |
| `objects deleted (reading Deleted Objects needs ...)` under *Not read* | the account may not list Deleted Objects | use an account in Domain Admins, or give the account read access to `CN=Deleted Objects` (Microsoft documents it: `dsacls` with `/takeownership`, then `/G <account>:LCRP`) |
| "By" is `-` everywhere | the change was made on a DC not in the job, or the audit policy does not record it | add the DC to `win_dc`; turn on the audit subcategories above |
| Replication **amber** every night on site links | the link replicates on a schedule longer than `ad_repl_warn_hours` | raise `ad_repl_warn_hours` above the site link's interval |
| Many **5719** / **5807** events | clients in subnets AD does not know | add the subnets in *Active Directory Sites and Services* (the *Clients in no site* table lists them) |
| GPO **version AD x, SYSVOL y** | SYSVOL has not caught up with an edit, or a GPO is broken on this DC | wait for the next run; if it stays, check DFSR on this DC (`dfsrdiag backlog`) |

## Limits

- Group membership is **direct**: a member added to a group that is itself in Domain Admins shows as a change of that group only if that group is watched (add it to `ad_privileged_groups_extra`).
- "Created" and "changed" times of accounts are AD's own; "changed" GPOs use the reading DC's `whenChanged` (a GPO whose links changed shows under *links*, not as changed).
- A forest with several domains: put the DCs of every domain in `win_dc`; each domain is read by one of its own DCs. Schema and forest-wide FSMO roles are reported by the forest root domain.
- The report covers what the DCs say about themselves and the domain. It does not test logons from clients, and it does not read Azure AD / Entra ID.

## Account lockout

`playbooks/ad_lockout.yml` answers the service desk's question: *is this account locked out, and what keeps locking it out?* Give it the account (`jdoe`, `CORP\jdoe` or `jdoe@corp.example.mil`); leave it empty and it lists **every account locked out now**, each with its source.

**How it works.** Every DC of the job (the group `win_dc`, or `target`) runs a PowerShell script over WinRM that asks only itself, as the health report does:

1. The **PDC emulator** finds the account (or every account locked out now) and the lockout policy that applies to it - the domain's, or its fine-grained password policy.
2. **Every DC** reads its own `badPwdCount`, last bad password and last logon of the account (these are **not replicated**: each DC counts the bad passwords it was given itself), and its **Security log** of the last `ad_lockout_hours` for the account:

| Event | What it says | Where it came from |
|---|---|---|
| 4740 | the account was locked out | *Caller Computer Name* |
| 4767 | the account was unlocked | by whom |
| 4771 | Kerberos pre-authentication failed (`0x18` = bad password) | the client's address |
| 4768 | a Kerberos ticket was refused (`0x12` = asked for while locked out) | the client's address |
| 4776 | NTLM credential validation failed (`0xC000006A` = bad password) | the workstation |
| 4625 | a logon to the DC failed | workstation, address, logon type, process |

Each address is resolved to a name (reverse DNS, asked of the DC), so the Kerberos and the NTLM failures of one computer are counted together.

3. The report ranks the **source computers**: the one that locked it out first, then the most failed sign-ins. For each it says what that usually means, in plain words - a steady pace ("about every 5 minutes") is a saved password retrying, not a person typing; attempts *after* the lock mean it will lock out again right after an unlock; logon type 4 is a scheduled task, 5 a service, 3 a mapped drive or an application, 10 Remote Desktop; a source that is one of your mail servers (`ad_lockout_exchange_servers`) is a phone or a mail app with the old password; an empty caller is a server signing in for the user (Exchange / OWA, NPS / RADIUS for Wi-Fi or VPN, a web application).

The report: **what to do** first, then the account (locked since, enabled, password last set or expired), **where the failed sign-ins came from** (source, address, bad passwords, after the lock, lockouts, first, last, pace, how, which DCs saw them, whether it is in the inventory), the **lockouts and unlocks**, and **each DC**'s own count (an unreachable DC in red). It is printed in the job's output, published as artifacts (`ad_lockout`) and emailed when `report_email_to` is set. The job stays green: it fails only when no DC could read its domain, an account asked for is in no domain (it stops before reading the Security logs), or an unlock was refused.

**Optional:**

- `ad_lockout_unlock: true` unlocks the account on the PDC emulator after reading, and reads it back - only an account asked for by name, and only when it is locked out. A dry run (Check) says *would be unlocked* and changes nothing. The report says it first: it locks out again as soon as the source tries its old password, so fix the source.

- `ad_lockout_check_sources: true` also looks on each source computer that is a host of the job's inventory (WinRM, the same credential, which needs administrator rights there) for what runs as the account: services (*Log On As*), scheduled tasks (*Run as*), its sessions - a **disconnected** Remote Desktop session keeps the old password - processes, and drives it has mapped. Read-only. A source that is not in the inventory (most workstations) is listed with what to check by hand.

### Before you start

- **The DCs** in the group `win_dc`, as for the health report (step 1 above). Put every DC there: a DC that is not in the job does not show its bad password count or its Security log, and a lockout recorded there is missed.
- **The audit policy** on the DCs (*Advanced Audit Policy Configuration*, in the Default Domain Controllers Policy): **Account Lockout** (Failure; 4740 is in *Audit User Account Management*, Success), **Kerberos Authentication Service** (Failure: 4771, 4768) and **Credential Validation** (Failure: 4776). The DISA STIG asks for all of them. Without them the report says *No failed sign-in … the DCs may not audit*.
- **A Security log big enough** to hold `ad_lockout_hours` of events (on a busy DC 24 h can be 1 GB or more); an older lockout has rolled out of it.
- **The account** for the job. The health report's account works. A smaller one is enough for this job alone - it only reads, and the unlock is one delegated right:
- **Reading the Security log** of the DCs: the built-in group **Event Log Readers** (of the domain: it applies on the DCs), plus **Remote Management Users** for WinRM.
- **Reading the accounts**: any domain user can.
- **Unlocking** (`ad_lockout_unlock`): *Active Directory Users and Computers*, right-click the OUs of the users > **Delegate Control** > the job's account > *Create a custom task* > *User objects* > the property permissions **Read lockoutTime** and **Write lockoutTime**. Nothing else.
- **Looking on the source computers** (`ad_lockout_check_sources`): an administrator of those computers (other users' processes and scheduled tasks are not readable otherwise). Use a separate template with that credential if your DC account must not log on to workstations.

### Set it up for the service desk

1. **Templates > Create template > Create job template:**
   - Name: `Active Directory - account lockout`
   - Inventory: `Windows servers`. Playbook: `playbooks/ad_lockout.yml`. Execution environment: your Windows EE. Credentials: the account above.
   - Variables (ready to paste: aap/extra_vars/ad_lockout.yml):

```yaml
ad_lockout_exchange_servers: [EXCH01, EXCH02]   # your mail servers, if any
```

   - Leave Limit empty (with no `target` it reads the group `win_dc`). Save.
2. **Survey** (template > *Survey* > *Add*), then switch the survey **on**:
   - *Account (jdoe, CORP\jdoe or a UPN; empty = every account locked out now)* - answer variable `ad_lockout_user`, **Text**, **not required**.
   - *Unlock it?* - answer variable `ad_lockout_unlock`, **Multiple Choice (single select)**, choices `false` and `true`, default `false`, not required. Leave this question out for a "look only" template, and keep the unlock template for the people allowed to unlock.

- Optionally *Look on the source computers?* - `ad_lockout_check_sources`, the same way.

3. **Access** (template > *User Access / Team Access*): give the service desk team the **Execute** role only - they launch it and read the output, they cannot change it.
4. **Try it** on your own account: type a wrong password three times on a test machine, then launch with your name. The source table should show that machine.

### Settings

| Setting | Default | What it does |
|---|---|---|
| `ad_lockout_user` | `""` | the account(s): `jdoe`, `CORP\jdoe`, `jdoe@corp.example.mil`; several separated by commas; empty = every account locked out now |
| `ad_lockout_hours` | `24` | how far back the Security logs are read |
| `ad_lockout_unlock` | `false` | `true` = unlock it on the PDC emulator (an account asked for by name, locked out) |
| `ad_lockout_check_sources` | `false` | `true` = look on the source computers that are hosts of the inventory |
| `ad_lockout_check_max` | `5` | source computers looked on per account (the worst first) |
| `ad_lockout_exchange_servers` | `[]` | your mail servers by name or address (`*` and `?` allowed) |
| `ad_lockout_max_users` | `25` | accounts per run (asked for, or locked out now) |
| `ad_lockout_max_events` | `2000` | Security log entries read per account and DC (the newest) |
| `ad_lockout_resolve_max` | `50` | addresses resolved to names per DC |
| `ad_lockout_max_rows` | `50` | rows per table (the artifacts have all) |

**All settings** of this job, with their defaults: ad_lockout.

## Troubleshooting

| You see | It means | Do |
|---|---|---|
| `No account named jdoe in corp.example.mil` | the name is not the sign-in name, or the account is in another domain | the *User logon name (pre-Windows 2000)*, `CORP\jdoe` or the UPN; for another domain, put its DCs in `win_dc` |
| `No failed sign-in ... the DCs may not audit` | the audit subcategories above are off, or the events have rolled out of the Security log | turn them on; raise the Security log's size; a shorter `ad_lockout_hours` for a fresh lockout |
| `Security log for jdoe: Attempted to perform an unauthorized operation` under *Not read* | the account may not read the Security log | add it to **Event Log Readers** |
| A source shown only as an **address** | no reverse DNS (PTR) record for it | look the address up in DHCP; add PTR records |
| The source is the **DC itself** or the caller is **(not recorded)** | a server signed in for the user: NPS / RADIUS, Exchange, a web application, an LDAP bind | look at that server's own logs (NPS: *Network Policy Server* events 6273) |
| `NOT unlocked: Insufficient access rights` (the job fails) | the account has no *Write lockoutTime* on that user's OU | delegate it (above); an account in a protected group (Domain Admins) is protected by AdminSDHolder - unlock it by hand |
| It locks out again right after the unlock | the source still has the old password | fix the source first (the report's *what to do*), then unlock |

## Limits

- Failed sign-ins the DCs of the job did not see are missed: every DC should be in `win_dc`.
- A sign-in with cached credentials (a laptop off the network) does not reach a DC: it is counted only once the laptop reaches a DC again.
- Credential Manager entries, saved passwords in applications and phones cannot be read remotely: the report says where to look.
- One forest at a time per job: a trusted domain's lockouts are on its own DCs (put them in the inventory, or run the job there).

# 28. Horizon 8 (VDI): reports, broken desktops, user help, image push

Nine job templates for Horizon 8 with instant clones. They talk to a Connection Server's REST API (`https://<connection-server>/rest/`) from the AAP side: no inventory, no login to the desktops, no collection to add to the execution environment.

| Job template | Playbook | What it does | Changes anything? |
|---|---|---|---|
| Horizon - API test | `horizon_api_test.yml` | the first job to run: login, each API path and version that answers, the settings against the pools, one user | no |
| Horizon - health report | `horizon_health_report.yml` | Connection Servers (status, services, certificate, replication), gateways, vCenter and AD from each Connection Server, the event database, SAML and True SSO, the license, pools that cannot provision, broken desktops | no |
| Horizon - capacity report | `horizon_capacity_report.yml` | per pool: maximum, free, in use, broken, sessions, how full | no |
| Horizon - desktops check | `horizon_desktops_check.yml` | the broken desktops, what fixes each, and why some are held back (job 1 of the broken desktops workflow) | no |
| Horizon - fix the approved desktops | `horizon_desktops_fix_approved.yml` | after the approval: fixes exactly the approved desktops (job 2) | yes |
| Horizon - user help | `horizon_user_help.yml` | for the service desk: a user's sessions and desktops; log off, disconnect, reset, restart or recover them | only when asked |
| Horizon - session cleanup | `horizon_session_cleanup.yml` | logs off sessions disconnected for 8 hours or more | yes |
| Horizon - image push plan | `horizon_image_push_plan.yml` | the plan for a new golden image: pools, VM, snapshot, start (job 1 of the image push workflow) | no |
| Horizon - push the approved image | `horizon_image_push_approved.yml` | after the approval: schedules exactly the approved push (job 2) | yes |

Every job reads first and says what it found. A dry run (Job type **Check**) changes nothing and says what it would do. No job emails anything unless `report_email_to` is set on its template.

## Your Horizon version

The jobs do not need to know it. Horizon's REST API has a version in each path (`/rest/inventory/v2/machines`), and newer releases add newer versions. For each path a job asks the newest version first (`horizon_api_newest`, v9) and works down to v1. The first version that answers is used, and the reports print which ones answered. If your release does not have a part at all (the license on older releases, for example), the jobs leave it out and say so once.

**Run *Horizon - API test* first.** It prints a line for each part of the API:

```
PASS machines: rest/inventory/v2/machines - 412 record(s) and more
----  license: not in this Horizon (HTTP 404 at every API version, v9 to v1)
FAIL sessions: the account may not read rest/inventory/v1/sessions (HTTP 403): give its Horizon
  role the privileges in docs/HORIZON.md
```

`----` is fine: that part is not in your release. `FAIL` means the part is there but the account may not read it (the role below), or something else went wrong (the line says what).

**What was built from the API documentation and not yet tried on a real Horizon.** The tests run against a stand-in (`tests/horizon/hzmock.py`) written from the Horizon Server REST API reference. Check these on yours. Your Connection Server documents its own API at `https://<connection-server>/rest/swagger-ui.html`.

- **Paths.** The jobs use:
- `monitor/connection-servers`, `gateways`, `virtual-centers`, `ad-domains`, `event-database`, `saml-authenticators`, `true-sso`
- `config/licenses` (also tried: `config/license`, `monitor/licenses`)
- `inventory/desktop-pools`, `machines`, `sessions`, `farms`
- `external/base-vms`, `base-snapshots`, `ad-users-or-groups`
- the actions: `inventory/v1/machines/action/{recover,reset,restart,rebuild}`, `DELETE inventory/v1/machines` (remove), `inventory/v1/sessions/action/{logoff,disconnect}`, `inventory/v1/desktop-pools/{id}/action/schedule-push-image`
- **Field names.** Each value is looked for under the names the API versions use, for example `pattern_naming_settings.max_number_of_machines`, `provisioning_status_data.instant_clone_pending_image_state`, `session_state` and `disconnected_time`. A missing value is never an error, but a report column can stay empty. If one does, tell us the field name your swagger page shows.
- **The user search.** `external/ad-users-or-groups` is searched with a filter on `login_name`, and also on `name` for versions that take that instead. *API test* with `horizon_test_user: <someone>` shows whether your release finds users that way.
- **A push already running.** A pool counts as busy when `instant_clone_pending_image_state` or `instant_clone_operation` is set (and not `NONE`). The plan refuses a busy pool.
- **The actions answer.** Horizon answers each ID of a bulk action with `status_code` and `errors`. A refusal is shown with Horizon's own reason.

## Step 1. The service account and its Horizon role

Ask for an AD account (for example `svc-horizon-aap`) whose password does not expire, or put it on your rotation. Then, in Horizon Console, **Settings › Administrators**:

- **For the reports and the API test**: the built-in role **Administrators (Read only)** on the root access group `/`. It reads everything and changes nothing.
- **For the jobs that change something** (fix, user help, session cleanup, image push): a role of your own, under **Roles › Add Role**, named for example `AAP automation`, with these privileges:
- **Console Interaction** (to log in)
- **Manage Machine** and **Manage Reboot Operation** (reset, restart, recover, remove)
- **Manage Sessions** (log off, disconnect)
- **Manage Desktop Pool** (schedule an image push)

Give it on the access group of the pools the jobs may change, or on `/` for all of them. Privilege names differ a little between releases: pick the closest ones your Console lists.

Two accounts, each with one credential, keep the reports read-only: a read-only one for the reports and the API test, the other for the jobs that act. One account with the second role also works.

From the execution environment, the Connection Server (or the load balancer in front of the Connection Servers) must answer on **443**. Its certificate must be trusted by the execution environment (your DoD CA in the EE, docs/EXECUTION_ENVIRONMENT.md), or set `horizon_ca_path`.

## Step 2. The credential type and the credentials

1. **Automation Execution › Infrastructure › Credential Types › Create credential type**, name `Horizon API`. Paste the `inputs` and `injectors` blocks of aap/credential_types/horizon_api.yml.
2. **Credentials › Create credential**, type `Horizon API`:
   - **Connection Server URL** e.g. `https://horizon.example.mil`
   - **AD domain** the NetBIOS name, e.g. `CORP`
   - **User name** without the domain
   - **Password**
   - **Verify the Connection Server's certificate** ticked

A second credential, e.g. `Horizon API (read only)`, for the read-only account.

## Step 3. The job templates

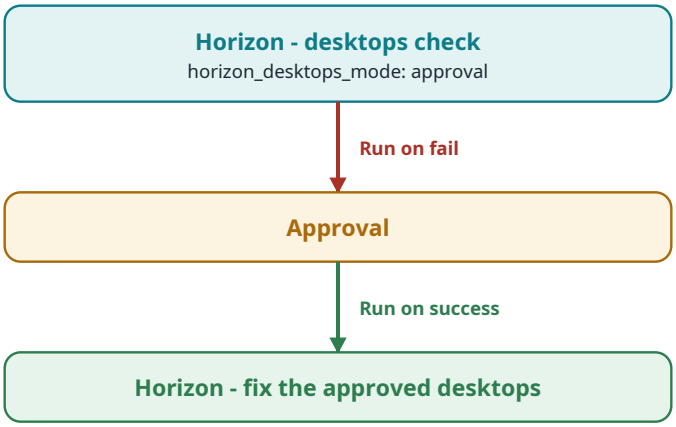
Create one template per row: **Inventory** any (the jobs run on the AAP side, against Horizon), **Project** this repository, the **Playbook**, the credential. Paste the template's file from `aap/extra_vars/` into **Variables** and switch on what you change.

| Template | Credential | Survey | Notes |
|---|---|---|---|
| Horizon - API test | read only | `horizon_test_user` (text, optional) | run it after any change to the account or the Connection Server |
| Horizon - health report | read only | none | |
| Horizon - capacity report | read only | none | |

| Template | Credential | Survey | Notes |
|---|---|---|---|
| Horizon - desktops check | read only | none | `horizon_desktops_mode: approval` when it is the workflow's first node |
| Horizon - fix the approved desktops | acting | none | **Prompt on launch** unticked for Variables; Job type: tick Prompt on launch to rehearse with Check |
| Horizon - user help | acting | `horizon_user` (text, **required**), `horizon_user_action` (multiple choice: `look`, `logoff`, `disconnect`, `reset`, `restart`, `recover`; default `look`) | give the service desk Execute on this template only |
| Horizon - session cleanup | acting | none | first run it as Check |
| Horizon - image push plan | read only | none: the survey is on the workflow | |
| Horizon - push the approved image | acting | none | **Prompt on launch** unticked for Variables |

The protected pools, `horizon_protected_pools` (e.g. `[VIP-*]`), belong in `playbooks/group_vars/all.yml`. No job fixes, logs off or pushes an image to those pools.

## Workflow: fix broken desktops, with an approval




1. **Templates › Create template › Create workflow job template**, name `Horizon - fix broken desktops (workflow)`. Save, and the visualizer opens.
2. **Add step**: *Horizon - desktops check*. Its Variables already say `horizon_desktops_mode: approval`, from Step 3.
3. On it, **+** › **Run on fail** › **Approval**:
   - name `Fix these desktops?`
   - timeout e.g. 60 minutes
   - description `Open the desktops check job: the NEEDS APPROVAL line lists each desktop and its fix.`

4. On the approval, **+** › **Run on success** › *Horizon - fix the approved desktops*. **Save**.

What happens:

- **Nothing broken**: the check is green, and the workflow ends there.
- **Desktops to fix**: the check fails on purpose with `NEEDS APPROVAL: 3 broken desktop(s) to fix - w11-004 (AGENT_UNREACHABLE: recover), ...` and hands the list to the next steps (`set_stats horizon_desktops_fix`). The approver reads it and approves or denies.
- **After the approval**, the fix job reads each desktop again. It fixes a desktop only when it is still broken, no session is on it and its pool is not protected, with exactly the approved action. A desktop that recovered meanwhile is left alone, and the report says so.

A check that fails for another reason, a refused login for example, also takes the "Run on fail" path. Its approval then shows no NEEDS APPROVAL line, and the fix job refuses with `Nothing to fix: no approved list`. Deny it, and fix the login.

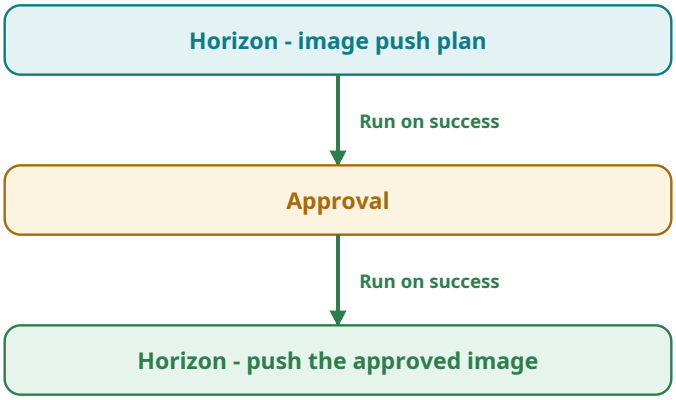
**What fixes a desktop.**

- **Recover** (`horizon_fix_default_action`): the instant clone is deleted and made again from the pool's image.
- **By state** (`horizon_fix_actions`): another action for some states, `restart` for `AGENT_ERR_NEED_REBOOT` by default.
- **Held back for a person**:
- a desktop with a session on it
- one in a protected pool
- one in a pool that is not instant clones (recover is for instant clones)
- `remove` of a desktop assigned to a user
- anything over `horizon_fix_max` (20 per run)

**Schedule** the workflow e.g. every hour in working hours. For a list without the approval, make a second template, *Horizon - desktops check (report)*: the same playbook, without `horizon_desktops_mode` in its Variables (`report` is the default), on its own schedule. The workflow's template keeps `approval`.

**Rehearse it.** Tick **Prompt on launch** for Job type on *Horizon - fix the approved desktops*. In the visualizer, set its node's Job type to **Check**. Run the workflow and approve: the fix job prints `w11-004: would recover (a dry run)` and changes nothing.

## Workflow: push a new golden image, with an approval



1. Create a workflow job template, `Horizon - image push (workflow)`, with this **survey**:
   - `horizon_push_pools`: textarea, required. The pools, one per line.
   - `horizon_push_base_vm`: text, required. The golden image VM as vCenter names it.
   - `horizon_push_snapshot`: text, required. The snapshot's name, or its path when two snapshots have that name.
   - `horizon_push_start`: text, optional. Empty = once approved. Or `2026-11-01 22:00` (UTC).
   - `horizon_push_logoff_policy`: multiple choice, `WAIT_FOR_LOGOFF` (default) or `FORCE_LOGOFF`.
2. Nodes: *Horizon - image push plan* › **Run on success** › **Approval** › **Run on success** › *Horizon - push the approved image*. The approval: name `Push this image?`, timeout e.g. 8 hours, description `Open the image push plan job: the READY FOR APPROVAL line says what, where and when.`
3. **Rehearse it** before the first real push: tick **Prompt on launch** for Job type on *Horizon - push the approved image*, set its node's Job type to **Check**, launch and approve. It prints what it would schedule and schedules nothing.

The plan job checks everything in Horizon:

- each pool is an instant-clone pool, not protected, with no push already scheduled or running
- the VM and the snapshot are found in each pool's vCenter
- the start is not in the past

A plan with a problem **fails**, and nothing reaches the approval. A good plan prints `READY FOR APPROVAL: Push W11-Gold / /2026-10/2026-11 to IC-Win11, IC-Finance, starting ...; users: each desktop is replaced when its user logs off.` It also prints any warnings, such as Horizon flagging the VM or the same image being pushed again.

The plan reaches the push job as the job artifact `horizon_push_plan` (on the plan job: **Details** > **Artifacts**). The push job reads each pool again and schedules exactly the approved push. A start time that has passed by the approval starts in a minute. Progress is in Horizon Console (the pool's **Summary**), and the next health report shows a failed push in red.

## User help (the service desk)

*Horizon - user help* with `horizon_user` = `jdoe`, `CORP\jdoe`, `jdoe@corp.example.mil` or a SID:

```
CORP\jdoe (Jane Doe), S-1-5-21-...: 2 session(s), 1 desktop(s)
Session: desktop on w11-002 (pool IC-Win11), connected, started 2026-10-20 08:02 UTC, from
  LAPTOP-12, BLAST
Session: application on (an RDS host) (pool farm RDS-Farm), disconnected since 2026-10-19 17:30
  UTC
Desktop: w11-002 (pool IC-Win11) is CONNECTED
```

Then, with `horizon_user_action`:

- **logoff**: all their sessions; a locked one too, `horizon_session_forced_logoff`
- **disconnect**: their connected sessions
- **reset**: their desktops, like the power button
- **restart**: their desktops; the OS shuts down first
- **recover**: their instant-clone desktops, made again from the image

It never acts on a desktop another user has a session on, nor on one in a protected pool. A user with no session and no desktop is usually not entitled to a pool: the job says where to look.

## Session cleanup

*Horizon - session cleanup* logs off every session disconnected for `horizon_session_max_disconnected_hours` (8) or more. Such a session holds a desktop, and an instant clone is only made fresh at log off. The oldest are logged off first, at most `horizon_session_cleanup_max` (100) per run.

It never logs off sessions in these:

- a protected pool
- `horizon_session_cleanup_exclude_pools`
- `horizon_session_cleanup_exclude_users`, given as login names or SIDs

Nor a session whose disconnect time Horizon does not give. It sends no message first: no one sees a message in a disconnected session.

Run it as Check first. Then schedule it, e.g. every night at 02:00. For a list only, set `horizon_session_cleanup_report_only: true`.

## Reports and schedules

| Template | Schedule (suggested) | Red when |
|---|---|---|
| Horizon - health report | every day 06:00 | a Connection Server, service, gateway or vCenter link down; a certificate within `horizon_cert_crit_days` (7) or not valid; the event database not connected; the license expired; a pool that stopped provisioning or whose image failed |
| Horizon - capacity report | every Monday 07:00 | a pool at `horizon_capacity_crit_pct` (95 %) in use, or with no free desktop (amber at 80 %, or fewer free desktops than its spares) |

Both stay green when they find problems: the report says what. With `horizon_health_fail: true` the job turns **red** when it finds a problem: put a notification on the template (its **Notifications** tab: on failure) to hear about it. The *ServiceNow tickets* job does not take this report's results - it only works after the checks that hand it findings (JOB_TEMPLATES.md, "What one job hands to the next"). To email them, set `report_email_to` on the template (EMAIL_REPORTS.md).

## Messages and what to do

| Message | What to do |
|---|---|
| `No Horizon connection: attach a credential of type "Horizon API"` | attach the credential to the template |
| `Horizon did not answer at https://...: ...` | the address, a firewall (443 from the execution environment), or the certificate (`horizon_ca_path`) |
| `Horizon refused the login of CORP\svc-... (HTTP 401 ...)` | the password, the domain, or the account has no Horizon role (Step 1) |
| `the account may not read rest/... (HTTP 403)` | the role lacks a privilege for that part (Step 1) |
| `Horizon refused N action(s) ...` / `REFUSED - recover refused (HTTP 400): ...` | Horizon's own reason follows; often the desktop is busy (a task already running): the next run tries again |
| `Nothing to fix: no approved list` / `Nothing to push: no approved plan` | the job was started outside its workflow, or the check failed for another reason: run the workflow |
| `2 users are sam (CORP\sam, LAB\sam): write it DOMAIN\user` | give the domain |

**All settings** of these jobs, with their defaults: horizon.

# Appendix: credential types to paste into AAP

Each block below is one file in `aap/credential_types/`. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**. Paste the part under `inputs:` into **Input configuration** and the part under `injectors:` into **Injector configuration** (without those two top-level lines). Copy from the repository file where you can; PDF copy can change spaces.

## ServiceNow API (servicenow_api.yml)

Used by playbooks/servicenow_tickets.yml, servicenow_health.yml and servicenow_test_ticket.yml (REST Table API, basic auth only). The same variable names as the certified servicenow.itsm collection, so this credential also works if you move to that collection later.

YAML

```yaml
inputs:
  fields:
    - id: instance
      type: string
      label: Instance URL
      help_text: e.g. https://servicenow.example.mil
    - id: username
      type: string
      label: API user name
    - id: password
      type: string
      label: API password
      secret: true
  required: [instance, username, password]
injectors:
  env:
    SN_HOST: "{{ instance }}"
    SN_USERNAME: "{{ username }}"
    SN_PASSWORD: "{{ password }}"
```

## MariaDB monitor (mariadb_monitor.yml)

For the mariadb health check. Needed when MariaDB runs in a container (the official images give root a password); optional on a host install, where the check can log in as root through the local socket. The password reaches the client only through its environment (MYSQL_PWD), never a command line. The account needs no data access at all (docs/MARIADB.md section 4 has the GRANT).

```yaml
inputs:
  fields:
    - id: username
      type: string
      label: Monitor account
    - id: password
      type: string
      label: Password
      secret: true
  required: [username, password]
injectors:
  env:
    MARIADB_MONITOR_USER: "{{ username }}"
    MARIADB_MONITOR_PASSWORD: "{{ password }}"
```

## Keystore password (keystore_password.yml)

Optional, for the certs check on a PKCS12 keystore (JKS keystores list without a password). In the inventory, name the variable, never the password: check_certs_keystores: [{path: /path/to/keystore, password_env: KEYSTORE_PASSWORD}]

```yaml
inputs:
  fields:
    - id: password
      type: string
      label: Keystore password
      secret: true
  required: [password]
injectors:
  env:
    KEYSTORE_PASSWORD: "{{ password }}"
```

## STIG Manager API (stigman_api.yml)

Optional, for the STIG Manager cross-check in playbooks/poam_status.yml and the checklist comparison in playbooks/stig_drift_report.yml (both only read). The client is created by your Keycloak realm owner (client-credentials grant, scope stig-manager:collection:read, and a grant in the collections to read) - this repository never changes the realm.

```yaml
inputs:
  fields:
    - id: token_url
      type: string
      label: Token URL
      help_text: https://keycloak.example.mil/realms/<realm>/protocol/openid-connect/token
    - id: client_id
      type: string
      label: Client ID
    - id: client_secret
      type: string
      label: Client secret
      secret: true
  required: [token_url, client_id, client_secret]
injectors:
  env:
    STIGMAN_TOKEN_URL: "{{ token_url }}"
    STIGMAN_CLIENT_ID: "{{ client_id }}"
    STIGMAN_CLIENT_SECRET: "{{ client_secret }}"
```

## ACT model key (act_model_key.yml)

```yaml
inputs:
  fields:
    - id: genai_key
      type: string
      label: GenAI API key
      secret: true
    - id: asksage_key
      type: string
      label: AskSage API key
      secret: true
injectors:
  env:
    GENAI_KEY: "{{ genai_key }}"
    ASKSAGE_API_KEY: "{{ asksage_key }}"
```

## ACT GenAI beta key (act_genai_beta_key.yml)

Only for ACT with site_act_provider=genai-beta (the GenAI.mil beta proxy). A separate type, so an "ACT model key" type that is already in use does not have to change (AAP refuses to change the fields of a credential type that credentials use). Attach it next to "ACT model key".

```yaml
inputs:
  fields:
    - id: genai_beta_key
      type: string
      label: GenAI beta API key
      secret: true
  required: [genai_beta_key]
injectors:
  env:
    GENAI_BETA_KEY: "{{ genai_beta_key }}"
```

## STIG Manager database (stigman_database.yml)

```yaml
inputs:
  fields:
    - id: mysql_root_password
      type: string
      label: MySQL root password
      secret: true
      help_text: Used only when the database is first created (MYSQL_ROOT_PASSWORD_FILE).
    - id: stigman_db_password
      type: string
      label: STIG Manager database user password
      secret: true
  required: [mysql_root_password, stigman_db_password]
injectors:
  env:
    STIGMAN_MYSQL_ROOT_PASSWORD: "{{ mysql_root_password }}"
    STIGMAN_DB_PASSWORD: "{{ stigman_db_password }}"
```

## TLS certificate (tls_certificate.yml)

AAP writes each value to a private temporary file inside the job's execution environment and puts the file's PATH in the environment variable; the playbook copies the files to the host.

```yaml
inputs:
  fields:
    - id: certificate
      type: string
      label: Certificate (PEM; server certificate first, then intermediates)
      multiline: true
    - id: private_key
      type: string
      label: Private key (PEM)
      multiline: true
      secret: true
    - id: ca_chain
      type: string
      label: CA chain (PEM) - also used to trust your Keycloak/other internal services
      multiline: true
  required: [certificate, private_key]
injectors:
  file:
    template.cert: "{{ certificate }}"
    template.key: "{{ private_key }}"
    template.ca: "{{ ca_chain }}"
  env:
    TLS_CERT_FILE: "{{ tower.filename.cert }}"
    TLS_KEY_FILE: "{{ tower.filename.key }}"
    TLS_CA_FILE: "{{ tower.filename.ca }}"
```

## Container registry login (hosts) (registry_login.yml)

For podman on the MANAGED HOSTS (the built-in "Container Registry" credential type is for pulling execution-environment images on AAP itself, not for the hosts).

```yaml
inputs:
  fields:
    - id: registry
      type: string
      label: Registry (e.g. docker.io, quay.io)
    - id: username
      type: string
      label: Username
    - id: password
      type: string
      label: Password or access token
      secret: true
  required: [registry, username, password]
injectors:
  env:
    REGISTRY_HOST: "{{ registry }}"
    REGISTRY_USERNAME: "{{ username }}"
    REGISTRY_PASSWORD: "{{ password }}"
```

## Teams webhook (teams_webhook.yml)

```yaml
inputs:
  fields:
    - id: webhook_url
      type: string
      label: Teams Workflows webhook URL
      secret: true
  required: [webhook_url]
injectors:
  env:
    TEAMS_WEBHOOK_URL: "{{ webhook_url }}"
```

## SMTP relay (smtp_relay.yml)

Only when your mail relay needs a login to send. The jobs that email a report (report_email_to, roles/site_email) read SMTP_USERNAME / SMTP_PASSWORD from it. A relay that accepts mail from the AAP server without a login needs no credential.

```yaml
inputs:
  fields:
    - id: username
      type: string
      label: SMTP user name
    - id: password
      type: string
      label: SMTP password
      secret: true
  required: [username, password]
injectors:
  env:
    SMTP_USERNAME: "{{ username }}"
    SMTP_PASSWORD: "{{ password }}"
```

## Report website (report_web.yml)

The login for the report website (roles/site_email, docs/WEB_REPORTS.md): - on a Windows server (report_web_host): the user name (DOMAIN\user) and password the job logs in with over WinRM to write the pages - without this credential, the template's Machine credential; - for uploads (report_web_dest: https://...): a token, sent as "Authorization: Bearer <token>", or the user name and password, as basic authentication. A report folder (a mounted share) needs no credential.

YAML

```
inputs:
  fields:
    - id: username
      type: string
      label: User name (DOMAIN\user for a Windows server)
    - id: password
      type: string
      label: Password
      secret: true
    - id: token
      type: string
      label: Upload token (instead of a user name and password)
      secret: true
injectors:
  env:
    REPORT_WEB_USERNAME: "{{ username | default('') }}"
    REPORT_WEB_PASSWORD: "{{ password | default('') }}"
    REPORT_WEB_TOKEN: "{{ token | default('') }}"
```

## Horizon API (horizon_api.yml)

Used by playbooks/horizon_*.yml (the Connection Server's REST API, https://<connection-server>/rest/). The account: an AD account with a Horizon administrator role - the least it needs is in docs/HORIZON.md.

YAML

```yaml
inputs:
  fields:
    - id: url
      type: string
      label: Connection Server URL
      help_text: e.g. https://horizon.example.mil (a Connection Server, or the load balancer in
        front of them)
    - id: domain
      type: string
      label: AD domain
      help_text: The account's domain, NetBIOS name (e.g. CORP)
    - id: username
      type: string
      label: User name
      help_text: The account's login name, without the domain (e.g. svc-horizon-aap)
    - id: password
      type: string
      label: Password
      secret: true
    - id: verify_ssl
      type: boolean
      label: Verify the Connection Server's certificate
      default: true
  required: [url, domain, username, password]
injectors:
  env:
    HORIZON_URL: "{{ url }}"
    HORIZON_DOMAIN: "{{ domain }}"
    HORIZON_USERNAME: "{{ username }}"
    HORIZON_PASSWORD: "{{ password }}"
    HORIZON_VERIFY_SSL: "{{ verify_ssl }}"
```